
generatingfunctionology (生成函数学)

第三版 (Third Edition)

作者: Herbert S. Wilf

序言 (Preface)

本书是关于生成函数 (generating functions) 及其在离散数学中的一些应用的。这个主题极其宏大, 因此我并没有试图进行面面俱到的全面讨论, 而是仅仅试图传达其中一些核心的思想。

生成函数是一座桥梁, 它的一端是离散数学, 另一端则是连续分析 (特别是复变函数理论)。我们可以仅仅将它们作为解决离散问题的工具来学习。就此而言, 生成函数为处理这类问题提供了一种统一的方法, 这种方式既强大又神奇。如果读者想跳过本学科中关于分析 (解析) 的部分, 可以直接跳过第5章以及前面章节中的部分相关内容。

然而, 如果省略掉这门学科的分析部分, 就像是只用左声道去听贝多芬第九交响曲的立体声广播一样。

只有同时调频接收“离散”和“连续”这两个声道, 生成函数这门学科的全部魅力才会显现出来。看它们如何让求解差分方程变得像小孩子做游戏一样简单; 再看复变函数理论如何让你几乎只需“看一眼”就能得出解的渐近规模 (近似大小)。这两个声道之间的相互交织, 对于欣赏这首“数学乐曲”来说至关重要。

近年来, 寻找组合定理的**双射证明 (bijective proofs)**成为了一种强劲的趋势。也就是说, 如果我们想证明两个集合具有相同的基数 (元素个数), 我们就应该能够通过展示这两个集合之间的一个显式的双射来完成证明。在许多情况下, 这两个集合具有相同基数的**事实**, 最初正是通过生成函数的论证被发现的。此外, 即使双射的论证方法已知, 生成函数的证明往往更简短或更优雅。

双射证明会给人一种令人满足的感觉, 让人觉得自己“真正”理解了为什么定理是成立的。而生成函数的论证常常能给人一种自然而然的满足感, 让人发出“哦, 我本来也能想到那个”的感叹, 同时, 它通常也是寻找目标序列的精确公式或渐近公式的最佳途径。

本书曾作为宾夕法尼亚大学离散数学高年级课程的教材进行了测试。感谢那门课的学生们帮助我找出了手稿中的部分错误, 也感谢我的许多同事提出了许多有益的建议。如果有哪位好心的读者能给我发送纠错信息, 他将收到一份最新的完整勘误表, 并获得我最诚挚的谢意。

Herbert S. Wilf

宾夕法尼亚州，费城

1989年9月1日

(注：第二版和第三版序言主要说明了新版增加了使用计算机代数系统（如 Maple 和 Mathematica）的附录，修复了错误，并因为网络下载量巨大而重新发行了纸质版。此处略过详细翻译，直接进入正文。)

1. 介绍性概念与示例 (Introductory Ideas and Examples)

“生成函数就像一根晾衣绳，我们把一串数字挂在上面以便展示。”

(A generating function is a clothesline on which we hang up a sequence of numbers for display.)

这句话的意思是这样的：假设我们有一个问题，其答案是一个数字序列 $a_0, a_1, a_2, \dots$ 。我们想要“知道”这个序列是什么。我们能期望得到什么样的答案呢？

对于 a_n 来说，一个简单的公式将是我们能期望的最好结果。如果我们发现对于每个 $n = 0, 1, 2, \dots$ ，都有 $a_n = n^2 + 3$ ，那么毫无疑问，我们已经“回答”了这个问题。

但是，如果这个未知序列的成员没有任何简单的公式呢？毕竟，有些序列是非常复杂的。举一个令人毛骨悚然的例子，假设未知序列是 2, 3, 5, 7, 11, 13, 17, 19, ...，其中 a_n 是第 n 个素数。这种情况下，期望找到任何形式的简单公式简直是不讲道理的。

生成函数为你的武器库又增添了一件利器。 尽管给出一个序列成员的简单公式可能是不现实的，但我们也许能够给出一个**幂级数求和 (sum of a power series)**的简单公式，而这个**幂级数的系数正是我们正在寻找的那个序列**。

例如，假设我们想求斐波那契数列 (Fibonacci numbers) $F_0, F_1, F_2, \dots$ ，而我们对它们的了解仅仅是它们满足如下递推关系：

$$F_{n+1} = F_n + F_{n-1} \quad (n \geq 1; F_0 = 0; F_1 = 1)$$

这个序列的前几项是 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

如我们稍后在 1.3 节将会看到的，对于 F_n 确实存在精确且不是非常复杂的公式。但是，仅仅为了让你理解生成函数的概念，以下是一位“生成函数学家 (generatingfunctionologist)”可能会如何回答这个问题：

第 n 个斐波那契数 F_n ，是函数 $x/(1 - x - x^2)$ 在 origin 处展开为幂级数时， x^n 的系数。

你可能会勉强承认这也是一种答案，但它会留下一一种某种程度上的不满足感。你可能会说，这不算是**真正**的答案，因为我们没有得到那个显式公式。这算是一个好答案吗？

在本书中，我们希望让你相信：**像这样的答案通常是极其出色的**。因为它们本身非常优雅，它们允许你对你的序列做几乎任何你想做的事情；而且，即使在精确公式可能极其复杂的情况下，生成函数也可以保持简单且易于处理。

以下是你通常能够用生成函数答案来做的一些事情：

(a) 找到序列成员的精确公式。 不总是能行。如果你的序列很复杂，找到的方式可能也不那么令人愉快。但至少，你会有一个寻找这种公式的绝佳机会。

(b) 找到递推公式。 生成函数最常来源于递推公式。然而，有时候从生成函数中，你会发现一个新的递推公式（不是你一开始用的那个），这能为你深入了解序列的本质提供新的视角。

(c) 找到序列的平均值及其他统计特性。 生成函数能够极其迅速地推导出，由未知序列所代表的问题中的各种概率统计特征。

(d) 找到序列的渐近公式 (Asymptotic formulas)。 该理论中最深刻、最强大的应用就体现在这里。通常，我们面对的是一个非常困难的序列，由于寻找精确公式是不可能的，我们转而寻找**近似 (approximate)** 公式。例如，虽然我们不指望找到第 n 个素数的精确公式，但有一个极其优美的数学事实（“素数定理”）：在某种精确的意义上，第 n 个素数在 n 很大时大约等于 $n \ln n$ 。在第5章中，我们将讨论渐近性问题。

(e) 证明单峰性 (unimodality)、凸性 (convexity) 等。 如果一个序列先稳步上升，然后稳步下降，我们就称之为**单峰的**。许多组合序列都是单峰的，且有多种方法可以用来证明这类定理。生成函数在这方面能帮上大忙。有一些方法可以将生成函数的解析性质转化为关于系数序列升降的结论。当生成函数方法有效时，它通常是已知最简单的方法。

(f) 证明恒等式 (identities)。 在组合数学和数学的其他领域中，存在着许许多多的恒等式。我们所指的恒等式是指：对于自由变量的指定值，断言某个公式等于另一个公式。例如，众所周知的恒等式：

$$\sum_{j=0}^n \binom{n}{j}^2 = \binom{2n}{n} \quad (n = 0, 1, 2, \dots)$$

证明此类恒等式的一种方法是：考虑以宣称的恒等式左边为系数的生成函数，再考虑以右边为系数构造的生成函数，并证明这两个是同一个函数。这听起来可能显而易见，但当你从一位黑带级别的“生成函数学家”的视角来看时，你会惊叹于许多推导变得多么简单和透明。我们在后面的 4.3 节中介绍的“万金油 (Snake Oil)”方法将探索其中一些奇妙的境界。在 4.4 节中的有理函数方法则是较新的技术，能处理更多、更难的此类问题。

(g) 其他作用。 关于你的序列，你还想知道别的什么吗？生成函数也许能提供希望。一个例子可能是发现同余关系。另一种可能性是，你的生成函数可能与另一个已知的生成函数有着惊人的相似之处，这可能会引导你发现你的问题与另一个你以前从未怀疑过的问题密切相关。值得注意的是，通过这种方式，你可能会发现**你的问题的答案与另一个问题的答案简单相关，即使你并不知道这两个问题中任何一个的答案公式！**

在本章接下来的部分中，我们将给出一些可以通过生成函数视角进行有益思考的例子。我们希望在学完这些例子后，读者能至少在一定程度上被这种方法的强大力量所折服，并领略到这种统一方法的美感。

1.1 一个简单的两项递推关系 (An Easy Two Term Recurrence)

某个数字序列 $a_0, a_1, \dots$ 满足以下条件：

$$a_{n+1} = 2a_n + 1 \quad (n \geq 0; a_0 = 0). \quad (1.1)$$

求这个序列。

首先，试着计算该序列的前几项，看看它们是什么样子的。它以 0, 1, 3, 7, 15, 31, ... 开始。这些数字看起来极其可疑，它们就像是 2 的幂减 1。因此，我们可以猜想

$$a_n = 2^n - 1 \quad (n \geq 0),$$

然后基于递推关系 (1.1) 通过数学归纳法快速证明它。

但是，这是一本关于**生成函数**的书。所以，让我们忘掉所有这些，假装我们没有看出这个公式，然后使用生成函数方法。因此，我们不去寻找序列 $\{a_n\}$ ，而是去寻找它的生成函数

$A(x) = \sum_{n \geq 0} a_n x^n$ 。一旦我们知道那个函数是什么，我们就能通过将 $A(x)$ 展开成级数，直接“读出” a_n 的显式公式。

为了求 $A(x)$ ，将递推关系 (1.1) 的两边同时乘以 x^n ，并对递推关系成立的所有 n 值求和（在这个例子中，就是对 $n \geq 0$ 求和）。然后，试着将这些和式与那个未知的生成函数 $A(x)$ 联系起来。

如果我们首先对 (1.1) 的左边进行这个操作，结果是 $\sum_{n \geq 0} a_{n+1} x^n$ 。我们如何把它与 $A(x)$ 联系起来呢？它几乎与 $A(x)$ 相同。只是在每一项中，“ a ”的下标比 x 的幂次大 1。不过，很明显地：

$$\begin{aligned} \sum_{n \geq 0} a_{n+1} x^n &= a_1 + a_2 x + a_3 x^2 + a_4 x^3 + \dots \\ &= \{(a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots) - a_0\} / x \\ &= A(x) / x \end{aligned}$$

因为在这个问题中 $a_0 = 0$ 。因此，对 (1.1) 左边操作的结果就是 $A(x)/x$ 。

接下来对方程 (1.1) 的右边进行操作。将它乘以 x^n 并对所有 $n \geq 0$ 求和。结果是：

$$\begin{aligned} \sum_{n \geq 0} (2a_n + 1) x^n &= 2A(x) + \sum_{n \geq 0} x^n \\ &= 2A(x) + \frac{1}{1-x}, \end{aligned}$$

在这里，我们使用了大家熟悉的几何级数求和公式 $\sum_{n \geq 0} x^n = 1/(1-x)$ ，这个公式在 $|x| < 1$ 时成立。

如果我们将 (1.1) 两边操作的结果画上等号，我们发现：

$$\frac{A(x)}{x} = 2A(x) + \frac{1}{1-x},$$

对于未知生成函数 $A(x)$ 来说，这极其容易求解，形式为：

$$A(x) = \frac{x}{(1-x)(1-2x)}.$$

这就是该问题的生成函数。那些未知的数字 a_n 就整齐地挂在这条晾衣绳上： a_n 是上面 $A(x)$ 级数展开式中 x^n 的系数。

假设我们想求 a_n 的显式公式。那么我们就不得不把 $A(x)$ 展开成级数。在这个例子里并不难，因为部分分式展开（partial fraction expansion）为：

$$\begin{aligned} \frac{x}{(1-x)(1-2x)} &= x \left\{ \frac{2}{1-2x} - \frac{1}{1-x} \right\} \\ &= \{2x + 2^2x^2 + 2^3x^3 + 2^4x^4 + \dots\} \\ &\quad - \{x + x^2 + x^3 + x^4 + \dots\} \\ &= (2-1)x + (2^2-1)x^2 + (2^3-1)x^3 + (2^4-1)x^4 + \dots \end{aligned}$$

现在可以清晰地看出， x^n 的系数，即 a_n ，对每个 $n \geq 0$ 都等于 $2^n - 1$ 。

在这个例子中，我们本来不需要使用这么重型的“机械设备”，因为我们通过观察几乎能立即知道答案。关于生成函数学的令人印象深刻的一点是，**即使问题可能变得比这个难得多，求解的方法却与这里的方法极其相似**。因此，同样的重型设备能够在完全看不出答案的情况下得出结论。

1.2 一个稍难一点的两项递推关系 (A Slightly Harder Two Term Recurrence)

某个数字序列 $a_0, a_1, \dots$ 满足以下条件：

$$a_{n+1} = 2a_n + n \quad (n \geq 0; a_0 = 1). \quad (1.2)$$

求这个序列。

像以前一样，我们可能会计算序列的前几个成员以得到 1, 2, 5, 12, 27, 58, 121, ... 在这个例子中，似乎没有一个能够立刻显现出来的一般公式，所以我们使用生成函数的方法。也就是说，我们不去寻找序列 $a_0, a_1, \dots$ ，而是寻找函数 $A(x) = \sum_{j \geq 0} a_j x^j$ 。一旦我们找到了这个函数，该序列就可以被识别为该函数的幂级数系数序列。¹

¹ 如果你觉得自己的幂级数知识生疏了，请参阅第 2 章，那里包含了关于该主题的复习。

正如 1.1 节的例子一样，第一步是确保我们试图求解的递推关系，明确标明了下标取值的有效范围。在这个例子中，递推式 (1.2) 在括号注释中清楚地标记出对 $n = 0, 1, 2, \dots$ 有效。千万不要凑合使用一个带有未受限制自由变量的递推式。

下一步是定义你打算寻找的生成函数。在这个例子中，既然我们在寻找序列 $a_0, a_1, a_2, \dots$ ，一种自然的选择就是我们上面提到的函数 $A(x) = \sum_{j \geq 0} a_j x^j$ 。

接下来，取递推关系 (1.2)，将等式两边同时乘以 x^n ，并**针对该关系有效的所有 n 值求和**，这就意味着在这个例子中求和的范围是从 $n = 0$ 到 ∞ 。试着用你刚刚定义的函数 $A(x)$ 来表达这么做的结果。

如果我们对 (1.2) 的左边进行这个操作，结果是

$$\begin{aligned} a_1 + a_2 x + a_3 x^2 + a_4 x^3 + \dots &= (A(x) - a_0)/x \\ &= (A(x) - 1)/x. \end{aligned}$$

左边就是这样了。那么，如果我们将 (1.2) 的右边乘以 x^n 并对非负整数 n 求和会发生什么？显然，结果是 $2A(x) + \sum_{n \geq 0} n x^n$ 。我们需要识别这个级数

$$\sum_{n \geq 0} n x^n = x + 2x^2 + 3x^3 + 4x^4 + \dots$$

有两种方法可以继续进行：(a) 查表；(b) 算出来。为了把它算出来，我们使用下面这个绝招。如果你以前没见过这个绝招，它可能会显得很做作，但在你使用了 4993 次之后，它就会显得极其常规：

$$\sum_{n \geq 0} n x^n = \sum_{n \geq 0} x \left(\frac{d}{dx} \right) x^n = x \left(\frac{d}{dx} \right) \sum_{n \geq 0} x^n = x \left(\frac{d}{dx} \right) \frac{1}{1-x} = \frac{x}{(1-x)^2}. \quad (1.3)$$

换句话说，我们感兴趣的级数本质上是几何级数的导数，所以它的和本质上就是几何级数求和后的导数。这引发了一些麻烦的问题，我们将在这里提及，并在以后加以处理。(1.3) 式对于哪些 x 的值有效？几何级数仅在 $|x| < 1$ 时收敛，因此在 (1.3) 中对函数进行解析操作只对那些 x 才是合法的。然而，生成函数的解析性质通常并不会引起我们的兴趣；**我们喜欢它仅仅是因为它是挂我们的序列晾衣绳**。在这种情况下，我们可以将生成函数仅视为一个**形式幂级数 (formal power series)**，即作为一个代数对象而不是一个解析对象。那么 (1.3) 式将在形式幂级数环中作为一个恒等式成立（我们稍后将讨论形式幂级数环），此时变量 x 根本不需要受到任何条件限制。

无论如何，将 (1.2) 右边乘以 x^n 并对 $n \geq 0$ 求和的结果是 $2A(x) + x/(1-x)^2$ ，如果我们将它与之前 (1.2) 左边的结果等同起来，我们会发现

$$\frac{A(x) - 1}{x} = 2A(x) + \frac{x}{(1-x)^2}, \quad (1.4)$$

接下来进入容易的部分：求解 (1.4) 得到未知的 $A(x)$ ，即

$$A(x) = \frac{1 - 2x + 2x^2}{(1-x)^2(1-2x)}. \quad (1.5)$$

我们到底学到了什么？最初的问题是“找到”由递推式 (1.2) 决定的数字 $\{a_n\}$ 。我们在某种意义上已经“找到”了它们：数字 a_n 是函数 (1.5) 的幂级数展开式中的 x^n 系数。

这就是该方法中“寻找生成函数”部分的终点。我们已经找到了它。至于我们拿它做什么，那完全取决于我们当初为什么要寻找 (1.2) 的解。

例如，假设我们想要未知序列成员 a_n 的一个精确、简单的公式。那么部分分式的方法在这里也能发挥作用，就像在第一个例子中那样，只不过它的应用现在稍微有些棘手。让我们试一试，看看效果。

第一步是将 (1.5) 的右边展开为部分分式。这样的分式保证能够被展开成如下形式的部分分式：

$$\frac{1 - 2x + 2x^2}{(1 - x)^2(1 - 2x)} = \frac{A}{(1 - x)^2} + \frac{1}{1 - x} + \frac{2}{1 - 2x}, \quad (1.)$$

唯一的问题是如何找到常数 $A, ,$ 。

这里有一种快速的方法。首先将 (1.6) 的两边都乘以 $(1 - x)^2$ ，然后令 $x = 1$ 。立竿见影的结果是 $A = -1$ （别光听我说，自己试着算算！）。接下来将 (1.6) 等式全乘以 $1 - 2x$ 并令 $x = 1/2$ 。瞬间得到的结果是 $= 2$ 。最难找的一个是 $,$ ，所以我们来“作弊”找到它。因为我们知道 (1.6) 是一个恒等式，即对于所有的 x 值都为真，那么我们选一个简单的 x 值，比如说令 $x = 0$ ，并将该值代入 (1.6)。既然我们现在已经知道了 A 和 $,$ ，我们立刻就能求出 $= 0$ 。

现在我们回到 (1.6)，并将刚找到的 $A, ,$ 的值代入。结果就是下面这个关系式：

$$A(x) = \frac{1 - 2x + 2x^2}{(1 - x)^2(1 - 2x)} = \frac{(-1)}{(1 - x)^2} + \frac{2}{1 - 2x}. \quad (1.7)$$

我们正在努力尝试做的，是为 (1.7) 左侧中 x^n 的系数找到一个显式公式。我们正在将这个问题**转换成两个更容易的问题**，也就是在 (1.7) 右侧的每个求和项中，寻找 x^n 的系数。为什么它们更容易呢？比如，项 $2/(1 - 2x)$ 会展开成一个几何级数。那里的 x^n 的系数仅仅是 $2 \cdot 2^n = 2^{n+1}$ 。而级数 $(-1)/(1 - x)^2$ 在上面的 (1.3) 中处理过了，它对应的 x^n 的系数是 $-(n + 1)$ 。如果我们把这些结果结合起来，我们就会发现未知的序列是

$$a_n = 2^{n+1} - n - 1 \quad (n = 0, 1, 2, \dots).$$

在做了这么多工作之后，我必须要坦白：其实有更好的办法来处理类型为 (1.2) 的递推式，而不必使用生成函数。2 然而，这道题仍然是展示如何使用生成函数的一个极佳例子，它强调了一个事实：**在有关序列的问题中，一种单一的、统一的方法可以取代大量各自孤立的特殊技巧**。无论如何，用不了多久，我们就会碰到一些不使用生成函数就基本上无法处理的问题了。

2 例如，参见我的著作 [Wi1] 的第一章。

现在是时候引入一些符号来帮助我们在后续的内容中节省大量文字了。

定义 1.1 设 $f(x)$ 是一个关于 x 的幂级数。那么符号 $[x^n]f(x)$ 代表级数 $f(x)$ 中 x^n 的系数。

这里是一些使用这个符号的例子：

$$[x^n]^x = 1/n!; \quad [r]\{1/(1-3)\} = 3^r; \quad [u^m](1+u)^s = \binom{s}{m}.$$

这个符号有一个显而易见的特性，我们将反复使用：

$$[x^n]\{x^a f(x)\} = [x^{n-a}]f(x). \quad (1.)$$

这个符号的另一个属性是一个约定：如果 a 是任何实数，那么

$$[x^n]f(x) = (1/)[x^n]f(x), \quad (1.)$$

因此，例如，对所有 $n \geq 0$ 都有 $[x^n/n!]^x = 1$ 。

在我们进行下一个例子之前，这里对我们到目前为止使用的生成函数方法进行一个总结。

1.2.1 方法总结 (The Method)

已知： 试图通过生成函数的方法求解的一个递推公式。

1. **明确范围：** 确保清楚地划定了已知递推关系为真的自由变量（比如 n ）的值域集合。
2. **定义函数：** 为你打算寻找的生成函数起一个名字，并用未知的序列来写出该函数（例如，叫它 $A(x)$ ），并定义它为 $\sum_{n \geq 0} a_n x^n$ 。
3. **求和方程：** 将递推式的两边同时乘以 x^n ，并对递推式成立的所有 n 的值求和。
4. **化简表达式：** 将得到的等式两边明确地（explicitly）用你的生成函数 $A(x)$ 来表示。
5. **解出函数：** 解出得到的方程，求出未知的生成函数 $A(x)$ 。
6. **展开取系数：** 如果你想要得到由给定的递推关系所定义的序列的显式公式，那么尝试通过你能想到的任何方法将 $A(x)$ 展开为幂级数来得到这样一个公式。特别是，如果 $A(x)$ 是一个有理函数（两个多项式的商），那么展开为部分分式，然后分别处理每一项，将会带来成功。

1.3 一个三项递推关系 (A Three Term Recurrence)

现在让我们来处理斐波那契递推式

$$F_{n+1} = F_n + F_{n-1} \quad (n \geq 1; F_0 = 0; F_1 = 1). \quad (1.10)$$

按照“方法总结”，我们将求解生成函数

$$F(x) = \sum_{n \geq 0} F_n x^n.$$

为了做到这一点，将 (1.10) 式乘上 x^n 并对 $n \geq 1$ 求和。我们在左侧得到

$$F_2 x + F_3 x^2 + F_4 x^3 + \cdots = \frac{F(x) - x}{x},$$

而在右侧我们得到

$$\{F_1x + F_2x^2 + F_3x^3 + \cdots\} + \{F_0x + F_1x^2 + F_2x^3 + \cdots\} = \{F(x)\} + \{xF(x)\}.$$

(**重点注意：** 请不要偷看上面的内容，试着自己完成上述操作，看看你是否能得到相同的答案。) 由此可知 $(F - x)/x = F + xF$ ，因此，未知的生成函数现在已经揭晓，它就是：

$$F(x) = \frac{x}{1 - x - x^2}.$$

现在，我们将通过将 $x/(1 - x - x^2)$ 展开成部分分式来寻找斐波那契数的一些公式。部分分式方法的成功在很大程度上取决于分母中是否只有线性（一次）因子。然而，现在我们拥有的是一个二次因子。所以，让我们进一步把它分解。我们发现：

$$1 - x - x^2 = (1 - xr_+)(1 - xr_-) \quad \left(r_{\pm} = \frac{1 \pm \sqrt{5}}{2} \right)$$

因此，

$$\begin{aligned} \frac{x}{1 - x - x^2} &= \frac{x}{(1 - xr_+)(1 - xr_-)} \\ &= \frac{1}{(r_+ - r_-)} \left(\frac{1}{1 - xr_+} - \frac{1}{1 - xr_-} \right) \\ &= \frac{1}{\sqrt{5}} \left\{ \sum_{j \geq 0} r_+^j x^j - \sum_{j \geq 0} r_-^j x^j \right\}, \end{aligned}$$

这要归功于几何级数的魔力。很容易就能挑出 x^n 的系数，并找到：

$$F_n = \frac{1}{\sqrt{5}} (r_+^n - r_-^n) \quad (n = 0, 1, 2, \dots) \quad (1.11)$$

这就是斐波那契数 F_n 的显式公式。

这个例子给了我们一个机会，让我们能更进一步地深入探究“生成函数能告诉我们关于序列的什么信息”。也就是说，我们**不仅能获得精确的答案，还能获得一个在 n 很大时有效的近似答案**。确实，当 n 很大时，由于 $r_+ > 1$ 且 $|r_-| < 1$ ，(1.11) 式中的第二项与第一项相比将显得微不足道，因此对于 F_n 来说，一个极好的近似（极好地趋近于 F_n ）将会是

$$F_n \sim \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n. \quad (1.12)$$

但是，你可能会问，当一个精确的公式就在眼前时，为什么还会有人想要一个近似公式呢？当然，答案之一是，有时精确的答案异常复杂，而近似的答案更能说明问题。即使是在这个精确答案并不是很复杂的例子里，我们仍然可以从近似中有所收获。读者应该花一点时间来验证：如果在 (1.11) 式中忽略掉第二项，那么我们忽略掉的那个量的绝对值将永远不会超过 0.5。因此不仅 F_n 近似由 (1.12) 给出，**它还确切地等于最接近 (1.12) 右侧的那个整数**。由此可见，对近似公式的探讨，竟然为我们找到了一个更简单的精确公式！

1.4 一个三项边值问题 (A Three Term Boundary Value Problem)

这个例子将与之前的例子有所不同，因为这里涉及的递推关系不允许我们直接计算出序列的成员，尽管它确实唯一地决定了这个序列。这种情况类似于：假设我们有斐波那契的递推关系，但附加的已知条件变成了 $F_0 = 1$ 和 $F_{735} = 1$ 。那么，序列 $\{F_n\}$ 将被唯一确定，但你无法通过递推直接计算它，因为你没有启动递推所需的两个连续的值。

我们将考虑一个稍微更一般的情况。它由以下递推关系组成：

$$au_{n+1} + bu_n + cu_{n-1} = d_n \quad (n = 1, 2, \dots, N-1; u_0 = u_N = 0) \quad (1.13)$$

其中，正整数 N ，常数 a, b, c ，以及序列 $\{d_n\}_{n=1}^{N-1}$ 是预先给定的。正如我们将要看到的，方程组 (1.13) 唯一地决定了序列 $\{u_i\}_0^N$ 。对于这类边值问题，生成函数方法为我们提供了一种强大的攻击手段。这类问题在众多应用中都有出现，比如样条函数 (spline functions) 的插值理论。

首先，我们要定义两个生成函数。一个是未知的 $U(x) = \sum_{j=0}^N u_j x^j$ ，另一个是 $D(x) = \sum_{j=1}^{N-1} d_j x^j$ 。这里 $D(x)$ 被视为一个已知的函数（我们是不是遗漏了 d_j 的某些给定值，比如 d_0 或 d_N ？为什么？）。

接下来，按照一贯的“秘籍”，我们将递推式 (1.13) 乘以 x^n ，并对使递推式成立的 n 值求和。在这个例子中，这意味着我们要从 $n = 1$ 加到 $N - 1$ 。这会得到：

$$a \sum_{n=1}^{N-1} u_{n+1} x^n + b \sum_{n=1}^{N-1} u_n x^n + c \sum_{n=1}^{N-1} u_{n-1} x^n = \sum_{n=1}^{N-1} d_n x^n.$$

如果我们现在用之前定义的生成函数来表达这个等式，它就会变成这种形式：

$$\frac{a}{x} \{U(x) - u_1 x\} + bU(x) + cx\{U(x) - u_{N-1} x^{N-1}\} = D(x). \quad (1.14)$$

接下来，虽然因为 u_1 和 u_{N-1} 依然未知而让人有一丝挥之不去的疑虑，但我们还是继续按照“秘籍”操作。秘籍的下一步要求我们从 (1.14) 中解出未知的生成函数 $U(x)$ 。这并不太难，我们立刻就能得出：

$$\{a + bx + cx^2\}U(x) = x\{D(x) + au_1 + cu_{N-1}x^N\}. \quad (1.15)$$

除了两个尚未知晓的常数 u_1 和 u_{N-1} 之外，未知的生成函数 $U(x)$ 现在已经算作已知了。而 (1.15) 式也暗示了找到这两个常数的方法。(1.15) 式左边的二次多项式有两个使其为零的 x 值，我们称之为 r_+ 和 r_- 。暂时先假设 $r_+^N \neq r_-^N$ 。如果我们在 (1.15) 中令 $x = r_+$ ，我们就会得到一个关于两个未知数 u_1, u_{N-1} 的方程；如果令 $x = r_-$ ，我们会得到另一个方程。这两个方程是：

$$\begin{aligned} au_1 + (cr_+^N)u_{N-1} &= -D(r_+) \\ au_1 + (cr_-^N)u_{N-1} &= -D(r_-). \end{aligned} \quad (1.1)$$

一旦从中解出了 u_1 和 u_{N-1} ，方程 (1.15) 就能极其明确且完整地给出 $U(x)$ 。至于 $r_+^N = r_-^N$ 的特殊情况，我们就留给读者自己去处理了。

这里是这些结果在**样条插值理论** (spline interpolation) 中的一个应用。3

3 这个应用稍微有些专业，初读时可以跳过。

假设我们给定了一个函数 (x) 在一组等距节点 $i = 0 + ih$ ($0 \leq i \leq n$) 处的一组值 $0, 1, \dots, n$ 。我们想要构造一个平滑的函数 $S(x)$ 来拟合这些数据，并满足以下条件：

- (i) 在每个区间 $(i, i+1)$ ($i = 0, \dots, n-1$) 内，我们的函数 $S(x)$ 必须是一个三次多项式（在每个区间可以是一个不同的三次多项式！）；
- (ii) 函数 $S(x)$ 及其一阶导数 $S'(x)$ 和二阶导数 $S''(x)$ 必须在整个区间 $[0, n]$ 上是连续的；
- (iii) 对于 $i = 0, \dots, n$ ，有 $S(i) = i$ 。

满足这些条件的函数 $S(x)$ 称为**三次样条 (cubic spline)**。假设我们未知的样条函数 $S(x)$ 在 $x \in [0, 1]$ 时由 $S_0(x)$ 给出，在 $x \in [1, 2]$ 时由 $S_1(x)$ 给出，……，在 $x \in [n-1, n]$ 时由 $S_{n-1}(x)$ 给出。现在我们要求出所有的三次多项式 $S_0, \dots, S_{n-1}$ 。为了做到这一点，我们有 $2n$ 个插值条件：

$$S_{i-1}(i) = i = S_i(i) \quad (i = 1, \dots, n-1); \quad S_0(0) = 0; \quad S_{n-1}(n) = n \quad (1.17)$$

以及 $2n-2$ 个连续性条件：

$$S'_{i-1}(i) = S'_i(i); \quad S''_{i-1}(i) = S''_i(i) \quad (i = 1, \dots, n-1). \quad (1.18)$$

总共有 $4n-2$ 个条件需要满足。我们需要确定 n 个三次多项式，每个多项式有四个系数，总计 $4n$ 个未知参数。由于这些条件都是线性的，这样的样条函数 $S(x)$ 肯定存在，并且我们可以预期它有两个自由参数。通常约定这样选择这两个参数，使得 $S(x)$ 在 0 和 n 处有一个拐点（即二阶导数为0的自然样条条件）。

现在给出解。函数 $S_i(x)$ 可由下式给出：

$$S_i(x) = \frac{1}{h} \left(i(i+1-x)^3 + (i+1)(x-i)^3 + (i+1-h^2_{i+1})(x-i) + (i-h^2_i)(i+1-x) \right) \quad (i = 0, 1, \dots, n-1), \quad (1.19)$$

前提是，数字 $1, \dots, n-1$ 满足如下联立方程组：

$$i-1 + 4i + i+1 = \frac{1}{h^2} (i+1 - 2i + i-1) \quad (i = 1, 2, \dots, n-1) \quad (1.20)$$

其中 $0 = n = 0$ 。这很容易检验，只要把 $x = i$ 和 $x = i+1$ 代入 (1.19)，就能验证 (1.17) 和 (1.18) 是满足的。因此，剩下的工作仅仅是求解方程组 (1.20)。

方程组 (1.20) 正是 (1.13) 的形式，因此我们可以通过 (1.15) 和 (1.16) 求出解。为此，从给定的要进行插值的数据点 $\{(i, i)\}_{i=0}^n$ 出发，用它们写出：

$$D(x) = \frac{1}{h^2} \sum_{i=1}^{n-1} (i_{+1} - 2i + i_{-1})x^i. \quad (1.21)$$

因为在这个例子中 $(a, b, c) = (1, 4, 1)$ ，所以我们有 $r_{\pm} = -2 \pm \sqrt{3}$ 。现在我们未知的生成函数 $U(x)$ 由 (1.15) 给出，也就是：

$$U(x) = \frac{x(D(x) + 1 + x^{n-1})}{1 + 4x + x^2}, \quad (1.22)$$

其中未知数 $1, n-1$ 由“(1.22) 右侧必须是一个多项式”这一要求所决定，或者等价于由方程组 (1.16) 决定，在这个例子中也就是：

$$\begin{aligned} 1 + (\sqrt{3} - 2)^n n_{-1} &= -D(\sqrt{3} - 2) \\ 1 + (-\sqrt{3} - 2)^n n_{-1} &= -D(-\sqrt{3} - 2). \end{aligned} \quad (1.23)$$

一旦我们知道了 $U(x)$ （它毕竟就是 $\sum_{i=1}^{n-1} i x^i$ ），我们就可以读出它的系数来找到所有的 i ，然后把它们代入 (1.19) 就能得到插值样条函数了。

例 1.2 现在让我们尝试一个带有实实在在数字的例子。假设我们试图用三次样条在区间 $[0, 5]$ 上拟合 2 的幂。我们的输入数据是 $i = 2^i$ 对 $i = 0, 1, \dots, 5$ ，步长 $h = 1$ 且 $n = 5$ 。根据 (1.21)，我们发现 $D(x) = x(1 + 2x + 4x^2 + x^3)$ 。然后解方程 (1.23)，得到 $1 = 204/20$ 和 $4 = 2370/20$ 。接着 (1.22) 告诉我们

$$U(x) = \frac{204x}{20} + \frac{43x^2}{20} + \frac{552x^3}{20} + \frac{2370x^4}{20},$$

现在我们就知道了所有的 i 。最后，(1.19) 给出了构成该样条函数的那些精确的三次多项式。例如，定义在子区间 $[0, 1]$ 上的 $S_0(x)$ 就是：

$$S_0(x) = 1 + \frac{175}{20}x + \frac{34}{20}x^3.$$

请注意 $S_0(0) = 1$ 且 $S_0(1) = 2$ ，所以它正确地拟合了其子区间端点处的数据；并且 $S_0(0) = 0$ ，所以拟合曲线在原点处会有一个拐点。在此强烈建议读者在这个例子中求出所有的 $S_i(x)$ ($i = 0, 1, \dots, 5$)，并检查它们在点 1, 2, 3, 4 处是否都能“平滑地相互融合”（即函数本身及其前二阶导数都是连续的）。

人们想要用样条函数去拟合数值数据的一个原因，是因为想要对数据代表的函数进行积分。将 (1.19) 从 $i = ih$ 积分到 $i_{+1} = (i+1)h$ ，可得：

$$\int_i^{i+1} S_i(x) dx = \frac{h}{2}(i + i_{+1}) - \frac{h^3}{24}(i + i_{+1}). \quad (1.24)$$

因此，用样条拟合数据并对样条积分，这就等同于使用了带有三阶修正项的**梯形法则**

(trapezoidal rule) 进行数值积分。如果我们把 (1.24) 式从 $i = 0$ 到 $n-1$ 加起来，就能得到整体的积分：

$$\int_0^{nh} S(x)dx = -\frac{h}{12}(0 - 1 - n_{-1} + n) - \frac{h^3}{72}(1 + n_{-1}) \quad (1.25)$$

其中“trap”代表基础的梯形法则结果，而 $1, n_{-1}$ 满足 (1.23)。

样条函数插值是一个非常重要的话题。它出现在计算机字体的存储中，比如你现在正在阅读的这款字体。你有没有想过字体中字母的形状实际上是如何存储在计算机里的？其中一种方法就是存储那些拟合字母轮廓的样条函数的参数。

1.5 两个独立变量 (Two Independent Variables)

在本节中，我们将看到生成函数如何在涉及两个离散变量的问题中发挥作用。我们也将借此机会引入**二项式系数 (binomial coefficients)**，因为它们无疑是最重要的组合计数序列之一。

设 n 和 k 为满足 $0 \leq k \leq n$ 的整数。从集合 $\{1, 2, \dots, n\}$ 中选取 k 个对象的子集，有多少种方法？让我们假装不知道结果会怎样，让生成函数来帮我们寻找答案。

假设问题的答案是 $f(n, k)$ 。我们想象从这 n 个对象中选出 k 个的所有可能的子集集合都摆在面前，我们将把它们分成两堆。在第一堆中，我们放入所有**包含**对象“ n ”的子集，而在第二堆中，我们放入所有**不包含**“ n ”的子集。这第一堆显然包含 $f(n-1, k-1)$ 个子集。第二堆包含 $f(n-1, k)$ 个子集。这两堆原本加起来包含 $f(n, k)$ 个子集。因此，我们未知的数字 $f(n, k)$ 一定满足递推关系：

$$f(n, k) = f(n-1, k) + f(n-1, k-1) \quad (f(n, 0) = 1). \quad (1.2)$$

为了找到这些数字的公式，我们（还能用什么？）使用生成函数。对于每个 $n = 0, 1, 2, \dots$ ，定义生成函数：

$${}_n(x) = \sum_{k \geq 0} f(n, k)x^k.$$

现在将 (1.26) 整体乘以 x^k ，并对 $k \geq 1$ 求和。结果是，对于 $n \geq 1$ ，有 ${}_n(x) - 1 = ({}_{n-1}(x) - 1) + x {}_{n-1}(x)$ ，且 ${}_0(x) = 1$ 。因此

$${}_n(x) = (1+x){}_{n-1}(x) \quad (n \geq 1; {}_0(x) = 1). \quad (1.27)$$

于是，对于所有的 $n \geq 0$ ，我们有 ${}_n(x) = (1+x)^n$ 。我们未知的数字 $f(n, k)$ 就这样真相大白了：它就是多项式 $(1+x)^n$ 中 x^k 的系数。为了求出 $f(n, k)$ 的公式，比如我们可以使用泰勒公式。泰勒公式告诉我们 $f(n, k)$ 就是 $(1+x)^n$ 的第 k 阶导数在 $x=0$ 处的值，最后除以 $k!$ 。这个求导很容易做。确实， $(1+x)^n$ 的第 k 阶导数是 $n(n-1)\cdots(n-k+1)(1+x)^{n-k}$ 。如果我们令 $x=0$ 并除以 $k!$ ，我们就能立刻发现，从 n 个事物中选取 k -子集的数量 $f(n, k)$ 可由下式给出：

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!} \quad (1.2)$$

对于满足 $0 \leq k \leq n$ 的整数 n, k 成立。

这非常好地处理了当 $0 \leq k \leq n$ 且 n, k 均为整数时的二项式系数 $\binom{n}{k}$ 。当 k 是一个负整数时，二项式系数 $\binom{n}{k} = 0$ 。

尽管如果 n 不是非负整数，方程 (1.28) 的第二部分将很难理解（因为阶乘不直接适用），但第三部分却一点也不难理解，即使 n 是一个复数，只要 k 是一个非负整数，这就完全讲得通。这为我们提供了一种将二项式系数推广到任意复数 n 的方法，即：

$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!} \quad (\text{整数 } k \geq 0). \quad (1.2)$$

因此 $\binom{-3}{3} = (-3)(-4)(-5)/6 = -10$ ，而且 $\binom{i}{2} = i(i-1)/2 = (-1-i)/2$ 等等。生成函数

$$n(x) = (1+x)^n = \sum_{k \geq 0} \binom{n}{k} x^k = 1 + nx + \frac{n(n-1)}{2} x^2 + \cdots \quad (1.30)$$

对于所有的复数 n 依然有效：如果 n 是一个非负整数，这个级数就是有穷的（有限项）；无论在哪种情况下，它在 $|x| < 1$ 时都是收敛的。

符号 $\binom{n}{k}$ 的**非零支撑集 (support)** 是什么？也就是说，对于哪些 n, k 的值， $\binom{n}{k} \neq 0$ 成立？首先， k 必须是一个非负整数。如果 n 不是一个非负整数，那么根据 (1.29)， $\binom{n}{k}$ 肯定是非零的。如果 n 是一个非负整数，那么 (1.29) 表明 $\binom{n}{k} = 0$ 当且仅当 $0 \leq k \leq n$ （这也完全符合组合学上的定义！）。

这些事实带来了一个极其重要的推论：如果 n 是一个非负整数，我们就不必写成形如 $\sum_{k=0}^n \binom{n}{k} x^k$ 这样，而完全可以写成 $\sum_{k=-\infty}^{\infty} \binom{n}{k} x^k$ 。因为在求和的第二种形式中出现的那些看似多余的 k 的值上，二项式系数都会变为零。**二项式系数自己就会“切断”求和，因此我们根本没必要用求和范围再来切断一次。**

既然如此，我们在这里引入两个在全书中都将严格遵守的约定。

约定 1.3 当被求和的变量的范围没有被明确指出时，应当理解为该求和的范围是从 $-\infty$ 到 $+\infty$ 。

约定 1.4 当方程式中自由变量的范围没有被明确指出时，应当理解为该方程式对于该变量的**所有整数值**均成立。

例如，我们将写成 $\sum_k \binom{n}{k} x^k = (1+x)^n$ 。这些约定将在接下来的内容中为我们节省大量的工作，主要体现在：当我们通过一个常数偏移来改变求和变量时，我们再也不必去担心如何改变求和的上下限了。

让我们再看看一些其他类型的生成函数。如果我们将 ((1.30) 式) 中的 $n(x)$ 乘以 x^n ，并**仅对** $n \geq 0$ 求和，我们发现

$$\sum_{n \geq 0} n(x)^n = \sum_{n \geq 0} \sum_k \binom{n}{k} x^{kn} = \sum_{n \geq 0} (1+x)^{nn} = \frac{1}{1-(1+x)}.$$

因此对于整数 $n \geq 0$, $\binom{n}{k} = [x^{kn}](1 - (1+x))^{-1}$ 。

再作为一个练习, 对于非负整数 k , 让我们来计算和式 $\sum_n \binom{n}{k} n$ 。请注意下标 n 跑遍了所有的整数, 但除非 $n \geq k$, 否则被加项为零。那个和式显然是:

$$\begin{aligned} \sum_{n \geq 0} \binom{n}{k} x^{kn} &= [x^k] \frac{1}{1-(1+x)} = \frac{1}{1-} [x^k] \frac{1}{1-(\frac{1}{1-})x} \\ &= \frac{1}{1-} \left(\frac{1}{1-} \right)^k = \frac{k}{(1-)^{k+1}}. \end{aligned}$$

为了以后参考方便, 我们将二项式系数的这两种幂级数生成函数并排放在一起:

$$\sum_k \binom{n}{k} x^k = (1+x)^n; \quad \sum_n \binom{n}{k} n = \frac{k}{(1-)^{k+1}}. \quad (1.31)$$

1.6 另一个双变量的情形 (Another 2-Variable Case)

这个例子将比前面的例子具有更浓厚的组合学风味。它涉及到集合的**划分 (partitions)**。对于一个集合 S , 它的划分是指一组非空且两两不相交的子集, 这些子集的并集恰好是 S 。集合 S 的划分的另一个名称是 S 上的**等价关系 (equivalence relation)**。 S 被划分成的那些子集被称为该划分的**类 (classes)**。

例如, 我们可以用几种不同的方式来划分 $[5]$ 。其中一种方式是 $\{123\}\{4\}\{5\}$ 。在这个划分中, 共有三个类, 其中一个类包含了 1、2 和 3, 另一个类只包含 4, 而剩下的一个类只包含 5。类内部元素的顺序没有任何意义, 各类之间的顺序也没有任何意义。唯一要紧的只是“谁和谁在一起, 谁和谁分开了”。

4 回顾一下, $[n]$ 指的是集合 $\{1, 2, \dots, n\}$ 。

下面是将 $[4]$ 划分为两个类的**所有**可能情况的列表:

$$\{12\}\{34\}; \{13\}\{24\}; \{14\}\{23\}; \{123\}\{4\}; \{124\}\{3\}; \{134\}\{2\}; \{1\}\{234\}. \quad (1.32)$$

恰好有 7 种将 $[4]$ 划分为两个类的方法。

在这个例子中, 我们要解决的问题是: 发现究竟有多少种将 $[n]$ 划分为 k 个类的方法。我们用 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 来表示这个数字。它被称为**第二类斯特林数 (Stirling number of the second kind)**。我们上面的列表表明 $\left\{ \begin{smallmatrix} 4 \\ 2 \end{smallmatrix} \right\} = 7$ 。

为了找出关于这些数字的更多信息，我们将遵循生成函数的方法。首先我们要找到一个递推关系，然后是几个生成函数，接着是一些精确的公式，等等。

我们从 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 的递推公式开始，它的推导过程与前一个例子中二项式系数的推导非常相似。

给定正整数 n, k 。想象在你面前摆放着将 $[n]$ 划分为 k 个类的所有可能划分的集合。恰好有 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 个这样的划分。和二项式系数那个例子一样，我们将把这个大集合分成两堆；在第一堆中，我们放入所有满足“数字 n 自己单独构成一个类”的将 $[n]$ 划分为 k 个类的划分。在第二堆中，放入所有其他的划分，也就是那些最大的数字 n 与其他数字位于同一个类中的划分。

问题是，（用斯特林数来表示）这两堆中分别有多少个划分？

考虑第一堆。在那里，每一个划分中 n 都是“独居”的。想象一下你在这一堆中巡视，把每一项划分中单独出现的类“ $\{n\}$ ”都擦掉。如果你这么做了，那么擦除之后剩下的，将完完全全是把 $[n-1]$ 划分为 $k-1$ 个类的所有划分的完整集合。由于这样的划分有 $\left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\}$ 个，所以第一堆中必定原本就有 $\left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\}$ 个划分。

刚才那个很简单，但现在考虑第二堆。在那里，数字 n 总是和其他数字生活在一个类里。因此，如果我们在那一堆中巡视，并在 n 出现的任何地方将它擦除，我们将**不会**影响类的数量；我们依然会看着有 k 个类的划分。在把所有的 n 都擦除之后，我们这一堆现在包含的是将 $n-1$ 个数字划分为 k 个类的划分。然而，这些划分中的每一个现在出现的次数不再是只有一次，而是好几次。

例如，在 (1.32) 的列表中，第二堆包含了如下划分：

$$\{12\}\{34\}; \{13\}\{24\}; \{14\}\{23\}; \{124\}\{3\}; \{134\}\{2\}; \{1\}\{234\}, \quad (1.33)$$

而在我们把“4”从所有的划分中删掉之后，我们得到了如下列表：

$$\{12\}\{3\}; \{13\}\{2\}; \{1\}\{23\}; \{12\}\{3\}; \{13\}\{2\}; \{1\}\{23\}.$$

我们现在看到的，是将 $[3]$ 划分为两类的所有划分的列表，只不过每个划分被写了两次。因此这个列表包含了恰好 $2 \left\{ \begin{smallmatrix} 3 \\ 2 \end{smallmatrix} \right\}$ 个划分。

在一般情况下，在第二堆里把一切 n 都擦除之后，我们将看到的是把 $[n-1]$ 划分为 k 个类的所有划分的列表，其中每一个这样的划分都会被写下 k 次。因此这个列表将包含恰好 $k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\}$ 个划分。

因此，在擦除 n 之前，第二堆必然也包含了 $k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\}$ 个划分。

所以，原本含有 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 个划分的列表被分成了两堆，第一堆包含 $\left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\}$ 个划分，而第二堆包含 $k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}$ 个划分。

因此，下式必定成立：

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\} \quad ((n, k) \neq (0, 0)).$$

为了确定 n 和 k 的范围，让我们将 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 的定义扩展到所有整数对。如果 $k > n$ 或 $n < 0$ 或 $k < 0$ ，我们规定 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = 0$ 。此外，如果 $n = 0$ ，我们规定 $\left\{ \begin{matrix} n \\ 0 \end{matrix} \right\} = 0$ ，并且我们将取 $\left\{ \begin{matrix} 0 \\ 0 \end{matrix} \right\} = 1$ 。有了这些约定，上面的递推式对于除 $(0, 0)$ 之外的所有 (n, k) 都是成立的，于是我们有

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\} \quad ((n, k) \neq (0, 0); \left\{ \begin{matrix} 0 \\ 0 \end{matrix} \right\} = 1). \quad (1.34)$$

寻找生成函数的舞台现已搭建完毕。这里又有三个很自然、而且似乎有可能求得出来的候选生成函数，即：

$$\begin{aligned} A_n() &= \sum_k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} k \\ {}_k(x) &= \sum_n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^n \\ (x,) &= \sum_{n,k} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^{nk}. \end{aligned} \quad (1.35)$$

在我们一头扎进计算之前，让我们先停下来片刻，培养一点直觉，看看这三个选择中哪一个最有可能成功。为了求 $A_n()$ ，我们需要将 (1.34) 乘以 k 并对 k 求和。你看出这么做会有什么问题吗？嗯，确实有一些问题，它们源自右边第二项中的因子 k 。事实上，在乘以 k 并对 k 求和之后，我们将不得不去处理类似 $\sum_k k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} k$ 这样的东西。这当然是可以考虑的，因为它与 $A_n()$ 的导数有关，但我们在这里确实遇到了一个麻烦。

如果我们转而选择寻找 ${}_k(x)$ ，我们将 (1.34) 乘以 x^n 并对 n 求和。此时，看起来有些麻烦的因子 k 并没有参与到求和中，我们可以把那个 k 作为一个乘法因子提取到求和号的外面。

对比这两种方法，我们投后者一票，也就是尝试寻找函数 ${}_k(x)$ ($k \geq 0$)。因此，将 (1.34) 乘以 x^n 并对 n 求和，得到：

$${}_k(x) = x {}_{k-1}(x) + k {}_k(x) \quad (k \geq 1; {}_0(x) = 1).$$

这导致：

$${}_k(x) = \frac{x}{1 - kx} {}_{k-1}(x) \quad (k \geq 1; {}_0(x) = 1)$$

最后求得结果：

$${}_k(x) = \sum_n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^n = \frac{x^k}{(1-x)(1-2x)(1-3x)\cdots(1-kx)} \quad (k \geq 0). \quad (1.3)$$

因此，如果我们能找到 (1.36) 中出现的函数的幂级数展开式，那么寻找斯特林数显式公式的问题就迎刃而解了。反过来，这需要一剂“部分分式”的解药，而不是“泰勒公式”！

我们讨论的这个部分分式展开式具有如下形式：

$$\frac{1}{(1-x)(1-2x)\cdots(1-kx)} = \sum_{j=1}^k \frac{j}{(1-jx)}.$$

为了求出 's，固定一个 r , $1 \leq r \leq k$ ，将两边同时乘以 $1-rx$ ，然后令 $x = 1/r$ 。结果是：

$$\begin{aligned} r &= \frac{1}{(1-1/r)(1-2/r)\cdots(1-(r-1)/r)(1-(r+1)/r)\cdots(1-k/r)} \\ &= (-1)^{k-r} \frac{r^{k-1}}{(r-1)!(k-r)!} \quad (1 \leq r \leq k). \end{aligned} \quad (1.37)$$

结合 (1.36) 和 (1.37)，对于 $n \geq k$ ，我们得到：

$$\begin{aligned} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} &= [x^n] \left\{ \frac{x^k}{(1-x)(1-2x)\cdots(1-kx)} \right\} \\ &= [x^{n-k}] \left\{ \frac{1}{(1-x)(1-2x)\cdots(1-kx)} \right\} \\ &= [x^{n-k}] \sum_{r=1}^k \frac{r}{1-rx} \quad (k \geq 1) \\ &= \sum_{r=1}^k r [x^{n-k}] \frac{1}{1-rx} \\ &= \sum_{r=1}^k r r^{n-k} \\ &= \sum_{r=1}^k (-1)^{k-r} \frac{r^{k-1}}{(r-1)!(k-r)!} r^{n-k} \\ &= \sum_{r=1}^k (-1)^{k-r} \frac{r^n}{r!(k-r)!} \quad (n, k \geq 0), \end{aligned} \quad (1.3)$$

这恰恰就是我们想要的： $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 的一个显式公式。请务必检查一下这个公式是否能算出 $\left\{ \begin{matrix} 4 \\ 2 \end{matrix} \right\} = 7$ ，这是我们之前就知道的。同时，注意这个公式表示 $\left\{ \begin{matrix} n \\ 2 \end{matrix} \right\} = 2^{n-1} - 1 \quad (n > 0)$ 。你能给出这个事实的一个独立的证明吗？

接下来，我们打算尝试解决斯特林数递推问题的另一种途径，也就是研究 (1.35) 中的函数 $A_n()$ 。把这个方法彻头彻尾地执行到底，要比我们刚刚使用的方法困难得多，但事实证明，从理论的

角度来看，这些生成函数具有非常重要的其他用途。

因此，固定 $n > 0$ ，将 (1.34) 乘以 k 并对 k 求和。结果是：

$$\begin{aligned} A_n() &= \sum_k \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} k + \sum_k k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\} \\ &= A_{n-1}() + \left(\frac{d}{d} \right) A_{n-1}() \\ &= \{(1 + D)\} A_{n-1}() \quad (n > 0; A_0() = 1). \end{aligned} \quad (1.3)$$

这其中新奇的地方在于，由于递推关系中因子 k 的存在，不可避免地出现了微分算子 d/d 。

因此，每个函数 A_n 都是通过对其前驱应用算子 $(1 + D)$ 来获得的。从 $A_0 = 1$ 开始，我们依次得到 $1, 1 + 2, 1 + 3^2 + 3, \dots$ ，但是就求一个显式公式而言，通过这种方法我们只发现：

$$A_n() = \{+D\}^n 1 \quad (n \geq 0). \quad (1.40)$$

然而，相比于 $n(x)$ ，有些东西从这些生成函数中能够被看得更清楚。其中之一我们将在 4.5 节讨论，它涉及固定 n 、当 k 从 1 到 n 变化时，序列 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 的**形状 (shape)**。事实证明，该序列先增大后减小。也就是说，它只有一个最大值。许多组合序列和这个序列一样是**单峰的**

(unimodal)，但在某些情况下证明这种性质可能会非常困难。在这个例子里，多亏了公式 (1.40)，我们将会看到证明这一点一点也不难。

作为 (1.38) 的一个应用，回顾一下斯特林数 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 是将一个包含 n 个元素的集合划分为 k 个类的方法数。假设我们不是特别关心有多少个类，而只是想知道**划分一个包含 n 个元素的集合的方法总数**。令这些数字为 $\{b(n)\}_0$ 。它们被称为**贝尔数 (Bell numbers)**。通常约定取 $b(0) = 1$ 。贝尔数序列的开头是 1, 1, 2, 5, 15, 52, ...

我们能找到贝尔数的显式公式吗？小菜一碟。在 (1.38) 中我们已经有了 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 的显式公式。如果我们把那个公式从 $k = 1$ 加到 n ，我们就能得到 $b(n)$ 的显式公式。然而，我们还可以注意到一件非常有利可图的事情。公式 (1.38) 对 n 和 k 的所有正整数值都是有效的。特别地，如果 $k > n$ 它也是有效的。但是如果 $k > n$ ， $\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = 0$ 。这意味着公式 (1.38) 不需要我们“告诉”它 $\left\{ \begin{matrix} 13 \\ 1 \end{matrix} \right\} = 0$ ；它自己**知道**；也就是说，如果我们心无旁骛地把 $n = 13, k = 1$ 代入那个庞大的求和公式中并计算出结果，我们将得到 0。

因此，为了计算贝尔数，我们可以将 (1.38) 的最后部分从 $k = 1$ 加到 n ，其中 n 是你喜欢的任何一个 $\geq n$ 的数字。我们来算一下。结果是：

$$\begin{aligned}
b(n) &= \sum_{k=1}^{\infty} \sum_{r=1}^k (-1)^{k-r} \frac{r^{n-1}}{(r-1)!(k-r)!} \\
&= \sum_{r=1}^{\infty} \frac{r^{n-1}}{(r-1)!} \sum_{k=r}^{\infty} \frac{(-1)^{k-r}}{(k-r)!} \\
&= \sum_{r=1}^{\infty} \frac{r^{n-1}}{(r-1)!} \left\{ \sum_{s=0}^{\infty} \frac{(-1)^s}{s!} \right\}.
\end{aligned}$$

但是现在的数字 r 是任意的，只要满足 $r \geq n$ 即可。由于花括号里指数级数的部分和看起来是如此诱人，让我们保持 n 和 r 固定，令 $s = k - r$ 。这给出了关于贝尔数的以下令人惊叹的公式（你可以自己用 $n = 1$ 检验一下）：

$$b(n) = \frac{1}{n!} \sum_{r=0}^{\infty} \frac{r^n}{r!} \quad (n \geq 0). \quad (1.41)$$

这个贝尔数公式虽然具有某种特殊的魅力，但并不适合用来计算。然而，我们可以从中推导出贝尔数的一个生成函数，它出乎意料地简单和优雅。我们将寻找如下形式的生成函数：

$$(x) = \sum_{n=0}^{\infty} \frac{b(n)}{n!} x^n. \quad (1.42)$$

这是我们第一次发现有必要在生成函数的系数中引入一个额外的因子 $1/n!$ 。然而，这种情况经常发生，我们将在第 2 章讨论如何识别什么时候像这样引入额外因子是有用的。形式为 (1.42)（系数中扔进去了 $1/n!$ ）的生成函数被称为**指数生成函数（exponential generating function）**。例如，我们会说“ (x) 是贝尔数的指数生成函数”。

当我们希望区分各种生成函数时，我们可能会使用这样的短语：序列 $\{a_n\}$ 的**普通幂级数生成函数（ordinary power series generating function）**是 $\sum_n a_n x^n$ ，或者序列 $\{a_n\}$ 的**指数生成函数**是 $\sum_n a_n x^n / n!$ 。

为了显式地求出 (x) ，将对 $n \geq 1$ 有效的公式 (1.41) 乘以 $x^n / n!$ （别忘了除以 $n!$ ），并对所有 $n \geq 1$ 求和。这给出：

$$\begin{aligned}
(x) - 1 &= \left(\frac{1}{n!} \right) \sum_{n=1}^{\infty} \frac{x^n}{n!} \sum_{r=1}^{\infty} \frac{r^{n-1}}{(r-1)!} \\
&= \left(\frac{1}{n!} \right) \sum_{r=1}^{\infty} \frac{1}{r!} \sum_{n=1}^{\infty} \frac{(rx)^n}{n!} \\
&= \left(\frac{1}{n!} \right) \sum_{r=1}^{\infty} \frac{1}{r!} (e^{rx} - 1) \\
&= \left(\frac{1}{n!} \right) \{ e^x - 1 \} \\
&= e^x - 1.
\end{aligned}$$

因此我们已经证明了：

定理 1.5 贝尔数的指数生成函数是 x^{-1} ，也就是说， x^{-1} 幂级数展开式中 $x^n/n!$ 的系数恰好是包含 n 个元素的集合的划分数。

这个结果无疑是生成函数方法威力的一个杰出代表。贝尔数本身极其复杂，但是生成函数却非常简单且易于记忆。

这个故事中的下一个新奇元素是：**我们可以从生成函数反推回到递推公式**，尽管到目前为止在我们所有的例子中，操作的方向都是正好相反的。我们现在打算从定理 1.5 中推导出贝尔数的一个递推公式，有了它，无论你想查看多少个贝尔数，计算起来都会非常容易。

首先，定理告诉我们：

$$\sum_{n \geq 0} \frac{b(n)}{n!} x^n = x^{-1}. \quad (1.43)$$

我们将对这个等式执行一个非常标准的操作。虽然当这个操作第一次出现时，它看起来一点也不标准。

1.6.1 $x(d/dx)$ 技巧 (The $x(d/dx)$ operation)

1. 对等式两边同时取自然对数 (1)。
2. 对两边求导，然后乘以 x 。
3. 清除方程中的分母。
4. 针对每个 n ，找出等式两边 x^n 的系数并使它们相等。

对于上面这个流程，最具有说服力的动机就是：**它就是管用！** 但在我们执行它之前，让我们先停下来花点时间看看它为什么可能会管用。取对数的意义在于简化函数 x^{-1} ，在取对数之前它的幂级数系数非常神秘，而在取对数之后就变得显而易见了。这种简化的代价是，在等式左边我们面临着“**求和的对数**”，这是一个让人看着就头大 (awesome thing to have) 的东西。接下来的求导操作，将“求和的对数”变成了两个和式相除的商，这就漂亮多了。乘以 x 的原因是由于求导使 x 的幂次下降了 1 阶，所以顺手把它补回去是很方便的。清除分母之后，我们看到的将只是两个彼此相等的和式，然后我们的工作就大功告成了。

在这个例子中，在对 (1.43) 应用了步骤 1 之后，我们有：

$$\ln \left\{ \sum_{n \geq 0} \frac{b(n)}{n!} x^n \right\} = x^{-1}.$$

步骤 2 给出：

$$\frac{\sum_n \frac{nb(n)x^n}{n!}}{\sum_n \frac{b(n)x^n}{n!}} = x^{-1}.$$

为了清除分母，将等式两边同时乘以左边的分母，得到：

$$\sum_n \frac{nb(n)x^n}{n!} = (x^x) \sum_n \frac{b(n)x^n}{n!}.$$

最后，我们必须识别这个等式两边 x^n 的系数。在左边这很简单。在右边，我们首先要将两个幂级数相乘，然后再找出系数。由于在第 2 章中我们将制定一个通用的、非常易于使用的规则来处理这类事情，我们现在暂且把这个计算推迟到那时再做，而仅仅在这里直接引用结果。结果就是，贝尔数满足如下递推关系：

$$b(n) = \sum_k \binom{n-1}{k} b(k) \quad (n \geq 1; b(0) = 1). \quad (1.44)$$

我们现在已经看到了好几个如何使用生成函数来寻找递推公式的例子。经常会发生这样的情况：生成函数方法找到了一个递推关系，而直到后来我们才能够对这个递推关系给出一个直接的、纯组合学的解释。在某些情况下，即使找到的递推关系看起来似乎应该有一个简单的组合学解释，但至今也没人找到。

1.7 习题 (Exercises)

1. 寻找以下各个序列的普通幂级数生成函数，要求结果为简单的闭合形式 (closed form)。在每种情况下，序列对所有 $n \geq 0$ 均有定义。

- (a) $a_n = n$
- (b) $a_n = n +$
- (c) $a_n = n^2$
- (d) $a_n = n^2 + n +$
- (e) $a_n = P(n)$ ，其中 P 是一个给定的多项式，次数为 m 。
- (f) $a_n = 3^n$
- (g) $a_n = 5 \cdot 7^n - 3 \cdot 4^n$

2. 对于习题 1 中给出的每一个序列，寻找其指数生成函数，要求结果为简单的闭合形式。

3. 如果 $f(x)$ 是序列 $\{a_n\}_{n \geq 0}$ 的普通幂级数生成函数，那么请用 $f(x)$ 简单地表示出以下序列的普通幂级数生成函数。在每种情况下， n 的取值范围都是 $0, 1, 2, \dots$

- (a) $\{a_n + c\}$
- (b) $\{a_n + c\}$
- (c) $\{na_n\}$
- (d) $\{P(n)a_n\}$ ，其中 P 是一个给定的多项式。
- (e) $0, a_1, a_2, a_3, \dots$
- (f) $0, 0, 1, a_3, a_4, a_5, \dots$
- (g) $a_0, 0, a_2, 0, a_4, 0, a, 0, a, 0, \dots$
- (h) $a_1, a_2, a_3, \dots$
- (i) $\{a_{n+h}\}$ (h 为给定常数)

(j) $\{a_{n+2} + 3a_{n+1} + a_n\}$

(k) $\{a_{n+2} - a_{n+1} - a_n\}$

4. 设 $f(x)$ 是序列 $\{a_n\}$ 的指数生成函数。对于习题 3 中的每一个序列，请用 $f(x)$ 简单地表示出它的指数生成函数。

5. 求：

(a) $[x^n]^{2x}$

(b) $[x^n/n!]^x$

(c) $[x^n/n!] \ln x$

(d) $[x^n]\{1/((1-ax)(1-bx))\}$ ($a \neq b$)

(e) $[x^n](1+x^2)^m$

6. 在下面每个小题中，序列 $\{a_n\}_{n \geq 0}$ 满足给定的递推关系。求该序列的普通幂级数生成函数。

(a) $a_{n+1} = 3a_n + 2$ ($n \geq 0; a_0 = 0$)

(b) $a_{n+1} = a_n + 1$ ($n \geq 0; a_0 = 0$)

(c) $a_{n+2} = 2a_{n+1} - a_n$ ($n \geq 0; a_0 = 0; a_1 = 1$)

(d) $a_{n+1} = a_n/3 + 1$ ($n \geq 0; a_0 = 0$)

7. 请给出递推公式 (1.44) 的直接组合学证明，过程如下：给定 n ；考虑集合 $[n]$ 的所有可能划分的集合。共有 $b(n)$ 种划分。将这个大集合分拣到编号为 $k = 0, 1, \dots, n-1$ 的几堆中，其中第 k 堆包含 $[n]$ 的所有满足如下条件的划分：包含字母“ n ”的那个类中，恰好还包含其他 k 个字母。计算出第 k 堆中的划分数，你的证明就大功告成了。

8. 对于习题 6 中的每一个小题，求出该序列的指数生成函数（为了做到这一点，你可能需要解一个微分方程！）。

9. 一个函数 f 对所有 $n \geq 1$ 都有定义，且满足以下关系：(a) $f(1) = 1$ ；(b) $f(2n) = f(n)$ ；(c) $f(2n+1) = f(n) + f(n+1)$ 。令

$$F(x) = \sum_{n \geq 1} f(n)x^{n-1}$$

为该序列的生成函数。证明

$$F(x) = (1+x+x^2)F(x^2)$$

并由此证明

$$F(x) = \prod_{j \geq 0} \{1 + x^{2^j} + x^{2^{j+1}}\}.$$

10. 设 X 为一个随机变量，其取值为 $0, 1, 2, \dots$ ，对应的概率分别为 $p_0, p_1, p_2, \dots$ ，其中 p_i 是给定的非负实数，且总和为 1。令 $P(x)$ 为序列 $\{p_n\}$ 的普通幂级数生成函数 (opsgf)。

(a) 直接用 $P(x)$ 来表示 X 的均值和标准差。

(b) 独立抽取 X 的两个值。它们的和等于 n 的概率 $\binom{2}{n}$ 是多少？用 $P(x)$ 来表示序列 $\{\binom{2}{n}\}$ 的普通幂

级数生成函数 $P_2(x)$ 。

(c) 独立抽取的 k 个值。令 $\binom{k}{n}$ 为它们的和等于 n 的概率。用 $P(x)$ 来表示序列 $\{\binom{k}{n}\}_{n \geq 0}$ 的普通幂级数生成函数 $P_k(x)$ 。

(d) 使用 (a) 和 (c) 的结果，用 μ 和 σ 来表示 k 个独立选取的值的和的均值与标准差。

(e) 设 $A(x)$ 为一个满足 $A(0) = 1$ 的幂级数，并设 $(x) = A(x)^k$ 。我们希望计算 (x) 的系数，而完全不用对 $A(x)$ 进行任何幂次运算。请使用正文中介绍的“xD1”技巧，推导出一个满足 (x) 各项系数的递推公式。

(f) 一枚灌铅的骰子掷出 1, 2, 3, 4, 5, 6 点的概率分别为 0.1, 0.2, 0.1, 0.2, 0.2, 0.2。现在将这枚骰子投掷 100 次，我们想要计算所有 100 次投掷的点数总和 300 的概率。请将 (x) 识别为某个函数幂级数展开式中 x^{300} 的系数。准确写出该函数是什么（不要求你实际计算出 (x) 的值）。使用 (e) 部分的结果来准确说明：如果你非算不可，你将如何计算。

(g) 一个随机变量取 $1, 2, \dots, m$ 中每个值的概率均为 $1/m$ 。令 S_n 为独立抽取 n 个 x 的值并将它们相加的结果。证明对于 $n = 1, 2, \dots$,

$$\{S_n, j\} = \frac{1}{m^n} \sum_r (-1)^r \binom{n}{r} \binom{j - mr}{n}.$$

11. 对于整数 n ，令 $f(n)$ 为集合 $[n]$ 中**不包含任何两个连续元素**的子集的数量。求出这些数字满足的递推关系，然后“求出”这些数字本身。

12. 对于给定的整数 n, k ，令 $f(n, k)$ 为集合 $[n]$ 中**不包含任何两个连续元素**的 k -子集的数量。求出这些数字满足的递推关系，找出一个合适的生成函数，并求出这些数字本身。将 $f(n, k)$ 对于 n 的数值以帕斯卡三角形排列的形式展示出来。

13. 通过比较上面两道习题的结果，推导出一个恒等式。画一张图，标出该恒等式在帕斯卡三角形中涉及的元素。

14. 将整数 $1, 2, \dots, n$ 依次排成一个圆圈，令 $g(n)$ 为从中选取一个子集，且在圆圈上没有任意两个被选元素相邻的方法数。也就是说， g 与习题 11 中的 f 的区别在于，现在认为 n 和 1 也是连续（相邻）的。求 $g(n)$ 。

15. 与上一题类似，仿照上面的习题 12，求出从排成圆圈的 n 个元素中选取 k 个元素，使得在圆圈上没有任意两个被选元素相邻的方法数 $g(n, k)$ 。

16. 求出函数

$$f(x) = \frac{1}{(1 - x^2)^2}$$

幂级数中 x^n 的系数。首先使用部分分式的方法来求；其次，发挥一点“狡猾”的聪明才智，给出一个简单得多的推导过程。

17. 集合 $[n]$ 的排列的一个**逆序 (inversion)** 是指一对字母 i, j ，满足 $i < j$ 且 $(i) > (j)$ 。在排列的双行书写形式中，逆序表现为在第二行中处于“错误顺序”的一对数字。例如，对于 $\begin{bmatrix} 1 & 2 & 3 & 4 \\ 3 & 1 & 4 & 2 \end{bmatrix}$ 的排列：

$$= \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & & 7 \\ 4 & & 2 & 5 & & 1 & 7 & 3 \end{pmatrix}$$

有 19 个逆序，例如数对 (4,2)、(4,1)、...、(7,3)。令 $b(n, k)$ 为恰好有 k 个逆序的 n 字母排列的数量。为生成函数 $n(x) = \sum_k b(n, k)x^k$ 找出一个“简单”公式。制作一个对于 $n \leq 5$ 的 $b(n, k)$ 数值表。

18.

- (a) 给定 n, k 。在 n 个字母的排列中，有多少个排列满足它的前 k 个值是递减的？
 (b) 对于一个随机的 n -排列，其开头递减序列的平均长度是多少？
 (c) 如果 $f(n, k)$ 是恰好有 k 个**升程 (ascending runs)** 的排列数量，求出 $f(n, k)$ 满足的类似帕斯卡三角形的递推关系。这些数字被称为欧拉数 (Euler numbers)。作为一个例子，排列：

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & & 7 \\ 4 & 1 & & & 2 & 5 & 3 & 7 \end{pmatrix}$$

包含四个升程，即 4, 1 6 9, 2 5 8, 和 3 7。

19. 考虑下面这种形式的 256 种可能的和：

$$1 + 2 + 2_3 + 5_4 + 10_5 + 10 + 20_7 + 50 \quad (1.45)$$

其中每个 $_i$ 都是 0 或 1。

- (a) 对于每个整数 n ，令 a_n 为表示出 n 的不同的和的数量。请将生成多项式

$$1 + 1x + 2x^2 + 3x^3 + \cdots + x^n$$

写成乘积的形式。

- (b) 接下来，考虑由 (1.45) 构成的所有可能的和，其中 a_i 现在可以取 $-1, 0, 1$ 这三个值中的任意一个。对于每个整数 n ，令 D_n 为表示出 n 的不同的和的数量。证明：存在某个整数 n ，它至少能用 33 种不同的方式表示出来。然后将生成函数

$$\sum_{n=-\infty}^{\infty} D_n x^n$$

写成乘积的形式。

- (c) 通过将这组特定的权重替换为一般的权重集合，推广本题 (a) 和 (b) 的结果。将出现的多项式进行因式分解。
 (d) 在本题 (c) 的一般情况下，用这组权重的形式准确陈述生成多项式的所有零点是什么，并准确说明每个零点的重数。

(注：原书习题编号直接从19跳到了21，此处遵照原书排版)

21.

- (a) 我们想要找到函数 x^n 的第 n 阶导数的公式。求导几次，研究其中的规律，然后对一般的 n 猜想答案的形式，其中包括一些待定的常数。接着找到这些常数满足的递推公式，并识别出它们就是我们曾经学习过的一些“著名”的数字。

(b) 接下来令 $f(x_1, \dots, x_n)$ 为某一个 n 元函数。寻找一个混合偏导数的公式

$$\frac{\partial^n f}{\partial x_1 \partial x_2 \cdots \partial x_n}$$

用 f 本身的各种偏导数来表示它。

第 2 章 级数 (Series)

本章致力于研究作为生成函数被广泛使用的不同类型的级数。

2.1 形式幂级数 (Formal Power Series)

讨论幂级数的**形式 (formal)** 理论，而不是它的解析 (analytic) 理论，意味着我们将这些级数纯粹视为**代数对象**，仅仅把它们当成挂载序列的“晾衣绳”，而不去使用该级数可能代表的函数的任何解析性质；甚至，我们根本不需要知道是否存在这样一个函数。

我们之所以研究形式级数，是因为在生成函数理论中经常会发生这样的情况：我们试图求解一个递推关系，于是我们引入了一个生成函数，接着我们进行了一系列各种各样的推导操作；但我们常常带着一种**负罪感**，因为我们并不确定我们正在处理的这些级数是否真的会收敛。此外，我们可能还会发现自己正在对一个生成函数求导，而依然完全不知道这个级数是否收敛于某个函数。

本节的重点就是：完全没有必要感到内疚。 因为各种推导操作完全可以在**形式幂级数环 (ring of formal power series)** 中进行，在这个环里，收敛性问题根本不存在。我们可以执行整套方法，最终得到生成级数，然后才去探究它是否收敛并因此代表一个真实存在的函数。如果它不收敛，我们依然可以从这个形式级数中获取大量信息，尽管可能无法获得解析信息（例如系数大小的渐近公式）。但是，有关目标序列的精确公式很可能依然能够得出，哪怕这些方法在那些情况下纯粹是建立在代数的、形式的基础之上的。

例如，下面这个级数

$$f = 1 + x + 2x^2 + x^3 + 24x^4 + 120x^5 + \cdots + n!x^n + \cdots, \quad (2.1)$$

作为一个形式幂级数，它有着完美而合理的存在性，尽管除了 $x = 0$ 之外它在任何 x 的值处都不收敛，因此也不提供任何使用解析方法进行研究的的可能性。不仅如此，这个级数在某些自然的组合计数问题中还扮演着重要的角色。

一个**形式幂级数**就是一个具有如下形式的表达式：

$$a_0 + a_1x + a_2x^2 + \cdots$$

其中，序列 $\{a_n\}_0$ 被称为**系数序列**。说两个级数**相等**，就是说它们的系数序列完全相同。

我们可以对形式幂级数执行某些特定类型的操作。例如，我们可以对它们进行**加法或减法**。这是按照以下规则进行的：

$$\sum_n a_n x^n \pm \sum_n b_n x^n = \sum_n (a_n \pm b_n) x^n.$$

幂级数可以按照通常的**柯西乘积 (Cauchy product)** 规则进行相乘：

$$\sum_n a_n x^n \sum_n b_n x^n = \sum_n c_n x^n \quad \left(c_n = \sum_k a_k b_{n-k} \right). \quad (2.2)$$

正是这条乘法规则解释了为什么级数方法在组合问题中有着如此广泛的适用性。这是因为，我们经常可以通过选取一个 k 型的对象和一个 $n - k$ 型的对象，然后把它们“缝合”在一起构成一个 n 型对象，以此来构造某个家族中所有 a_n 个 n 型的对象。这么做的组合方法数将是 $a_k b_{n-k}$ ，如果我们对 k 求和，就会发现两个形式级数的柯西乘积与我们正在研究的问题直接相关。

例如，如果我们遵循乘法规则，我们将得到：

$$(1 - x)(1 + x + x^2 + x^3 + \cdots) = 1.$$

因此我们可以说级数 $(1 - x)$ 拥有一个**倒数 (reciprocal)**，且该倒数是 $1 + x + x^2 + \cdots$ （反之亦然）。

命题 2.1 一个形式幂级数 $f = \sum_{n \geq 0} a_n x^n$ 存在倒数，当且仅当 $a_0 \neq 0$ 。在这种情况下，其倒数是唯一的。

证明： 假设 f 有一个倒数，即 $1/f = \sum_{n \geq 0} b_n x^n$ 。那么 $f \cdot (1/f) = 1$ ，并且根据 (2.2) 式， $c_0 = 1 = a_0 b_0$ ，所以必定有 $a_0 \neq 0$ 。此外，在这种情况下，(2.2) 式告诉我们对于 $n \geq 1$ ， $c_n = 0 = \sum_k a_k b_{n-k}$ ，由此我们求得

$$b_n = (-1/a_0) \sum_{k \geq 1} a_k b_{n-k} \quad (n \geq 1). \quad (2.3)$$

这唯一地确定了 $b_1, b_2, \dots$ ，正如我们所宣称的那样。

反之，假设 $a_0 \neq 0$ 。那么我们可以从 (2.3) 中依次确定 $b_0, b_1, \dots$ ，并且由此产生的级数 $\sum_n b_n x^n$ 就是 f 的倒数。

在上述算术规则下，形式幂级数的集合构成了一个**环 (ring)**，在这个环中，可逆元素（可求倒数的元素）正是那些常数项不为零的级数。

上面关于形式幂级数**倒数 (reciprocal, 即 $1/f$)** 的概念，不应与更微妙的**反函数/逆级数 (inverse)** 的概念相混淆。如果级数 f 存在逆级数，那么它是一个级数 g ，使得

$f(g(x)) = g(f(x)) = x$ 。这样的逆级数何时存在呢？首先我们需要能够定义符号 $f(g(x))$ 的意义，然后我们才能去关心它是否等于 x 。

如果 $f = \sum_n a_n x^n$ ，那么 $f(g(x))$ 的意思是：

$$f(g(x)) = \sum_n a_n g(x)^n. \quad (2.4)$$

如果级数 $g(x)$ 有一个非零的常数项 g_0 ，那么 (2.4) 式中的每一项都可能会对 x 的每个幂次的系数产生贡献。另一方面，如果 $g_0 = 0$ ，那么我们将能够仅仅通过查看级数的前 58 项，就能计算出 (2.4) 中 x^{57} 的系数。事实上，请注意每一项

$$\begin{aligned} a_n g(x)^n &= a_n (g_1 x + g_2 x^2 + \dots)^n \\ &= a_n x^n (g_1 + g_2 x + \dots)^n \end{aligned}$$

当 $n > 57$ 时，将只包含高于 57 次的 x 的幂，因此我们不需要去看这些项就能找到 x^{57} 的系数。

因此，如果 $g_0 = 0$ ，那么计算 $f(g(x))$ 的每一个系数都是一个有限的过程 (finite process)，因此所有这些系数都是明确定义的，该级数也是明确定义的。然而，如果 $g_0 \neq 0$ ，那么除非 f 是一个多项式，否则计算 $f(g(x))$ 的每一个系数将是一个无限的过程，因此只有当级数“收敛”时，它才有意义。但在一个形式的、代数的理论中，收敛的概念没有立足之地。因此：**两个形式幂级数的复合 $f(g(x))$ 有定义，当且仅当 $g_0 = 0$ 或者 f 是一个多项式。**

例如，级数 x^{-1} 是一个定义良好的形式级数，而级数 x^x 则是没有定义的，至少从复合函数的一般定义来看是这样的。

回到寻找给定级数 f 的逆级数的问题，我们看到如果这样一个逆级数 g 存在，那么

$$f(g(x)) = g(f(x)) = x \quad (2.5)$$

必须既有意义，又要成立。我们断言：如果 $f(0) = 0$ ，那么逆级数存在当且仅当级数 f 中 x 的系数非零。

命题 2.2 设形式幂级数 f, g 满足 (2.5) 且 $f(0) = 0$ 。那么 $f = f_1 x + f_2 x^2 + \dots$ ($f_1 \neq 0$)，并且 $g = g_1 x + g_2 x^2 + \dots$ ($g_1 \neq 0$)。

证明： 假设 $f = f_r x^r + \dots$ 且 $g = g_s x^s + \dots$ ，其中 $r, s \geq 0$ 且 $f_r g_s \neq 0$ 。那么 $f(g(x)) = x = f_r g_s^r x^{rs} + \dots$ ，由此推得 $rs = 1$ ，进而有 $r = s = 1$ ，证毕。

在形式幂级数环中，还定义了其他操作，这些操作映射了函数微积分中的相应操作，但没有任何求极限的操作。

形式幂级数 $f = \sum_n a_n x^n$ 的**导数 (derivative)** 是级数 $f' = \sum_n n a_n x^{n-1}$ 。求导遵循微积分的常用规则，例如和法则、乘积法则和商法则。其中许多性质在形式级数中证明起来，甚至比在微积分函数中证明更容易。

例如：

命题 2.3 如果 $f' = 0$ ，那么 $f = a_0$ 是一个常数。

证明： 再看一下假设条件 $f' = 0$ 中的“=”号。它意味着形式幂级数 f' 与形式幂级数 0 完全相同，这意味着形式级数 f' 的每一个系数都是 0。但 f' 的系数是 $a_1, 2a_2, 3a_3, \dots$ ，所以它们每一个都是 0，因此对于所有 $j \geq 1$ ， $a_j = 0$ ，这就等同于说 f 是常数。

接下来试试这个：

命题 2.4 如果 $f' = f$ ，那么 $f = c^x$ 。

证明： 因为 $f' = f$ ，所以对于所有的 $n \geq 0$ ， x^n 在 f 中的系数必然与在 f' 中的系数相同。因此对于所有的 $n \geq 0$ ，有 $(n+1)a_{n+1} = a_n$ ，从而 $a_{n+1} = a_n/(n+1)$ ($n \geq 0$)。通过对 n 使用数学归纳法，对于所有的 $n \geq 0$ ， $a_n = a_0/n!$ ，因此 $f = a_0^x$ 。

2.2 形式普通幂级数生成函数的微积分

(The Calculus of Formal Ordinary Power Series Generating Functions)

对形式级数进行的操作，对应着对它们的系数进行的操作。如果级数实际收敛并代表函数，那么对这些函数的操作就对应着对这些函数的幂级数展开系数的某些操作。在本节中，我们将探索其中的一些关系。它们对于帮助我们一眼看出哪种类型的生成函数适合哪种类型的递推关系或其他组合情况，具有巨大的重要性。

定义 2.5 符号 $\sum_{n=0}^{\infty} a_n x^n$ 意味着级数 f 是序列 $\{a_n\}_0$ 的**普通幂级数** (ordinary power series, 简称 'ops') 生成函数。也就是说，它意味着 $f = \sum_{n=0}^{\infty} a_n x^n$ 。

假设 $\sum_{n=0}^{\infty} a_n x^n$ 。那么什么能生成 $\{a_{n+1}\}_0$ 呢？为了回答这个问题，我们做一个小计算：

$$\sum_{n=0}^{\infty} a_{n+1} x^n = \frac{1}{x} \sum_{m=1}^{\infty} a_m x^m = \frac{(f(x) - f(0))}{x}.$$

因此

$$\sum_{n=0}^{\infty} a_{n+1} x^n = \frac{f(x) - a_0}{x}. \quad (2.)$$

因此，下标偏移 1 个单位，会将对应的级数转变为差商 (difference quotient) $(f - a_0)/x$ 。如果偏移 2 个单位，我们当然只需迭代差商操作，就会发现

$$\sum_{n=0}^{\infty} a_{n+2} x^n = \frac{((f - a_0)/x) - a_1}{x} = \frac{f - a_0 - a_1 x}{x^2}.$$

注意，这种观点如何让我们“一眼 (at a glance)”就能看出，斐波那契递推关系

$F_{n+2} = F_{n+1} + F_n$ ($n \geq 0; F_0 = 0; F_1 = 1$) 会直接转化为普通幂级数生成函数关系：

$$\frac{f - x}{x^2} = \frac{f}{x} + f.$$

实际上，本节的目的就是培养这种从**序列关系 (sequence relations)** 快速、便捷地过渡到**级数关系 (series relations)** 的能力。

法则 1 (Rule 1) 如果 $\sum_{n=0}^{\infty} a_n x^n$ ，那么，对于整数 $h > 0$ ，

$$\sum_{n=0}^{\infty} a_{n+h} x^n = \frac{f - a_0 - \cdots - a_{h-1} x^{h-1}}{x^h}.$$

接下来让我们看看将序列乘以 n 的幂次会有什么效果。同样，假设 $\sum_{n=0}^{\infty} a_n x^n$ 。那么什么能生成序列 $\{na_n\}_0$ 呢？这个问题意思是：我们能否用级数 $f = \sum_{n=0}^{\infty} a_n x^n$ 以某种简单的方式来表达级数 $\sum_{n=0}^{\infty} na_n x^n$ ？答案很简单，因为前一个级数恰好是 xf' 。因此，**将序列的第 n 项乘以 n ，会导致其 ops 生成函数被“乘以” $x(d/dx)$ ，我们将把这个操作符写为 xD 。**用符号表示就是，

$$\sum_{n=0}^{\infty} na_n x^n = (xD) \sum_{n=0}^{\infty} a_n x^n. \quad (2.7)$$

作为一个例子，考虑递推式：

$$(n+1)a_{n+1} = 3a_n + 1 \quad (n \geq 0; a_0 = 1).$$

如果 f 是序列 $\{a_n\}_0$ 的 opsgf，那么根据法则 1 和 (2.7)，

$$f' = 3f + \frac{1}{1-x},$$

这是一个关于未知生成函数的一阶微分方程，可以通过标准方法求解。

接着假设 $\sum_{n=0}^{\infty} a_n x^n$ 。那么什么能生成序列 $\{n^2 a_n\}_0$ 呢？

显然，我们再次应用“乘以 n ”的操作符 xD ，所以答案是 $(xD)^2 f$ 。一般而言，

$$(xD)^k \sum_{n=0}^{\infty} a_n x^n = \sum_{n=0}^{\infty} n^k a_n x^n.$$

好了，什么能生成 $\{(3-7n^2)a_n\}_{n \geq 0}$ 呢？同样非常明显，我们把对 n 做的操作同样施加在 xD 上，即答案是 $(3-7(xD)^2)f$ 。一般的处方是：

法则 2 (Rule 2) 如果 $\sum_{n=0}^{\infty} a_n x^n$ ，且 P 是一个给定的多项式，那么

$$P(xD) \sum_{n=0}^{\infty} a_n x^n = \sum_{n=0}^{\infty} P(n) a_n x^n.$$

例 2.6 寻找以下级数求和的闭合公式：

$$\sum_{n \geq 0} (n^2 + 4n + 5)/n!.$$

根据法则，答案就是级数 $\{(xD)^2 + 4(xD) + 5\}^x$ 在 $x = 1$ 处的值：

$$\begin{aligned}\{(xD)^2 + 4(xD) + 5\}^x &= \{x^2 + x\}^x + 4x^x + 5^x \\ &= (x^2 + 5x + 5)^x.\end{aligned}$$

因此这个问题的答案是 11。

但我们作弊了。你抓住这个非法的招式了吗？ 我们把生成函数在 $x = 1$ 处进行了求值，不是吗？在形式级数环中，这种操作是不存在的。在那里，级数在特定的 x 值处没有“值”。字母 x 纯粹是一个形式符号，它的幂仅仅是在晾衣绳上标记出夹子的位置。

能在一个特定的 x 的数值处求值的，是在那个 x 处收敛的幂级数，这是一个解析 (analytic) 的概念，而不是形式的。在经常发生此类情况时，我们用来平息内疚感的方法是这样的：**如果我们在写出递推关系并通过形式幂级数生成函数求解它之后，我们发现得到的级数在复平面内的某个圆盘内收敛于一个解析函数，那么我们刚才形式上做的一整套推导实际上在分析层面上对那个圆盘内的所有复数 x 都是有效的。** 因此，如果能够让我们感到高兴的话，我们完全可以切换档位，将该级数视为一个收敛的解析怪物。

例 2.7 寻找前 N 个正整数平方和的闭合公式。

为此，首先从这个事实开始：

$$\sum_{n=0}^N x^n = \frac{x^{N+1} - 1}{x - 1},$$

并注意到，如果我们对这个关系的左右两边都应用操作符 $(xD)^2$ ，然后令 $x = 1$ ，等式的左边就是我们要找的平方和，而右边就会是答案！因此

$$\sum_{n=1}^N n^2 = (xD)^2 \left\{ \frac{x^{N+1} - 1}{x - 1} \right\}_{x=1}.$$

在进行两次求导并做大量代数化简之后，答案浮出水面：

$$\sum_{n=1}^N n^2 = \frac{N(N+1)(2N+1)}{6} \quad (N = 1, 2, \dots),$$

这个结果你毫无疑问早就知道了。然而，请注意，**生成函数机器有能力极其机械地处理许多涉及求和的、看起来令人生畏的问题。**

我们的第三条法则将重述两个 opsgf 相乘的方式。

法则 3 (Rule 3) 如果 $\overset{f}{\{a_n\}_0}$ 且 $\overset{g}{\{b_n\}_0}$ ，那么

$$s \quad fg \quad (2.)$$

$$\left\{ \sum_{r=0}^n a_r b_{n-r} \right\}_{n=0}.$$

现在考虑多于两个级数相乘的情况。例如，在三个级数的情况下，如果 f, g, h 是这三个级数，且它们分别生成序列 $\{a_n\}, \{b_n\}, \{c_n\}$ ，那么简短的计算表明 fgh 生成的序列是

$$\left\{ \sum_{r+s+t=n} a_r b_s c_t \right\}_{n=0}. \quad (2.)$$

对比上面的法则 3，就能得出适用于任意数量幂级数乘积的一般公式。其中一种情况值得写下来，也就是一个级数的 k 次方的表达式。

法则 4 (Rule 4) 设 $\sum_{n=0}^{\infty} a_n x^n$ ，并设 k 为一个正整数。那么

$$s \quad f^k \quad (2.10)$$

$$\left\{ \sum_{n_1+n_2+\cdots+n_k=n} a_{n_1} a_{n_2} \cdots a_{n_k} \right\}_{n=0}.$$

例 2.8 令 $f(n, k)$ 表示非负整数 n 被写为 k 个非负整数有序和 (ordered sum) 的方法数。求 $f(n, k)$ 。例如， $f(4, 2) = 5$ ，因为 $4 = 4 + 0 = 3 + 1 = 2 + 2 = 1 + 3 = 0 + 4$ 。

为了寻找 f ，考虑幂级数 $1/(1-x)^k$ 。因为 $\sum_{n=0}^{\infty} \frac{1}{k!} \frac{d^n}{dx^n} \frac{1}{(1-x)^k} \bigg|_{x=0} x^n$ ，根据 (2.10) 我们有

$$\sum_{n=0}^{\infty} \frac{1}{k!} \frac{d^n}{dx^n} \frac{1}{(1-x)^k} \bigg|_{x=0} x^n = \sum_{n=0}^{\infty} f(n, k) x^n.$$

根据 (1.31) 式， $f(n, k) = \binom{n+k-1}{n}$ ，我们就大功告成了。

接下来考虑将一个幂级数乘以 $1/(1-x)$ 的效果。假设 $\sum_{n=0}^{\infty} a_n x^n$ 。那么 $f(x)/(1-x)$ 会生成什么序列呢？为了找出答案，我们有：

$$\begin{aligned} \frac{f(x)}{(1-x)} &= (a_0 + a_1 x + a_2 x^2 + \cdots)(1 + x + x^2 + \cdots) \\ &= a_0 + (a_0 + a_1)x + (a_0 + a_1 + a_2)x^2 \\ &\quad + (a_0 + a_1 + a_2 + a_3)x^3 + \cdots. \end{aligned}$$

这清楚地引导我们得出：

法则 5 (Rule 5) 如果 $\sum_{n=0}^{\infty} a_n x^n$ ，那么

$$\frac{f}{s(1-x)} \left\{ \sum_{j=0}^n a_j \right\}_{n \geq 0}.$$

也就是说，将一个 opsgf 除以 $(1-x)$ 的效果是，将被生成的序列替换为其前缀和 (partial sums) 序列。

例 2.9 这里有前 n 个自然数平方和公式的另一种推导方法。既然 $\frac{1}{s}(1-x) \{1\}_{n \geq 0}$ ，根据法则 2 我们有 $(xD)^2(1/(1-x)) \{n^2\}_{n \geq 0}$ ，再根据法则 5，

$$\frac{1}{s1-x} (xD)^2 \frac{1}{1-x} \left\{ \sum_{j=0}^n j^2 \right\}_{n \geq 0}.$$

也就是说，前 n 个正整数的平方和是下面这个级数中 x^n 的系数：

$$\frac{1}{1-x} (xD)^2 \frac{1}{1-x} = \frac{x(1+x)}{(1-x)^4}.$$

然而，根据 $k=3$ 时的 (1.31) 式，

$$[x^n] \left(\frac{1}{(1-x)^4} \right) = \binom{n+3}{3}.$$

因此，根据 (1.8)，

$$\begin{aligned} \frac{x(1+x)}{(1-x)^4} &= \binom{n+2}{3} + \binom{n+1}{3} \\ &= \frac{n(n+1)(2n+1)}{6}, \end{aligned}$$

所以这必定就是前 n 个正整数的平方和。

例 2.10 调和数 (harmonic numbers) $\{H_n\}_1$ 定义为

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \quad (n \geq 1).$$

我们如何求它们的 ops 生成函数？根据法则 5，该函数是 $1/(1-x)$ 乘以正整数倒数序列 $\{1/n\}_1$ 的 opsgf。那么 $f = \sum_{n \geq 1} x^n/n$ 是什么呢？嗯，它的导数是 $1/(1-x)$ ，所以它一定是 $-1/(1-x)$ 。这意味着调和数的 opsgf 是

$$\sum_{n=1} H_n x^n = \frac{1}{1-x} \ln \left(\frac{1}{1-x} \right).$$

例 2.11 证明斐波那契数满足

$$F_0 + F_1 + F_2 + \cdots + F_n = F_{n+2} - 1 \quad (n \geq 0).$$

根据法则 5，左侧序列的 opsgf 是 $F/(1-x)$ ，其中 F 是斐波那契数的 opsgf，我们在 1.3 节中发现它是 $x/(1-x-x^2)$ 。根据法则 1，右侧序列的 opsgf 是

$$\frac{F-x}{x^2} - \frac{1}{1-x},$$

验证这两个式子相等只需花一点点时间即可。

例 2.12 由硬币喷泉 (fountain of coins)，我们指的是将 n 枚硬币排列成若干行，使得第一行的硬币形成一个单一连续的块，并且在所有更高排的行中，每枚硬币恰好接触它下方那一行的两枚硬币。如果第一行包含 k 枚硬币，我们称之为 (n, k) -喷泉。在图 2.1 中我们展示了一个 $(28, 12)$ 喷泉。

(此处图2.1：一个 $(28, 12)$ 的硬币喷泉堆叠图)

在所有可能的喷泉中，我们区分出一种特殊类型：那些每一行都只包含单一连续硬币块的喷泉。我们称这些为**块状喷泉 (block fountains)**。

这里的问题是：有多少个块状喷泉的第一行恰好包含 k 枚硬币？

设 $f(k)$ 为这个数量，对于 $k = 0, 1, 2, \dots$ 。如果我们从这样的一个块状喷泉中剥去第一行，那么我们看到的是另一个块状喷泉，它的硬币数量少了 k 个。反过来，如果我们想要构造第一行有 k 枚硬币的所有可能的块状喷泉，那么首先铺下那一行。然后选择一个数字 j , $0 \leq j \leq k-1$ 。在包含 k 枚硬币的这一行上方，我们将放置一个第一行有 j 枚硬币的块状喷泉。如果 $j = 0$ 只有一种方法可以做到这一点。否则有 $k-j$ 种方法可以做到这一点，这取决于我们将 j 枚硬币的这一行在 k 枚硬币行的上方缩进了多远。由此得出 $f(0) = 1$ 且

$$f(k) = \sum_{j=1}^k (k-j)f(j) + 1 \quad (k = 1, 2, \dots). \quad (2.11)$$

定义 opsgf 为 $F(x) = \sum_{j \geq 0} f(j)x^j$ 。在 (2.11) 的求和号下，出现了一个 $k-j$ 的函数乘以一个 j 的函数，这应该会触发条件反射，告诉你**法则 3** 在此适用，并且这里涉及到了两个普通幂级数生成函数的乘积。所涉及的两个级数分别是整数序列 $\{j\}_1$ 和未知序列 $\{f(j)\}_1$ 的 opsgf。

然而，前一个 opsgf 是 $x/(1-x)^2$ ，而后者是 $F(x) - 1$ 。因此，在将方程 (2.11) 乘以 x^k 并对 $k \geq 1$ 求和后，我们得到

$$F(x) - 1 = \frac{x}{(1-x)^2} (F(x) - 1) + \frac{x}{1-x},$$

由此得到

$$F(x) = \frac{1-2x}{1-3x+x^2}. \quad (2.12)$$

序列 $\{f(k)\}_0$ 的开头是 1, 1, 2, 5, 13, 34, 89,... 如果这些数字看起来非常像斐波那契数列, 请参阅习题 19。

2.3 形式指数生成函数的微积分

(The Calculus of Formal Exponential Generating Functions)

在本节中, 我们将研究上一节中应用于普通幂级数的法则, 在指数生成函数情况下的类比 (analogues)。

定义 2.13 符号 $\overset{f}{g}f$ $\{a_n\}_0$ 表示级数 f 是序列 $\{a_n\}_0$ 的指数生成函数 (exponential generating function, egf), 也就是说

$$f = \sum_{n \geq 0} \frac{a_n}{n!} x^n.$$

让我们假设 $\overset{f}{g}f$ $\{a_n\}_0$, 然后问出与上一节相同的问题。那么, 序列 $\{a_{n+1}\}_0$ 的 egf 是什么呢? 我们断言答案是 f' (f 的导数), 因为

$$\begin{aligned} f' &= \sum_{n=1} \frac{n a_n x^{n-1}}{n!} \\ &= \sum_{n=1} \frac{a_n x^{n-1}}{(n-1)!} \\ &= \sum_{n=0} \frac{a_{n+1} x^n}{n!}, \end{aligned}$$

这与 $\overset{f'}{g}f$ $\{a_{n+1}\}_0$ 的断言是完全等价的。

因此, 在这方面, 指数生成函数的情况比普通幂级数的相应情况要稍微简单一些。序列中下标偏移 1 个单位等同于对生成函数施加算子 D (求导), 而在此前的普通幂级数 (opsgf) 的情况中则是算子 $(f(x) - f(0))/x$ 。因此, 通过归纳, 我们有:

法则 1' (Rule 1') 如果 $\overset{f}{g}f$ $\{a_n\}_0$, 那么, 对于整数 $h \geq 0$,

$$\overset{f}{g}f \{a_{n+h}\}_0 = D^h f. \quad (2.13)$$

邀请读者将这个法则 1' 与第 36 页的法则 1 进行比较。

例 2.14 为了体会这种观点在解决问题时的威力, 让我们来寻找斐波那契数的 egf。现在, 仅仅只需看一眼递推式:

$$F_{n+2} = F_{n+1} + F_n \quad (n \geq 0),$$

我们从法则 1' 就能看出，该 egf 满足如下微分方程：

$$f = f' + f.$$

在解决这个问题的 ops（普通幂级数）版本的相应阶段，我们有一个包含 f 但**不涉及任何导数**的方程需要求解。我们解出了它，然后为了得到斐波那契数的精确公式，不得不处理部分分式展开。而在现在这个版本中，我们求解微分方程，得到：

$$f(x) = c_1 r_+^x + c_2 r_-^x \quad (r_{\pm} = (1 \pm \sqrt{5})/2)$$

其中 c_1 和 c_2 要由初始条件（我们还没用过呢！）来确定： $f(0) = 0$ 和 $f'(0) = 1$ 。在应用了这两个条件之后，我们发现 $c_1 = 1/\sqrt{5}$ 且 $c_2 = -1/\sqrt{5}$ ，由此得出斐波那契数列的 egf 是：

$$f = (r_+^x - r_-^x)/\sqrt{5}. \quad (2.14)$$

现在获得精确公式变得更加容易了，因为**不需要进行部分分式展开**。只需对 (2.14) 两边应用提取系数的操作符 $[x^n/n!]$ ，公式 (1.11) 就会立刻显现出来。

比较这两种方法，我们发现：在这个例子中，ops 方法涉及解一个更容易的**函数方程**

(functional equation) 来寻找生成函数，即它是一个代数方程而不是微分方程。而 egf 方法则涉及从生成函数到精确公式的**更容易的旅程 (easier trip)**，因为不需要部分分式展开。两种方法都行得通，而这毕竟是我们首要追求的目标 (primary desideratum)。

继续，接下来我们讨论适用于 egf 的**法则 2 的类比**，这个非常简单：它是一样的。也就是说，将序列的成员乘以关于 n 的多项式，等同于对 egf 施加作用在算子 xD 上的同一个多项式。我们有：

法则 2' (Rule 2') 如果 $\overset{f}{g} \{a_n\}_0$ ，且 P 是一个给定的多项式，那么

$$\overset{P(xD)f}{g} \{P(n)a_n\}_{n \geq 0}.$$

接下来让我们思考法则 3 的类比，也就是说，当序列的 egf 相乘时，序列会发生什么变化。具体

来说，假设 $\overset{f}{g} \{a_n\}_0$ 且 $\overset{g}{g} \{b_n\}_0$ 。问题是， fg 是哪个序列的 egf？

这被证明有一个非常漂亮，而且**极其有用 (uncommonly useful)** 的答案。为了找到它，我们执行乘法 fg 并尝试识别出 $x^n/n!$ 的系数。我们得到：

$$\begin{aligned}
fg &= \left\{ \sum_{r=0}^{\infty} \frac{a_r x^r}{r!} \right\} \left\{ \sum_{s=0}^{\infty} \frac{b_s x^s}{s!} \right\} \\
&= \sum_{r,s \geq 0} \frac{a_r b_s}{r! s!} x^{r+s} \\
&= \sum_{n \geq 0} x^n \left\{ \sum_{r+s=n} \frac{a_r b_s}{r! s!} \right\}.
\end{aligned}$$

$x^n/n!$ 的系数显然是：

$$\begin{aligned}
\left[\frac{x^n}{n!} \right] (fg) &= \sum_{r+s=n} \frac{n! a_r b_s}{r! s!} \\
&= \sum_r \binom{n}{r} a_r b_{n-r}.
\end{aligned}$$

我们将此结果陈述为：

法则 3' (Rule 3') 如果 $\overset{f}{g}f \{a_n\}_0$ 且 $\overset{g}{g}f \{b_n\}_0$ ，那么 fg 生成的序列是

$$\left\{ \sum_r \binom{n}{r} a_r b_{n-r} \right\}_{n=0}. \quad (2.15)$$

这个法则应当与第 38 页上的法则 3（即普通幂级数 opsgf 相乘的相应法则）进行对比，opsgf 相乘生成的是如下序列：

$$\left\{ \sum_r a_r b_{n-r} \right\}_{n=0}. \quad (2.1)$$

我们早先指出过，在 (2.16) 中展示的序列的卷积（convolution），对于解决这样一类计数问题非常有用：尺寸为 n 的结构是通过将尺寸为 r 的结构和尺寸为 $n-r$ 的结构以所有可能的方式“缝合”在一起而获得的。

相对应地，卷积 (2.15) 在以下的组合情况中非常有用：**我们不仅要两个这样的结构缝合在一起，而且还要对这些结构进行重新标记 (relabel)**。因为，粗略地说，有 $\binom{n}{r}$ 种方法来选择尺寸为 r 的结构的元素的新标签，同时有 a_r 种方法来选择那个结构，还有 b_{n-r} 种方法来选择另一个结构。

由于这些听起来可能非常抽象，让我们用几个例子来使它变得具体。

例 2.15 在 (1.44) 中，我们找到了贝尔数（Bell numbers）的递推公式，我们可以将其写为如下形式：

$$b(n+1) = \sum_k \binom{n}{k} b(k) \quad (n \geq 0; b(0) = 1). \quad (2.17)$$

我们现在将应用本节的方法来寻找贝尔数的 egf。这将给出对定理 1.5 的一个独立的证明，因为 (2.17) 式可以直接推导出来，如第 1 章的习题 7 所述。

设 $B(x)$ 为所求的 egf。根据法则 1'，(2.17) 左侧的 egf 是 $B'(x)$ 。如果我们将 (2.17) 的右侧与 (2.15) 进行比较，我们发现 (2.17) 右侧序列的 egf 是 e^x 与项全是 1 的序列的 egf 的乘积。后者的 egf 显然是 e^x ，因此我们得到：

$$B'(x) = e^x B(x)$$

这是为了找到未知的 egf 我们必须求解的方程。但显然解是 $B(x) = c e^{x-1}$ ，由于 $B(0) = 1$ ，我们必定有 $c = e^{-1}$ ，由此得出 $B(x) = e^{x-1}$ ，从而完成了对定理 1.5 的重新证明。

例 2.16 为了突出普通生成函数与指数生成函数的各自优势，让我们做一个问题，其中出现的序列卷积的形式暗示了我们应该使用生成函数的 ops（普通幂级数）形式。

我们将计算把 n 对括号（每对包含一个左括号和一个右括号）排列成一个**合法字符串（legal string）**的方法数。括号的合法字符串是指具有以下属性的字符串：当我们从左到右扫描该字符串时，我们看到的右括号的数量**永远不会超过**左括号的数量。

刚好有 5 种由 3 对括号组成的合法字符串，即：

$$((())); ((())); (())(); ()()(); ()(()) . \quad (2.1)$$

设 $f(n)$ 为由 n 对括号组成的合法字符串的数量 ($f(0) = 1$)，对于 $n \geq 0$ 。

我们将每个合法字符串与一个唯一的非负整数 k 关联起来，方法如下：当我们从左到右扫描该字符串时，可以肯定的是，在看完了所有 n 对括号之后，左括号的数量将等于右括号的数量。然而，这两个数字可能在更早的时候就已经相等了。例如，在 (2.18) 中的最后一个字符串里，在仅仅扫描了 $k = 1$ 对括号之后，我们发现所有打开的括号都已经被闭合了。一般而言，对于任何合法字符串，**我们与它关联的整数 k ，是使得该字符串的前 $2k$ 个字符自身构成一个合法字符串的最小正整数。**

(2.18) 中每个字符串关联的 k 值分别是 3, 3, 2, 1, 1。如果一个由 $2n$ 个括号组成的合法字符串的 $k = n$ ，我们将称其为**本原的（primitive）**。(2.18) 中的前两个字符串是本原的。

由 $2n$ 个括号组成的合法字符串中，有多少个具有给定的 k 值？设 $g(k)$ 为这样的字符串的数量。的前 $2k$ 个字符是一个本原字符串，而 $g(k)$ 的后 $2n - 2k$ 个字符是一个任意的合法字符串。由于有恰好 $f(n - k)$ 种方法来选择后 $2n - 2k$ 个字符，但我们有多少种方法来选择前 $2k$ 个字符呢？也就是说，长度为 $2k$ 的本原字符串有多少个？

引理 2.17 如果 $k \geq 1$ ， $g(k)$ 是本原合法字符串的数量，且 $f(k)$ 是长度为 $2k$ 的所有括号合法字符串的数量，那么

$$g(k) = f(k - 1).$$

证明： 给定任意一个由 $k - 1$ 对括号组成的合法字符串，通过在它的前面添加一个初始的左括号并在末尾添加一个右括号，就能将其变成一个长度为 $2k$ 的本原字符串。反过来，给定一个长度

为 $2k$ 的本原字符串，如果将其初始的左括号和末尾的右括号删除，剩下的就是一个长度为 $2k - 2$ 的任意合法字符串。因此，长度为 $2k$ 的本原字符串的数量与长度为 $2k - 2$ 的所有合法字符串的数量一样多，也就是说，共有 $f(k - 1)$ 个。

因此，长度为 $2n$ 且具有特定 k 值的合法字符串的数量是 $f(k - 1)f(n - k)$ 。由于每个合法字符串都有一个唯一的 k 值，所以必然有：

$$f(n) = \sum_k f(k - 1)f(n - k) \quad (n \geq 0; f(0) = 1) \quad (2.1)$$

并约定在所有负数参数处 $f = 0$ 。

该递推式使我们很容易就能算出这些值：1, 1, 2, 5, 14, ...

现在让我们来寻找这些数字的生成函数。**到底该用哪种生成函数？** 线索来自于递推式 (2.19) 的形式。右侧的求和显然与两个普通幂级数生成函数乘积的系数有关，所以我们将使用这种类型 (opsgf)。

设 $F = \sum_k f(k)x^k$ 是 $\{f(n)\}_{n \geq 0}$ 的 opsgf。那么 (2.19) 的右侧几乎就是级数 F^2 中 x^n 的系数。确切地说，它到底是什么？它是级数 F 和级数 $\sum_k f(k - 1)x^k$ 乘积中 x^n 的系数。而后面这个级数与 F 的关系是什么呢？它恰恰就是 xF 。

因此，如果我们将 (2.19) 的右侧乘以 x^n 并在 $n \geq 0$ 上求和，我们得到 xF^2 。如果我们将左侧乘以 x^n 并在 $n \geq 0$ 上求和，我们得到 $F - 1$ 。因此我们未知的生成函数满足方程：

$$F(x) - 1 = xF(x)^2. \quad (2.20)$$

在这里我们遇到了一个新花样。我们已经习惯了从一个序列的递推关系，转变为必须求解以找出生成函数的方程。在前面的例子中，那些函数方程要么是简单的一次线性方程，要么是微分方程。而在 (2.20) 中，**我们得到了一个满足二次方程的生成函数**。当我们求解它时，我们得到：

$$F(x) = \frac{1 \pm \sqrt{1 - 4x}}{2x}.$$

我们想要哪个符号？如果我们选择“+”号，那么当 $x \rightarrow 0$ 时，分子将趋近于 2，所以这个比值在 0 处将趋于无穷大。但我们的生成函数在 0 处的值是 1，所以那不可能是对的。如果我们选择“-”号，那么使用一下洛必达法则 (L'Hospital's rule) 就能证明我们确实将得到 $F(0) = 1$ 。因此我们的生成函数是：

$$F(x) = \frac{1 - \sqrt{1 - 4x}}{2x}. \quad (2.21)$$

这无疑是组合数学中最著名的生成函数之一。数字 $f(n)$ 是**卡特兰数 (Catalan numbers)**，在 (2.42) 中有一个它们的显式公式。目前，我们宣布这个旨在展示“如何由递推式的形式来引导我们选择哪种生成函数”的练习到此结束。

例 2.18 我们所说的 n 个字母的**错排 (derangement)**，指的是一个没有任何不动点 (fixed

points) 的排列。令 D_n 表示 n 个字母的错排数量，并令 $\{D_n\}_0$ 。我们将找到该序列的递推式，然后找到 $D(x)$ ，最后找到该序列成员的显式公式。

n 个字母的排列中，将某个特定的、包含 k n 个字母的集合作为其所有不动点集合的排列数量，显然是 D_{n-k} 。一共有 $\binom{n}{k}$ 种方法来选择由 k 个不动点组成的集合，因此恰好有 $\binom{n}{k}D_{n-k}$ 个拥有正好 k 个不动点的 n 字母排列。由于**每个**排列都具有**某个**不动点集合，因此必定有：

$$n! = \sum_k \binom{n}{k} D_{n-k} \quad (n \geq 0).$$

如果我们对等式两边取 egf，根据法则 3'，我们得到：

$$\frac{1}{1-x} = {}^x D(x)$$

(你看这有多简单?)，由此得到 $D(x) = -x/(1-x)$ 。接着，根据法则 5，如果在两边同时取 $[x^n]$ 操作，我们发现：

$$\frac{D_n}{n!} = 1 - 1 + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!},$$

我们就大功告成了。

就像在普通幂级数生成函数的情况下一样，当我们考虑三个或更多指数生成函数的乘积时，会发生令人愉快且有用的事情。例如，如果我们将分别生成 f, g 和 h 的三个函数相乘，那么我们发现：

$${}^{fgh} \left\{ \sum_{r+s+t=n} \frac{n!}{r!s!t!} a_r b_s c_t \right\}_{n=0}, \quad (2.22)$$

因此可以预期，这种运算在处理涉及多项式系数 (multinomial coefficients) 的求和时会很有帮助。

如果 $\{a_n\}_0$ ，那么

$${}^{f^k} \left\{ \sum_{r_1+\cdots+r_k=n} \frac{n!}{r_1!r_2!\cdots r_k!} a_{r_1} a_{r_2} \cdots a_{r_k} \right\}_{n=0}. \quad (2.23)$$

2.4 幂级数，解析理论 (Power Series, Analytic Theory)

幂级数的**形式 (formal)** 理论向我们表明，我们可以操作递推关系并求解函数方程（比如微分方程）来得到幂级数，而不必非得去担心由此产生的级数是否收敛。然而，如果它们确实收敛，并且代表了真实的函数，那就是一个巨大的优势。因为这样一来，我们就有可能找到关于该递推关系的解析信息，而这些信息通过其他方式往往很难获得。

在本节中，我们将复习幂级数及其系数序列的基本解析性质。

首先，假设给定了一个幂级数

$$f = \sum_{n \geq 0} a_n z^n,$$

在这里我们现在使用字母 z 来鼓励大家往**复变量 (complex variables)** 的方向思考。问题是：确切地说，对于什么样的复数 z 的取值集合，该级数 f 是收敛的？我们想要对这个问题给出一个相当完整的回答，并用其系数序列 $\{a_n\}_0$ 来表达这个答案。

定理 2.19 存在一个数字 R ， $0 \leq R \leq +\infty$ ，称为该级数 f 的**收敛半径 (radius of convergence)**，使得该级数对于所有满足 $|z| < R$ 的 z 均收敛，而对于所有满足 $|z| > R$ 的 z 均发散。数字 R 可以通过级数的系数序列 $\{a_n\}_0$ 以下式表达：

$$R = \frac{1}{\limsup_{n \rightarrow \infty} |a_n|^{1/n}} \quad (1/0 = +\infty; 1/+\infty = 0). \quad (2.24)$$

在证明该定理之前，我们先回顾一下序列的**上极限 (limit superior)** 的定义。设 $\{x_n\}_0$ 是一个实数序列，并设 x 为一个实数（可能为 $\pm\infty$ ）。

定义 2.20 如果满足以下条件，我们就说 x 是序列 $\{x_n\}$ 的**上极限 ('upper limit')**：

(a) x 是有限的，并且

(i) 对于任意 $\epsilon > 0$ ，序列中除了有限个成员外，其余所有成员均满足 $x_n < x + \epsilon$ ，并且

(ii) 对于任意 $\epsilon > 0$ ，序列中有无限多个成员满足 $x_n > x - \epsilon$ ，或者

(b) $x = +\infty$ ，并且对于任意 $M > 0$ ，存在一个 n 使得 $x_n > M$ ，或者

(c) $x = -\infty$ ，并且对于任意 x ，仅有有限个 n 使得 $x_n > x$ 。

如果 x 是序列 $\{x_n\}_0$ 的上极限，那么我们写成 $x = \limsup_{n \rightarrow \infty} x_n$ ，或者在上下文足够清晰时，直接写成 $x = l\{x_n\}$ 。

上极限具有以下性质：

- 在扩展的实数系统中（即包含 $\pm\infty$ ），每个实数序列都有且仅有一个上极限。
- 如果一个序列有极限 L ，那么 L 也是该序列的上极限。
- 如果 S 是序列 $\{x_n\}_0$ 的**聚点 (cluster points)** 集合，那么 $l\{x_n\}$ 是集合 S 中所有数字的最小上界 (least upper bound)。

证明 (定理 2.19)： 设 R 为 (2.24) 中所示的数字，首先假设 $0 < R < +\infty$ 。选取 ϵ 使得 $|z| < R$ 。我们将证明级数在 z 处收敛。

对于给定的 ϵ ，我们可以找到 $N > 0$ 使得

$$|a_n|^{1/n} < \frac{1}{1 + \epsilon}.$$

现在，根据上极限的定义，存在一个 N ，使得对于所有的 $n > N$ ，我们有

$$|a_n|^{1/n} < \frac{1}{1 + \epsilon}.$$

因此，对于这些相同的 n ，

$$|a_n|^{1/n} < \left\{ \left(\frac{1}{1 + \epsilon} \right) \right\}^n.$$

令 ϵ 表示花括号中的数字。那么根据我们对 ϵ 的选择，我们有 $\epsilon < 1$ 。因此，通过与一个收敛的几何级数（等比级数）的各项进行比较，级数 $\sum a_n^n$ 是绝对收敛的。因此我们的级数在 $|z| < 1 + \epsilon$ 处绝对收敛，因此它对于所有 $|z| < 1 + \epsilon$ 都绝对收敛。

接下来我们断言，如果 $|z| > 1$ ，级数发散。确实，我们将证明对于这样的 z ，级数的各项构成的序列并不趋于零。因为 $|z| > 1$ ，我们可以选择 $\epsilon > 0$ ，使得若 $\epsilon = |z| - 1$ ，则 $|z| > 1 + \epsilon$ 。根据上极限的定义，对于无限多个 n 的值，我们有 $|a_n|^{1/n} > (1 + \epsilon)$ 。因此，对于这些 n 的值，

$$|a_n^n| > \left(\frac{1}{1 + \epsilon} \right)^n = \frac{1}{(1 + \epsilon)^n},$$

由于 $|z| > 1$ ，它将无界增长。因此，幂级数各项的那个子序列不趋于零，级数发散。这完成了在 $0 < |z| < 1 + \epsilon$ 情况下的定理证明。 $|z| = 0$ 或 $|z| = +\infty$ 的情况非常相似，留给读者自行完成。

定理 2.21 假设幂级数 $\sum a_n^n$ 对于所有位于 $|z| < 1$ 中的 z 收敛，并令 $f(z)$ 表示它的和。那么 $f(z)$ 在 $|z| < 1$ 中是一个**解析函数 (analytic function)**。如果进一步该级数对于 $|z| > 1$ 发散，那么函数 $f(z)$ 在收敛圆周 $|z| = 1$ 上必定至少有一个**奇点 (singularity)**。

换句话说：一个幂级数会一直收敛下去，直到被某种东西拦住去路，那个东西就是它所代表的函数的奇点。

证明： 如果 f 在其收敛圆周 $|z| = 1$ 上没有任何奇点，那么在该圆周上的每一个点周围，我们都可以画出一个开圆盘，在这些圆盘内 f 仍然保持解析。根据海涅-博雷尔定理 (Heine-Borel theorem, 即有限覆盖定理)，有限个这样的圆盘就能覆盖住圆周 $|z| = 1$ ，因此 f 必定在某个更大的圆盘 $|z| < 1 + \epsilon$ 内保持解析。由柯西不等式可知，用于表示 f 的泰勒系数满足 $|a_n| \leq (1 + \epsilon)^n$ (对所有 n 成立)，因此该级数必定在一个更大的圆盘内收敛——这就产生了矛盾。

例 2.22 级数 $\sum z^n$ 在 $|z| < 1$ 时收敛，在 $|z| > 1$ 时发散。因此，它所代表的函数必定在圆周 $|z| = 1$ 的某处存在一个奇点。那个函数是 $1/(1 - z)$ ，果然，它在 $z = 1$ 处有一个奇点。

例 2.23 考虑函数 $f(z) = 1/(2 - z)$ 。假设我们将 $f(z)$ 在 $z = 0$ 处展开为幂级数。该级数的收敛半径将是多少？

根据定理，该级数将在使 f 保持解析的最大的圆盘 $|z| < 2$ 内收敛。该函数仅在满足 $z = 2$ 的点 z 处

不解析。这些点的形式为 $z = 12 + 2ki$ (其中 k 为整数)，其中距离原点最近的一个是 $12i$ 。因此 $f(z)$ 在圆盘 $|z| < 12$ 内解析，且在任何更大的圆盘内均不解析。因此该级数的收敛半径将是 $R = 12$ 。

请记住，如果给定了 $f(z)$ ，求它在原点处的幂级数展开式的收敛半径的最好方法，很可能就是去寻找它距离原点最近的那个奇点。

例 2.24 考虑函数 $f(z) = 1/(z-1)$ ($f(0) = 1$)。请直接根据该函数的解析性质，估计其在原点处的幂级数展开式的系数大小。

这就是事情开始变得更加有趣的地方。除了可能在 $z = 1$ 的点 (即除了可能在 $z = 2ki$ 处，其中 k 为整数) 之外，这个 $f(z)$ 处处解析。这些点中距离原点最近的是原点本身 ($k = 0$)。然而， f 在 $z = 0$ 处**并不是**奇点，因为虽然那里 f 的分母为 0，但分子也为 0，使用洛必达法则 (或任何其他方法) 会揭示其值为 $f(0) = 1$ ，从而消除了这个“奇点”(这是一个可去奇点)。因此，这个函数距离原点最近的奇点在 $z = 2i$ 处。该幂级数

$$\frac{1}{z-1} = \sum_{n=0}^{\infty} a_n z^n$$

因此它的收敛半径为 $R = 2$ 。

该问题要求估计系数 $\{a_n\}_0$ 的大小。由于收敛半径是 2，根据定理 2.19，我们有：

$$\limsup_{n \rightarrow \infty} |a_n|^{1/n} = \frac{1}{2}.$$

由此可见，首先，对于所有足够大的 n 值，我们有

$$|a_n|^{1/n} < \frac{1}{2} + \epsilon,$$

并且对于无限多个 n 的值，我们有

$$|a_n|^{1/n} > \frac{1}{2} - \epsilon.$$

因此，我们发现了关于系数的以下事实：对于任意 $\epsilon > 0$ ，存在一个 N 使得

$$|a_n| < \left(\frac{1}{2} + \epsilon\right)^n \quad (n > N)$$

并且进一步，对于无限多个 n 的值，

$$|a_n| > \left(\frac{1}{2} - \epsilon\right)^n.$$

因此，这个级数的系数呈**指数级快速趋向于零**，对于大的 n ，衰减的速率大约是 $1/(2)^n$ 。能在完全不需要计算出任何一个系数的情况下，就找出关于系数大小的这么多信息，这是相当了不起的！

为了方便将来参考，我们将在这个例子中学到的东西总结成一个一般的命题。

定理 2.25 设 $f(z) = \sum a_n z^n$ 在包含原点的某个区域内解析，设 $f(z)$ 模长最小的奇点位于点 z_0 处，并给定 $\delta > 0$ 。那么存在 N ，使得对于所有 $n > N$ ，我们有

$$|a_n| < \left(\frac{1}{|z_0|} + \delta \right)^n.$$

进一步，对于无限多个 n 的值，我们有

$$|a_n| > \left(\frac{1}{|z_0|} - \delta \right)^n.$$

在第 5 章中，我们将学习如何基于级数所代表函数的解析性（或解析性的缺失），来对幂级数的系数大小做出精确得多的估计。例如，Darboux 方法（定理 5.11）就是一种用于对生成函数的系数序列进行渐近分析的强大技术。存在这类方法，正是我们应当精通生成函数这门学科的解析面（而不仅仅是形式面）的一个绝佳理由。

另一条通往系数序列渐近分析的路径，源于**柯西公式 (Cauchy's formula)**：

$$a_n = \frac{1}{2\pi i} \oint \frac{f(z) dz}{z^{n+1}} \quad (n = 0, 1, 2, \dots) \quad (2.25)$$

它将泰勒级数展开式 $f(z) = \sum a_n z^n$ 的第 n 个系数表达为了一个涉及函数 f 的围道积分（contour integral）。该围道可以是任何一条包围原点、且完全位于 f 的解析区域内的简单闭曲线。

从 (2.25) 我们立刻就能得到**柯西不等式 (Cauchy's inequality)**，它声明：

$$|a_n| \leq \frac{(r)^n}{r^n},$$

并且该式对于所有的 $n \geq 0$ 以及所有的 $0 < r < R$ 均成立，其中 R 是级数的收敛半径，且

$$(r) = \max_{|z|=r} |f(z)| = \max_{|z|=r} |f(z)|. \quad (2.2)$$

正如上面例 2.24 的分析被 Darboux 方法改进为一种估计系数序列增长的精确得多的方法一样，柯西不等式也被 Hayman 方法（定理 5.15）改进为另一种用于同一目的的极其精确的工具。

如果一个幂级数实际收敛于一个函数，那么我们可以使用**单位根 (roots of unity)** 从一个级数中提取出以等差数列（progression）形式跳跃出现的项。例如，我们如何才能从一个幂级数中只挑选出偶数次幂？如果该级数代表一个函数 f ，那么众所周知， $(f(x) + f(-x))/2$ 恰好只包含 $f(x)$ 级数中涉及 x 的偶数次幂的项，而 $(f(x) - f(-x))/2$ 恰好只包含奇数项。

但是假设，我们不想保留级数的每第二项，而是想只保留**每第三项**呢？例如，如果我们取指数级数，并且只保留其中 x 的幂是 3 的倍数的项，我们会得到什么函数？换句话说，是谁等于：

$$g(x) = \sum_{n \geq 0} \frac{x^{3n}}{(3n)!} \quad (2.27)$$

嗯，让 $(f(x) + f(-x))/2$ 起作用的原因是，两个单位平方根（也就是 ± 1 ）具有这样的性质：

$$\frac{1^n + (-1)^n}{2} = \begin{cases} 1, & \text{如果 } n \text{ 是偶数;} \\ 0, & \text{如果 } n \text{ 是奇数.} \end{cases}$$

现在，三个**单位立方根 (cube roots of unity)** $1, \omega, \omega^2$ 同样具有一个对应且非常有用的属性：

$$\frac{(1^n + \omega^n + \omega^{2n})}{3} = \begin{cases} 1, & \text{如果 } 3 \mid n; \\ 0, & \text{其他.} \end{cases} \quad (2.2)$$

既然情况如此，对于任何一个收敛的幂级数 $f = \sum_r a_r x^r$ ，我们有：

$$\frac{f(x) + f(\omega x) + f(\omega^2 x)}{3} = \sum_r a_{3r} x^{3r}. \quad (2.2)$$

由于 $\omega = e^{2\pi i/3}$ 且 $\omega^2 = e^{4\pi i/3}$ ，我们可以揭开 (2.27) 中神秘函数 $g(x)$ 的面纱，它就是：

$$\begin{aligned} g(x) &= \frac{1}{3} (x + \omega x + \omega^2 x) \\ &= \frac{1}{3} \left(x + 2^{-x/2} \left(\frac{\sqrt{3}x}{2} \right) \right). \end{aligned} \quad (2.30)$$

例 2.26 对于固定的 n ，求：

$$a_n = \sum_k (-1)^k \binom{n}{3k}.$$

如果我们知道下面这个函数，我们就能做这道题：

$$f(x) = \sum_k \binom{n}{3k} x^{3k},$$

因为 $a_n = f(-1)$ 。但是 $f(x)$ 挑选出的是级数 $F(x) = (1+x)^n$ 中的每第三项，所以

$$\begin{aligned} f(x) &= (F(x) + F(\omega x) + F(\omega^2 x))/3 \\ &= \{(1+x)^n + (1+\omega x)^n + (1+\omega^2 x)^n\}/3. \end{aligned}$$

因此，要求我们找的那个数字就是，对于 $n > 0$ ，

$$\begin{aligned} a_n &= f(-1) = \{(1-\omega)^n + (1-\omega^2)^n\}/3 \\ &= \frac{1}{3} \left\{ \left(\frac{3 - \sqrt{3}i}{2} \right)^n + \left(\frac{3 + \sqrt{3}i}{2} \right)^n \right\} \\ &= 2 \cdot 3^{(n/2-1)} \binom{n}{n/2}. \end{aligned} \quad (2.31)$$

序列 $\{a_n\}_{n \geq 0}$ 的前几个值是 1, 1, 1, 0, -3, -9, -18,

为了完善这个例子，我们要证明一下单位立方根那个有用的性质 (2.28)。实际上，对于任何 $r > 1$ ， r 次单位根也会做同样的事情，即：

$$\frac{1}{r} \sum_{n=1}^r = \begin{cases} 1 & \text{如果 } r \mid n; \\ 0 & \text{其他.} \end{cases} \quad (2.32)$$

确实，左侧就是

$$\frac{1}{r} \sum_{j=0}^{r-1} \omega^{(2ij n)/r},$$

这是一个有限的几何级数，它的和很容易求出，且正如同 (2.32) 中所述。因此，多多少少费点力气，我们总是可能从一个收敛级数中，挑选出具有算术级数（等差数列）指数的项构成的子集。见习题 25。

2.5 一些有用的幂级数 (Some Useful Power Series)

生成函数学家们 (Generatingfunctionologists) 需要一份已知幂级数以及在理论应用中经常出现的其他级数的参考列表。这里就是这样一份列表。对于每个级数，我们都给出了级数本身及其对应的和函数。发生收敛的、以原点为中心的最大开圆盘的半径，当然也就是距离原点最近的那个函数奇点的模长。考虑到这些函数的形式相对简单，那些奇点的位置是非常明显的，所以我们在这里没有明确标出收敛半径。

$$\frac{1}{1-x} = \sum_{n \geq 0} x^n \quad (2.33)$$

$$-\ln(1-x) = \sum_{n \geq 1} \frac{x^n}{n} \quad (2.34)$$

$$e^x = \sum_{n \geq 0} \frac{x^n}{n!} \quad (2.35)$$

$$\cos x = \sum_{n \geq 0} (-1)^n \frac{x^{2n}}{(2n)!} \quad (2.36)$$

$$\sin x = \sum_{n \geq 0} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \quad (2.37)$$

$$(1+x)^k = \sum_{k \geq 0} \binom{k}{n} x^n \quad (2.38)$$

$$\frac{1}{(1-x)^{k+1}} = \sum_{n \geq 0} \binom{n+k}{k} x^n \quad (2.39)$$

$$\frac{x}{x-1} = \sum_{n \geq 0} \frac{n x^n}{n!} \quad (2.40)$$

$$\mathbf{n}^{-1} x = \sum_{n \geq 0} (-1)^n \frac{x^{2n+1}}{2n+1} \quad (2.41)$$

$$\begin{aligned} \frac{1}{2x}(1 - \sqrt{1-4x}) &= \sum_n \frac{1}{n+1} \binom{2n}{n} x^n \\ &= 1 + x + 2x^2 + 5x^3 + 14x^4 + 42x^5 \\ &\quad + 132x^6 + 42x^7 + 1430x^8 \\ &\quad + 42x^9 + \dots \end{aligned} \quad (2.42)$$

$$\begin{aligned} \frac{1}{\sqrt{1-4x}} &= \sum_k \binom{2k}{k} x^k \\ &= 1 + 2x + x^2 + 20x^3 + 70x^4 + 252x^5 + 24x^6 \\ &\quad + 3432x^7 + 1270x^8 + 420x^9 + \dots \end{aligned} \quad (2.43)$$

$$\begin{aligned} x x &= \sum_{k \geq 0} \frac{(-4)^k 2k}{(2k)!} x^{2k} \\ &= 1 - \frac{x^2}{3} - \frac{x^4}{45} - \frac{2x}{45} - \frac{x}{4725} - \frac{2x^{10}}{3555} - \dots \end{aligned} \quad (2.44)$$

$$\begin{aligned} \mathbf{n} x &= \sum_{r \geq 1} (-1)^{r-1} \frac{2^{2r} (2^{2r} - 1)_{2r}}{(2r)!} x^{2r-1} \\ &= x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{17x^7}{315} + \frac{2x}{235} + \frac{132x^{11}}{15525} + \dots \\ &\quad + \frac{2144x^{13}}{01075} + \frac{25x^{15}}{351275} + \dots \end{aligned} \quad (2.45)$$

$$\begin{aligned} \frac{x}{\mathbf{n} x} &= \sum_{r \geq 0} (-1)^{r-1} \frac{(4^r - 2)_{2r}}{(2r)!} x^{2r} \\ &= 1 + \frac{x^2}{30} + \frac{7x^4}{30} + \frac{31x}{15120} + \dots \end{aligned} \quad (2.4)$$

$$\frac{1}{\sqrt{1-4x}} \left(\frac{1 - \sqrt{1-4x}}{2x} \right)^k = \sum_n \binom{2n+k}{n} x^n \quad (2.47)$$

$$\left(\frac{1 - \sqrt{1-4x}}{2x} \right)^k = \sum_{n \geq 0} \frac{k(2n+k-1)!}{n!(n+k)!} x^n \quad (k \geq 1) \quad (2.4)$$

$$\mathbf{n}^{-1}(x) = x + \frac{1}{2} \frac{x^3}{3} + \frac{1 \cdot 3}{2 \cdot 4} \frac{x^5}{5} + \frac{1 \cdot 3 \cdot 5}{2 \cdot 4 \cdot} \frac{x^7}{7} + \dots \quad (2.4)$$

$$\begin{aligned} x \mathbf{n} x &= \sum_{n \geq 1} \frac{2^{\frac{n}{2}} \mathbf{n} \frac{n}{4}}{n!} x^n \\ &= x + x^2 + \frac{x^3}{3} - \frac{x^5}{30} - \frac{x}{0} - \frac{x^7}{30} + \dots \end{aligned} \quad (2.50)$$

$$\begin{aligned}
\frac{1}{2} \mathbf{n}^{-1}(x) \mathbf{l}(1+x^2) &= \sum_{r \geq 1} (-1)^{r-1} H_{2r} \frac{x^{2r+1}}{2r+1} \\
&= \frac{x^3}{2} - \frac{5x^5}{12} + \frac{7x^7}{20} - \frac{71x}{2520} + \dots
\end{aligned} \tag{2.51}$$

$$\begin{aligned}
\frac{1}{4} \mathbf{n}^{-1}(x) \mathbf{l} \frac{1+x}{1-x} &= \sum_{r \geq 0} \frac{x^{4r+2}}{4r+2} \left(1 - \frac{1}{3} + \frac{1}{5} - \dots + \frac{1}{4r+1} \right) \\
&= \frac{x^2}{2} + \frac{13x}{0} + \frac{23x^{10}}{3150} + \dots
\end{aligned} \tag{2.52}$$

$$\begin{aligned}
\frac{1}{2} \left\{ \mathbf{l} \frac{1}{1-x} \right\}^2 &= \sum_{r \geq 2} \frac{H_{r-1}}{r} x^r \\
&= \frac{x^2}{2} + \frac{x^3}{2} + \frac{11x^4}{24} + \frac{5x^5}{12} + \frac{137x}{30} + \frac{7x^7}{20} + \dots
\end{aligned} \tag{2.53}$$

$$\begin{aligned}
\frac{\sqrt{1-\sqrt{1-x}}}{x} &= \sum_{k=0} \frac{(4k)!}{1^k \sqrt{2} (2k)! (2k+1)!} x^k \\
&= \frac{1}{\sqrt{2}} \left(1 + \frac{x}{2} + \frac{7x^2}{12} + \frac{33x^3}{1024} + \frac{715x^4}{327} + \frac{41x^5}{22144} \right. \\
&\quad + \frac{52003x}{414304} + \frac{334305x^7}{33554432} + \frac{17735x}{2147434} \\
&\quad \left. + \frac{114075x}{171714} + \frac{141030105x^{10}}{27477044} + \dots \right)
\end{aligned} \tag{2.54}$$

$$\begin{aligned}
\mathbf{n} x &= \sum_{k=0} \frac{j=0^{k-1} (4j^2+1)}{(2k)!} x^{2k} + \sum_{k=0} \frac{4^k j=1^k (\frac{1}{2} - j + j^2)}{(2k+1)!} x^{2k+1} \\
&= 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \frac{5x^4}{24} + \frac{x^5}{144} + \frac{17x}{144} + \frac{13x^7}{12} \\
&\quad + \frac{2x}{04} + \frac{325x}{453} + \frac{177x^{10}}{145152} + \dots
\end{aligned} \tag{2.55}$$

$$\begin{aligned}
\left(\frac{\mathbf{n} x}{x} \right)^2 &= \sum_{k=0} \frac{4^k k!^2}{(k+1)(2k+1)!} x^{2k} \\
&= 1 + \frac{x^2}{3} + \frac{x^4}{45} + \frac{4x}{35} + \frac{12x}{1575} + \frac{12x^{10}}{207} + \dots
\end{aligned} \tag{2.5}$$

$$\begin{aligned}
(x + \sqrt{1+x^2})^a &= \sum_{k=0}^{\infty} \frac{2^k \cdot \left(\frac{a}{2} - \frac{k}{2} + 1\right)^k}{(1+k/a)k!} x^k \\
&= 1 + ax + \frac{a^2 x^2}{2} + \left(\frac{-a}{1} + \frac{a^3}{2}\right) x^3 + \left(\frac{-a^2}{2} + \frac{a^4}{24}\right) x^4 \\
&\quad + \frac{a(-10a^2 + a^4)}{120} x^5 + \frac{a^2(4 - 20a^2 + a^4)}{720} x^6 \\
&\quad + \frac{a(-225 + 25a^2 - 35a^4 + a)}{5040} x^7 \\
&\quad + \frac{a^2(-2304 + 74a^2 - 5a^4 + a)}{40320} x^8 + \dots
\end{aligned} \tag{2.57}$$

在上述公式中， $\{a_n\}$ 是**伯努利数 (Bernoulli numbers)**，由公式 (2.40) 定义。前 16 个伯努利数 $\{a_n\}_0^1$ 的值依次为：

$$1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, 0, -\frac{1}{30}, 0, \frac{5}{66}, 0, -\frac{1}{42}, 0, \frac{7}{78}, 0, -\frac{317}{510}.$$

$\{H_n\}$ 是在 2.2 节中定义的调和数。在 (2.57) 中的符号 m^k ，表示上升阶乘 $m(m+1)\cdots(m+k-1)$ 。(2.54)–(2.57) 的展开式取自 [Ko]。

2.6 狄利克雷级数，形式理论 (Dirichlet Series, Formal theory)

我们已经讨论了两种略有不同的生成函数形式，即普通幂级数形式和指数生成函数形式。我们曾指出，在一个具体的问题中，当面临选择使用哪种形式时，这种选择往往取决于问题中出现的两个序列的**乘法卷积 (multiplicative convolution)** 的形式。如果形式如法则 3' 和 (2.15) 所示，我们会选择 egf；而如果形式如 (2.16) 所示，opsgf 可能就是首选。

为了帮助凸显做出此类选择的基础，我们现在将讨论另一种类型的生成函数，它与另一种常常在组合数学和数论问题中自然出现的序列卷积相匹配。

定义 2.27 给定一个序列 $\{a_n\}_1$ ；我们称如下形式的形式级数

$$\begin{aligned}
f(s) &= \sum_{n=1}^{\infty} \frac{a_n}{n^s} \\
&= a_1 + \frac{a_2}{2^s} + \frac{a_3}{3^s} + \frac{a_4}{4^s} + \dots
\end{aligned} \tag{2.5}$$

为该序列的**狄利克雷级数生成函数 (Dirichlet series generating function, 简称 Dsgf)**，并且我们将其记为

$$f_{Dir}(s) \quad \{a_n\}_1.$$

狄利克雷级数的重要性直接来源于它们的乘法规则。假设 $\sum_{n=1}^{\infty} a_n n^{-s}$ 且 $\sum_{n=1}^{\infty} b_n n^{-s}$ 。问题是， $f(s)g(s)$ 生成的序列是什么？

为了找出答案，考虑这些级数的乘积：

$$\begin{aligned} fg &= (a_1 + a_2 2^{-s} + a_3 3^{-s} + \cdots)(b_1 + b_2 2^{-s} + b_3 3^{-s} + \cdots) \\ &= (a_1 b_1) + (a_1 b_2 + a_2 b_1) 2^{-s} + (a_1 b_3 + a_3 b_1) 3^{-s} \\ &\quad + (a_1 b_4 + a_2 b_2 + a_4 b_1) 4^{-s} + \cdots \end{aligned}$$

一般规律是什么？在乘积 fg 中， n^{-s} 的系数是多少？它是所有满足下标乘积等于 n 的 a 和 b 的乘积之和，即它等于

$$\sum_{rs=n} a_r b_s.$$

现在如果 $rs = n$ ，那么 r 和 s 就是 n 的因数（约数），所以上面的和也可以写成

$$\sum_{dn} a_d b_{\frac{n}{d}},$$

其中符号“ dn ”读作“ d 整除 n ”（译注：现代更标准的数学符号写作 $d \mid n$ ）。我们将其正式地表述为：

法则 1" (Rule 1") 如果 $\sum_{n=1}^{\infty} a_n n^{-s}$ 且 $\sum_{n=1}^{\infty} b_n n^{-s}$ ，那么

$$\sum_{n=1}^{\infty} (f(s)g(s)) n^{-s} = \sum_{dn} a_d b_{\frac{n}{d}} n^{-s}. \quad (2.5)$$

$$\sum_{dn} a_d b_{\frac{n}{d}} n^{-s}.$$

我们要赶紧说明一下是怎样的问题会导致这种序列卷积的产生。粗略地说，这种情况通常是：大小为 n 的所有对象，是通过将 d 个大小为 n/d 的对象缝合在一起获得的，其中 d 是 n 的某个约数。在我们接触此类现象的例子之前，由于这种乘法太重要了，让我们先来看看它的一些其他性质。

如果我们求一个狄利克雷级数的 k 次方，生成的序列会发生什么变化？让我们像下面这样把它算出来：

$$\begin{aligned} f(s)^k &= \left(\sum_{n \geq 1} a_n n^{-s} \right)^k = \sum_{n_1, \dots, n_k \geq 1} a_{n_1} \cdots a_{n_k} (n_1 n_2 \cdots n_k)^{-s} \\ &= \sum_{n \geq 1} n^{-s} \left\{ \sum_{n_1 \cdots n_k = n} a_{n_1} \cdots a_{n_k} \right\}. \end{aligned}$$

这表明：

法则 2" (Rule 2") 如果 $\overset{f(s)}{D_{ir}} \{a_n\}_1$, 那么 $\overset{f(s)}{D_{ir}} (s)^k$ 生成这样一个序列, 其第 n 项是将 n 分解为 k 个因子的所有有序因数分解 (ordered factorizations) 上, 对应的以因子为下标的序列成员的乘积之和。

那么什么级数 f 能够生成所有项均为 1 的序列 $\{1\}_1$ 呢? 当我们在 opsgf 和 egf 的情况下问出这个问题时, 答案被证明是那些“著名”的函数。对于 opsgf, 它是 $1/(1-x)$, 对于 egf, 它是 x 。在目前的情况下, 系数全部为 1 的形式狄利克雷级数与微积分中的任何简单函数都没有关系; 它是一个全新的生物, 并获得了一个新名字: **黎曼 函数 (Riemann zeta function)**。它是如下的狄利克雷级数:

$$\begin{aligned} (s) &= \sum_{n=1} \frac{1}{n^s} \\ &= 1^{-s} + 2^{-s} + 3^{-s} + 4^{-s} + \dots, \end{aligned} \quad (2.0)$$

并且它是分析数学中最重要的函数之一。

现在, 既然 $\overset{(s)}{D_{ir}} \{1\}_1$, 那么 $^2(s)$ 会生成什么序列呢? 直接根据 (2.59),

$$[n^{-s}]^2(s) = \sum_{dn} 1 \cdot 1 = d(n),$$

其中 $d(n)$ 是整数 n 的约数个数。序列 $d(n)$ 非常不规则, 开头是

$$1, 2, 2, 3, 2, 4, 2, 4, 3, 4, 2, \dots$$

然而, 根据法则 2", 它的狄利克雷级数生成函数就是 $^2(s)$ 。

同样地, $(s)^k$ 生成将 n 进行有序因数分解为 k 个因子的方法数。如果认为因子 1 是不被允许的, 那么 $((s) - 1)^k$ 将生成把 n 进行有序因数分解且 k 个因子全都 ≥ 2 的方法数。

人们还可以继续研究由黎曼 函数的各种“亲戚”生成的其它有趣的数论序列的例子, 但是存在一个稍微有些令人惊叹的推广, 它能够一口气把所有这些例子都包含进去, 所以让我们为它准备一下基础。

定义 2.28 数论函数 (number-theoretic function) 是指其定义域为正整数集合的函数。如果一个数论函数 f 对于所有互质 (relatively prime) 的正整数对 m 和 n , 都具有 $f(mn) = f(m)f(n)$ 的性质, 那么我们称该函数是**乘性的 (multiplicative)**。

由于每个正整数 n 除了次序外都可以唯一地表示为不同素数的幂的乘积,

$$n = \underset{1}{a_1} \underset{2}{a_2} \cdots \underset{r}{a_r}, \quad (2.1)$$

所以可以推导出, 一个乘性数论函数完全由它在所有素数幂上的取值所决定。确实有:

$$f(n) = f(\underset{1}{a_1})f(\underset{2}{a_2}) \cdots f(\underset{r}{a_r}). \quad (2.2)$$

例如，假设我在脑海里构思了某个函数 f 。它是乘性的，并且进一步，对于每个素数 p 和正整数 m ，我们有 $f(p^m) = 2^m$ 。那么对于所有的 n ，必定有 $f(n) = n^2$ ，因为如果 n 如 (2.61) 所示，那么

$$\begin{aligned} f(n) &= f\left(\prod_i a_i\right) = \prod_i f(a_i) \\ &= \prod_i 2^{a_i} = \left\{\prod_i a_i\right\}^2 \\ &= n^2, \end{aligned}$$

正如我们所声称的那样。

另一个乘性函数的例子（稍微没那么显然）是 $d(n)$ ，即 n 的约数个数。例如，

$$= d(12) = d(3 \cdot 4) = d(3)d(4) = 2 \cdot 3 = 6.$$

为了看出 $d(n)$ 在一般情况下是乘性的，设 m 和 n 为互质的正整数。那么 mn 的每个约数 d 都可以**唯一地**表示为 m 的某个约数 d' 和 n 的某个约数 d'' 的乘积。确实，我们可以取 $d' = (d, m)$ 且 $d'' = (d, n)$ 。因此 mn 的约数个数就是 m 的约数个数与 n 的约数个数的乘积，得证。

因此，虚构出乘性函数的例子是很容易的：让你的函数 f 在素数的幂上随便取什么它喜欢的值，然后宣布它是乘性的，拍拍手走人就行了。

乘性数论函数满足一个令人惊叹的恒等式，我们将先陈述它，再证明它，然后使用它。

定理 2.29 设 f 是一个乘性数论函数。那么我们有如下形式恒等式：

$$\sum_{n=1}^{\infty} \frac{f(n)}{n^s} = \prod_p \{1 + f(p)p^{-s} + f(p^2)p^{-2s} + f(p^3)p^{-3s} + \dots\} \quad (2.3)$$

其中右侧的乘积跨越所有的素数。

证明： 想象一下（如果你愿意的话）把 (2.63) 右侧出现的乘积展开。乘积中的每一个因子都是一个无穷级数。当我们把这个乘积详细展开时，它看起来像这样：

$$\begin{aligned} &(1 + f(2)2^{-s} + f(2^2)2^{-2s} + f(2^3)2^{-3s} + \dots) \\ &(1 + f(3)3^{-s} + f(3^2)3^{-2s} + f(3^3)3^{-3s} + \dots) \\ &(1 + f(5)5^{-s} + f(5^2)5^{-2s} + f(5^3)5^{-3s} + \dots) \\ &(1 + f(7)7^{-s} + f(7^2)7^{-2s} + f(7^3)7^{-3s} + \dots) \dots \end{aligned} \quad (2.4)$$

要把一堆这样的形式无穷级数乘开，我们先伸手到第一个括号里抽出一项，比如 $f(2^3)2^{-3s}$ 。然后我们伸手到第二个括号里，抽出一项，比方说 $f(3)3^{-s}$ ，并将它乘以我们之前得到的项。这给了我们（到目前为止）的累积乘积：

$$f(2^3)f(3)2^{-3s}3^{-s} = \frac{f(2^3)f(3)}{(24)^s}. \quad (2.5)$$

比方说，作为个例子，假设在后面的所有括号中，我们都行使我们的选择权，统统抽出项“1”。那么，作为做出了所有这些选择（每个括号选出一项）的结果，对最终答案的贡献将正是上面 (2.65) 中显示的单个项。

现在有趣的部分来了。这组特定的选择产生了一个包含 $(24)^{-s}$ 的项。每个括号选出一项的其他什么组合能**同样**导致一个包含 $(24)^{-s}$ 的净贡献？答案是：**没有任何其他选择组合能做到这一点。**

确实，如果我们从第一个括号中选择除 $f(2^3)2^{-3s}$ 之外的任何一项，那么无论我们从后面的括号中抽出什么项，我们永远也无法找到在 $(24)^{-s}$ 中出现的 2 的幂，也就是 2^{-3s} 。我们需要三个 2，而除了第一个括号之外，没有任何其他括号能提供任何一个 2，所以我们最好在有机会时就拿走它们。

同样地，我们需要一个因子 3^{-s} 来完成对项 $(24)^{-s}$ 的构造。除了第二个括号，其他括号里没有 3 可选，而且在那里，为了得到正确数量的 3，也就是一个 3，我们最好还是选择我们实际选择的项 $f(3)3^{-s}$ 。

因此，(2.63) 右侧的 $(24)^{-s}$ 的系数正好就是我们所发现的，即 $f(2^3)f(3)$ 。现在由于 f 是乘性的，它就等于 $f(24)$ 。因此 $(24)^{-s}$ 的系数就是 $f(24)$ 。但这正是 (2.63) 左侧所声明的。

让我们再用一般的“ n ”代替“24”重说一遍。设 n 是某个固定的整数，令 (2.61) 为其分解为素数幂的形式。为了获得包含 n^{-s} 的项，即包含 $i^{-a_i s}$ 的项，我们**被迫**在 (2.64) 右侧的每一个括号中选择“1”项，**除非**这些括号属于实际在 n 中出现的素数 i 。在属于 i 的括号内，我们必须选择唯一的那一项：其 i 的幂次刚好等于它在 n 中实际出现的幂次，否则我们就没有机会得到 n^{-s} 。因此，我们被迫从属于 i 的括号中选出项 $f(i)^{a_i} i^{-a_i s}$ 。这意味着，到最后 n^{-s} 的系数将是

$$\prod_i f(i)^{a_i} = f(n),$$

根据 (2.62)。

让我们再看一眼 (2.63)。非常明显的一点是，**一个乘性函数完全由它在所有素数幂上的值所决定**。确实，在 (2.63) 的右侧我们只看到 f 在素数幂上的值，而在左侧，所有的值都出现了。

来试试这个定理的一个例子。取乘性函数 $f(n) = 1$ （对于所有 n ）。那么 (2.63) 说

$$\begin{aligned} (s) &= \prod \{1 + i^{-s} + i^{-2s} + \dots\} \\ &= \prod \left\{ \frac{1}{1 - i^{-s}} \right\} \\ &= \frac{1}{(1 - i^{-s})}, \end{aligned} \tag{2.}$$

这就是 zeta 函数的一个极其基础的因子分解形式（欧拉乘积公式）。

作为另一个例子，取乘性函数 $f(n)$ ，其在素数幂上的值为：

$$\binom{a}{n} = \begin{cases} +1, & \text{如果 } a = 0; \\ -1, & \text{如果 } a = 1; \\ 0, & \text{如果 } a \geq 2. \end{cases}$$

将这个函数代入 (2.63) 中的 f 后，我们可以看到花括号里曾经令人生畏的无穷级数现在只有两项了，(2.63) 变成了

$$\sum_{n \geq 1} \frac{\binom{n}{s}}{n^s} = \prod \{1 - \frac{1}{n^s}\}. \quad (2.7)$$

比较 (2.66) 和 (2.67)，浮现出一个重要的事实：级数 (s) 和 (2.67) 左侧的级数互为倒数。因此，

$$\frac{1}{(s)} = \sum_{n \geq 1} \frac{\binom{n}{s}}{n^s},$$

或者，也是一样的意思， $\frac{1}{Dir(s)} \{(n)\}_1$ 。

函数 $\binom{n}{s}$ 就是**莫比乌斯函数 (Möbius function)**，它在解析数论中扮演着核心角色，这是因为它是由黎曼 zeta 函数的倒数所生成的。例如，看这个：假设我们有两个序列 $\{a_n\}_1$ 和 $\{b_n\}_1$ ，并且假设这两个序列通过以下方程相互关联：

$$a_n = \sum_{dn} b_d \quad (n \geq 1). \quad (2.)$$

问题是，我们如何能够**反演 (invert)** 这些方程，并解出用 a 来表示的 b 呢？

小菜一碟。设这两个序列的 Dsgf 为 $A(s)$ 和 (s) 。然后，如果我们踏入生成函数的国度，我们就能看出 (2.68) 的意思是：

$$A(s) = (s)(s)$$

这利用了法则 1"。因此 $(s) = A(s)/(s)$ ，然后再次使用法则 1"，

$$b_n = \sum_{dn} \left(\frac{n}{d}\right) a_d \quad (n = 1, 2, 3, \dots). \quad (2.)$$

这就是著名的**莫比乌斯反演公式 (Möbius Inversion Formula)**。这两个序列的互逆关系 (2.68) 和 (2.69)，完美地映射了它们的 Dsgf 即 (s) 和 $1/(s)$ 的互逆关系。

例 2.30 (本原位串 / Primitive bit strings)

长度为 n 、由 0 和 1 组成的串中，有多少个是**本原的 (primitive)**？本原指的是，这样的字符串无法被表示为几个完全相同且更小的字符串拼接 (concatenation) 而成。

例如，100100100 不是本原的，但 1101 是本原的。

长度为 n 的字符串总共有 2^n 个。假设其中有 $f(n)$ 个是本原的。每一个长度为 n 的字符串都可以**唯一地**表示为某种数量（即 n/d 个）完全相同的本原字符串（长度为 d ）拼接而成，其中 d 是 n 的约数。

因此我们有：

$$2^n = \sum_{d|n} f(d) \quad (n = 1, 2, \dots).$$

由 (2.69) 的莫比乌斯反演我们得到：

$$f(n) = \sum_{d|n} \left(\frac{n}{d}\right) 2^d \quad (n = 1, 2, \dots). \quad (2.70)$$

这就是所求的数量。

例 2.31 (分圆多项式 / Cyclotomic polynomials)

在方程 $x^n = 1$ 的 n 个根中，**本原 (primitive)** n 次单位根是指那些不能同时成为某个 $m < n$ 的 m 次单位根的根。因此 4 次单位根是 $\pm 1, \pm i$ ，但 ± 1 是 $x^2 = 1$ 的根，所以它们不是本原的 4 次单位根。

一般而言， n 次单位根是 $\{e^{2\pi i r/n}\}_{r=0}^{n-1}$ ，而本原根是：

$$\{e^{2\pi i r/n}\} \quad (0 \leq r \leq n-1; (r, n) = 1).$$

所以对于每个 n ，恰好有 $\phi(n)$ 个（欧拉函数）本原 n 次单位根。

根正好为所有这 n 个 n 次单位根的方程显然是 $x^n - 1 = 0$ 。问题是：次数为 $\phi(n)$ 、且其根恰好是本原 n 次单位根集合的那个多项式 $\Phi_n(x)$ 是什么？换句话说，关于多项式

$$\Phi_n(x) = \prod_{\substack{0 \leq r \leq n-1 \\ (r, n) = 1}} (x - e^{2\pi i r/n}) \quad (n = 1, 2, 3, \dots)$$

我们能说些什么？

多项式 $\Phi_n(x)$ 被称为**分圆 (cyclotomic, "circle-cutting") 多项式**。

回答这个问题的关键事实是：

$$\prod_{d|n} \Phi_d(x) = 1 - x^n \quad (n = 1, 2, 3, \dots). \quad (2.71)$$

确实，等式右侧是所有可能的因子 $(1 - x^d)$ 的乘积，其中 随便一个 n 次单位根，不论是不是本原的。但**每一个** n 次单位根都恰好是唯一的一个 $d | n$ 的本原 d 次单位根，并且那个 d 是 n 的约数。

详细来说，如果我们有某个 n 次单位根 $\zeta = e^{2\pi i r/n}$ ，那么令 $g = (r, n)$ ， $d = n/g$ ，且 $r' = r/g$ 。因为 $\zeta = e^{2\pi i r'/d}$ ，我们看到 ζ 是一个本原的 d 次单位根，并且 $d | n$ 。因此，(2.71) 右侧的每一个线性因子，在 (2.71) 左侧的分圆多项式中，必定有且仅出现一次，这证明了上述断言。

从 (2.71) 中，我们将通过反演这个方程来解出，从而得到 ${}_n(x)$ 的一个相当显式的公式。这个方程的形式让我们想起了莫比乌斯反演公式的设置 (2.68)，只是我们这里是对约数求乘积，而不是对约数求和。然而，稍微用一点对数就能把乘积转换成求和，所以我们对 (2.71) 两边取对数，得到：

$$\sum_{dn} \ln_d(x) = \ln(1 - x^n) \quad (n = 1, 2, 3, \dots). \quad (2.72)$$

这现在精确地变成了 (2.68) 的形式，所以我们可以使用 (2.69) 来反演它，结果是：

$$\ln_n(x) = \sum_{dn} \left(\frac{n}{d}\right) \ln(1 - x^d) \quad (n = 1, 2, 3, \dots).$$

最后，我们对两边取指数，以获得我们的“相当显式的公式”：

$${}_n(x) = \prod_{dn} (1 - x^d)^{(n/d)} \quad (n = 1, 2, 3, \dots). \quad (2.73)$$

现在正是回想莫比乌斯函数的值只能是 ± 1 或 0 的好时机。所以 (2.73) 右边指数部分告诉我们的是：是否要忽略某个特定的因子（如果 $= 0$ ），将其放在分子中（如果 $= 1$ ），还是将其放在分母中（如果 $= -1$ ）。例如， ${}_{12}(x)$ 就是：

$$\begin{aligned} & (1 - x)^{(12)} (1 - x^2)^{(-1)} (1 - x^3)^{(4)} (1 - x^4)^{(-3)} (1 - x)^{(2)} (1 - x^{12})^{(-1)} \\ &= (1 - x)^0 (1 - x^2)^1 (1 - x^3)^0 (1 - x^4)^{-1} (1 - x)^{-1} (1 - x^{12})^1 \\ &= \frac{(1 - x^2)(1 - x^{12})}{(1 - x^4)(1 - x)} \\ &= 1 - x^2 + x^4, \end{aligned}$$

直到最后一步之前，它看起来一点也不像是个多项式！

关于这些多项式，一个重要且美丽的事实是：方程 ${}_n(x) = 0$ 总是可以通过**根式 (radicals)** 求解的。也就是说，解总是可以通过有限次的开根号和有理运算来获得。这对于一般的多项式方程来说肯定不是成立的。

作为这种性质的一个例子，我们注意到了这个壮丽的事实：

$$\frac{2}{17} = \frac{1}{1} \left\{ -1 + \sqrt{17} + \sqrt{2(17 - \sqrt{17})} + 2\sqrt{17 + 3\sqrt{17} - \sqrt{2(17 - \sqrt{17})} - 2\sqrt{2(17 + \sqrt{17})}} \right\}.$$

这个证明相当困难，可以在 Rademacher [Ra] 的著作中找到。

分圆多项式的一些应用将在 4.9 节中出现。

2.7 习题 (Exercises)

1. 计算以下函数幂级数的**倒数 (reciprocals, 即 $1/f(x)$)**的前三个系数：

(a) x

(b) $(1+x)^m$

(c) $1 + 2 + 3 + 5 + 7 + 11 + \dots$

2. 计算以下函数幂级数的逆级数 (inverses, 即反函数 $f^{-1}(x)$) 的前三个系数:

(a) $\ln x$

(b) $\ln x$

(c) $x + x^2\sqrt{1+x}$

(d) $x + x^3$

(e) $\ln(1-x)$

3. 设 f 是一个形式幂级数, 使得 $f + f = 0$ 。请给出严谨的证明: $f = A \ln x + x$ 。

4. 为以下序列的普通幂级数生成函数 (opsgf) 寻找简单的闭合公式:

(a) $\{n+7\}_0$

(b) $\{1\}_4$

(c) $\{1, 0, 1, 0, 1, 0, 1, 0, \dots\}$

(d) $\{1/(n+1)\}_2$

(e) $\{1/(n+5)!\}_0$

(f) $F_1, 2F_2, 3F_3, 4F_4, \dots$ (其中 F 为斐波那契数)

(g) $\{(n^2 + n + 1)/n!\}_1$

5. 使用生成函数来证明 $\sum_k \binom{n}{k} = 2^n$ 。

6. 给定正整数 n, k ; 按如下方式定义 $f(n, k)$: 对于将 n 写成恰好 k 个非负整数的有序和的每一种方式, 令 S 为这 k 个整数的乘积。然后令 $f(n, k)$ 为通过这种方式获得的所有 S 的总和。求 f 的 opsgf, 并为其求出一个显式、简单的公式。

7. 令 $f(n, k, h)$ 表示将 n 写成恰好 k 个整数的有序和的方法数, 其中每个整数都 $\geq h$ 。求 $\sum_n f(n, k, h)x^n$ 。

8. 寻找下列每个序列的上极限 (limit superior)。在每种情况下, 请给出严谨的证明以说明你的答案是正确的。

(a) $1, 0, 1, 0, 1, 0, \dots$

(b) $\{(-1)^n\}_0$

(c) $\{(n/k)\}_{n \geq 0}$ ($k \neq 0$ 为一个固定整数)

(d) $\{1 + ((-1)^n/n)\}_{n \geq 1}$

(e) $\{n^{1/n}\}_{n \geq 1}$

9. 证明: 如果一个序列存在极限, 那么它的上极限等于该极限。

10. 证明: 一个序列不可能有两个不同的上极限。

11. 寻找以下每一个幂级数的收敛半径:

(a) $\sum_{n \geq 1} x^n/(n^2)$

- (b) $1 + x^3 + x + x + x^{12} + \dots$
 (c) $1 + 5x^2 + 25x^4 + 125x + \dots$
 (d) $1 + 2!x^2 + 4!x^4 + !x + \dots$
 (e) $\sum_{n \geq 0} x^{n!}$

12. 补全定理 2.19 在 $= 0$ 和 $=$ 情况下的证明。

13. 证明：如果 $\{f(n)\}_1$ 是一个乘性函数，那么

$$g(n) = \sum_{dn} f(d) \quad (n = 1, 2, \dots)$$

也是一个乘性函数。

14. 欧拉函数 $\phi(n)$ 是满足 $1 \leq m \leq n$ 且 m 与 n 互质的整数的个数。请用直接的计数论证（直接数数的方式）来证明：

$$\sum_{dn} \phi(d) = n \quad (n = 1, 2, \dots).$$

15. 证明以下每个函数都是乘性的。在每种情况下，求出该函数在 n 是素数幂时的值，并由此推导出它在任意整数 n 上取值的公式。

- (a) 欧拉函数 $\phi(n)$ （使用上面习题 13, 14 的结果）。
 (b) $\sigma(n)$ ，表示 n 的所有约数之和。
 (c) 函数 $\mu(n)$ ，如果 n 不被任何完全平方数整除则为 1，否则为 0。

16. 对于上面习题 15 中定义的每一个函数，使用定理 2.29 求出它们的狄利克雷级数生成函数 (Dsgf)。首先将该函数在素数幂处的值代入 (2.63) 中。然后尝试对出现的幂级数求和，化为闭合形式。最后，通过将得到的结果乘积与 (2.66) 进行比较，尝试将你的答案简单地用黎曼函数表示出来。在每种情况下，Dsgf 都可以用 (s) 或其微小变形简单地表示。

17. 求以下各个序列的 Dsgf：

- (a) $\{n\}_1$
 (b) $\{n\}_1$
 (c) $\{1/n\}_1$
 (d) $\left\{\sum_{dn} d\right\}_{n=1}$

18. 对于下面的每一个恒等式：首先通过计算当 $n = 1, 2, 3, 4, 5, 7$ 时据称等式的两边，来检查该恒等式有时是正确的；接下来，寻找所声明恒等式两边序列的 Dsgf，观察它们是相同的，从而证明该恒等式。请使用上面习题 16 的结果。

- (a) $\sum_{dn} \phi(d) = n \quad (n \geq 1)$
 (b) $\sum_{dn} \mu(d) = 0$ （如果 $n \geq 2$ ）且 $= 1$ （如果 $n = 1$ ）
 (c) $\sum_n \mu(n) d(n) = 1 \quad (n \geq 1)$

19. 如果 $\{f(k)\}$ 是例 2.12 中的序列, 证明对于 $k \geq 1$, 有 $f(k) = F_{2k-1}$, 其中 $\{F_k\}$ 是斐波那契数。

20. 通过比较方程 $(x+)^n = x$ 两边 $n/n!$ 的系数, 来证明二项式定理

$$(x +)^n = \sum_k \binom{n}{k} x^{kn-k}$$

使用类似的手段证明多项式定理 (multinomial theorem)

$$(x_1 + \cdots + x_k)^n = \sum_{r_1 + \cdots + r_k = n} \frac{n!}{r_1! \cdots r_k!} x_1^{r_1} \cdots x_k^{r_k}.$$

21.

(a) 设 S 是一个固定的非负整数集合。令 $f(n, k, S)$ 为将 n 表示为从 S 中选出的 k 个整数的有序和的表示方法数。求 $\sum_n f(n, k, S) x^n$ 。

(b) 令 $g(n, k, S)$ 为将 n 表示为从 S 中选出的 k 个不同整数的有序和的表示方法数。求 $\sum_n g(n, k, S) x^n$ 。

(c) 最后, 设 S 是两个固定的非负整数集合。令 $f(n, k, S, S')$ 为将 n 表示为从 $S \cup S'$ 中选出的 k 个整数的有序和的方法数, 其中每一个选出的整数的重复次数 (multiplicity) 都属于集合 S 。求 $\sum_n f(n, k, S, S') x^n$ 。

22. 令 $f(n)$ 表示“将 n 表示为偶数个正整数之和的方法数”减去“将 n 表示为奇数个正整数之和的方法数”的差额。通过求 $\sum_n f(n) x^n$ 并读出其系数来求 $f(n)$ 。

23. 设 $\{B_n\}$ 为由 (2.40) 定义的伯努利数 (Bernoulli numbers) 序列, 并设 m 为一个正整数。通过用两种方法考虑生成函数

$$\frac{x(mx - 1)}{x - 1}$$

求出前 N 个正整数的 r 次方之和, 将其表示为关于 N 的一个次数为 $r + 1$ 的多项式, 其系数由伯努利数相当显式地给出。

24.

(a) 制作一张经典莫比乌斯函数 $\mu(n)$ 在 $n = 1, 2, \dots, 30$ 时的数值表。

(b) 制作一张公式 (2.70) 中的函数 $f(n)$ 在 $n = 1, 2, \dots, 12$ 时的数值表。

(c) 列出长度为 6 的本原字符串 (primitive strings), 并验证你对 $f()$ 计算的值。

25. 本练习旨在说明生成函数是如何出现在编码理论 (coding theory) 中的。编码理论中一个重要的问题是这样的: 对于固定的整数 n 和 d , 满足“任意两个不同码字总是在至少 d 个比特位置上不同”的、由 0 和 1 组成的 n 比特字符串 (码字/codewords) 构成的列表, 其最长长度 $A(n, d)$ 是多少?

(a) 按如下方式为 2^n 个码字 $(c_1, \dots, c_n)$ 中的每一个分配一种颜色 (color): c_i 的颜色为 $\sum_j j_j$ 模 $2n$ 的余数。证明: 如果两个码字仅在 1 个或 2 个坐标上不同, 那么在此方案中它们被分配了不同的

颜色。

(b) 由 (a) 部分, 证明 $A(n, 3) \geq 2^n / (2n)$ 。

(c) 令 a_j 为满足 $\sum_r r_r = j$ 的码字数, 对于每个 j 。将 opsgf $f() = \sum_j a_j j^j$ 显式地表达为一个乘积。

(d) 如果 r 是颜色为 r 的码字数, 那么用上面的 a_j 来表示 r 。然后使用 2.4 节中的单位根方法, 证明对于每个 $r = 0, 1, \dots, 2n - 1$, 有

$$r = \frac{1}{2n} \sum_{j=1}^n 2^{2^a(j,n) - \frac{2irj}{n}}$$

这里 a 和 n 的定义使得 $n = 2^a n$, 其中 n 是奇数, 且 (b, c) 表示 b 和 c 的最大公约数。

(e) 推导出 r_0 是所有 r 中最大的, 并由此得到更强的下界

$$A(n, 3) \geq \frac{1}{2n} \sum_{j=1}^n 2^{2^a(j,n)} \geq \frac{2^n}{2n}.$$

(f) 使用帕塞瓦尔恒等式 (Parseval's identity) 和 (d) 部分的结果, 求占有数 (occupancy numbers) $r_0, \dots, r_{2n-1}$ 的方差。做一个估计, 表明该方差在某种意义上是非常小的, 从而证明这种着色方案将码字非常均匀地分布到了各个颜色类中。

26. 从 (2.38) 推导 (2.39)。也就是说, 证明

$$\binom{-n}{k} = (-1)^k \binom{n+k-1}{k}.$$

27. 令 $D(n)$ 为例 2.18 中讨论的 n 个字母的错排 (derangements) 的数量。

(a) 用简单显式的形式, 求 $\{D(n)\}_0$ 的 egf。

(b) 用任何方法证明

$$D(n+1) = (n+1)D(n) + (-1)^{n+1} \quad (n \geq 0; D(0) = 1).$$

(c) 用任何方法证明

$$D(n+1) = n(D(n) + D(n-1)) \quad (n \geq 1; D(0) = 1; D(1) = 0).$$

(c) (注: 原书有两道(c)题, 此处保留编号) 证明: 恰好有一个不动点的 n 字母排列数, 与没有不动点的排列数, 相差 ± 1 。

(d) 令 $D_k(n)$ 为恰好有 k 个不动点的 n 字母排列数。证明

$$\sum_{k,n \geq 0} D_k(n) \frac{x^{nk}}{n!} = \frac{-x(1-x)}{1-x}.$$

28. 证明莫比乌斯反演公式的以下变体。设 $\{a_n(x)\}$ 和 $\{b_n(x)\}$ 是两个函数序列, 它们通过关系式连接:

$$a_n(x) = \sum_{d|n} b_{\frac{n}{d}}(x^d) \quad (n = 1, 2, 3, \dots).$$

那么我们有

$$b_n(x) = \sum_{d|n} \left(\frac{n}{d}\right) a_d (x^{n/d}) \quad (n = 1, 2, 3, \dots).$$

29.

- (a) 制作一张 (n) 在 $1 \leq n \leq 25$ 时的数值表。
- (b) 在你的表格范围内, 通过实际计算来验证 (n) 是一个乘性函数。然后再根据你的表格, 通过实际计算来检查, 当 $n = 20$ 和 $n = 24$ 时, 上面习题 14 中陈述的结果是正确的。
- (c) 令 $n = a^k$, 其中 a 为素数。 (n) 是什么?
- (d) 使用上面的结果找到一个关于 (n) 的一般公式, 用 n 的素数分解式 $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ 来表达。用你的结果计算 (252) 。
- (e) 利用 (2.63) 寻找 (n) 的狄利克雷级数生成函数, 并将其用黎曼 ζ 函数表达出来。
- (f) 将莫比乌斯反演公式应用于习题 18(a) 的结果中, 从而“解出” 18(a) 中的 (n) , 得到一个包含莫比乌斯函数各种值的总和的 (n) 的显式公式。

30.

- (a) 求序列 $\{\sqrt{n}\}_{n=1}^{\infty}$ 的 Dsgf。
- (b) 求序列 $\{ |(n)| \}_{n=1}^{\infty}$ 的 Dsgf, 其中 (n) 是莫比乌斯函数。
- (c) 如果一个数论函数 $f(n)$ 对于所有的正整数对 m, n 都满足 $f(mn) = f(m)f(n)$, 那么我们称它是**强乘性的 (strongly multiplicative)**。令 (n) 为在每个素数上都取值 -1 且 $(1) = 1$ 的强乘性函数。求它的 Dsgf, 然后证明

$$\sum_{d|n} (d) = \begin{cases} 1 & \text{如果 } n \text{ 是一个完全平方数;} \\ 0 & \text{其他.} \end{cases}$$

31. 兰伯特级数 (Lambert series) 是一种形如下式的级数:

$$f(x) = \sum_{n \geq 1} a_n \frac{x^n}{1 - x^n},$$

此时我们就说 f 是序列 $\{a_n\}$ 的兰伯特级数生成函数。(兰伯特级数很少被使用, 因为它们很难进行解析。)

- (a) 假设 f 是序列 $\{a_n\}_{n=1}^{\infty}$ 的兰伯特级数生成函数, 同时相同的 f 又是序列 $\{b_n\}_{n=1}^{\infty}$ 的 opsgf (普通幂级数生成函数)。请用 a 来表示 b 。
- (b) 由此证明下面这个惊人的恒等式:

$$\sum_{n \geq 1} \frac{(n)x^n}{1 - x^n} = x,$$

其中 (n) 又是莫比乌斯函数。

- (c) 寻找欧拉 ϕ 函数的兰伯特级数生成函数。

32. 设 $\{a_n\}_{n \geq 0}$ 是一个给定的序列。令 S 为将 $\{a_n\}$ 转换为其前缀和序列的操作符: 对于 $n \geq 0$, $(S)_n = a_0 + \cdots + a_n$ 。

- (a) 如果 f 是 的 opsgf, 那么 S 的 opsgf 是什么?
- (b) 如果 f 是 的 opsgf, 且 $r \geq 0$, 那么 S^r 的 opsgf 是什么?
- (c) 如果 是全为 1 的序列, 那么 S^r 是什么?
- (d) 对于一般的序列, 寻找一个包含单个求和号的显式公式, 用来表示序列 S^r 的第 n 项。
- (e) 一个未知的序列 具有以下性质: 如果从 开始, 我们迭代 r 次操作 S (即每次将序列替换为其前缀和序列), 我们获得了序列 $\{1, 0, 0, \dots\}$ 。求。

33.

- (a) 写出前十二个分圆多项式 (cyclotomic polynomials)。
- (b) 如果 $n = a$ 是一个素数幂, ${}_n(x)$ 是什么?
- (c) 证明对于 $n \geq 1$,

$${}_n(1) = \begin{cases} 1, & \text{如果 } n > 1 \text{ 不是素数幂;} \\ , & \text{如果 } n = k \text{ 是一个素数幂;} \\ 0, & \text{如果 } n = 1. \end{cases}$$

34. 考虑下面的多项式序列:

$${}_n(x) = \sum_{\substack{1 \leq m \leq n \\ (m,n)=1}} x^m \quad (n = 1, 2, \dots).$$

因此 ${}_1 = x, {}_2 = x, {}_3 = x + x^2, {}_4 = x + x^3$, 等等。

(a) 证明

$$\sum_{dn} \frac{n}{d} (x^d) = \frac{x(1-x^n)}{1-x} \quad (n = 1, 2, \dots).$$

(b) 使用习题 28 的结果来证明

$${}_n(x) = (1-x^n) \sum_{dn} (d) \frac{x^d}{1-x^d} \quad (n = 1, 2, \dots).$$

(c) 证明: 在每一个本原的 n 次单位根 处, 我们都有 ${}_n() = (n)$, 因此多项式 ${}_n(x) - (n)$ 能被第 n 个分圆多项式 ${}_n(x)$ 整除。

3 卡牌、牌叠与手牌: 指数公式

(Cards, Decks, and Hands: The Exponential Formula)

3.1 简介 (Introduction)

在这一章中，我们将讨论生成函数理论在计数问题中一条极其丰富的应用脉络。我们在此的主要目标——**指数公式 (exponential formula)**，是计数艺术的一块基石。它处理的是计算那些**由连通的 (connected) 部件构建而成的结构**的数量问题。结构本身不需要是连通的，但它的部件始终是连通的。问题在于，如果我们知道每种尺寸的部件有多少个，我们能用这些部件构建出每种尺寸的结构各有多少个？

我们先从一个小例子开始。有 1 个包含 1 个顶点的连通标号图 (connected labeled graph)，有 1 个包含 2 个顶点的连通标号图，并且有 4 个包含 3 个顶点的连通标号图。这些小家伙都在下面的图 3.1 中展示出来了。

(此处图 3.1：展示了 3 个顶点的六种连通标号图)

现在想想那些包含恰好三个标号顶点、但**不一定**连通的图。共有 8 个这样的图，如图 3.2 所示。

(此处图 3.2：展示了三个顶点的八种不一定连通的标号图)

问题是，我们如何发展出一套理论，向我们展示数字 8（即 3 个顶点的**所有图**的数量）与数字 1, 1, 4（分别是一、二、三个顶点的**连通标号图**的数量）之间的联系？毕竟，这样的一套理论应该是存在的，因为连通图正是构建所有图的“积木 (building blocks)”。那么，这些积木究竟是如何被使用的呢？

假设我们想要构造一个包含 n 个顶点和 k 个连通分量的图 G 。我们可以首先选择要使用哪 k 个连通图，唯一的限制条件是它们的顶点数总和必须等于 n 。

其次，在决定了要使用哪些连通图之后，我们需要对它们所有的顶点**重新标号 (relabel)**。这是因为我们使用的连通图（如图 3.1 中的那些），每一个都有它们自己私有的顶点标签集。一个包含 5 个顶点的连通图将在其顶点上有标签 1, 2, 3, 4, 5 等等。然而，我们正在用这些连通部件制造的、最终组装好的图 G ，将使得每个顶点标签 $1, 2, 3, \dots, n$ 恰好被使用一次，如图 3.2 所示。请注意，例如，图 3.2 的第一幅图（包含三个孤立顶点）中，包含 1 个顶点的连通图就出现了三次，每次带有不同的顶点标签。

因此，我们的计数理论将必须考虑到：用作积木的连通图的**选择 (choices)**，以及为了获得最终产品而对这些连通图的顶点进行**重新标号 (relabel)** 的方法数。

现在我们要加大筹码了。相比于继续直接回答图论情况下的这些计数问题，从一开始就变得更具一般性会被证明是更好的选择。这是因为有许多漂亮的应用不能完全归入图论的范畴。所以，我们将在“**打牌 (playing cards)**”和“**手牌 (hands)**”的背景下发展这套理论，而不是用“连通图”和“所有图”。

稍后你会看到一堆基本术语的定义。在所有这些定义之后，毫无疑问你会非常欢迎几个例子的出现，并且这些例子很快就会到来。然后我们将继续发展这套理论及其众多的应用。

3.2 定义与一个问题 (Definitions and a Question)

我们假设给定了一个抽象的“图片 (pictures)”集合 P 。

定义 3.1 一张卡牌 (**card**) $(S, \cdot)$ 是一个数对，由一个正整数组成的有限集合 S （称为“标签集/label set”）和一张图片 P 组成。卡牌的**权重 (weight)** 是 $n = |S|$ 。如果一张权重为 n 的卡牌的标签集恰好是 $[n]_1$ ，那么它被称为**标准卡牌 (standard)**。

1 回顾一下， $[n]$ 是集合 $\{1, 2, \dots, n\}$ 。

定义 3.2 一手手牌 (**hand**) 是一组卡牌的集合，这些卡牌的标签集构成某个集合 $[n]$ 的一个**划分 (partition)**。

这意味着，如果 n 表示手牌中各卡牌权重的总和，那么 $[n]$ 中各卡牌的标签集是两两不相交的、非空的，并且它们的并集是 $[n]$ 。

定义 3.3 一手手牌的**权重 (weight of a hand)** 是该手牌中所有卡牌权重的总和。

定义 3.4 对卡牌 $(S, \cdot)$ 使用集合 S' 进行的**重新标号 (relabeling)**，在 $|S| = |S'|$ 时是有定义的，其结果是卡牌 $(S', \cdot)$ 。如果 $S' = [n]$ ，那么我们就得到了该卡牌的**标准重新标号 (standard relabeling)**。

定义 3.5 一副牌叠 (**deck**) 是由一组**标准卡牌**组成的有限集合，它们的权重全部相同，且图片互不相同。牌叠的**权重 (weight of the deck)** 即为其内部所有卡牌的共同权重。

定义 3.6 一个**指数族 (exponential family)** 是一组牌叠 $d_1, d_2, \dots$ 的集合，其中对于每个 $n = 1, 2, \dots$ ，牌叠 d_n 的权重是 n 。

如果 $\{d_n\}_{n \geq 1}$ 是一个指数族，我们将写下 d_n 表示牌叠 d_n 中的卡牌数量，并将序列 $\{d_n\}_{n \geq 1}$ 的 egf（指数生成函数） (x) 称为该族的**牌叠枚举器 (deck enumerator)**。

问题 3.7 给定一个指数族。对于每个 $n \geq 0$ 和 $k \geq 1$ ，令 $h(n, k)$ 表示权重为 n 、由恰好 k 张卡牌组成的**手牌** 的数量，且满足：手牌中的每一张卡牌，都是 $[n]$ 中某个牌叠里某张卡牌的重新标号 (relabeling)。**允许重复 (Repetitions are allowed)**。也就是说，我们允许从同一个牌叠中拿出同一张卡牌的多个副本，并用不同的标签集对这些副本进行重新标号。

我们如何用 $d_1, d_2, d_3, \dots$ （其中 d_i 是牌叠 d_i 中不同卡牌的数量，对于 $i \geq 1$ ）来表示 $h(n, k)$ 呢？

如果 $h(n, k)$ 是权重为 n 且恰好有 k 张卡牌的手牌 的数量，那么我们引入双变量生成函数 (2-variable generating function)：

$$(x, y) = \sum_{n, k \geq 0} h(n, k) \frac{x^n y^k}{n!}. \quad (3.1)$$

这是一个混合类型的生成函数；对于变量 y ，它是普通幂级数 (opsgf)；对于 x ，它是指数生成函数 (egf)。我们将称其为该族的**双变量手牌枚举器 (2-variable hand enumerator)**。

如果 $h(n) = \sum_k h(n, k)$ 是权重为 n 、但不计卡牌数量的**所有手牌**的数量，那么我们用 (x) 来表示 $\{h(n)\}$ 的 egf，而不是 $(x, 1)$ 。它是的**单变量手牌枚举器 (1-variable hand enumerator)**。

回答上面提出的问题的一种方式，就是展示生成函数 $(x,)$ 和 (x) 之间的一个简单的关系式。而这，当然也就是我们接下来立刻要去做的事情（如果你想提前看一眼答案，请看下面的 (3.5) 式）。

3.3 指数族的例子 (Examples of Exponential Families)

在我们着手回答上一节提出的问题之前，这里有几个指数族的例子，它们在组合理论中扮演着重要的角色。

例 3.8 我们将描述的第一个指数族是**所有带有顶点标号的无向图**的族。我们将这个族称为 $\mathcal{I}_1$ 。

一个图 G 是一个顶点集合，其中一些顶点对被指定为边 (edges)。一个**标号图 (labeled graph)** 是一个图，其每个顶点都关联着一个正整数。所有的整数（“标签”）互不相同。如果其顶点标签的集合是 $[n]$ ，其中 n 是 G 的顶点数，则该图具有**标准标号 (standard labeling)**。

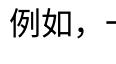
在具有 n 个顶点的图中，存在 $\binom{n}{2}$ 条可能的边，因此存在 $2^{\binom{n}{2}}$ 种带有 n 个顶点的标号图。例如，有 8 种带有三个顶点的标号图，它们显示在图 3.2 中（画图的方式是：首先画出 n 个顶点，然后，对于每一对被指定为边的顶点，画一条线连接它们）。

有些图是**连通的 (connected)**，有些则是**不连通的 (disconnected)**。如果给定任意一对顶点，我们都可以沿着图的画法中的边从一个顶点走到另一个顶点，那么这个图就是连通的。否则，这个图就是不连通的。在图 3.2 所示的 8 个 3 顶点图中，有 4 个是连通的，也就是那里画出来的最后四个图。

现在让我们来描述我们的指数族。

首先，我们描述一张卡牌 $(S,)$ 。**每一个连通标号图 G 对应一张卡牌**。集合 S 是图中所使用的顶点标签的集合。

在描述卡牌上的“图片”之前，我们需要说明什么是一个图的标准重新标号 (standard relabeling)。设 G 为一个有 n 个顶点的图，用标签集 S 标号。然后使用 $[n]$ 重新标号这些顶点，**保持标签的顺序**。也就是说， S 中具有最小标签的顶点将获得标签 1，等等。因此，标准重新标号是唯一确定的。

现在，如果 G 是一个连通的标号图，卡牌 $(S,)$ 上对应的图片 就是 G 的**标准重新标号**。因此，在一张卡牌 上，我们看到两样东西：一幅带有标准标签的连通图  的图片，以及另一个具有相等基数的标签集。

例如，一张权重为 3 的卡牌可能是：

$$(S,) = (\{5, , 11\}, \quad \begin{array}{c} 1 \text{---} 3 \text{---} 2 \end{array})$$

这对应于如下的连通标号图：

所以，卡牌对应的是**不一定具有标准标签集的连通图**。

那么**手牌 (hand)** 是什么呢？一手手牌就是一组卡牌，其标签集划分了 $[n]$ ，其中 n 是手牌的权重，也就是说，它是手牌中所有卡牌上所有连通图的顶点总数。但这非常有用；**一手手牌 对应于一个带有标准标签的不一定连通的图！** 它的各个连通分量可能带有非标准的标签，但整个图本身精确地使用了标签 $1, 2, \dots, n$ ，其中 n 是它的顶点数量。

总结来说，所有带有顶点标号的图的集合构成了一个指数族。每张卡牌是一个标号连通图，每个牌叠 $_n$ 是所有 n 个顶点的**连通标准标号图**的集合，每一手手牌是一个**标准的（不一定连通的）标号图**。第 n 个牌叠中的卡牌数量 d_n 是 n 个顶点的标准连通标号图的数量，而权重为 n 且包含 k 张卡牌的手牌数量 $h(n, k)$ ，是具有 n 个顶点和 k 个连通分量的标准标号图的数量。在上一节末尾提出的问题，在这种情况下，询问的是**所有大小的全部标号图的数量与所有连通标号图的数量之间的关系**。

例 3.9 在这个例子中，我们将发现：所有的排列 (permutations) 也可以被认为是一个指数族。

首先，让我们说说卡牌是什么。在一张卡牌上，图片将显示呈圆形排列的 n 个点，这些点以某种顺序用集合 $[n]$ 标号，并且圆圈周围会有指向顺时针方向的箭头，告诉我们这些点是以顺时针圆形序列排列的。

这便是卡牌的“图片”部分。此外，卡牌上还有一个包含 n 个正整数的集合 S 。

读者会认出这样一张卡牌对应于 S 元素的**循环排列 (cyclic permutation)**，也就是说，它对应于 S 上一个**仅包含单个循环 (single cycle)** 的排列。例如，图 3.3 中展示了一张卡牌的图片 (此处图 3.3：卡牌中展示的一个循环排列的示意图，由数字 1, 2, 3, 4, 5 按顺时针排列构成)

如果它的集合是 $S = \{2, 4, 7, 10\}$ ，这代表了集合 S 的如下循环排列：

$$2 \ 7 \ 4 \ 10 \ 2$$

那么这些卡牌的**牌叠 (deck)** 是什么呢？牌叠里的卡牌是标准卡牌，它们由给定权重的每一张不同的标准卡牌的一个样本组成。在这个例子中，第 n 个牌叠 $_n$ 恰好包含 $(n-1)!$ 张卡牌，每张卡牌对应 $[n]$ 的一个循环排列。

到目前为止，我们已经解释了只包含一个循环的排列。它们是构造所有排列的积木，而构造的方式就是使用卡牌的“手牌”。

那么在这个例子中，什么是**手牌 (hand)** 呢？一手手牌是一组卡牌，它们的标签集划分了 $[n]$ ，对于某个 n 。因此，在我们手中有一组卡牌，假设其标签集为 $S_1, \dots, S_k$ 。这些代表了 S 的各个循环排列，并且这些 S 彼此不相交，它们的并集是 $[n]$ ，其中 n 是所有卡牌上点的总数。下面展示了一手典型的手牌：

(此处示意图：展示了一手手牌，三张卡牌分别带有标签集 $\{1, 5, 6\}$, $\{2, 7\}$, $\{3, 4\}$ ，组成了集合 $[7]$ 的一个排列)

由于 n 个字母的每一个排列都有一个唯一的、分解为循环 (cycles) 的表示方式，我们看到**权重为 n 的手牌精确地对应于 n 个字母的排列。**

因此，所有排列的集合是一个指数族。我们称其为 $\mathcal{S}$ 。

牌叠 $\mathcal{S}_n$ 中有多少张卡牌？有 $d_n = (n-1)!$ 张。上一节末尾提出的问题，在这里询问的是，由 k 张卡牌组成的、权重为 n 的手牌数量 $h(n, k)$ 。由于这样的一手手牌代表了一个具有 k 个循环的 n 个字母的排列，所以在这种情况下， $h(n, k)$ 就是具有 k 个循环的 n 字母排列的数量。当我们得出一般的定理（给出 d_n 和 $h(n, k)$ 之间关系的定理）之后，我们将学到很多关于各种带有特定循环数量和大小的排列的知识。稍后，在第 5 章中，我们将回到这个主题，并重新使用这些生成函数，来获取有关排列及其循环的渐近信息。

3.4 主要的计数定理 (The Main Counting Theorems)

在本节中，我们将陈述并证明指数公式的各种形式。在下一节中，将包含该方法的 10（夸张的修辞，意为“无数”）个应用。

首先，假设给定了两个指数族 (exponential families)。我们将说明**合并 (merge)** 它们意味着什么。粗略地说，它意味着形成一个新的指数族，其各个权重的牌叠 (decks) 是两个给定指数族中对应权重的牌叠的并集。然而，为了确保合并后的牌叠拥有全部互不相同的卡牌，必须小心一点，因此我们现在给出一个精确的定义。

设 $\mathcal{S}'$ 和 $\mathcal{S}$ 是两个图片集 P' 和 P 不相交的指数族。我们通过如下方式构造第三个指数族 $\mathcal{S} = \mathcal{S}' \cup \mathcal{S}$ ，并写成 $\mathcal{S} = \mathcal{S}' \cup \mathcal{S}$ ：固定 $n \geq 1$ 。从 $\mathcal{S}'$ 中取出 d'_n 牌叠里的所有 d'_n 张卡牌，把它们放进一个新牌堆里。然后从 $\mathcal{S}$ 中取出 d_n 牌叠里的所有 d_n 张卡牌，把这 d_n 张卡牌也加到牌堆里，现在这个牌堆就包含了 $d_n = d'_n + d_n$ 张不同的卡牌。对每个 $n \geq 1$ 重复这个过程。

引理 3.10 (标号计数的基本引理 / The fundamental lemma of labeled counting)

设 $\mathcal{S}'$ 和 $\mathcal{S}$ 是两个指数族，并设 $\mathcal{S} = \mathcal{S}' \cup \mathcal{S}$ 为它们的合并。进一步，设 $'(x,)$ ， $(x,)$ ， $(x,)$ 分别为这些族的双变量手牌枚举器 (2-variable hand enumerators)。那么

$$(x,) = '(x,)(x,).$$

证明： 考虑合并后的族 $\mathcal{S}$ 中的一手手牌 σ 。它的一些卡牌来自 $\mathcal{S}'$ ，另一些来自 $\mathcal{S}$ 。那些来自 $\mathcal{S}'$ 的卡牌集合构成了一手子手牌 (sub-hand) σ' ，假设其权重为 n' ，包含 k' 张卡牌，并且这手子手牌使用某个标签集合 $S' [n']$ 以保持顺序 (order-preserving) 的方式进行了重新标号。

合并后的族中的所有手牌 σ ，都是由以下三个因素唯一决定的：来自 $\mathcal{S}'$ 的特定的子手牌 σ' ；用来对这手牌进行重新标号的新标签集 S 的选择；以及来自 $\mathcal{S}$ 的剩余子手牌 σ'' （它必须用 $[n] - S'$ 进行重新标号，同样要保持标签的顺序）。

因此，在合并后的族中，权重为 n 且恰好包含 k 张卡牌的手牌数量是：

$$\begin{aligned}
h(n, k) &= \sum_{n', k'} \binom{n}{n'} h'(n', k') h(n - n', k - k') \\
&= \left[\frac{x^n}{n!} \right]_k (x,) (x,),
\end{aligned} \tag{3.2}$$

证明完毕。

核心思想是：**合并指数族的过程与指数生成函数（egf）相乘的过程是完全对应的。**在公式 (3.2) 的求和中，变量 n' 带出了一个二项式系数 $\binom{n}{n'}$ ，而 k' 则没有；这解释了为什么我们选择的生成函数具有混合性质（mixed nature）：带有变量“ x ”的部分像是指形的（egf-like），带有变量“”的部分像是普形的（ops-like）。

这个基本引理将允许我们非常容易地，以一种“魔法师学徒（Sorcerer's Apprentice）”式的方法建立起牌叠枚举器和手牌枚举器之间的一般关系。我们将从“涓涓细流”开始，最终迎来“洪水”。我们从一个极其简单、赤裸裸的指数族开始，该族仅包含一个非空牌叠，且该牌叠里只有一张卡牌。它的手牌枚举器是显而易见的。然后，我们考虑一个在一个牌叠里有多张卡牌，但在其他牌叠里没有卡牌的族。最后，我们跳跃到最一般的情况，在此过程中的每一步我们都会使用基本引理，因为我们执行的实际上就是合并操作。

第一步：涓涓细流 (The Trickle).

固定一个正整数 r 。令第 r 个牌叠 r 恰好包含一张卡牌，并令所有其他的牌叠为空。那么牌叠计数为 $d_r = 1$ ，且所有其他的 $d_j = 0$ 。牌叠枚举器为 $(x) = x^r/r!$ 。一手手牌是由这唯一存在的卡牌的某个数量的副本（比方说 s 个）组成的。的权重将是 rs 。因此在这个指数族中，包含 k 张卡牌且权重为 n 的手牌数量 $h(n, k)$ ，除非 $n = kr$ 否则为 0。如果 $n = kr$ ，那么权重为 n 的手牌有多少手呢？我们可以有 $\binom{n}{r}$ 种方法为第一张卡牌选择标签， $\binom{n-r}{r}$ 种方法为第二张选择标签，等等，为第 k 张卡牌选择标签的方法有 $\binom{n-(k-1)r}{r} = 1$ 种。由于带有标签的卡牌之间的顺序是无关紧要的，所以手牌的数量因此是：

$$h(kr, k) = \frac{1}{k!} \frac{n!}{r!^k}.$$

这个基本指数族的手牌枚举器因此是

$$\begin{aligned}
(x,) &= \sum_{n, k} h(n, k) x^{nk} / n! \\
&= \sum_k \frac{x^{krk}}{k! r!^k} \\
&= \left\{ \frac{x^r}{r!} \right\}.
\end{aligned} \tag{3.3}$$

为了获得一般性的结果，我们不需要再做任何计算了；基本引理会替我们完成。

第二步：溪流 (The Flow).

固定正整数 r 和 d_r ，考虑一个指数族，它的第 r 个牌叠 r 中有 d_r 张卡牌，并且没有其他的非空牌叠。我们断言该族的手牌枚举器是

$$(x,) = \left\{ \frac{d_r x^r}{r!} \right\}. \quad (3.4)$$

证明过程是对 d_r 进行归纳。当 $d_r = 1$ 时该断言正确，因为那就是 (3.3)。假设该断言对于 $d_r = 1, 2, \dots, m-1$ 是正确的，并设指数族 的第 r 个牌叠里有 m 张卡牌。那么 可以看作是：一个第 r 牌叠里有 $m-1$ 张卡牌的族，与一个该牌叠里有 1 张卡牌的族**合并 (merging)** 的结果。根据归纳假设和基本引理，手牌枚举器就是乘积：

$$\{(m-1)x^r/r!\} \{x^r/r!\} = \{mx^r/r!\},$$

断言得证。

第三步：洪水 (The Flood).

我们现在准备好证明主要的计数定理了。

定理 3.11 (指数公式 / The exponential formula) 设 为一个指数族，其牌叠和手牌的枚举器分别为 (x) 和 $(x,)$ 。那么

$$(x,) = (x). \quad (3.5)$$

详细地说，权重为 n 且包含 k 张卡牌的手牌数量是

$$h(n, k) = \left[\frac{x^n}{n!} \right] \left\{ \frac{(x)^k}{k!} \right\}. \quad (3.)$$

证明： 在 (3.4) 中，我们已经在这个族只有一个非空牌叠的特殊情况下证明了这个结果。但是，一个拥有完整非空牌叠序列 $1, 2, \dots$ 的一般指数族，正是特殊族 r ($r = 1, 2, \dots$) 的**合并 (merger)**，其中每个 r 都只有一个单一的非空牌叠 r 。根据基本引理，一般族的手牌枚举器就是这些特殊族的手牌枚举器的乘积。而定理中所宣称的生成函数 (3.5)，确实就是特殊族 r 的枚举器 (3.4) 乘积的形式，证明完毕。

通过将 (3.6) 式对所有的 k 求和，我们得出以下结论：

推论 3.12 设 为一个指数族，令 (x) 为牌叠大小序列 $\{d_n\}_1$ 的 egf，并令 $\overset{gf}{(x)} \{h_n\}_0$ ，其中 h_n 是权重为 n 的所有手牌数量。那么

$$(x) = \overset{gf}{(x)}. \quad (3.7)$$

如果我们仅仅对位于某个给定集合 中的那些 k 对 (3.6) 式求和，我们获得：

推论 3.13 (限制卡牌数量的指数公式) 设 为一个正整数集合，令 $(x) = \sum_n x^n/n!$ ，并令 $h_n()$ 为权重为 n 且所含卡牌数量属于允许集合 的手牌数量。那么

$$\overset{gf}{(x)} \{h_n()\}_0 = ((x)). \quad (3.)$$

接下来的几个小节，将包含指数公式的具体应用。

3.5 排列及其循环 (Permutations and Their Cycles)

我们将定理应用于例 3.9 中描述的关于排列 (permutations) 的指数族₂。在那里我们观察到牌叠_n 包含 $d_n = (n-1)!$ 张卡牌。序列 $\{(n-1)!\}_1$ 的指数生成函数是

$$\begin{aligned}(x) &= \sum_{n \geq 1} (n-1)! \frac{x^n}{n!} \\ &= \sum_{n \geq 1} \frac{x^n}{n} \\ &= 1 \frac{1}{1-x}.\end{aligned}$$

现在根据定理 3.11 我们有：

$$\begin{aligned}(x,) &= \left\{ 1 \frac{1}{1-x} \right\} \\ &= \frac{1}{(1-x)}.\end{aligned}\tag{3.}$$

在这个指数族中， $h(n, k)$ 是包含 k 个循环的 n 字母排列的数量，它被称为**第一类斯特林数 (Stirling number of the first kind)**。我们将使用一种标准符号， $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ ，来表示这些数字²，而将 $h(n, k)$ 保留给一般的情况。

2 译注：关于 $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ ，正如有很多关于组合数学的书一样，符号也多得不可胜数。它被叫作 $(-1)^k s(n, k)$ 或 $s_1(n, k)$ ，或者 $s(n, k)$ ，或者 $c(n, k)$ ，或其它诸如此类的名字。同样地， $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 被叫作 $s_2(n, k)$ 或 $S(n, k)$ 等。Wilf 书中默认采用中括号。

现在，

$$\begin{aligned}\sum_k \left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] x^k &= \left[\frac{x^n}{n!} \right] (1-x)^{-} \\ &= n! \binom{n-1}{n} \quad (\text{根据(2.3)}) \\ &= (+1) \cdots (+n-1),\end{aligned}\tag{3.10}$$

因此，具有各种循环数量的 n 字母排列的数量，正好是“上升阶乘 (rising factorial)”函数 $(+1) \cdots (+n-1)$ 展开式中的系数。

包含 k 张卡牌的手牌枚举器显然是

$$\frac{1}{k!} \left\{ 1 \frac{1}{1-x} \right\}^k \quad (k = 1, 2, \dots),$$

这就告诉我们，第一类斯特林数也可以由下式给出：

$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} x^n \end{bmatrix} \frac{1}{k!} \left\{ 1 \frac{1}{1-x} \right\}^k. \quad (3.11)$$

有一件事我们没有发现，那就是关于这些斯特林数的一个简单明了的小公式。我们确实可以为它们找到公式，但那些公式会相当讨厌，涉及到带正负号交替的双重求和等。但是，有了生成函数这套装置，**我们即使没有那样一个显式公式，也几乎能做到我们想做的任何事**。例如，为了计算 $\begin{bmatrix} n \\ k \end{bmatrix}$ 的数值，我们完全可以使用那些非常简单的、可从这些生成函数推导出来的递推关系（参见习题 8）。

3.6 集合的划分 (Set Partitions)

我们引入一个新的指数族 $\mathfrak{S}$ ，如下所述：首先，对于每个 $n \geq 1$ ，在牌叠 $\mathfrak{S}_n$ 中**只有一张**权重为 n 的卡牌。在那张卡牌上有一张“微笑的兔子”的图片³，并且带有标签集合 $[n]$ 。

3 为什么不呢？既然只有一张卡牌，图片是什么并不重要，所以不妨让它显得高兴点。

什么是手牌呢？这里的手牌 **对应于集合 $[n]$ 的每一个划分 (partition)**。确实，给定这样一个划分，取其中的每一个子集，让它们去对这手牌中各张卡牌的标签集重新标号 (relabel)。由于每个权重只有一张卡牌，这些卡牌除此之外就被唯一决定了。

因此在这个指数族中，权重为 n 且恰好有 k 张卡牌的手牌数量，**等于将集合 $[n]$ 划分为 k 个类的方法数**。但这是我们之前遇到过的，在 1.6 节中，我们将那些数字称为 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ ，即**第二类斯特林数 (Stirling numbers of the second kind)**。

为了应用指数公式，我们首先计算每个牌叠中卡牌数量 d_n 的 egf。但是如果 $n \geq 1$ 这些数字全部是 1，否则为 0，所以

$$(x) = \sum_n d_n \frac{x^n}{n!} = \sum_{n \geq 1} \frac{x^n}{n!} = x - 1.$$

现在，根据指数公式，手牌枚举器是：

$$(x,) = (x-1), \quad (3.12)$$

特别地，

$$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = \begin{bmatrix} x^n \end{bmatrix} \left\{ \frac{(x-1)^k}{k!} \right\}. \quad (3.13)$$

将这个结果与贝尔数的生成函数 (1.43) 进行比较，我们发现：我们在那里的那个生成函数的基础上得到了一种“细化 (refinement)”。 $x-1$ 不仅生成了 n -集合的划分数（即贝尔数），而且该展开式

的每一项：

$$x_{-1} = \sum_{k \geq 0} \frac{(x-1)^k}{k!}$$

对于划分中的类 (classes) 的数量 k 都具有清晰的统计意义。

3.7 排列的一个子类 (A Subclass of Permutations)

问题： 在 n 个字母的排列 中，有多少个排列满足这样的性质： 拥有偶数个循环，并且这些循环的长度全部为奇数？

这个问题发生在一个类似于排列的指数族₂的指数族里，区别在于它仅仅包含**奇数权重**的牌叠，即 $1, 3, \dots$ 。那些计算牌叠中卡牌数量的数字 $\{d_n\}_1$ 现在变成了 $1, 0, 2, 0, 24, 0, 720, \dots$ 。牌叠计数的 egf 是

$$\begin{aligned} (x) &= \sum_{n \text{ 是奇数}} (n-1)! \frac{x^n}{n!} \\ &= \sum_{r \geq 0} \frac{x^{2r+1}}{2r+1} \\ &= 1 \sqrt{\frac{1+x}{1-x}} \end{aligned}$$

(利用了公式 2.34)。

由于循环的数量被要求是偶数，所以在一手牌中**允许的卡牌数量集合** = 偶数集合。根据 (3.8) 式，答案的 egf 是

$$\begin{aligned} \left\{ 1 \sqrt{\frac{1+x}{1-x}} \right\} &= \frac{1}{\sqrt{1-x^2}} \\ &= \sum_{m \geq 0} \binom{2m}{m} \frac{x^{2m}}{2^{2m}}. \end{aligned}$$

满足问题中条件的排列数量是此展开式中 $x^n/n!$ 的系数，也就是：

$$\binom{n}{n/2} \frac{n!}{2^n}.$$

这是回答该问题的一种方式，但这个答案可以用一种更加引人入胜的形式重新陈述，就像下面这样：

定理 3.14 固定一个正整数 n 。以下两个事件的概率是相等的：

- (a) 从 n 字母排列中随机选择一个排列，它拥有偶数个循环，且所有循环的长度都是奇数。
- (b) 将一枚硬币抛掷 n 次，恰好出现 $n/2$ 次正面。

3.8 对合, 等等 (Involutions, etc.)

固定正整数 m, n 。有多少个 n 字母的排列 满足 $m = 1$, 其中“1”是恒等排列 (identity permutation) ?

要解决这个问题, 我们需要以下引理:

引理 3.15 排列 $m = 1$ 的充要条件是: 的所有循环长度 (cycle lengths) 都是 m 的约数。

证明: 考虑 的一个长度为 r 的循环。设 i 是 中的某个字母。那么, 根据循环的定义, $m(i)$ 就是: 从 i 开始沿着循环移动 m 步遇到的字母, 也就是从 i 开始绕着 移动 $m (r)$ 步的那个字母。但是 $m(i) = i$ 。因此 $m (r) = 0$, 即 r 整除 m 。所以 m 是 的每一个循环长度的倍数。反之显然成立, 证明完毕。

现在回到问题本身。考虑一个指数族 $_4$, 其中的卡牌和排列的循环中的卡牌是一样的, 但在这个族里, 只有那些权重是 m 的约数的牌叠才会出现。那么当 $r \nmid m$ 时, $d_r = (r-1)!$, 否则为 0。因此

$$(x) = \sum_{r \geq 1} d_r \frac{x^r}{r!} = \sum_{dm} \frac{x^d}{d}. \quad (3.14)$$

根据指数公式 (定理 3.11), 我们得到了以下优雅的结果:

定理 3.16 固定 $m > 0$ 。其 m 次幂为恒等排列的 n 字母排列数, 其生成函数为:

$$\sum_{dm} \frac{x^d}{d}. \quad (3.15)$$

让我们试试这个定理的一个特例。取 $m = 2$ 。那么我们讨论的就是**平方为恒等排列**的排列。这些排列被称为**对合 (involutions)**。根据上面的引理, 对合只能包含长度为 1 或 2 的循环。如果 n 是 n 字母的对合数量, 那么由 (3.15) 我们得到

$$\sum_{n \geq 0} \frac{n}{n!} x^n = x + \frac{1}{2} x^2. \quad (3.1)$$

3.9 2-正则图 (2-Regular Graphs)

在 n 个顶点上, 每个顶点的度数都为 2 的无向标号图 (此类图称为 **2-正则图**) 有多少个?

这样一个图是无向循环 (undirected cycles) 的**不相交并集**。因此我们有一个指数族 $_5$, 其中的卡牌代表**无向循环**, 而不是像排列情况下的有向循环。

对于固定的 $n \geq 2$ ，根本不存在无向循环。对于 $n \geq 3$ ，第 n 个牌叠中的卡牌数量 d_n 就是 n 个字母的无向环形排列数，这个数字是 $(n-1)!/2$ 。因此牌叠大小的生成函数是：

$$\begin{aligned} (x) &= \sum_{n \geq 3} \frac{(n-1)!}{2n!} x^n \\ &= \frac{1}{2} \sum_{n \geq 3} \frac{x^n}{n} \\ &= \frac{1}{2} \left\{ 1 \frac{1}{1-x} - x - \frac{x^2}{2} \right\}. \end{aligned}$$

通过指数公式 (3.5)，无向 2-正则标号图数量 $g(n)$ 的指数生成函数为：

$$\begin{aligned} \sum_{n \geq 0} g(n) \frac{x^n}{n!} &= \left\{ \frac{1}{2} 1 \frac{1}{1-x} - \frac{x}{2} - \frac{x^2}{4} \right\} \\ &= \frac{-\frac{1}{2}x - \frac{1}{4}x^2}{\sqrt{1-x}}. \end{aligned} \tag{3.17}$$

这个答案极好地证明了生成函数方法能以最少的工作量，巧妙地给出复杂计数问题答案的能力。

3.10 计算连通图 (Counting Connected Graphs)

包含 n 个顶点的标号连通图有多少个？

现在我们回到了标号图的指数族 $_1$ ，但这回有一两个小小的反转。指数公式可以告诉你，如果你知道了每种大小的连通小物件的数量，就能求出所有大小小物件的数量；**反之亦然 (vice versa)**。这个问题就是“反之亦然”。 n 个顶点的所有标号图的数量是 $2^{\binom{n}{2}}$ ，所以在方程“ $Hands = Decks^n$ ”中，我们已知的是“手牌 (Hands)”，我们想要找到的是“牌叠 (Decks)”，而不是相反。

这还有另一个反转。令 (x) 和 (x) 分别为牌叠和手牌的 egf。那么

$$(x) = \sum_{n \geq 0} \frac{2^{\binom{n}{2}}}{n!} x^n,$$

但这个级数除了在 $x = 0$ 处，在其他任何地方都不收敛。所以这仅仅是一个**形式 (formal)** 幂级数生成函数，我们不应该指望在这条路的尽头找到解析函数。

尽管说了这些，这套机制依然运转得非常好。我们现在将使用 1.6 节中的“ xDI ”技巧来找到连通图数量的递推公式。虽然对于这个特定情况找一个一般的递推关系并不难，但不管怎样，让我们从一般的情况来做吧。

定理 3.17 一个指数族中，牌叠和手牌的计数序列 $\{d_n\}$ 和 $\{h_n\}$ 满足如下递推式：

$$nh_n = \sum_k \binom{n}{k} kd_k h_{n-k} \quad (n \geq 1; h_0 = 1). \quad (3.1)$$

证明： 将 1.6 节中的“ $xD1$ ”技巧应用到指数公式 (3.7) 上。

由此可见，包含 n 个顶点的连通标号图的数量 d_n 满足递推式：

$$n2^{\binom{n}{2}} = \sum_k \binom{n}{k} kd_k 2^{\binom{n-k}{2}} \quad (n \geq 1). \quad (3.1)$$

通过这个公式我们能够，例如，计算出 n 较小时的 d_n 。对于 $n = 1, \dots$ ，我们求出的值分别是 1, 1, 4, 38, 728, 26704。

3.11 计算标号二分图 (Counting Labeled Bipartite Graphs)

包含 n 个顶点的顶点标号二分图 (bipartite graphs) 有多少个？

指数公式即使只需稍微哄一哄，也能搞定这个问题。**二分图** G 是一个将其顶点集 (G) 划分为 $= A$ ，使得 G 的每一条边都是形如 (a, b) 的图，其中 $a \in A$ 且 $b \notin A$ 。图 3.4 显示了一个包含 10 个顶点的二分图。

(此处图 3.4：一个二分图的示意图，左边集合 A ，右边集合 B ，边只在 A 和 B 之间连接)

现在，在 n 个顶点的 $2^{\binom{n}{2}}$ 个标号图中，有多少个是二分图呢？

嗯，有个小问题。如果你能数出牌叠的数量，指数公式就能算出手的数量；如果你能数出手牌的数量，它也能算出牌叠的数量。但它不能两样都干，而在这个问题中，不仅到底有多少个**连通的**二分图不明显，而且连二分图**总共有多少个**都不明显。

一个思路可能是先选好划分 $[n] = A \cup B$ 的集合 A ，然后再去数具有这个划分的二分图的数量。后者很简单；由于有 $|A|$ 条可能的边，就一定有 $2^{|A|}$ 种方法来行使“画还是不画这所有的边”的自由。

问题在于，一个固定的二分图在这个过程中可能会被计算多次。换句话说，对于同一个二分图，可能存在几种方法来展示其顶点集的划分，使得所有的边都在不同类的顶点之间运行。例如，图 3.4 中的图 G 将会出现多次：一次是 $A = \{1, 4, \dots\}$ ，另一次是 $A = \{2, 3, 5, 7, \dots, 10\}$ ，还有一次是 $A = \{1, 4, 5, \dots\}$ ，等等。

一般而言，我们正在考虑的这种构造法，将把具有 c 个连通分量的二分图构造 2^c 次。原因在于，对于 G 的每一个连通分量 G_i ，我们可以选择其顶点划分 A_i 或 B_i 中的哪一个被放到左侧的 A 中，哪一个被放到右侧的 B 中。

为了绕过这个难题，我们使用**稍有不同**的卡牌。由 **2-着色二分图 (2-colored bipartite graph)**，我们指的是一个带有两种颜色（“红”、“绿”）顶点着色的顶点标号二分图 G ，使得只要 (i, j) 是 G 的一条边，那么 i 和 j 就拥有不同的颜色。

例如，一个连通二分图会创造出两 (2) 个 2-着色图。一个带有 c 个连通分量的二分图会创造出 2^c 个这样的 2-着色图。

在我们正在建立的指数族中，将会有一张卡牌对应于每个 2-着色连通标号二分图。卡牌上印着的不仅有像往常一样的顶点标签集合 S ，还有一张包含 $|S|$ 个顶点的具有标准顶点标签的 2-着色连通二分图的照片。

通过给卡牌着色我们得到了什么？正是这个：**我们现在知道了有多少手权重为 n 的手牌了**。这个数量是

$$n = \sum_k \binom{n}{k} 2^{k(n-k)}, \quad (3.20)$$

因为每一手（并且是仅仅只有一手）手牌精确地从以下构造中产生一次：

- (i) 固定一个整数 k , $0 \leq k \leq n$;
- (ii) 从 $[n]$ 的元素中选择 k 个并把它们染成红色;
- (iii) 将 $[n]$ 剩余的元素染成绿色;
- (iv) 对每一对顶点 (i, j) (其中 i 是红色而 j 是绿色)，独立决定是否要让 (i, j) 成为一条边。

显然，(3.20) 式计算了这种构造的可能结果。

所以，尽管由于那些我们希望没有被着色的东西被着色了，导致我们处于一个错误的指数族中，但至少我们现在知道了一共有多少手手牌！

接下来，让我们使用指数公式来找出牌叠的 egf，它对应于**连通的** 2-着色二分图。它立刻告诉我们

$$(x) = 1 \left\{ \sum_{n \geq 0} \frac{n}{n!} x^n \right\}, \quad (3.21)$$

其中 n 由 (3.20) 定义。

现在我们既然已经把连通的带颜色的图给数出来了，那么计算连通的无颜色图还难吗？一点也不，因为未着色的连通图正好是着色且连通图数量的一半。所以普通的、未着色的连通二分图的 egf 是 $(x)/2$ ，其中 (x) 由 (3.21) 给出。

但现在我们在**正确的 (correct)** 指数族中，达成了我们以前没有达成的目标：我们知道了每个牌叠里有多少张卡牌。因此我们知道了指数公式所关联的两个项目中的一个，所以我们就能找到另一个。

因为 $(x)/2$ 生成牌叠的计数，那么必然有：

$$\begin{aligned} (x)^{1/2} &= \left\{ \frac{1}{2} \left\{ \sum_{n \geq 0} \frac{n}{n!} x^n \right\} \right\} \\ &= \sqrt{\sum_{n \geq 0} \frac{n}{n!} x^n} \end{aligned} \quad (3.22)$$

生成了手牌的计数，我们得到：

定理 3.18 令 (n) 表示 n 个顶点的顶点标号二分图的数量。那么

$$\sum_{n \geq 0} \frac{(n)}{n!} x^n = \sqrt{\sum_{n \geq 0} \frac{n}{n!} x^n}, \quad (3.23)$$

其中 n 由 (3.20) 给出。

因此，关于重复计算的所有复杂情况，都通过对我们一开始得到的生成函数进行简单的“开根号”操作给解决了！

3.12 计算标号树 (Counting Labeled Trees)

树 (tree) 是一个不含任何环的连通图。包含 n 个顶点的 (标准) 标号树有多少个？

在这个例子里，我们将以组合数学中最著名的结果之一的形式来推导这个问题的答案，即：

定理 3.19 (Cayley) 对于每个 $n \geq 1$ ，恰好有 n^{n-2} 个包含 n 个顶点的标号树。

尽管有许多已知的证明，但这个使用指数公式的生成函数证明尤为迷人，就在这里：

有根树 (rooted tree) 是一个有一特殊顶点被称为**根 (root)** 的树。显然，含有 n 个顶点的有标号有根树的数量，是无根树数量的 n 倍，所以只要我们能数出有根树的数量，我们就大功告成了。

令 t_n 为 (带有标准标签的) n 个顶点的有根树数量，对于 $n \geq 1$ 。我们按如下方式定义一个指数族 τ 。卡牌对应于有根标号树。在一张卡牌 (S, γ) 上， γ 是一张包含 $|S|$ 个顶点的标准有根树的图片，且 S 是一个标签集合。

在 τ 中，手牌是什么呢？一手手牌 对应于一个**有根标号森林 (rooted labeled forest)**，也就是一个其每个连通分量都是有根树的标号图。指数公式会告诉我们：如果我们知道树的数量，就能知道森林的数量；反之亦然。但这又是一个我们两边都不知道、令人不安的局面。解决办法？**硬着头皮往前走，保持信念 (Press on, and keep the faith)**。

根据指数公式，

$$(x) = (x)^{\tau}, \quad (3.24)$$

其中 $\binom{(x)}{gf} \{f_n\}$, $\binom{(x)}{gf} \{n\}$, 且 f_n 是包含 n 个顶点的有根森林的数量。现在 (3.24) 只是两个未知函数的一个方程。为了获得另一个方程, 我们使用由 Pólya 发现的一个事实, 即

$$n_{+1} = (n+1)f_n \quad (n \geq 0). \quad (3.25)$$

为了证明 (3.25), 设 F 为一个包含 n 个顶点的有根标号森林。引入一个新顶点, 并给它分配一个标签 j , 其中 $1 \leq j \leq n+1$ 。使用集合 $1, 2, \dots, j-1, j+1, \dots, n+1$ 保持标签顺序地重新标号 F 。然后在 F 和 F 中每个连通分量的所有根之间画出连线, 并将所得的树以 j 为根。其结果就是一棵包含 $n+1$ 个顶点的有根标号树。随着我们让标签 j 变化, 我们为每一个对应的有根森林 F 构造出了 $n+1$ 棵有根树。这个构造很容易逆转, 因此每一个 $n+1$ 顶点的有根树恰好出现一次, 这就证明了 (3.25)。

序列 $f_n = n_{+1}/(n+1)$ 拥有 egf 为:

$$\begin{aligned} \binom{(x)}{gf} &= \sum_{n \geq 0} \frac{f_n}{n!} x^n \\ &= \sum_{n \geq 0} \frac{n+1}{(n+1)!} x^n \\ &= \frac{1}{x} \binom{(x)}{gf}. \end{aligned}$$

如果我们把这个结果与 (3.24) 结合起来, 我们得到

$$\binom{(x)}{gf} = x^{\binom{(x)}{gf}}. \quad (3.2)$$

在之前遇到未知生成函数的问题中, 我们总是会得到某种形式的函数方程, 必须解开它才能找到该函数。我们见过方程是微分方程的情况, 也见过是二次方程的情况。在 (3.26) 中, 我们得到了一个需要求解 $\binom{(x)}{gf}$ 的函数方程, 它实际上唯一地决定了 $\binom{(x)}{gf}$, 但它既不是微分方程也不是代数方程, 它的解一点也不明显。

处理这类函数方程有一个强大的工具, 称为**拉格朗日反演公式 (Lagrange Inversion Formula, LIF)**, 我们将在 5.1 节中讨论它。在那里, 我们将完成树的枚举问题, 以此作为使用拉格朗日公式的一个绝佳示例。

3.13 指数族与“二项式类型”的多项式

(Exponential Families and Polynomials of ‘Binomial Type’)

与每个指数族相关联的, 都有一个多项式序列

$$h_n() = \sum_k h(n, k) k^n \quad (n = 0, 1, 2, \dots), \quad (3.27)$$

其中 $h(n, k)$ 是权重为 n 且包含 k 张卡牌的手牌数量。考虑到指数公式 (3.5)，这些多项式满足生成关系

$$x^{(n)} = \sum_{n \geq 0} \frac{n!}{n!} x^n. \quad (3.2)$$

满足 (3.28) 的多项式序列被 Rota 和 Mullin [RM] 称为**二项式类型的多项式 (polynomials of binomial type)**。之所以叫这个名字，是因为

$$\begin{aligned} & \frac{u(x)}{gf} \\ & \frac{gf}{gf} \{n(u)\}; \quad (x) \\ & \{n()\}; \end{aligned}$$

由此推导出

$$n(u +) = \sum_r \binom{n}{r} r(u)_{n-r}() \quad (n \geq 0),$$

这不禁让人联想到**二项式定理 (binomial theorem)**。

尽管许多作者已经为这类多项式序列给出了组合学解释，但上面出现的这种极其自然的解释似乎并未被讨论过。这种解释就是：**当二项式类型的多项式 $\{n()\}$ 的系数为非负时，那么必定存在一个指数族，使得对于每个 $n \geq 0$ ， $n()$ 都以卡牌数量为变量，生成了权重为 n 的手牌。反之，每一个指数族都有一族与之相关联的二项式类型多项式。**

3.14 无标号的卡牌与手牌 (Unlabeled Cards and Hands)

在本章的剩余部分，我们将考虑同样类型的问题，区别在于：这里不再有需要我们去操心的标签集 (label sets) 了。这看起来似乎能让事情变得简单，在某些方面确实如此，但在另外一些方面则不然。我们将关注的是，用给定的积木 (卡牌) 究竟能构建出多少种结构 (手牌)。

现在，一张卡牌 $= (n,)$ 仅仅拥有它的权重 n 和它的图片。对于每个 $n = 1, 2, \dots$ ，都有一个包含 d_n 张卡牌的牌叠 n ，这些卡牌的权重全都是 n 。

一手手牌 (hand) 是一个卡牌的多重集 (multiset)。 也就是说，我们可以把手伸进牌叠 r 中，抽取出一定数量的同一张卡牌 $(r, ')$ 的副本，然后再抽出若干张卡牌 $(r,)$ 的副本，等等；然后我们再从另一个牌叠中拿出更多卡牌，等等。

手牌中卡牌的顺序没有任何意义。**重要的是哪些卡牌被选中了，以及它们被选中的多重性**

(multiplicities, 即重复次数)。 一手手牌的权重是该手牌中所有卡牌权重的总和 (需计算它们的重复次数)。像以前一样，我们令 $h(n, k)$ 为权重为 n 且恰好包含 k 张卡牌的手牌数量，并且我们令

$$(x,) = \sum_{n,k} h(n, k) x^{nk}. \quad (3.2)$$

请注意，在这个假设的生成函数形式中，“ $n!$ ”不见了。代替适用于标号计数的混合“egf-ops”生成函数，**纯粹的 ops（普通幂级数）才是无标号计数该走的路。**

我们需要为我们正在构建的系统起一个通用名称。我们将称它们为 **预制件（prefabs）**（而不是在标号情况下的“指数族”），并且我们将使用像 这样的字母来表示它们。

因此，一个预制件 由一系列牌叠 $1, 2, \dots$ 组成，正如上面所描述的那样，我们可以从中组成手牌。在 中，我们令 $\{d_n\}_1$ 。

主要的问题依然是找到 $(x,)$ 和 (x) 之间的函数关系，所以我们现在就来做这件事。我们将再次使用“魔法师学徒（Sorcerer's Apprentice）”的方法。

对于“涓涓细流（trickle）”，考虑一个预制件，它只包含一个非空的牌叠 r ，并且假设 r 只包含**一张（a single）** 卡牌。

在这个预制件中，一手手牌 H 是一件相当简单直白的事情。它由某个数量（比如 k 个）的那个唯一存在的卡牌的副本组成，并且它的权重将是 $n = rk$ 。因此，在这个预制件中，权重为 n 且恰好有 k 张卡牌的手牌数量 $h(n, k)$ ，在 $n = rk$ 时是 1，在其他情况下是 0。因此

$$\begin{aligned} (x,) &= \sum_{n,k} h(n, k) x^{nk} \\ &= \sum_{k \geq 0} 1 \cdot x^{rk} \\ &= \frac{1}{1 - x^r}. \end{aligned} \quad (3.30)$$

接下来，正如在 3.4 节中那样，我们定义**合并（merge）**操作。如果 ' 和 是图片集互不相交的预制件，那么它们的合并 $= ' \cup$ 就是这样一个预制件：对于每个 n ，它的牌叠 $_n$ 是 ' 和 对应牌叠的并集。如果那两个牌叠中分别有 d'_n, d_n 张卡牌，那么 $_n$ 中就有 $d_n = d'_n + d_n$ 张卡牌。

引理 3.20 (无标号计数的基本引理) 设 $'(x,)$ 、 $(x,)$ 和 $(x,)$ 分别是预制件 '、 以及 $= ' \cup$ 的手牌枚举器。那么 $= ' \cup$ 。

证明：考虑合并后的族 中一手权重为 n 、且恰好包含 k 张卡牌的手牌 H 。其中的某 k' 张卡牌来自于 '，它们的总权重为，比方说， n' ；而剩余的 $k - k'$ 张卡牌来自于 ，它们的总权重必须是 $n - n'$ 。因此

$$h(n, k) = \sum_{k', n'} h'(n', k') h(n - n', k - k'),$$

但由于一个奇妙的巧合，这恰好就是幂级数 $'$ ，系数之间所满足的关系。

武装了这个基本引理之后，我们现在可以考虑一个稍微复杂一点的预制件 r ，它依然只包含一个非空牌叠 r ，但现在那个牌叠里包含了 d_r 张不同的卡牌。通过对 $d_r = 1, 2, \dots$ 进行归纳，我们立

刻就能看出这个预制件的手牌枚举器是：

$$(x,) = \frac{1}{(1 - x^r)^{d_r}}. \quad (3.31)$$

最后（是不是有种似曾相识的感觉？），在一个一般的预制件 中，其中对于每个 $n = 1, 2, 3, \dots$ ，牌叠 $_n$ 中有 d_n 张卡牌，我们观察到 $= \prod_{n=1}^{\infty} (1 - x^n)^{-d_n}$ ，其中 $_n$ 如上一段所定义。我们立刻得出：

定理 3.21 在一个手牌枚举器为 $(x,)$ 的预制件 中，我们有

$$(x,) = \prod_{n=1}^{\infty} \frac{1}{(1 - x^n)^{d_n}}, \quad (3.32)$$

其中 d_n 是第 n 个牌叠中的卡牌数量 ($n \geq 1$)。

这就是没有标签情况下的指数公式类比（大名鼎鼎的 **欧拉乘积/Euler product** 形式）。与指数公式一样，这个公式也拥有数量惊人的优雅应用，我们将在后文中讨论其中的一些。在我们开始之前，让我们先使用 1.6 节中的“DI”技巧，将 (3.32) 式转化为一个可以让我们根据 d 实际计算出 h 的公式。

如果我们对 (3.32) 两边取对数，

$$\begin{aligned} \ln(x,) &= \sum_{s=1}^{\infty} \ln \frac{1}{(1 - x^s)^{d_s}} \\ &= \sum_{s=1}^{\infty} d_s \ln \frac{1}{(1 - x^s)} \\ &= \sum_{s=1}^{\infty} d_s \sum_{m=1}^{\infty} \frac{-x^{sm}}{m} \\ &= \sum_{n,m=1}^{\infty} d_{\frac{n}{m}} x^n \frac{-1}{m}, \end{aligned}$$

这里，如果 d_j 的下标不是正整数，则将其解释为 0。

接下来我们对 求导并乘以 ，得到

$$\frac{(x,)}{(x,)} = (x,) \sum_{n,m=1}^{\infty} x^{nm} d_{\frac{n}{m}}.$$

最后，我们取两边的 $[x^{nm}]$ ，这得出

$$mh(n, m) = \sum_{r,m' \geq 1} h(n - rm', m - m') d_r \quad (n, m \geq 1; h(n, 0) = _{n,0}). \quad (3.33)$$

这个递推式在任何预制件中都成立，它允许我们从牌叠计数中数值计算出手牌计数。

通常双变量牌叠枚举器 $(x,)$ 或 $\{h(n, k)\}_{n,k \geq 0}$ 给出的细节超出了我们的需要。如果 $h_n = \sum_k h(n, k)$ 是权重为 n 的手牌数量（无论它们包含多少张卡牌），并且如果

$$\begin{pmatrix} x \\ s \end{pmatrix} = \{h_n\}_0,$$

那么，由于我们通过形式上将 s 替换为 1 就可以从 $\begin{pmatrix} x \\ s \end{pmatrix}$ 获得 (x) ，一般计数定理 (3.32) 就变成了

$$(x) = \prod_{r=1}^{\infty} \frac{1}{(1-x^r)^{d_r}}. \quad (3.34)$$

递推式 (3.33) 可以被替换为

$$nh_n = \sum_{m \geq 1} D_m h_{n-m} \quad (n \geq 1; h_0 = 1), \quad (3.35)$$

其中 $D_m = \sum_{r|m} r d_r$ ($m = 1, 2, \dots$)。

只需稍微多费一点力气，就能获得详细得多的信息。**假设我们限制卡牌在手牌中被使用的重复次数 (multiplicities)。** 例如，假设我们规定：手牌中出现的每一张卡牌，其出现的重复次数必须能被 3 整除，等等。那么对于手牌的数量，我们能说些什么呢？

设 S 为一个固定的、包含 0 的非负整数集合。对于每个 n 和 k ，令 $h(n, k; S)$ 为权重为 n 、包含恰好 k 张卡牌（计算重复次数！）的手牌数量，其中每张卡牌出现的重复次数都属于集合 S 。令

$$\begin{pmatrix} x, ; \\ S \end{pmatrix} = \sum_{n,k} h(n, k; S) x^{nk}.$$

最后，令

$$\begin{pmatrix} x, ; \\ S \end{pmatrix} = \sum_k \begin{pmatrix} x, ; \\ S \end{pmatrix}_k. \quad (3.3)$$

在合并图片集不相交的预制件时，生成函数再次是相乘的 (multiplicative)。考虑一个只有 1 张权重为 r 的卡牌，并且没有其他牌叠的预制件。那么如果 k 且 $n = kr$ ，则 $h(n, k; S) = 1$ ，否则为 0，因此

$$\begin{pmatrix} x, ; \\ S \end{pmatrix} = \sum_k x^{krk} = (x^r).$$

如果在第 r 个牌叠里有 d_r 张卡牌并且没有其他卡牌，那么 $\begin{pmatrix} x, ; \\ S \end{pmatrix} = (x^r)^{d_r}$ ，最终我们得到：

定理 3.22 设预制件 包含大小为 $d_1, d_2, \dots$ 的牌叠，并设 S 为一个非负整数集合， $0 \in S$ 。如果 $h(n, k; S)$ 是包含 k 张卡牌、权重为 n 、且每张卡牌出现的重复次数都属于 S 的手牌数量，那么

$$\begin{pmatrix} x, ; \\ S \end{pmatrix} = \sum_{n,k} h(n, k; S) x^{nk} = \prod_{r \geq 1} (x^r)^{d_r}, \quad (3.37)$$

其中 $\begin{pmatrix} x, ; \\ S \end{pmatrix}$ 由 (3.36) 给出。

请注意，在 $S = \mathbb{N}$ （所有非负整数集合）的情况下，该定理就退化为定理 3.21。

一个值得注意的特例是 $\mathcal{S} = \{0, 1\}$ ，这意味着我们可以选择某张卡牌加入手牌，也可以不选，但是不能拿超过一张同一张卡牌的副本。在那种情况下 (3.37) 给出

$$\begin{aligned} (x, ; \{0, 1\}) &= \prod_{r \geq 1} (1 + x^r)^{d_r} \\ &= \frac{1}{(x, -; +)}. \end{aligned} \quad (3.3)$$

我们将继续探讨使用这些公式的几个例子。

3.15 换零钱问题 (The Money Changing Problem)

假设在某国的铸币系统中，只有 5 分、11 分和 37 分硬币。我们有多少种方法可以凑出 \$17.19 美元？

用一般的话来说，我们给定 a 个正整数

$$1 \leq a_1 < a_2 < \cdots < a_n,$$

并且我们问以下问题：对于每个正整数 n ，我们有多少种方法可以写出

$$n = x_1 a_1 + x_2 a_2 + \cdots + x_n a_n \quad (i : x_i \geq 0), \quad (3.3)$$

其中 x_i 都是整数？这个问题在包括纯数学和应用数学在内的众多领域都具有巨大的重要性，并且它拥有一个非常美丽的理论，我们将在此给出其中的一部分。

对于给定的 $a_1, \dots, a_n$ ，我们将能够写成 (3.39) 形式的所有 n 构成的集合记为 $\mathcal{S} = (a_1, \dots, a_n)$ 。是一个非负整数的半群 (semigroup)。

首先让我们来识别一下所有事情将发生其中的那个预制件 $\mathcal{S}$ 。它的牌叠几乎全都是空的。唯一不为空的牌叠是那个牌叠： $a_1, \dots, a_n$ 。这些牌叠中的每一个都仅包含**单单一张**卡牌。因此牌叠枚举序列为：

$$d_n = \begin{cases} 1 & \text{如果 } n = a_1, \dots, a_n \\ 0 & \text{其他.} \end{cases}$$

从某种意义上说，问题到此就已经结束了。如果 $h(n, k)$ 表示恰好使用了 k 枚硬币来凑零钱的方法数，即在表示式 (3.39) 中满足 $\sum_i x_i = k$ 的表示方法数，那么根据主要的计数定理（方程 (3.32)），我们有：

$$(x, \mathcal{S}) = \frac{1}{(1 - x^{a_1})(1 - x^{a_2}) \cdots (1 - x^{a_n})}. \quad (3.40)$$

如果 h_n 是表示出 n 的方法数，而不考虑硬币的总数，那么从更粗略的公式 (3.34) 可得：

$$(x) = \frac{1}{(1-x^{a_1})(1-x^{a_2})\cdots(1-x^a)}. \quad (3.41)$$

尽管生成函数已经知道了，但仍有实质性的问题存在。这里列出其中几个。

我们该如何描述集合？也就是说，**哪些金额的钱是可以被凑出来的**？既然只给你 8 分和 12 分硬币，指望能凑出 53 分的零钱显然是不合理的。一般而言，如果集合 $\{a_1, \dots, a\}$ 的最大公约数是 $g > 1$ ，那么只有 g 的倍数才能被表示出来。但是假设 $g = 1$ ，即 a_i 们是**互素的 (relatively prime)**。那么哪些整数是可以表示的呢？这个主题的核心结果归功于 I. Schur。它声明：此时集合包含了**所有足够大的整数**，即存在一个整数 N ，使得每一个整数 $n \geq N$ 都可以用 (3.39) 的形式表示出来。

具有定理中所述属性的最小整数 N 将被称为集合 $= \{a_1, \dots, a\}$ 的**导子 (conductor)**，并记为符号 $= ()$ 。

例如，每一个 $\geq$ 的整数都可以被表示为 3 和 5 的非负整数线性组合，而 7 则不能被这样表示，所以 $(\{3, 5\}) =$ 。

准确求出一个集合的导子的问题，似乎极端困难。如果 ≥ 3 ，目前并没有计算导子的一般“公式”；如果 ≥ 4 ，也没有好的算法来计算它。 $= 2$ 的情况已经非常漂亮了，并且答案是已知的，所以它们就在这里：

定理 3.23 设 a 和 b 为互素的正整数。那么

(a) 每个大于等于 $= (a-1)(b-1)$ 的整数 n 都可以表示为 $n = xa + b$ ，其中 $x, \geq 0$ ，且

(b) 整数 -1 不能写成那种形式，且

(c) 在整数 $0, 1, 2, \dots, -1$ 中，恰好有一半是可以表示的，一半是不可表示的。

定理 3.23 的证明：（我们的证明遵循 [NW]）因为 $(a, b) = 1$ ，如果允许 x ，取任意正负号，我们肯定能把每个整数 m 写成 $xa + b$ 的形式。如果我们要求 $0 \leq x < b$ ，那么这种表示法是唯一的。此时，如果 ≥ 0 ，则 m ；如果 < 0 ，则 m 。因此，最大的不能被表示出来的整数，是在选取 $x = b-1, = -1$ 时得到的。因此 $()$ 比 $(b-1)a - b$ 大 1 个单位，定理的 (a) 和 (b) 部分得证。

为了证明 (c)，设 $0 \leq m < ()$ ，再次考虑 $m = xa + b$ 的唯一写法，其中 $0 \leq x < b$ 。那么

$$m' = -1 - m = (b-1-x)a + (-1-b).$$

既然 $0 \leq b-1-x < b$ ，所以如果 ≥ 0 ，则 m 是可表示的，而 m' 是不可表示的；如果 < 0 ，则 m' 是可表示的，而 m 是不可表示的。因此数字 $0, 1, \dots, -1$ 中恰好有一半是可表示的。

现在我们要去证明 **Schur 定理**。这个证明的思想是，我们将考虑（尽管根本不去写出它）公式 (3.41) 右侧的部分分式展开。在出现的众多项中，我们将找出一项，它的幂级数系数比任何其他项的系数增长得都要快，而这将给出我们想要的结果。

公式 (3.41) 中的生成函数 (x) 是一个有理函数，它的极点 (poles) 全都落在单位圆 $|x| = 1$ 上。事实上，这些极点位于各种**单位根**处。

这些极点的**重数 (multiplicities)** 是多少？点 $x = 1$ 是一个重数为 r 的极点，因为 (x) 的分母能被 $(1 - x)$ 整除。令 $\zeta = e^{2\pi i r/s}$ 为一个本原的（即 $(r, s) = 1$ ） s 次单位根。这个点 $x = \zeta$ 作为 (x) 极点出现的重数是多少呢？它等于能被 s 整除的 a_i 的数量。既然 a_i 们是互素的，它们**不可能**全都能被 s 整除。

因此， $x = 1$ 是 (x) 的一个 r 阶极点，而**每一个其他的极点，其重数都 $< r$** 。

假设 ζ 是一个 r 阶极点。那么 (x) 的部分分式展开式中来自于 ζ 的那部分具有如下形式：

$$\frac{c_1}{(1 - x/\zeta)^r} + \frac{c_2}{(1 - x/\zeta)^{r-1}} + \cdots$$

现在回忆一下幂级数展开式 (2.39)，我们在这里重写一遍：

$$\frac{1}{(1 - x)^{k+1}} = \sum_{n \geq 0} \binom{n+k}{k} x^n.$$

如果 $k = 1$ ，这个展开式的系数是 n 的一次（线性）函数。如果 $k = 2$ ，它们是 n 的二次函数。一般而言， x^n 的系数随着 n ，其增长速度类似于 $n^k/k!$ 。

因此， (x) 的系数序列中，来自于一个固定的 r 阶极点的贡献，其增长速度类似于 cn^{r-1} 。在 $x = 1$ 处存在唯一的一个 r 阶极点。它的那部分部分分式展开式，为 (x) 的第 n 个系数贡献了 $\sim cn^{r-1}$ 的增长。由于所有其他的极点都有着严格较低的重数，它们都不能改变由主极点 $x = 1$ 所贡献的渐近增长率。因此，当 $n \rightarrow \infty$ 时，我们有 $h_n \sim cn^{r-1}$ 。这当然就意味着，对于所有足够大的 n 的值，我们都会有 $h_n > 0$ ，从而完成了证明。然而，既然我们都已经算到这一步了，为什么不顺便把 c 的值也求出来呢？

(x) 的部分分式展开式具有如下形式：

$$\begin{aligned} (x) &= \frac{1}{(1 - x^{a_1})(1 - x^{a_2}) \cdots (1 - x^a)} \\ &= \frac{c}{(1 - x)^r} + ((1 - x)^{-1}) \end{aligned}$$

为了计算 c ，等式两边同时乘以 $(1 - x)$ 并令 $x \rightarrow 1$ 。这给出了 $c = 1/(a_1 \cdots a)$ 。因此，在定理的证明之外，我们还得到了一个增长估计式。

定理 3.24 (Schur 定理) 如果 h_n 表示将 n 表示为 $a_1, \dots, a$ 的非负整数线性组合的表示法数量，其中这些给定的正整数构成一个互素的集合，那么

$$h_n \sim \frac{n^{r-1}}{(r-1)!a_1a_2 \cdots a} \quad (n \rightarrow \infty). \quad (3.42)$$

特别地，存在一个整数 N ，使得每一个 $n \geq N$ 都能至少以一种方式被这样表示。

例 3.25 给定两个互素的整数 a, b 。找出一个显式公式来计算 $f(n)$ ，即使用这些硬币来凑齐 n 分钱的方法数。

从 (3.41) 中我们有

$$\sum_n f(n)x^n = \frac{1}{(1-x^a)(1-x^b)}, \quad (3.43)$$

所以剩下的就是一个部分分式展开。我们发现

$$\frac{1}{(1-x^a)(1-x^b)} = \frac{A}{(1-x)^2} + \frac{1}{(1-x)} + \sum_{a=1}^{\infty} \frac{1}{1-x^a} + \sum_{b=1}^{\infty} \frac{D}{1-x^b}. \quad (3.44)$$

关于这些常数，从 (3.42) 我们已经知道 $A = 1/(ab)$ 。为了求 B ，将 (3.44) 乘以 $(1-x)^2$ ，求导，并令 $x = 1$ 。这给出 $B = (a+b-2)/(2ab)$ 。为了求 C ，乘以 $(1-x)$ 并令 $x = 1$ 。结果是 $C = 1/(a(1-b))$ ，同样地求出 D 。最后我们在 (3.44) 的全式中提取 x^n 的系数，得到公式

$$f(n) = \frac{n}{ab} + \frac{a+b}{2ab} + \sum_{a=1}^{\infty} \frac{1}{n} + \sum_{b=1}^{\infty} \frac{D}{n}. \quad (3.45)$$

如果我们把 (3.45) 中出现的这两个和式作为 n 的函数来考察，我们会发现它们每一个都是 n 的周期函数。第一个和式的周期为 a ，第二个和式的周期为 b 。因此，这两个和式的总和就是一个周期为 ab 的周期函数。所以我们发现，将 n 分钱换成 a 分和 b 分面额硬币的方法数是

$$f(n) = \frac{n}{ab} + \frac{a+b}{2ab} + r(n), \quad (3.4)$$

其中 $r(n)$ 是周期为 ab 的周期函数，且它的平均值为 0。

我们可能想看看这个周期性是如何起作用的，所以让我们取 $a = 3$ 且 $b = 5$ 。计算数字 $f(n)$ 的一个好方法是利用隐藏在生成函数 (3.43) 中的递推公式。如果我们在 (3.43) 上使用 xD 技巧，我们会发现递推式的形式为：

$$nf(n) = 3 \sum_{j \geq 1} f(n-3j) + 5 \sum_{j \geq 1} f(n-5j) \quad (n \geq 1; f(0) = 1), \quad (3.47)$$

并且我们约定，如果 $m < 0$ ，则 $f(m) = 0$ 。表 3.1 展示了 $n, f(n)$ 以及 $15(f(n) - (n/15) - (4/15))$ （根据 (3.46)，它是一个周期为 15 的周期函数）。

n	0	1	2	3	4	5	6	7	8	9	10	11
$f(n)$	1	0	0	1	0	1	1	0	1	1	1	1
r_{15}	11	-5	-6	8	-8	6	5	-11	3	2	1	0

n	15	16	17	18	19	20	21	22	23	24	25	26
$f(n)$	2	1	1	2	1	2	2	1	2	2	2	2
r_{15}	11	-5	-6	8	-8	6	5	-11	3	2	1	0

3.16 整数的划分 (Partitions of Integers)

正整数 n 的一个划分 (partition) 是指如下的一种表示形式：

$$n = r_1 + r_2 + \cdots + r_k \quad (r_1 \geq r_2 \geq \cdots \geq r_k \geq 1). \quad (3.4)$$

数字 $r_1, \dots, r_k$ 称为划分的**部分 (parts)**。因此 (3.48) 是将 n 划分为 k 个部分的划分。

数字 5 有 7 种划分，即：

$$5, \quad 4 + 1, \quad 3 + 2, \quad 3 + 1 + 1, \quad 2 + 2 + 1, \quad 2 + 1 + 1 + 1, \quad 1 + 1 + 1 + 1 + 1.$$

n 的划分数记为 (n) ，而 (n, k) 是将 n 划分为 k 个部分的划分数。对 (n) 更深层性质的研究是 20 世纪数学分析中一颗璀璨的明珠，这涉及到了哈代 (Hardy) 和拉马努金 (Ramanujan) 的研究，以及 Rademacher 的后续工作，最终的结果是得到了一个计算 (n) 的精确闭合公式，该公式同时也是一个完整的渐近级数。完整的故事可以在 Andrews 的著作 [An] 中找到。

从我们的角度来看，**划分理论就是换零钱问题的一种情况，只不过在这个问题中，各种正整数面额的硬币全都可以使用。**

因此，定理 3.21 立刻为我们给出了划分函数的普通幂级数生成函数 (opsgf)，它是一个无穷乘积的倒数形式：

$$\sum_{n,k \geq 0} (n, k) x^{nk} = \frac{1}{(1-x)(1-x^2)(1-x^3)(1-x^4) \cdots} \quad ((0, k) = 0, k). \quad (3.4)$$

令 $x = 1$ ，我们得到：

$$\sum_{n \geq 0} (n) x^n = \frac{1}{(1-x)(1-x^2)(1-x^3)(1-x^4) \cdots} \quad ((0) = 1) \quad (3.50)$$

这就是 $\{(n)\}$ 本身的生成函数 (大名鼎鼎的欧拉划分恒等式)。

在另一个极端，我们可以考虑**受限于特定部分以及部分的特定重复次数**的划分。设给定两个集合，非负整数集合 (包含 0)，以及正整数集合。令 $(n, k; ,)$ 表示将 n 划分为 k 个部分的划分数，满足：所有的部分都位于 中，且所有这些部分的重复次数都位于 中。那么由 (3.37) 式可得：

$$\sum_{n,k} (n, k; ,) x^{nk} = \prod_r \left(\sum_k x^{kr} \right). \quad (3.51)$$

从这个生成函数出发，我们可以证明许多关于划分的定理。

例 3.26 设 $\mathcal{P} = \{0, 1\}$, $\mathcal{K} = \{1, 2, \dots\}$ 。那么 $(n, k; \mathcal{P}, \mathcal{K})$ 就是将 n 划分为 k 个不同 (distinct) 部分的划分数, 我们有:

$$\sum_{n,k} (n, k; \{0, 1\}, \mathcal{K}) x^{nk} = \prod_{r \geq 1} (1 + x^r). \quad (3.52)$$

如果我们令 $k = 1$, 我们得到

$$\begin{aligned} \sum_n (n; \{0, 1\}, \mathcal{K}) x^n &= \prod_{r \geq 1} (1 + x^r) \\ &= \prod_{r \geq 1} \frac{1 - x^{2r}}{1 - x^r} \\ &= \frac{(1 - x^2)(1 - x^4) \cdots}{(1 - x)(1 - x^2)(1 - x^3)(1 - x^4) \cdots} \\ &= \frac{1}{(1 - x)(1 - x^3)(1 - x^5)(1 - x^7) \cdots}. \end{aligned}$$

然而, 这最后一部分生成的是将 n 划分为奇数部分 (odd parts) 的划分数, 所以我们得到了以下定理的一个生成函数证明:

定理 3.27 (欧拉的奇划分与互异划分定理) 对于每一个 $n = 1, 2, 3, \dots$, 将 n 划分为全都是奇数部分的划分数, 等于将 n 划分为各不相同部分的划分数。

例如, 将 5 划分为奇数部分有: 5, 3+1+1, 以及 1+1+1+1+1, 而划分为不同部分的有: 5, 4+1, 以及 3+2。定理 3.27 是由欧拉发现的。后来人们给出了许多它的证明。其中最有趣的一些证明是**双射证明 (bijective)**; 也就是说, 它们给出了明确的构造法, 将每一个划分为奇数部分的划分, 与一个划分为各不相同部分的划分相匹配 (例如, 格莱舍双射)。

例 3.28 现在令 $\mathcal{P} = \{0, 1, \dots, r-1\}$ 且 $\mathcal{K} = \mathcal{K}$ 。那么 (3.51) 的右侧变成了

$$\prod_{r \geq 1} (1 + x^r + \cdots + x^{(r-1)r}) = \prod_{r \geq 1} \left(\frac{1 - x^{r^2}}{1 - x^r} \right).$$

这个乘积的分子中的每一个因子都会抵消分母中的一个因子, 分母中只留下那些 r 不能被 $r+1$ 整除的因子。这证明了以下结果 (当 $r = 1$ 时它就退化为定理 3.27):

定理 3.29 (Glaisher 定理) 固定 $r \geq 1$ 。对于每个 $n \geq 1$, 将 n 划分为不能被 $r+1$ 整除的部分的划分数, 等于将 n 划分为任何部分最多只出现 r 次的划分数。

3.17 有根树和森林 (Rooted Trees and Forests)

有根树是指顶点没有标号, 唯独其中一个顶点被特殊标记为“根”的树。

在 3.12 节中，我们使用指数公式计算了**带标号的树**。这里我们将计算**无标号的有根树**。在某个牌叠 n 中的每一张卡牌上，现在包含整数 n 和一棵 n 顶点的有根树的图片。牌叠 $_4$ 如图 3.5 所示。

(此处图 3.5: 4 个顶点的无标号有根树的四种可能形状)

在这种情况下，一手里包含 k 张卡牌且权重为 n 的手牌，是一个包含 n 个顶点和 k 个连通分量（有根树）的有根森林。如果 $h(n, k)$ 是这类森林的数量，并且如果 $h(n) = \sum_k h(n, k)$ 是所有 n 顶点的有根森林的数量，那么由 (3.34) 我们得到：

$$\sum_n h(n)x^n = \prod_{n \geq 1} \frac{1}{(1 - x^n)^{(n)}}, \quad (3.53)$$

其中 $(n) = h(n, 1)$ 是 n 顶点的有根树的数量。

接着，正如我们在带标号情况中发现的那样（见 (3.25)）， n 顶点的有根森林的数量和 $n + 1$ 顶点的有根树的数量之间有一个简单的关系：**它们是相等的**。只需向森林中添加一个新顶点 r ，称其为新的根，将它连接到森林中以前所有树的根上，你就得到了与给定森林相对应的那棵有根树。因此 $h(n) = (n + 1) \quad (n \geq 0)$ 。此时 (3.53) 采取以下形式：

$$\sum_n (n + 1)x^n = \prod_{n \geq 1} \frac{1}{(1 - x^n)^{(n)}}. \quad (3.54)$$

这个方程实际上决定了所有的数字 $\{(n)\}$ 。然而，这是一个相当令人生畏的方程，我们不应该指望这些数字存在什么简单的公式。

3.18 历史注记 (Historical Notes)

指数公式 (exponential formula) 最早出现在 Riddell [RU] 的论文中，其形式是：从所有带标号图的数量推导出连通带标号图的数量。此后，这一思想被几位研究人员推广和扩展。

在 [BG] 中，大约同时在 [FS] 中，Bender 和 Goldman，以及 Foata 和 Schützenberger 对该思想进行了重要扩展，将其应用于非常一般的带标号和无标号的场景。前者引入了“预制件 (prefabs)”的概念，后者使用了“复合划分 (composé partitionnel)”的概念。该方法的进一步发展可见于 Stanley [St1, St2]，他基于划分方法进行了研究；Joyal [Jo] 在他的“物种 (species)”理论中使用了函子方法 (functorial methods)；Beissinger [Bei]，以及 Garsia 和 Joni [GaJ] 也对此有所贡献。

本书中所采用的方法与**复合划分 (composé partitionnel)** 最为接近。在与 Adriano Garsia 的私人交谈中，他建议我用卡牌 (cards)、牌叠 (decks) 和手牌 (hands) 的术语来进行讨论。当时我向他展示了我基于图论观点编写的一套讲义。我认为，他的建议不仅最大程度地保证了概念的清晰性，同时还兼顾了应用的最大普适性。

3.19 习题 (Exercises)

1. 给出一个显式的 1-1 对应关系 (双射), 将 n 划分为各不相同部分 (distinct parts) 的划分与将 n 划分为奇数部分 (odd parts) 的划分对应起来。
2. 给定整数 n, k 。令 $f(n, k)$ 为: 其所有循环长度都能被 k 整除的 n 字母排列的数量。寻找序列 $\{f(n, k)\}_{n \geq 0}$ 的一个简单、显式的指数生成函数 (egf)。为 $f(n, k)$ 找出一个简单、显式的公式。(提示: 你可能需要回顾 3.4 节末尾的讨论。)
3. 寻找集合 $[n]$ 划分的 egf, 要求其所有类的元素个数都是一个素数。
4. 在一个群中, 一个元素 g 的阶 (order) 是满足 $g^m = 1$ 的最小正整数。
 - (a) 在所有 n 字母排列构成的群中, 用排列的各个循环长度来表示它的阶。
 - (b) 令 $g(n, k)$ 为阶为 k 的 n 字母排列数量。请用 $g(n, m)$ (即循环长度全都整除 m 的 n 字母排列数) 来表示 $g(n, k)$ 。
5. 令 a_n 为 n 字母的对合 (involutions, 即自身为其逆的排列) 数量。
 - (a) 寻找满足这些数字的一个递推公式。
 - (b) 计算 $a_1, \dots, a_{10}$ 。
 - (c) 对该递推给出组合学的、构造性的解释。也就是说, 在通过生成函数推导出它之后, 尝试不使用生成函数重新推导它。
6. 用简单的形式找出序列 $\{a_n\}$ 的 egf, 其中 a_n 是不包含任何长度 3 的循环的 n 字母排列数量。你的答案中不应包含任何无穷级数。
7. 找出所有顶点度数均为 1 或 2、且具有奇数个连通分量的标号图的生成函数。找出这些数字满足的递推公式, 计算前几个数值, 并画出涉及的那些图。
8. 从 (3.10) 中找出第一类斯特林数满足的一个三项递推关系。给出这个递推关系的直接组合学证明。也就是说, 不使用任何生成函数重新证明它。
9. 如同 3.7 节中那样, 寻找具有以下两个属性的 n 字母排列数 $\{g(n)\}_0$ 的 egf: (a) 它们有奇数个循环, 且 (b) 它们所有循环的长度都是偶数。为这些数字找出一个简单、显式的公式。
10. 通过用二项式定理展开 (3.13) 中出现的 k 次方, 为第二类斯特林数 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 找出一个显式公式。你的公式应该是一个单一有限求和的形式。
11. 设 S 为固定的正整数集合。令 $f(n; S,)$ 为 $[n]$ 的划分数, 其类的所有大小均位于 S 中, 且类的总数位于 S 中。证明 $\{f(n; S,)\}_{n \geq 0}$ 的 egf 为 $(s(x))$, 其中 $s(x) = \sum_{s \in S} x^s / s!$ 。
12. 设 $k > 0$ 。令 $f(n, k)$ 为其最长循环的长度为 k 的 n 字母排列的数量。在固定 k 的情况下, 寻找 $\{f(n, k)\}_{n \geq 0}$ 的 egf。

13. 如果 (x) 和 (x) 分别表示对合数量 (见 3.16 式) 和 2-正则图数量 (见 3.9.1 节, 原书这里可能指 (3.17) 式) 的 egf, 请观察到:

$$(x)(x)^2 = \frac{1}{1-x}.$$

(a) 写出上述生成函数关系所暗示的序列 $\{g(n)\}, \{n\}$ 之间的恒等式。

(b) 证明: 对于每个固定的 $n \geq 1$, 下列两者的数量完全相同:

(i) n 字母排列的数量;

(ii) 三元组 (σ, G_1, G_2) 的数量, 其中 σ 是集合 S 上的对合, G_1 是顶点集 S 上的 2-正则图, G_2 是顶点集 S 上的 2-正则图, 并且 σ, S 是 $[n]$ 的一个划分。

(c) 明确地找到 (b) 部分中描述的上述两者之间的 1-1 对应关系。

14. 设 $\{x_n\}$ 是一个具有相关二项式类型多项式 $\{x_n(x)\}$ 和牌叠枚举器 (x) 的指数族。

(a) 如果 D 表示微分算子 $\frac{d}{dx}$, 请证明:

$$(-1)(D)_n() = n_{n-1}() \quad (n \geq 0)$$

方法是直接将该算子应用于多项式序列的 egf。(这里 (-1) 表示函数复合意义上的反函数/逆函数。)

(b) 在按循环分类的排列构成的指数族中, 找到相关的二项式类型多项式, 并通过使用这些多项式直接进行计算来验证 (a) 部分中证明的恒等式。

15. 在一个指数族 $\{x_n\}$ 中, 令 $h(n)$ 为权重为 n 、且其所有卡牌都具有不同权重的手牌数量。

(a) 证明:

$$\sum_{n \geq 0} \frac{h(n)}{n!} x^n = \prod_{k=1}^{\infty} \left\{ 1 + \frac{d_k}{k!} x^k \right\}.$$

(b) 令 p_n 为 n 字母的排列中其所有循环长度都互不相同的概率。证明:

$$p_n = \prod_{k=1}^n \left\{ 1 + \frac{x^k}{k} \right\}.$$

(c) 如果 (x) 表示上述 (b) 部分中的生成函数, 通过在乘积中插入额外的因子 $-x^k/k$, 确定当 $x \rightarrow 1^-$ 时 (x) 的增长率。

16. 令数字 $\{c_n\}$ 由下式定义:

$$x^x = 1 + \sum_{n \geq 1} \frac{c_n}{n!} (x-1)^n.$$

证明每个 c_n 都是 n 的整数倍, 并且实际上它是 $n(n-1)$ 的倍数, 当且仅当 $n-1$ 能整除 $(n-2)!$ 。

17. 在这里我们要证明：在某种意义上，第一类和第二类斯特林数互为逆矩阵。

在前者的生成函数 (3.10) 中，将 x 替换为 $1/x$ ，并与后者的生成函数 (1.36) 进行比较。将这两个函数相乘，使得困难的部分相互抵消。读出余下部分中 x^n 的系数，并将其陈述为一个断言：即某一对各自包含斯特林数的矩阵，互为彼此的逆矩阵。

18. 令 a_n 为 n 个顶点的、其每个连通分量都是一条路径 (path) 或一个循环 (cycle) 的无向无标号图的数量。令 $F(x)$ 为序列 $\{a_n\}$ 的 opsgf (普通幂级数生成函数)。找出 $F(x)$ ，并将其用整数 n 的划分数序列 $\{p(n)\}$ 的欧拉 opsgf 来表示。

19. 令 a_n 为 n 个顶点的无标号有根树中，根的度数恰好为 2 (即恰好有 2 条边与根相连) 的数量。令 (x) 为计算所有 n 顶点有根树的序列 $\{t_n\}$ 的 opsgf。证明：

$$\sum_n a_n x^{n-1} = \frac{1}{2} ((x)^2 + (x^2)).$$

20. 找出不能写成 $x + 10 + 15$ (其中 $x, ,$ 为非负整数) 形式的最大整数。证明你的答案是正确的，也就是说，证明你的整数不能被这样表示，并且证明大于它的任何整数都可以被这样表示。

21. 在一个只有 1 分、2 分和 3 分硬币的国家，证明将 n 分钱找零的方法数恰好等于最接近 $(n+3)^2/12$ 的那个整数。

22. 这个练习发展了指数公式的一个相当大的锐化 (sharpening) 版本，该公式将在 4.7 节中再次使用。

(a) 在一个指数族中，包含恰好 a_1 张权重为 1 的卡牌，和 a_2 张权重为 2 的卡牌，以及 a_3 张权重为 3 的卡牌，等等，且 $a_1 + 2a_2 + \dots = n$ 的，权重为 n 的手牌数量，是展开式

$$\left\{ \sum_{i \geq 1} \frac{x_i d_i^i}{i!} \right\}$$

中 $(n x_1^{a_1} x_2^{a_2} \dots) / n!$ 的系数。

(b) 令 $f(n, r, s)$ 为集合 $[n]$ 的划分数，它恰好包含 r 个大小为 1 的类和恰好 s 个大小为 2 的类 (无论它们可能有多少其他大小的类)。证明：

$$\sum_{n,r,s} f(n, r, s) x^r s^s \frac{n!}{n!} = \left(x + \frac{2}{2} + -1 - -\frac{2}{2} \right).$$

4 生成函数的应用

(Applications of Generating Functions)

4.1 生成函数寻找平均值等

(Generating Functions Find Averages, etc.)

幂级数生成函数极其适合用来寻找分布的均值、标准差以及其他矩 (moments)，而且只需极少的工作量。假设在某个包含 N 个对象的集合 S 中，恰好拥有 n 个特定属性的对象数量是 $f(n)$ ，对于每个 $n = 0, 1, 2, \dots$ ，且 $\sum_n f(n) = N$ 。集合 S 中的一个对象**平均**拥有多少个属性？显然它是

$$= \frac{1}{N} \sum_n n f(n). \quad (4.1)$$

假设我们碰巧足够幸运，手里正好握着序列 $\{f(n)\}$ 的普通幂级数生成函数 (opsgf)，比方说 $F(x)$ $\{f(n)\}$ 。有没有什么方便的方法能用 F 来表示 (4.1) 式中的均值 呢？当然有。很明显， $= F'(1)/F(1)$ 。所以，平均值可以直接从生成函数计算出来。

让我们来看下一个矩，即该分布的**标准差 (standard deviation)**。它的定义如下：

$$^2 = \frac{1}{N} \sum_S (n() -)^2, \quad (4.2)$$

其中 $\text{}$ 代表集合 S 中的一个对象，而 $n()$ 是 $\text{}$ 所拥有的属性数量。 2 (即该分布的**方差 / variance**) 因此是每个对象拥有的属性数量与平均属性数量 之间差值的平方的平均值。

每一个恰好拥有 n 个属性的对象 (共有 $f(n)$ 个这样的对象)，都将为 (4.2) 中的求和贡献 $(n -)^2$ ，因此

$$\begin{aligned} ^2 &= \frac{1}{N} \sum_n (n -)^2 f(n) \\ &= \frac{1}{N} \sum_n (n^2 - 2n + ^2) f(n) \\ &= \frac{1}{N} \{(xD)^2 - 2(xD) + ^2\} F(x)_{x=1} \\ &= (F(1) + (1 - 2)F'(1) + ^2 F(1))/F(1) \\ &= F(1)/F(1) + F'(1)/F(1) - (F'(1)/F(1))^2 \\ &= \{(1F)'\} + (1F)\}_{x=1}. \end{aligned} \quad (4.3)$$

所以，标准差同样可以通过 F 及其前两阶导数在 $x = 1$ 处的值计算出来。

让我们在**指数族 (exponential families)** 中把它算出来。在一个指数族 中，一手权重为 n 的手牌中卡牌的平均数量 (n) 是多少？

如果 $h(n, k)$ 是权重为 n 且包含 k 张卡牌的手牌数量，那么平均值是

$$(n) = \frac{1}{h(n)} \sum_k k h(n, k). \quad (4.4)$$

现在，如果我们从指数公式开始：

$$\sum_{n,k} h(n,k) \frac{x^n}{n!} k = (x),$$

我们要做的就是应用算子 $\frac{d}{dx}$ ，然后令 $x = 1$ 。结果是

$$\sum_n \frac{x^n}{n!} \sum_k k h(n,k) = (x)' = (x)(x). \quad (4.5)$$

定理 4.1 在一个指数族中，权重为 n 的手牌中卡牌的平均数量是

$$\begin{aligned} (n) &= \left[\frac{h(n)x^n}{n!} \right] (x)(x) \\ &= \frac{1}{h(n)} \sum_r \binom{n}{r} d_r h(n-r). \end{aligned} \quad (4.)$$

例 4.2 (排列的循环 / Cycles of permutations)

如果在包含所有排列的族中 $h(n) = n!$ ，那么求均值的关系式 (4.6) 会变得特别让人高兴。在那里，(4.6) 式变成了

$$\begin{aligned} (n) &= \frac{1}{n!} \sum_r \binom{n}{r} (r-1)!(n-r)! \\ &= 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}. \end{aligned}$$

因此， n 字母排列中循环的平均数量就是调和数 (harmonic number) H_n 。

那标准差是多少呢？在排列的情况下，(4.3) 中出现的函数 $F(x)$ ，对于固定的 n ，是

$$F(x) = \sum_k h(n,k) x^k = x(x+1)(x+2) \cdots (x+n-1),$$

根据 (3.10)。在取对数并求导之后，遵循 (4.3)，我们发现 $F(1) = n!$ ， $(1F)'(1) = H_n$ ，以及

$$(1F)(1) = -1 - 1/4 - 1/9 - \cdots - 1/n^2.$$

如果我们把这个代入 (4.3)，我们发现 n 字母排列上循环分布的方差是

$$\begin{aligned} \sigma^2 &= H_n - 1 - 1/4 - 1/9 - \cdots - 1/n^2 \\ &= 1/n + 1/n^2 + (1). \end{aligned}$$

其中 γ 是欧拉常数。

因此，循环的平均数量是 $\sim \ln n$ ，标准差是 $\sim \sqrt{\ln n}$ 。

4.2 筛法（容斥原理）的生成函数学视角

(A Generatingfunctionological View of the Sieve Method)

筛法 (sieve method) 1 是组合数学中最强大的通用工具之一。它在大多数离散数学的教材中都有解释，然而，它最常以一连串对二项式系数的交替求和 (alternating sums) 的操作的形式出现。在这里，我们将强调一个事实：**生成函数可以极大地简化筛法使用者的生活。**

1 又名“容斥原理 (the principle of inclusion-exclusion)”，通常缩写为“p.i.e.”。

我们给定一个包含 N 个对象的有限集合，以及一个属性 (properties) 集合 P ，这些对象可能拥有或不拥有这些属性。2 在这种背景下，我们想要回答诸如此类的问题：有多少个对象没有任何属性？有多少个对象恰好拥有 r 个属性？对象拥有的平均属性数量是多少？等等，等等。

2 严格来说，一个属性仅仅是对象集合的一个子集，但在实践中，我们通常会对属性有简单的语言描述。

筛法能够处理的问题的特征风味 (characteristic flavor) 是：尽管很难看出有多少对象**恰好**拥有 r 个属性，但看出有多少对象**至少 (at least)** 拥有某组特定的属性 (也许更多) 相对容易。

该方法所做的，就是将“至少”的信息转化为“恰好”的信息。

为了了解它是如何工作的，如果 $S \subseteq P$ 是一个属性的集合，令 $N(S)$ 为**至少**拥有 S 中属性的对象数量。也就是说， $N(S)$ 是其拥有的属性集**包含 (contains)** S 的对象的数量。

对于固定的 $r \geq 0$ ，考虑以下求和：

$$N_r = \sum_{|S|=r} N(S). \quad (4.7)$$

引入符号 $P()$ 表示对象 拥有的属性集合。那么我们可以将 N_r 写成如下形式：

$$\begin{aligned} N_r &= \sum_{|S|=r} N(S) \\ &= \sum_{|S|=r} \sum_{S \subseteq P()} 1 \end{aligned} \quad (4.)$$

$$\begin{aligned} &= \sum_{|S|=r} \sum_{S \subseteq P()} 1 \\ &= \sum \binom{|P()|}{r}. \end{aligned}$$

因此，**每一个恰好拥有 r 个属性的对象都会对 N_r 贡献 $\binom{|P()|}{r}$ 。**

如果有 n_r 个对象恰好拥有 r 个属性，那么 (4.8) 就简化为

$$N_r = \sum_{\geq 0} \binom{\quad}{r} \quad (r = 0, 1, 2, \dots). \quad (4.)$$

回顾一下该方法的哲学理念： N_r 比 r 更容易计算，因为它们可以从 (4.7) 中求出。然而， r 才是我们想要的。因此，我们期望能够解出这些方程，用 N 's 来表示 's。但是我们该怎么做呢？毕竟，(4.9) 是一个包含许多方程的联立方程组。

乍一看，这似乎是一个难以完成的任务，但如果有一个友好的生成函数在你身边，这就变得很容易了。令 $N(x)$ 和 (x) 分别表示序列 $\{N_r\}$ 和 $\{r\}$ 的 opsgf。3 方程组 (4.9) 暗示了这两个生成函数之间有什么关系？

3 字母“N”和“E”旨在让人联想到 N_r 和单词“Exactly（恰好）”。

将 (4.9) 乘以 x^r 并对 r 求和。我们得到

$$\begin{aligned} N(x) &= \sum_r \sum \binom{\quad}{r} x^r \\ &= \sum \left\{ \sum_r \binom{\quad}{r} x^r \right\} \\ &= \sum (x + 1) \\ &= (x + 1). \end{aligned} \quad (4.10)$$

在生成函数的语言中，方程组 (4.9) 归结为 $N(x) = (x + 1)$ 这一事实。现在，用 N 's 来解出 's 的问题变得轻而易举，解显然就是

$$(x) = N(x - 1) \quad (4.11)$$

这就是筛法。在生成函数 $N(x)$ 中将变量 x 替换为 $x - 1$ 的行为，就把未经过滤的数据 $\{N_r\}$ 替换为了筛选后的数量 $\{r\}$ 。

如果 N 's 是已知的，那么原则上我们就可以把 's 作为 $N(x - 1)$ 的系数读出来。

例如， $_0$ 是没有任何属性的对象数量。根据 (4.11)，

$$_0 = (0) = N(-1) = \sum (-1) N. \quad (4.12)$$

通过观察 (4.11) 两边 x^j 的系数，很容易就能找出所有 $_j$ 的显式公式。结果是

$$_j = \sum (-1)^{-j} \binom{\quad}{j} N. \quad (4.13)$$

但是 (4.11) 以一种更简洁得多的方式说明了这一切。

我们现在将总结筛法，然后给出一些它使用的例子。

4.2.1 筛法 (The Sieve Method)

- (a) **(找出 和 P)** 给定一个计数问题，寻找一个对象集合和属性集合，使得如果我们知道了每种属性数量对应的对象数，该问题就能得到解决。
- (b) **(找出未过滤的计数 $N(S)$)** 对于每个属性集合 S ，找出 $N(S)$ ，即其属性集合包含 S 的对象数量。
- (c) **(求出系数 N_r)** 对于每个 $r \geq 0$ ，通过将 $N(S)$ 在所有包含 r 个属性的集合 S 上求和来计算 N_r ，如在 (4.7) 中那样。
- (d) **(答案就在这里)** 数字 N_r 就是多项式 $N(x-1)$ 中 x 次幂的系数。

在我们接触一些例子之前，我们想指出数字 N_1 扮演着一个特殊的角色。根据 (4.9)， $N_1 = \sum$ 。然而，这恰恰就是如果你试图计算对象拥有的**平均属性数量**时你想要知道的东西。因此，记住这一点是个好主意：**当在一个包含 N 个对象的集合上使用筛法时，一个对象拥有的平均属性数量就是 N_1/N 。**

例 4.3 (排列的不动点 / The fixed points of permutations)

在 n 个字母的 $n!$ 个排列中，有多少个恰好拥有 r 个不动点？

筛法的步骤 (a) 要求我们说明什么是对象集合，什么是属性集合。把这些说得非常明确是值得的。在目前的情况下，对象集合 就是 n 个字母的所有排列构成的集合。有 n 种属性：对于每个 $i = 1, \dots, n$ ，如果 i 是 的一个不动点，即如果 $(i) = i$ ，那么一个排列 就拥有属性 i 。有了对 和 P 的这些定义，我们确实很想知道恰好拥有 r 个属性的对象数量，对于每个 r 。

在步骤 (b) 中，我们必须找出 $N(S)$ 。因此设 S 是一个属性集合。那么 $S \subseteq [n]$ 是一组字母的集合，我们想知道有多少个 n 字母排列使得至少 S 中的这些字母保持固定不动。

如果一个排列让 S 中的字母保持固定，那么它就只能在剩余的 $n - |S|$ 个字母上自由作用，所以有 $(n - |S|)!$ 个这样的排列。因此

$$N(S) = (n - |S|)!.$$

对于步骤 (c) 我们计算 N_r 。但是，对于每个 $r = 0, \dots, n$ ，

$$N_r = \sum_{|S|=r} N(S) = \sum_{|S|=r} (n - |S|)! = \binom{n}{r} (n - r)! = \frac{n!}{r!}.$$

在步骤 (d) 中，我们准备好迎接答案了。如果我们引入截断指数级数 (truncated exponential series) 的缩写 $|$ ，能省点笔墨：

$$|(x) = \sum_{r=0}^n \frac{x^r}{r!}. \quad (4.14)$$

现在我们用刚找到的 N_r 来构造 opsgf $N(x)$ ：

$$N(x) = \sum_{r=0}^n \frac{n!}{r!} x^r = n! \sum_{r=0}^n \frac{x^r}{r!}.$$

那么 就是 $N(x-1)$ 中 x 的系数，即

$$(x) = \sum x = n! \sum_{r=0}^n \frac{(x-1)^r}{r!} = n! {}_n(x-1). \quad (4.15)$$

作为额外的红利， n 字母排列拥有的**平均**不动点数量是

$$\frac{N_1}{N} = \frac{n!}{n!} = 1.$$

平均而言，一个排列拥有一个不动点。

完全没有不动点（即错排）的排列数量是

$$_0 = (0) = N(-1) = n! {}_n(-1) \sim \frac{n!}{e}. \quad (4.1)$$

最后，如果我们真的想要一个 (n) 的公式，从 (4.15) 很容易就能找出

$$\begin{aligned} &= \frac{n!}{1!} {}_n(-1) \\ &\sim -1 \frac{n!}{1!} \quad (n). \end{aligned} \quad (4.17)$$

例 4.4 (排列中 k -循环的数量 / The number of k -cycles in permutations)

固定正整数 n, k 和 $r \geq 0$ 。在 n 个字母的排列中，有多少个恰好拥有 r 个长度为 k 的循环？

不管答案是什么，它至少应该要有礼貌地在 $k = 1$ 时还原为上一个例子的答案，因为不动点就是一个长度为 1 的循环。

对象和属性是什么？显然 (n) 是 n 个字母的所有排列构成的集合。进一步，属性集合 P 是从 n 个字母中选出的所有可能的 k -循环的集合。有多少个这样的 k -循环呢？ k 个字母可以通过 $\binom{n}{k}$ 种方法选出，并且它们可以按 $(k-1)!$ 种方法排列成一个循环，所以我们面对的是 $\binom{n}{k}(k-1)!$ 个属性构成的列表。

从 P 中选择一个 k -循环的集合 S 。有多少个排列**至少**拥有 S 中的属性呢？一个也没有，除非那些循环中的字母集合是**两两不相交 (pairwise disjoint)** 的。如果这些集合是两两不相交的，那么有 $N(S) = (n - k|S|)!$ 个排列至少拥有所有那些 k -循环。

接下来我们计算 N_r ，即 $N(S)$ 在所有由 r 个属性构成的集合 S 上的总和。这个和式中的各项要么是 0，要么是 $(n - kr)!$ 。所以我们实际上只需要知道其中有多少项不是 0，也就是说，我们有多少种方法可以从 n 个字母中选出 r 个 k -循环，使得这些循环作用在不相交的字母集合上。

第一个循环的字母有 $\binom{n}{k}$ 种选法，它们可以按 $(k-1)!$ 种方法排列成循环。第二个循环的字母可以有 $\binom{n-k}{k}$ 种选法，并按 $(k-1)!$ 种方法排列，等等。最后，由于构造这些循环的顺序是无关紧要的，我们要除以 $r!$ 。因此

$$\begin{aligned}
N_r &= \frac{(n - kr)!}{r!} \frac{n!(k - 1)!^r}{(k!)^r (n - kr)!} \\
&= \frac{n!}{k^r r!} \quad (0 \leq r \leq n/k).
\end{aligned} \tag{4.1}$$

在这里我们不用做任何额外的工作就能得到答案的一小部分： n 字母排列中 k -循环的**平均**数量是 $N_1/n! = 1/k$ 。

序列 $\{N_r\}$ 的 opsgf (普通幂级数生成函数) 是

$$\begin{aligned}
N(x) &= n! \sum_{0 \leq r \leq n/k} \frac{x^r}{k^r r!} \\
&= n! {}_{|(n/k)} \left(\frac{x}{k} \right).
\end{aligned} \tag{4.1}$$

最后，在筛选步骤中，我们通过将 x 替换为 $x - 1$ 来把它转换成精确信息，得到

$$(x) = n! {}_{|(n/k)} \left(\frac{x - 1}{k} \right). \tag{4.20}$$

例 4.5 (第二类斯特林数 / Stirling numbers of the second kind)

第二类斯特林数 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ ，我们在 1.6 节中研究过它，它是将一个包含 n 个元素的集合划分为 k 个类的方法数。如果我们能发明一套合适的对象和属性集合，我们就能用筛法找出关于它们的信息。

对于对象集合，我们取将 n 个**有标号**的球放入 k 个**有标号**的盒子里的所有 k^n 种安排方法。进一步，如果盒子 i 是空的 ($i = 1, \dots, k$)，那么这样一种安排就具有属性 P_i 。那么 $k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 就是恰好**没有任何属性** (即没有空盒子) 的对象数量。

设 S 是某个属性集合。有多少种球在盒子里的安排**至少**拥有属性集合 S 呢？如果 $N(S)$ 是那个数量，那么 $N(S)$ 计算的是将 n 个有标号的球仅仅放入 $k - |S|$ 个有标号的盒子里的安排数，因为所有被 S 标记的盒子都必须是空的。

显然有 $(k - |S|)^n$ 种这样的安排。因此

$$N(S) = \begin{cases} (k - |S|)^n & \text{如果 } |S| \leq k; \\ 0, & \text{其他.} \end{cases}$$

如果我们现在对所有包含 r 个属性的集合 S 求和，对于 $r \leq k$ ，我们得到

$$N_r = \binom{k}{r} (k - r)^n,$$

它的 opsgf 是

$$N(x) = \sum_{0 \leq r \leq k} \binom{k}{r} (k - r)^n x^r.$$

我们现在可以调用筛法来发现：恰好有 $\binom{n}{k}$ 个空盒子的安排数量是 $N(x-1)$ 中 x 的系数。另一方面，恰好有 $\binom{n}{k}$ 个空盒子的安排数量显然也是

$$\binom{n}{k} (k-1)! \left\{ \begin{matrix} n \\ k-1 \end{matrix} \right\} = \frac{k!}{1!} \left\{ \begin{matrix} n \\ k-1 \end{matrix} \right\}.$$

结果得到了这个恒等式：

$$\sum_{0 \leq k \leq n} \binom{n}{k} (k-1)^n (x-1)^k = k! \sum_{0 \leq k \leq n} \left\{ \begin{matrix} n \\ k-1 \end{matrix} \right\} \frac{x^k}{k!}. \quad (4.21)$$

如果我们令 $x = 0$ ，我们就再次得到了显式公式 (1.38)。

另一方面，如果我们将 (4.21) 与寻找两个 egf 乘积系数的法则 (2.15) 进行比较，我们发现了一个非凡的恒等式：

$$\sum_{1 \leq k \leq n} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} k = - \sum_{r \geq 1} \frac{r^n}{r!} r. \quad (4.22)$$

这表明 $-$ 乘以这个无穷级数竟然是一个多项式！当 $n = 1$ 时的特例我们已经在 (1.41) 中注意到过了。

例 4.6 (棋盘上的车 / Rooks on chessboards)

对于固定的 n ，棋盘 $[n] \times [n]$ 的一个子集。给定 S ，我们定义一个序列 $\{r_k\}$ 如下： r_k 是在 S 上放置 k 个互不攻击（即不在同一行或同一列）的“车（rooks）”的方法数。

接着，令 π 是 n 个字母的排列。对于每个 j ，我们令 a_j 表示“恰好在 j 个方格中遇到棋盘 S ”的排列数量，也就是说，如果事件 $(i, \pi(i)) \in S$ 对于恰好 j 个 i ($1 \leq i \leq n$) 的值发生。

问题是，我们如何能用 r_k 来表达 a_j 呢？

令对象集合 $\mathcal{A}$ 为 $[n]$ 的 $n!$ 个排列。每一个属于 $\mathcal{A}$ 的方格 s 都会对应一个属性 $P(s)$ 。如果排列 π 遇到了仅由单个方格 s 组成的微型棋盘，那么它就具有属性 $P(s)$ 。

令 S 为一个属性的集合，也就是 $\mathcal{A}$ 中的一组方格，并考虑求和 $N_k = \sum_{|S|=k} N(S)$ 。在 S 上布置 k 个互不攻击的车的每一种安排，都会为这个和贡献 $(n-k)!$ 。确实，当集合 S 对应于那些可以放置车的方格时，我们看到的是：在一个排列的 n 个值中，有 k 个在 S 中至少命中了 k 个方格。该排列的剩余部分可以在其余的 $n-k$ 行中以 $(n-k)!$ 种方式完成。

因此，对于每个 k , $0 \leq k \leq n$, $N_k = r_k (n-k)!$ 。因此

$$N(x) = \sum_k (n-k)! r_k x^k,$$

我们立刻发现，在 $\mathcal{A}$ 中恰好命中 j 个方格的 n 字母排列数量是

$$[x^j] \sum_k (n-k)! r_k (x-1)^k. \quad (4.23)$$

例 4.7 (关于子集的一个问题 / A problem on subsets)

这个例子与其说有多深刻，不如说很可爱 (cute)，但我们至少将以一个有趣恒等式的组合证明作为结尾，同时也说明了筛法中生成函数那一面的作用。

对于一个固定的正整数 n ，取我们的对象集合 为：从 $[2n]$ 中选择一个包含 n 个元素的子集的 $\binom{2n}{n}$ 种方法。对于属性集合 P ，我们选取包含 n （注意不是 $2n$ ）个属性的如下列表：对于每个 $i = 1, 2, \dots, n$ （注意我们只处理 S 的可能元素的前一半），如果 i ，那么 n -子集 就拥有属性 i 。

如果 S 是一个属性集合（即，是从 $[n]$ 中选出的字母的集合），那么至少拥有该组属性的（即，至少缺少了 S 中所有的 i 的）“对象” 的数量显然是

$$N(S) = \binom{2n - |S|}{n}.$$

因此

$$N_r = \sum_{|S|=r} N(S) = \binom{n}{r} \binom{2n-r}{n}.$$

如果我们将这些 N 's 代入筛法公式 (4.11)，我们发现

$$\sum_j j^j = \sum_r \binom{n}{r} \binom{2n-r}{n} (-1)^r. \quad (4.24)$$

这个公式告诉我们：恰好拥有 j 个属性的对象数量 j ，对于每个 j 是多少。

但是我们根本不需要它来告诉我们！

恰好拥有 j 个这些属性的对象，也就是 $[2n]$ 中恰好缺少元素 $1, 2, \dots, n$ 中 j 个元素的子集。显然，恰好有 $\binom{n}{j}^2$ 个这样的子集，因为我们可以用 $\binom{n}{j}$ 种方法选出它们缺少的 j 个元素，然后我们也可以用 $\binom{n}{j}$ 种方法从 $n+1, \dots, 2n$ 中选出填满该子集所需的其余 j 个元素。

因此，在没有任何筛法帮助的情况下，我们早就知道了对于所有的 j ， $j = \binom{n}{j}^2$ 。因此，根据 (4.24)，下面这个恒等式必定成立：

$$\sum_j \binom{n}{j}^2 j = \sum_r \binom{n}{r} \binom{2n-r}{n} (-1)^r. \quad (4.25)$$

因此，我们得到了一种奇特的关于恒等式 (4.25) 的组合学证明。读者应该怀疑，每当一个恒等式一边是围绕原点 ($= 0$) 的展开式，另一边是围绕 $= 1$ 的展开式时，这种事情可能正在发生。

4.3 轻松搞定组合恒等式的“万金油”法

(The ‘Snake Oil’ Method for Easier Combinatorial Identities)

组合数学里充满了令人眼花缭乱的恒等式。光是涉及二项式系数的恒等式，就能写满一本又一本的教科书和参考书（关于一些参考文献，见下文）。对于一个在一线工作的离散数学家来说，能够求出或者化简那些看起来错综复杂的、包含组合数的和式是一项非常出色的技能，因为它们总是在图论、算法、枚举等相关的问题中冒出来（而且它们本身也很有趣！）。

在过去，为了能够根据正确的场合拿出正确的工具，你必须建立起一个包含特定技巧的武器库，自然是越多越好。然而，近些年来，大量的系统化工作已经展开，现在有了**统一的方法（unified methods）**来处理上面提到的那些庞大“恒等式军团”中的很大一部分子集。

在本节中，我们将做两件事。首先我们将给出一个单一的方法（**万金油 / Snake Oil 方法**4），它利用生成函数来处理组合求和的计算。这一种方法就能处理包含二项式系数的种类繁多的和式，但在这一点上，二项式系数并没有什么特别的。该方法在它的局限范围内，同样极其优美地适用于包含其他组合数的和式。其哲学思想粗略地说是这样的：**不要试图去计算你正看着的那个特定的和式。相反，去寻找它们整个参数化家族的生成函数，然后再读出生成函数的系数。**

4 《兰登书屋英语词典》将“snake oil（蛇油/万金油）”定义为一种声称能包治百病的灵丹妙药，并给出了一个例句：“州长承诺减税，但那不过是同一副老旧的万金油（the same old snake oil）”。该表达的出现日期被标注为“1925–30年，美国”。

其次，我们要坦白，万金油并不能治好所有的病。有些组合求和真的是非常难的。许多最难的二项式系数求和现在都可以由计算机使用有理函数的方法（rational functions）来证明，我们接下来将讨论这个。不仅如此，计算机的使用还带来了一些经典恒等式的新证明。这些证明的显著特点是：(a) 它们与之前已知的证明相比非常短，(b) 对读者来说，它们似乎极度缺乏动机

(unmotivated)，但是 (c) 没有任何遗漏，它们确确实实是证明。计算机化的证明技术依赖于一个看起来极其简单的观察，我们将对其进行描述和说明。

因此，在本节中你可以期待看到一个能够处理大量相对简单的和式的统一方法，以及另一个能够处理更多种类二项式系数和式的统一方法，甚至包括一些极其困难的和式。

首先我们来谈谈万金油方法。

它的基本思想是我可能称之为处理恒等式的**外部方法（external approach）**，而不是通常的内部方法（internal method）。

为了解释这两种观点的不同，假设我们想证明某个涉及二项式系数的恒等式。通常这样的东西会断言：某个看起来相当吓人的和式，实际上等于 n 的某某简单的函数。

一种现在很流行的方法（这要归功于 Knuth 在 [Kn] 中，以及 Graham、Knuth 和 Patashnik 在 [GKP] 中熟练的阐述和巧妙的处理）主要是探视求和号的内部（“内部地/internally”），并使用二项式系数恒等式或对求和式内部的索引进行其他操作，以将和式变成可处理的形式。

我们即将讨论的方法是对内部方法的一种补充。在我们这里推销的**外部的（external）**或者说**生成函数学的（generatingfunctionological）**方法中，第一步就是快速瞥一眼求和号内部的表达

式，时间长到足够认出“自由变量 (free variables)”就行了。也就是说，在这个和式的所有“哑变量 (dummy variables)”被求和之后，这个和式还依赖于什么。假设这样一个自由变量被称为 n 。

那么，与其试图与这个和式进行缠斗，不如干脆把它全部扫到地毯底下，方法如下：

4.3.1 使用“万金油”法计算组合和式

(a) 识别出该和式所依赖的自由变量，比方说 n 。给你正在处理的这个和式起个名字；称它为 $f(n)$ 。

(b) 设 $F(x)$ 为其 $[x^n]$ 的系数正好是 $f(n)$ 的 opsgf (普通幂级数生成函数)，那个你梦寐以求想要计算出的和式。

(c) 将和式乘以 x^n ，并对 n 求和。你的生成函数现在被表达为了一个对 n 和对某个最初作为求和哑变量的变量的**双重求和 (double sum)**。

(d) **交换 (Interchange)** 你现在所看到的这两个求和的顺序，并将内层求和算成简单的闭合形式。为了达到这个目的，如果你手头有一份其和已知的级数目录 (比如本书第 2.5 节中的列表)，那将会非常有帮助。

(e) 尝试识别出答案 (生成函数) 的系数，因为这些系数正是你想要找到的东西。

如果这听起来很复杂，只需等你看了接下来的七个例子就明白了。到那时，它就会显得极其常规了。

该方法的成功取决于步骤 (d) 和 (e) 的有利结果。令人惊讶的是它的高成功率。它还具有一个“优势”，那就是它几乎不需要任何思考；当它有效时，你知道它有效；当它无效时，那也是显而易见的。

我们将严格遵守关于二项式系数和求和变量范围的惯例。这些是：首先，如果 $m < 0$ 或者如果 x 是一个小于 m 的非负整数，则二项式系数 $\binom{x}{m}$ 消失 (即等于 0)。其次，对于一个没有特别指明范围限制的求和变量，理解为从 $-$ 加到 $+$ 。因此对于整数 $n \geq 0$ ，我们有

$$\sum_k \binom{n}{k} = 2^n,$$

在这个意义上，和式覆盖了所有的正数、负数和 0 值的 k ，除了 $0 < k < n$ 之外被加项 (summand) 全都会变成 0，并且该和具有如上宣称的值。这些惯例将省去在更改作为求和哑变量的变量时，由于烦人地改变求和界限所带来的无尽麻烦。例如，我们发现

$$\sum_k \binom{n}{r+k} x^k = x^{-r} \sum_k \binom{n}{r+k} x^{r+k} = x^{-r} \sum_s \binom{n}{s} x^s = x^{-r} (1+x)^n,$$

对于非负整数 n 和整数 r ，在此过程中根本不用去想求和变量的范围。

在接下来的例子中，最有帮助的级数求值公式，首要的是

$$\sum_{r \geq 0} \binom{r}{k} x^r = \frac{x^k}{(1-x)^{k+1}} \quad (k \geq 0), \quad (4.2)$$

这基本上就是 (2.39) 的重写。另外同样有用的是二项式定理

$$\sum_r \binom{n}{r} x^r = (1+x)^n \quad (4.27)$$

以及 (2.42)，为了方便参考我们把它重写在这里：

$$\sum_n \frac{1}{n+1} \binom{2n}{n} x^n = \frac{1}{2x} (1 - \sqrt{1-4x}). \quad (4.2)$$

例 4.8 (开胃小菜 / Openers) 考虑这个和式

$$\sum_{k \geq 0} \binom{k}{n-k} \quad (n = 0, 1, 2, \dots).$$

自由变量是 n ，所以让我们把这个和式称为 $f(n)$ 。把它写成这样：

$$f(n) = \sum_{k \geq 0} \binom{k}{n-k}.$$

OK，现在把等式两边都乘以 x^n 并对 n 求和。你现在已经到达了一般方法的步骤 (c)，你正看着

$$F(x) = \sum_n x^n \sum_{k \geq 0} \binom{k}{n-k}.$$

准备好进入步骤 (d) 了吗？**交换求和顺序**，得到

$$F(x) = \sum_{k \geq 0} \sum_n \binom{k}{n-k} x^n.$$

我们想“做”那个内层的和式，也就是对 n 求和的那个。技巧是**让 x 的指数刚好与出现在二项式系数中的下标完全相同**。在这个例子中， x 的指数是 n ，而 n 以 $n-k$ 的形式卷入了二项式系数的下方部分 (downstairs part)。为了让它们相同，正确的“药方”是在和式内部乘以 x^{-k} ，在内层求和外部乘以 x^k ，以作补偿。结果是

$$F(x) = \sum_{k \geq 0} x^k \sum_n \binom{k}{n-k} x^{n-k}.$$

现在 x 的指数和出现在二项式系数下方的表达式一样了。因此，把 $r = n - k$ 当作内层和式新的求和哑变量。我们于是发现

$$F(x) = \sum_{k \geq 0} x^k \sum_r \binom{k}{r} x^r.$$

我们立刻认出内层的和式就是 $(1+x)^k$ 。因此

$$F(x) = \sum_{k \geq 0} x^k (1+x)^k = \sum_{k \geq 0} (x+x^2)^k = \frac{1}{1-x-x^2}.$$

右边的这个生成函数是一位老朋友了；它生成的是**斐波那契数列**（见 1.3 节）。因此 $f(n) = F_{n+1}$ ，于是我们发现：

$$\sum_{k \geq 0} \binom{k}{n-k} = F_{n+1} \quad (n = 0, 1, 2, \dots).$$

例 4.9 (再来一个 / Another one) 考虑和式

$$\sum_k \binom{n+k}{m+2k} \binom{2k}{k} \frac{(-1)^k}{k+1} \quad (m, n \geq 0). \quad (4.2)$$

有没有可能同样的方法不需要再注入任何独创性（ingenuity）就能搞定这个和式呢？**没错；只要往上面泼够多的万金油，它就会被治好。** 令 $f(n)$ 表示有问题的那个和式，令 $F(x)$ 为其 opsgf。二话不说，立即开始乘以 x^n 并对 $n \geq 0$ 求和，得到：

$$\begin{aligned} F(x) &= \sum_{n \geq 0} x^n \sum_k \binom{n+k}{m+2k} \binom{2k}{k} \frac{(-1)^k}{k+1} \\ &= \sum_k \binom{2k}{k} \frac{(-1)^k}{k+1} x^{-k} \sum_{n \geq 0} \binom{n+k}{m+2k} x^{n+k} \\ &= \sum_k \binom{2k}{k} \frac{(-1)^k}{k+1} x^{-k} \sum_{r \geq k} \binom{r}{m+2k} x^r \\ &= \sum_k \binom{2k}{k} \frac{(-1)^k}{k+1} x^{-k} \frac{x^{m+2k}}{(1-x)^{m+2k+1}} \quad (\text{根据(4.3.1)或(4.2)}) \\ &= \frac{x^m}{(1-x)^{m+1}} \sum_k \binom{2k}{k} \frac{1}{k+1} \left\{ \frac{-x}{(1-x)^2} \right\}^k \\ &= \frac{-x^{m-1}}{2(1-x)^{m-1}} \left\{ 1 - \sqrt{1 + \frac{4x}{(1-x)^2}} \right\} \\ &= \frac{-x^{m-1}}{2(1-x)^{m-1}} \left\{ 1 - \frac{1+x}{1-x} \right\} \\ &= \frac{x^m}{(1-x)^m}. \end{aligned}$$

原始和式现在被揭露了真面目：它是上面这个最后成员中 x^n 的系数。但这不正是 $\binom{n-1}{m-1}$ 吗（再一次根据 4.26 式）！我们得到了答案。想要看这个和式的推广，请参见习题 16。

如果这长长的一串操作看起来非常长，请考虑这样一个事实：**至少每当使用这种方法时，操作的那串套路都是一样的**，而且如果有必要的话，甚至可以通过一些努力，让计算机来帮你完成它！

例 4.10 (一个新发现 / A discovery)

有可能把这个和式

$$f_n = \sum_{k \geq \frac{n}{2}} (-1)^k \binom{n-k}{k} n^{-2k} \quad (n \geq 0) \quad (4.30)$$

写成一个更简单的闭合形式吗？

这个例子展示了这台机器再次发挥作用，并带有一些新的花样。第一步是令 $F = \{f_n\}$ ，然后尝试去寻找生成函数 F 而不是序列 $\{f_n\}$ 。

为了做到这一点，我们将 (4.30) 两边都乘以 x^n 并对 $n \geq 0$ 求和以获得

$$F(x) = \sum_{n \geq 0} x^n \sum_{k \geq \frac{n}{2}} (-1)^k \binom{n-k}{k} n^{-2k}.$$

下一步**毫无悬念地**是交换求和顺序并祈祷好运。为了试图让最内层的求和看起来尽可能干净，一定要把任何只依赖于 k 的因子提取到外层求和中。这产生

$$F(x) = \sum_k (-1)^{k-2k} \sum_{n \geq 2k} \binom{n-k}{k} x^{nn}.$$

现在把注意力集中在 (4.26) 上，试着让内层和式看起来像那样。如果我们在内层和式中 x 和 的幂次是 $x^{n-k}n^{-k}$ ，那么这些指数就会**精确地**匹配二项式系数 $\binom{n-k}{k}$ 的“上层 (upper story)”，所以在一个哑求和变量的替换之后，我们就会直接看到 (4.26) 左边的形式。

因此，我们接下来在内层和式内部乘以 x^{-k-k} ，并在内层和式外部乘以 x^{kk} 。现在我们有

$$\begin{aligned} F(x) &= \sum_k (-1)^k y^{-2k} x^k y^k \sum_{n \geq 2k} \binom{n-k}{k} x^{nn} \\ &= \sum_k (-1)^k x^k y^k \sum_{n \geq 2k} \binom{n-k}{k} x^{nn} y^{-2k} \end{aligned}$$

(问题：为什么在上面第三个等号之后， k 的范围变成了“ $k \geq 0$ ”？)

我们现在将 (4.31) 展开成部分分式 (partial fractions) 来获得和式 (4.30) 的闭合形式。这给出

$$\begin{aligned} F(x) &= \frac{1}{(1 - xx_+)(1 - xx_-)} \\ &= \frac{x_+}{(x_+ - x_-)(1 - xx_+)} - \frac{x_-}{(x_+ - x_-)(1 - xx_-)}, \end{aligned}$$

其中

$$x_{\pm} = \frac{\pm \sqrt{2-4}}{2}.$$

因此，对于 $n \geq 0$ ， x^n 的系数是

$$f_n = \frac{1}{\sqrt{2-4}} \left(\frac{+\sqrt{2-4}}{2} \right)^{n+1} - \left(\frac{-\sqrt{2-4}}{2} \right)^{n+1}.$$

我们现在已经得到了答案，但为了展示“清理工作”的有效性，让我们多花一点时间把答案弄得尽可能整洁。因为答案中随处可见 $\sqrt{2-4}$ ，我们在形式上把 $\sqrt{2-4}$ 替换为 $x + (1/x)$ 。那么

$$\sqrt{2-4} = x - \frac{1}{x},$$

于是我们的公式变成了

$$\sum_{k \leq \frac{n}{2}} (-1)^k \binom{n-k}{k} (x^2 + 1)^{n-2k} x^{2k} = \frac{x^{2n+2} - 1}{x^2 - 1} \quad (n \geq 0).$$

最后，我们记 $x = x^2$ ，就得到了这个非常漂亮的计算结果：

$$\sum_{k \leq \frac{n}{2}} (-1)^k \binom{n-k}{k} (+1)^{n-2k} = \frac{1 - x^{n+1}}{1 - x} \quad (n \geq 0). \quad (4.32)$$

例如，取值 $x = 1$ 时给出

$$\sum_{k \leq \frac{n}{2}} (-1)^k \binom{n-k}{k} 2^{n-2k} = n + 1 \quad (n \geq 0). \quad (4.33)$$

作为最后的一点润色，我们可以读出 (4.32) 中 x^m 的系数，从而发现这样一个有趣的事实：

$$\sum_{k \leq \frac{n}{2}} (-1)^k \binom{n-k}{k} \binom{n-2k}{m-k} = \begin{cases} 1, & \text{如果 } 0 \leq m \leq n; \\ 0, & \text{其他.} \end{cases} \quad (4.34)$$

试试令 $n = 2$ 看看这个恒等式会发生什么。

这里有同一技术的另一个例子。

例 4.11 计算和式

$$f_n = \sum_k \binom{n+k}{2k} 2^{n-k} \quad (n \geq 0). \quad (4.35)$$

想都不用想，令 F 为该序列的 opsgf（普通幂级数生成函数），将 (4.35) 的两边乘以 x^n ，对 $n \geq 0$ 求和，并交换右边的两个求和号。这产生：

$$\begin{aligned}
F &= \sum_k 2^{-k} \sum_{n \geq 0} \binom{n+k}{2k} 2^n x^n \\
&= \sum_k 2^{-k} (2x)^{-k} \sum_{n \geq 0} \binom{n+k}{2k} (2x)^{n+k} \\
&= \sum_{k \geq 0} 2^{-k} (2x)^{-k} \frac{(2x)^{2k}}{(1-2x)^{2k+1}} \quad (\text{根据(4.2)}) \\
&= \frac{1}{1-2x} \sum_{k \geq 0} \left\{ \frac{x}{(1-2x)^2} \right\}^k \\
&= \frac{1}{1-2x} \frac{1}{1 - \frac{x}{(1-2x)^2}} \\
&= \frac{1-2x}{(1-4x)(1-x)} \\
&= \frac{2}{3(1-4x)} + \frac{1}{3(1-x)}.
\end{aligned}$$

现在读出两边 x^n 的系数不过是小事一桩，并能发现答案是：

$$\sum_k \binom{n+k}{2k} 2^{n-k} = \frac{2^{2n+1} + 1}{3} \quad (n \geq 0). \quad (4.3)$$

例 4.12 我们的下一个例子是一个我们**无法**成功将其求值为一个整洁的闭合形式的求和。然而，我们得到的生成函数会非常整洁，而且对于这个家族的求和来说，这也是我们能期望得到的最好的结果了。

这个和式是

$$f_n() = \sum_k \binom{n}{k} \binom{2k}{k}^k \quad (n \geq 0). \quad (4.37)$$

遵循通常的处方。定义 $F(x, y) = \sum_{n \geq 0} f_n() x^n$ 。为了求 F ，将 (4.37) 乘以 x^n ，对 $n \geq 0$ 求和并交换内层和外层的求和号，获得

$$\begin{aligned} F(x, y) &= \sum_k \binom{2k}{k}^k y^k \sum_{n \geq 0} \binom{n}{k} x^n \quad \&= \sum_k \binom{2k}{k}^k y^k x^k \sum_{n \geq k} \binom{n}{k} x^{n-k} \end{aligned}$$

现在因为

$$\sum_k \binom{2k}{k}^k = \frac{1}{\sqrt{1-4}}, \quad (4.3)$$

(根据 (2.43))，我们得到

$$\begin{aligned} F(x, y) &= \frac{1}{(1-x) \sqrt{1 - \frac{4xy}{1-x}}} \quad \text{\tag{4.40}} \quad \&= \frac{1}{\sqrt{(1-x)^3 - 4xy(1-x)}} \end{aligned}$$

对于的一般值，这大概就是我们所能指望的了。有两个特殊的值能让我们走得更远。如果 $x = -1/4$ ，我们发现

$$\sum_k \binom{2k}{k} \binom{n}{k} \left(-\frac{1}{4}\right)^k = 2^{-2n} \binom{2n}{n} \quad (n \geq 0). \quad (4.41)$$

如果 $x = -1/2$ ，那么

$$\begin{aligned} F(x, -1/2) &= 1/\sqrt{1-x^2} \\ &= \sum_m \binom{2m}{m} (x/2)^{2m} \quad (\text{根据(2.5.11)}). \end{aligned}$$

因此我们得到了 **Reed Dawson** 的恒等式：

$$\sum_k \binom{2k}{k} \binom{n}{k} (-1)^k 2^{-k} = \begin{cases} \binom{n}{n/2} 2^{-n} & \text{如果 } n \geq 0 \text{ 是偶数;} \\ 0 & \text{如果 } n \geq 0 \text{ 是奇数.} \end{cases} \quad (4.42)$$

万金油再次获得了胜利。

例 4.13 假设我们有两个复杂的和式，我们想证明它们是相等的。那么生成函数方法（如果有效的话）应该非常容易执行。实际上，我们只需分别找出两个和式的生成函数，然后观察它们是否相同。

假设我们想证明

$$\sum_k \binom{m}{k} \binom{n+k}{m} = \sum_k \binom{m}{k} \binom{n}{k} 2^k \quad (m, n \geq 0)$$

而无需去计算这两个和式中任何一个的具体值。

将左边乘以 x^n ，对 $n \geq 0$ 求和并交换求和顺序，得到

$$\begin{aligned} \sum_k \binom{m}{k} x^{-k} \sum_{n \geq 0} \binom{n+k}{m} x^{n+k} &= \sum_k \binom{m}{k} x^{-k} \frac{x^m}{(1-x)^{m+1}} \\ &= \frac{x^m}{(1-x)^{m+1}} \left(1 + \frac{1}{x}\right)^m \\ &= \frac{(1+x)^m}{(1-x)^{m+1}}. \end{aligned}$$

如果我们将右边乘以 x^n ，等等，我们发现

$$\begin{aligned}
\sum_k \binom{m}{k} 2^k \sum_{n \geq 0} \binom{n}{k} x^n &= \frac{1}{(1-x)} \sum_k \binom{m}{k} \left(\frac{2x}{(1-x)} \right)^k \\
&= \frac{1}{(1-x)} \left(1 + \frac{2x}{1-x} \right)^m \\
&= \frac{(1+x)^m}{(1-x)^{m+1}}.
\end{aligned}$$

因此这两个和式相等，即使我们根本不知道它们具体等于什么！

例 4.14 在组合数学中，有许多**反演公式 (inversion formulas)**，而生成函数提供了一种证明其中许多公式的简单方法。一般来说，反演公式是一种用另一个序列来表示一个序列的关系式，同时还伴随着一个**逆关系 (inverse relation)**，该关系能从构造出来的序列中恢复出原始序列。

我们已经见过几个著名的例子了。其中一个莫比乌斯反演公式，即公式对 (2.68)、(2.69)。另一个是筛法中出现的公式对 (4.9)、(4.13)。为了方便查阅，我们在此重述那一对公式。它声明，如果我们通过以下关系从序列 $\{r\}$ 计算出序列 $\{N_r\}$ ：

$$N_r = \sum_{\geq 0} \binom{r}{r} \quad (r = 0, 1, 2, \dots), \quad (4.43)$$

那么我们就能够通过以下方式恢复（“反演/invert”）原始序列：

$$= \sum_j (-1)^{j-} \binom{j}{j} N \quad (\geq 0).$$

为了再给出一个这样公式对的例子，考虑以下关系式：

$$a_r = \sum_s \binom{r}{s} b_s \quad (r \geq 0), \quad (4.44)$$

它与上面那对公式的不同之处在于，求和是对二项式系数的**下指标 (lower index)** 进行的。我们如何找出与 (4.43) 互逆的关系呢？也就是说，我们如何用 a 's 来解出 b 's？

答案是：我们将两个序列之间的关系 (4.43) **转化**为它们各自的指数生成函数 (egf) 之间的关系，然后我们再对其进行反演。根据 (2.15)，我们有 $A(x) = {}^x(x)$ ，其中 A 和 x 是这两个序列的 egf。因此 $(x) = {}^{-x}A(x)$ ，由此可得

$$b_n = \sum_m \binom{n}{m} (-1)^{n-m} a_m \quad (n \geq 0). \quad (4.45)$$

一种更深层次的反演公式出现在公式 (5.6)、(5.7) 中。

例 4.15 (万金油法 对决 超几何函数 / Snake Oil vs. hypergeometric functions)

许多组合恒等式其实是超几何级数 (hypergeometric series) 理论中恒等式的特例 (我们稍后将简短地解释这句话)。然而, 万金油方法能够极其愉快地处理各种各样**本质上并非关于超几何函数**的恒等式。因此, 这两种方法是互补的。

一个**超几何级数**是指这样一种级数

$$\sum_k$$

其中任意两个连续项的比值是求和变量 k 的一个**有理函数 (rational function)**。那意味着

$$\frac{k+1}{k} = \frac{P(k)}{(k)},$$

其中 P 和 (k) 都是多项式, 这涵盖了很大的范围。许多二项式系数恒等式, 包括本章迄今为止的所有例子, 都属于这种类型。有一些通用的工具可用于处理这类求和, 并且考虑到它们在实践中出现的频率, 这些工具非常重要。有关其中一些工具的讨论, 请参见, 例如, Roy 的文章 [Ro]。

在这个例子中, 我们想强调的是, 万金油方法适用的范围**包含许多非超几何的求和**。例如, 考虑以下和式:

$$f(n) = \sum_k \begin{bmatrix} n \\ k \end{bmatrix}_k,$$

其中方括号 $\begin{bmatrix} n \\ k \end{bmatrix}$ 表示第一类斯特林数, 而 's' 是伯努利数。

现在, 看着这个和式, 至少有一点是清楚的: 它**不是**超几何的。两个连续项的比值绝对不是 k 的有理函数。然而, 万金油方法面对这种情况却面不改色。如果你完全按照之前的方法操作, 你可以定义 $F(x)$ 为序列 $\{f(n)\}$ 的 egf (指数生成函数), 乘以 $x^n/n!$, 对 n 求和, 交换下标, 等等, 并获得:

$$\begin{aligned} F(x) &= \sum_n \frac{f(n)x^n}{n!} \\ &= \sum_n \frac{x^n}{n!} \sum_k \begin{bmatrix} n \\ k \end{bmatrix}_k \\ &= \sum_k \sum_n \begin{bmatrix} n \\ k \end{bmatrix}_k \frac{x^n}{n!} \\ &= \sum_k \left\{ \frac{1}{k!} \left(1 - \frac{1}{1-x} \right)^k \right\} \quad (\text{根据(3.11)}) \\ &= \sum_k \frac{1}{k!} u^k \quad \left(u = 1 - \frac{1}{1-x} \right) \\ &= \frac{u}{u-1} \quad (\text{根据(2.40)}) \\ &= \frac{1-x}{x} \frac{1}{1-x}. \end{aligned}$$

如果我们现在在等式两边读出 $x^n/n!$ 的系数，我们发现那个未知的和式是：

$$\sum_k \begin{bmatrix} n \\ k \end{bmatrix}_k = -\frac{(n-1)!}{n+1} \quad (n \geq 1). \quad (4.4)$$

例 4.16 (万金油方法的适用范围 / The scope of the Snake Oil method)

万金油方法的成功，依赖于给定的要计算的和式中，**存在一个只在一个地方出现的自由变量 (free variable)**。然后，在交换求和顺序之后，我们就能发现可以通过基本幂级数 (4.26) 或 (4.27) 之一来进行求和。

冒着通过向该方法添加各种小花招而稍微削弱其魅力的风险，我们必须指出：在许多重要的情形下，这种对适用范围的限制是很容易克服的。这是因为，当出现一个带有重复多次的自由变量的恒等式时，**这个恒等式往往是一个更一般恒等式的特例，在那个更一般的恒等式中，每次重复出现的自由变量都被替换成了一个不同的自由变量**。在对某个给定问题放弃使用该方法之前，应该探索一下这种可能性。

考虑以下恒等式：

$$\sum_i \binom{n}{i} \binom{2n}{n-i} = \binom{3n}{n}.$$

乍一看，成功实施万金油疗法的可能性似乎很渺茫，因为 n 在被加项中出现了多次。然而，如果我们通过将 n 拆分成不同的自由变量来推广这个恒等式，我们可能会被引导去考虑这个和式：

$$\sum_i \binom{n}{i} \binom{m}{r-i},$$

而这可以毫不费力地用万金油方法计算出来（范德蒙德卷积）。“**证明特例通常比证明一般定理更难**”，这是恒等式学科的一个特征。自由变量的多次出现通常是一个暗示：人们应该尝试寻找一个合适的推广。

4.4 WZ 对证明更难的恒等式

(WZ Pairs Prove Harder Identities)

现在，计算机可以**找到 (find proofs of)** 组合恒等式的证明，包括我们在上一节中使用“万金油”方法证明的大多数恒等式，以及许许多多其他的恒等式。在本节中，我们将说明这是如何做到的。虽然用这种方式**寻找**证明需要比人类愿意付出的工作量多得多的计算量，但计算机完成后的**结果 (result)**，却是一个整洁紧凑的证明。人类通常可以轻松检查这个证明，并且总是可以借助现在个人电脑上众多可用的符号操作程序 (symbolic manipulation programs) 之一来检查它。因

此，根本不需要盲目信任计算机。你可以要求它找出一个恒等式的证明，并且你可以很容易地检查那个证明是否正确。

这些进展在最近才发生，它们肯定会改变我们对待例如二项式系数恒等式的态度。与其将每一个恒等式视为对我们聪明才智的挑战，我们**反而可以要求计算机去寻找一个证明**。它不一定每次都成功，但在绝大多数情况下它会成功。事实上，现在有更强大的方法可用，承诺在某些类别的恒等式中达到 100% 的成功率。

这并不意味着学习万金油方法是浪费时间。有些恒等式万金油能够处理，而 [WZ] 方法（我们即将讨论的）却处理不了。比如上一节的第一个例子 $\sum_{k \geq 0} \binom{k}{n-k} = F_{n+1}$ （对于 $n \geq 0$ ），这就让 WZ 方法束手无策。另一个让 WZ 方法绝望的是 (4.46)，它涉及斯特林数和伯努利数。此外，要使用万金油方法，人们不需要事先知道恒等式的右边是什么；该方法自己会**找到**它。而我们即将描述的 WZ 方法，虽然会**证明（prove）**一个给定的恒等式，但它本身并不会**发现（discover）**这个恒等式。

尽管有这些免责声明，但可以公平地说，该方法非常通用，并且似乎能够以统一的方式处理有史以来发现的一些最棘手的恒等式。

它源于一些完全显而易见的事实，以一种稍微不那么明显的方式串联在一起。假设我们想要证明这样一个恒等式：

$$\sum_k U(n, k) = rhs(n) \quad (n = 0, 1, 2, \dots)$$

成立。我们要做的第一件事就是等式两边同时除以右边（right-hand side, $rhs(n)$ ），以得到**标准形式（standard form）**

$$\sum_k F(n, k) = 1 \quad (n = 0, 1, 2, \dots). \quad (4.47)$$

所以，在标准形式下，我们实际上是在试图证明：对于 $n \geq 0$ ，某个特定的和式与 n 无关（等于 1）。

为了做到这一点，将 (4.47) 中的 n 替换为 $n + 1$ 重写一遍，然后减去 (4.47)，得到：

$$\sum_k \{F(n+1, k) - F(n, k)\} = 0 \quad (n = 0, 1, \dots). \quad (4.4)$$

如果有这样一个好的函数 $G(n, k)$ 满足

$$F(n+1, k) - F(n, k) = G(n, k+1) - G(n, k), \quad (4.4)$$

那不是很有帮助吗？因为如果那样的话，和式 (4.48) 就会发生“**望远镜相消（telescope）**”！详细地说，那将意味着

$$\begin{aligned}\sum_{k=-}^{k=}\{F(n+1, k) - F(n, k)\} &= \sum_{k=-}^{k=}\{G(n, k+1) - G(n, k)\} \\ &= G(n, +1) - G(n, -).\end{aligned}$$

嗯，既然我们都在“许愿”了，为什么不顺便许愿 $G(n, \pm) = 0$ 呢？因为那样的话，令， 我们就会发现 (4.48) 确实是真的！这种推理路线迅速引导出以下结论：

定理 4.17 (Wilf, Zeilberger) [WZ] 设 (F, G) 满足 (4.49)，并假设

$$\lim_{k \rightarrow \pm} G(n, k) = 0 \quad (n = 0, 1, 2, \dots). \quad (4.50)$$

那么恒等式

$$\sum_k F(n, k) = n. \quad (n = 0, 1, 2, \dots)$$

成立。

例 4.18 假设我们想要证明恒等式

$$\sum_k \binom{n}{k} = 2^n \quad (n \geq 0). \quad (4.51)$$

如果我们除以右边，我们发现 (4.47) 中的函数 $F(n, k)$ 是

$$F(n, k) = \binom{n}{k} / 2^n \quad (n \geq 0). \quad (4.52)$$

现在我们需要找到与这个 F 对应的配对函数 $G(n, k)$ 。嗯，它就是

$$G(n, k) = -\frac{\binom{n}{k-1}}{2^{n+1}}. \quad (4.53)$$

这引出了两个问题。这个 G 真的有效吗？它是从哪里来的？

让我们先回答简单的问题。要检查它是否有效，我们需要首先检查 (4.49) 是否成立，在这个例子中，它说的是：

$$2^{-n-1} \binom{n+1}{k} - 2^{-n} \binom{n}{k} = -2^{-n-1} \binom{n}{k} + 2^{-n-1} \binom{n}{k-1}.$$

只需片刻的运算，我们就能确信这是对的。此外，边界条件 (4.50) 很容易验证，这样我们就全都完成了。

定义 4.19 我们说一个恒等式 (4.47) 是被一对函数 (F, G) (“WZ 对 / WZ pair”) 所**认证** (**certified**) 的，如果条件 (4.49) 和 (4.50) 成立。

因此，简单的恒等式 (4.51) 就被 (4.52)、(4.53) 组成的对 (F, G) 认证了。

我们还可以按如下方式进一步削减这套机制的复杂性。事实证明，在我们这里讨论的这类恒等式中，配对函数 G 将总是具有形式 $G(n, k) = (n, k)F(n, k - 1)$ ，其中 (n, k) 是关于 n 和 k 的一个有理函数。因此，与其描述这一对 (F, G) ，**我们只需给出 F 和 G 即可**。但由于 F 就是直接把我们试图证明的恒等式中的被加项除以右边而得来的。所以如果这个恒等式摆在我们面前，那么**有理函数 就是我们需要的唯一额外证书**。

上述简化成立的恒等式类别是：在 (4.47) 形式中，函数 F 具有这样属性的那些恒等式：即

$$\frac{F(n+1, k)}{F(n, k)} \quad \text{和} \quad \frac{F(n, k-1)}{F(n, k)}$$

两者都是 n 和 k 的有理函数。这个类别包含了几乎每一个我们已经遇到过、或者将来会遇到的二项式系数恒等式。

让我们概括一下**对于由一个单一的有理函数 (n, k) 所认证的恒等式的完整证明过程**：

- (a) 给定一个恒等式 $\sum_k U(n, k) = rhs(n) \quad (n = 0, 1, 2, \dots)$ ，同时给定一个有理函数的证明证书 (n, k) 。
- (b) 定义 $F(n, k) = U(n, k)/rhs(n)$ ，对于 $n \geq 0$ 和整数 k 。
- (c) 定义 $G(n, k) = (n, k)F(n, k - 1)$ 。
- (d) 检查条件 (4.49) 和 (4.50) 是否得到满足。
- (e) 检查该恒等式在 $n = 0$ 时是否为真。
- (f) 恒等式的证明现在完成了。

现在这里有一些这种技术在实际应用中的更多例子。请务必花时间使用上面给出的完整证明技术 (a)-(f) 来检查其中的一个或多个。

定理 4.20 $\sum_k (-1)^k \binom{n}{k} \binom{2k}{k} 4^{n-k} = \binom{2n}{n} \quad (n \geq 0)$ 。

证明： 取 $(n, k) = (2k - 1)/(2n + 1)$ 。

让我们按照上面的证明程序，一步一步地检查这个声明。从步骤 (b) 我们发现

$$F(n, k) = (-1)^k \frac{\binom{n}{k} \binom{2k}{k} 4^{n-k}}{\binom{2n}{n}}.$$

从步骤 (c) 我们发现

$$G(n, k) = (-1)^{k-1} \frac{2k-1}{2n+1} \frac{\binom{n}{k-1} \binom{2k-2}{k-1} 4^{n-k+1}}{\binom{2n}{n}}.$$

现在我们知道了这对函数 (F, G) 。在步骤 (d) 中，我们必须首先检查条件

$F(n+1, k) - F(n, k) = G(n, k+1) - G(n, k)$ 是否满足。**在这一点上，如果能有一个符号运算程序可用将会非常有帮助**，因为那样我们只需把函数 F 和 G 敲进去，然后要求它验证条件是否成

立即可。否则，这将是一个需要用纸和笔花上五分钟进行的一个相当乏味的计算，我们就省略它了。

步骤 (d) 的第二部分是检查边界条件

$$\lim_{k \rightarrow \pm \infty} G(n, k) = 0 \quad (n = 0, 1, 2, \dots).$$

然而，这在此时是一个微不足道的事实，因为在这种情况下，不仅极限是 0，而且对于每一个大于 $n + 1$ 的 k 的单值，以及所有小于 1 的 k 的值，函数 $G(n, k)$ **本身就是 0**。

在步骤 (e) 中，我们快速检查并发现 $1 = 1$ ，证明完成。

定理 4.21 $\sum_k (-1)^k \binom{n}{k} / \binom{k+a}{k} = a/(n+a) \quad (n \geq 0)$ 。

证明： 取 $(n, k) = k/(n+a)$ 。

定理 4.22 $\sum_k (-1)^{n-k} \binom{2n}{k}^2 = \binom{2n}{n} \quad (n \geq 0)$ 。

证明： 取 $(n, k) = -\frac{10n^2 - kn + 17n + k^2 - 5k + 7}{2(2n - k + 2)^2}$ 。

在看了过去三个例子之后，读者可能会想知道：**究竟是如何找到这些 (n, k) 的？** 而不仅仅是去验证一个凭空抓出来的 (n, k) 似乎管用。所以我们现在就要讲讲那个故事。这不仅是为了达成眼前的目的，对于一般意义上的符号计算来说，这也是一个非常重要的故事。

问题是这样的：函数 $F(n, k)$ 是已知的，我们想要找到 $G(n, k)$ ，使得恒等式 (4.49) 成立。既然 F 已知，那么 $F(n+1, k) - F(n, k)$ 也就已知了，所以我们不妨把它叫做 $f(n, k)$ 。接下来，观察到在这一刻，索引 n 是一个“沉默合伙人 (silent partner)”。也就是说，我们正在寻找 G ，使得 $f(n, k) = G(n, k+1) - G(n, k)$ 。我们看到 k 是一个活跃的索引，而 n 只是一个参数，因为在整个过程中 n 都保持相同的值。

所以我们不妨完全忽略 n 的出现，并以这种方式陈述问题：**如果 f_k 是给定关于 k （以及其他参数）的函数，我们如何能找到 g_k ，使得对于所有的整数 k ， $f_k = g_{k+1} - g_k$ 呢？**

我们现在还没有问对问题，但我们已经快接近了。照原样问出的这个问题其实是毫无意义的。因为总是存在这样的 g_k ，它恰恰就是 $\sum_{j < k} f_j$ 。例如，取函数 $f_k = k$ 。那么我们可以取 g_k 为 $\sum_{0 \leq j < k} j$ 。

要问对问题，我们必须加上一个条件：**表示 g_k 的那个和式必须能被求成“闭合形式 (closed form)”**。整个这个问题就是关于闭合形式的。什么是闭合形式？粗略地说，它意味着答案应该看起来很顺眼，而且里面没有遗留任何求和号。不过这个概念太模糊了，没法进行操作。所以我们将使用一种将其精确化的方法，事实证明这种方法是富有成效的。

定义 4.23 如果整数 k 的函数 f_k 满足 f_{k+1}/f_k 是 k 的有理函数，那么 f_k 称作一个**超几何项 (hypergeometric term)**。

因此 $k!$ 是一个超几何项。 $(3k+2)!/(5k-)!$ 也是，下面这个也是：

$$(-1)^k 4^{n-k} \frac{\binom{2k}{k}}{\binom{n+k}{k}}.$$

函数 k^k 不是一个超几何项， $\sqrt{k}$ 也不是。函数

$$f_k = \sum_{0 \leq j \leq k} \binom{n}{j}$$

并不明显地是一个超几何项，但它也并非明显地不是一个超几何项。这个表达式被用来定义该函数，但它并没有立即揭示出这个“怪物”的本质。

现在我们准备好提出正确的问题了。

设 f_k 对整数 k 有定义，并且是一个超几何项。是否存在一个超几何项 g_k ，使得对于所有整数 k 都有 $f_k = g_{k+1} - g_k$ ？

一种归功于 **R. W. Gosper, Jr. [Gos]** 的算法为这个问题给出了完整的算法解答。也就是说，如果我们把 f 输入给 Gosper 算法，它要么会返回一个具有所需性质的函数 g ，要么会返回一个没有任何这种函数存在的保证。**它永远不会返回“我不知道”的声明。**我们在这里不会描述 Gosper 算法，因为那会使我们偏离生成函数这个主题太远。但是，强烈建议读者查阅原始参考文献 [Gos] 或者在 [GKP] 中那极其清晰的解释。

Gosper 算法已经被内置到了一些商业化的符号计算软件包中。该算法在 Macsyma 中被完全实现，在那里它可以通过 `nusum` 命令被调用，在 Mathematica 中也被部分实现。毫无疑问，随着它的用处逐渐被认识到，它将被更广泛地提供。

现在，这里是另外两个更一般、更困难的恒等式的陈述和证明，它们使用了 [WZ] 有理函数认证证明法。

定理 4.24 (Pfaff-Saalschütz 恒等式)

$$\sum_k \frac{(a+k)!(b+k)!(c-a-b+n-1-k)!}{(k+1)!(n-k)!(c+k)!} = \frac{(a-1)!(b-1)!(c-a-b-1)!(c-a+n)!(c-b+n)!}{(c-a-1)!(c-b-1)!(n+1)!(c+n)!}$$

证明： 取

$$(n, k) = -\frac{(b+k)(a+k)}{(c-b+n+1)(c-a+n+1)}.$$

定理 4.25 (Dixon 恒等式)

$$\sum_k (-1)^k \binom{n+b}{n+k} \binom{n+c}{c+k} \binom{b+c}{b+k} = \frac{(n+b+c)!}{n!b!c!}.$$

证明： 取

$$(n, k) = \frac{(c+1-k)(b+1-k)}{2(n+k)(n+b+c+1)}.$$

4.5 生成函数与单峰性、凸性等

(Generating Functions and Unimodality, Convexity, etc.)

二项式系数是**单峰序列 (unimodal sequences)**的典型代表。如果一个序列的项上升到一个最大值然后下降，那么这个序列就是单峰的。二项式系数 $\left\{\binom{n}{k}\right\}_{k=0}^n$ 恰好就是这样。如果 n 是偶数，二项式系数的最大值（“众数/mode”）出现在 $k = n/2$ 处；如果 n 是奇数，则出现在 $k = (n \pm 1)/2$ 处。

一般而言，如果存在指标 r, s 使得

$$c_0 \leq c_1 \leq c_2 \leq \cdots \leq c_r = c_{r+1} = \cdots = c_{r+s} \geq c_{r+s+1} \geq \cdots \geq c_n \quad (4.54)$$

那么序列 $c_0, c_1, \dots, c_n$ 就是单峰的。

在组合数学中出现的许多序列都是单峰的。有时很容易证明一个给定序列是单峰的，但有时却非常困难。生成函数可以帮助解决这类问题，尽管它们远非万能药（panacæa）。

一个比单峰性更强的性质是**对数凹性 (logarithmic concavity)**。首先回顾一下，如果只要 $x < y$ 就有 $f((x+y)/2) \geq (f(x) + f(y))/2$ ，那么实轴上的函数 f 就是**凹的 (concave)**。这意味着该函数的图像在其每一条弦的上方凸起。

类似地，如果 $\ln c$ 是关于 n 的凹函数，即

$$(\ln c_{n-1} + \ln c_{n+1})/2 \leq \ln c_n$$

那么一个正数序列 $c_0, c_1, \dots, c_n$ 就是对数凹的。

如果我们对上式的两边取指数，以消除所有的对数，我们发现如果满足以下条件，该序列就是对数凹的：

$$c_{n-1}c_{n+1} \leq c_n^2 \quad (n = 1, 2, \dots, n-1). \quad (4.55)$$

如果在 (4.55) 中我们能将“ $\leq$ ”替换为“ $<$ ”，那么我们将称该序列是**严格 (strictly) 对数凹的**。

命题 4.26 设 $\{c_r\}_0^n$ 为一个正数构成的对数凹序列。那么该序列是单峰的。

证明： 如果该序列不是单峰的，那么它必定有三个连续的成员满足 $c_{r-1} > c_r < c_{r+1}$ ，这与假设的对数凹性相矛盾。

在许多有趣的例子中，生成函数可以帮助证明序列的对数凹性，从而也证明了单峰性。这类结果的来源通常是以下定理的某种变体：

定理 4.27 (牛顿不等式的一个推论) 设 $(x) = c_0 + c_1x + c_2x^2 + \cdots + c_nx^n$ 是一个多项式，它的所有零点(根)都是实数且为负数。那么其系数序列 $\{c_r\}_0^n$ 是严格对数凹的。

为了证明这个定理，我们需要回顾基础微积分中的**罗尔定理 (Rolle's theorem)**。它指出：如果 $f(x)$ 在 (a, b) 内连续可导，且如果 $f(a) = f(b)$ ，那么在 a 和 b 之间的某处，导数 f' 必定为零。

如果 f 是一个多项式，这个定理可以被相当程度地强化。设 u 和 v 为 f 的两个连续的、互不相同的零点。那么根据罗尔定理，在 (u, v) 内存在 f' 的一个零点。假设 f 的次数为 n ，**仅有实数零点**，并且恰好有 r 个互不相同的实数零点。那么罗尔定理可以解释 f' 的 $r - 1$ 个零点，因为我们在 f 的每对连续且不同的零点之间都找到了一个。

f 剩下的 $n - r$ 个零点是那些不同零点的“副本 (copies)”。但是，如果 x_0 是 f 的一个重数为 $m > 1$ 的根，那么 $(x - x_0)^m$ 就是 f 的一个因子，因此 $(x - x_0)^{m-1}$ 就是 f' 的一个因子。因此 x_0 是 f' 的一个重数为 $m - 1$ 的零点。这解释了 f' 的另外 $n - 1 - (r - 1) = n - r$ 个零点。特别地，**如果 f 的所有零点都是实数，那么 f' 的所有零点也都必定是实数。** 为了在接下来的讨论中发挥最大效用，我们将这种讨论总结如下：

引理 4.28 设

$$f(x,) = c_0x^n + c_1x^{n-1} + \cdots + c_nx^0 \quad (4.5)$$

为一个多项式，其所有的根 $x/$ 都是实数。设 $g(x,)$ 为对 f 连续对 x 和 求导若干次的结果。如果 g 不恒为零，那么它的所有零点也都是实数。

证明 (定理 4.27)： 由于 f 的零点全都是负的，我们有

$$f(x) = c_0 + c_1x + \cdots + c_nx^n = \prod_{j=1}^n (x + x_j), \quad (4.57)$$

其中 x_j 都是正实数。因此没有一个 c_i 能为零。现在将微分算子 $D_x^m D_{n-m-2}$ 应用于 (4.56) 中的齐次多项式 $f(x,)$ 。那么只有三项能存活下来，即：

$$\frac{c_{n-m-2}(m+2)}{n-m-1}x^2 + 2c_{n-m-1}x + \frac{(n-m)c_{n-m-2}}{m+1}. \quad (4.5)$$

我们可以通过记 $c_j = \binom{n}{j}$ (即提取二项式系数，转化为所谓的牛顿系数)，将这个式子写成更干净的形式。在这种情况下，结果 (4.58) 变成了：

$$\binom{n}{m+1} ({}_{n-m-2}x^2 + 2{}_{n-m-1}x + {}_{n-m-2}).$$

但是，根据上面的引理 4.28，这个二次多项式必定有两个实数根，因此它的**判别式 (discriminant)** 必定是非负的，也就是说，

$${}_{n-m-1}^2 \geq {}_{n-m-2}{}_{n-m},$$

这说明 的序列是对数凹的。如果我们把 c 代回去，我们发现

$$c_{n-m-1}^2 \geq \frac{(m+2)(n-m)}{(m+1)(n-m-1)} c_{n-m-2} c_{n-m} > c_{n-m-2} c_{n-m},$$

从而确立了严格对数凹性。

推论 4.29 二项式系数序列 $\left\{\binom{n}{k}\right\}_{k=0}^n$ 是对数凹的，因此也是单峰的。

证明： 生成多项式 $(1+x)^n$ 的零点显然是实数且为负数。

推论 4.30 第一类斯特林数序列 $\left[n \atop k\right]_{k=1}^n$ 是对数凹的，因此也是单峰的。

证明： 根据 (3.10)，这些斯特林数的 opsgf 是多项式：

$$\sum_j \left[n \atop j\right] x^{j-1} = (x+1)(x+2) \cdots (x+n-1),$$

其零点显然是实数且为负数。

推论 4.31 第二类斯特林数序列 $\left\{n \atop k\right\}_{k=1}^n$ 是对数凹的，因此也是单峰的。

证明： 对于这个我们要稍微费点力气，因为多项式

$$A_n(x) = \sum_j \left\{n \atop j\right\} x^j$$

的零点并不容易找到。然而，它们确实是实数且为负数，这里提供一种方法来证明这一点：由 (1.39) 我们有递推公式

$$A_n() = \{(1+D)\} A_{n-1}() \quad (n > 0; A_0 = 1),$$

它可以被改写为

$$A_n() = (A_{n-1}())' \quad (n > 0; A_0 = 1). \quad (4.5)$$

我们断言，对于每一个 $n = 0, 1, 2, \dots$ ，函数 $A_n()$ 恰好有 n 个零点，除了 $= 0$ 处的一个零点外，其余全是**互不相同、为负的实数 (real, distinct, and negative)**。

这对于 $n = 0$ 是正确的，如果它对于 $0, 1, \dots, n-1$ 为真，那么 (4.59) 以及罗尔定理保证了 $(A_{n-1}())'$ 有 $n-2$ 个互不相同的负数零点，其中在 $A_{n-1}()$ 的每一对相邻零点之间都有一个。在如 (4.59) 中那样乘以 x 之后，我们为 $A_n()$ 得到了 $n-1$ 个互不相同的负数零点，但我们还需要再找一个。然而， $A_{n-1}()$ 当 $x \rightarrow -\infty$ 时显然趋于零。因此，它的导数必定在 $A_{n-1}()$ 的**最左侧 (leftmost) 零点的左边**还有另外一个零点，这样我们就证明完毕了。

这个定理非常强大，但是我们绝对不应该留下这样的印象：单峰性或对数凹性与生成多项式的零点为实数这件事有着什么**必然**的联系。我们知道许多序列是单峰的，但其生成多项式的零点全都分布在单位圆上，并且在角度上相当均匀地分布在圆周周围。在这些情况下，我们的定理将毫无帮助。

例如， n 字母排列 的一个**逆序 (inversion)** 是一对满足 $1 \leq i < j \leq n$ 且 $(i) > (j)$ 的 (i, j) 。一个排列可能拥有在 0 到 $\binom{n}{2}$ 之间的逆序数。众所周知，如果 $b(n, k)$ 是恰好拥有 k 个逆序的 n 字母排列的数量，那么

$$\sum_{k=0}^{\infty} \{b(n, k)\} x^k = (1+x)(1+x+x^2) \cdots (1+x+x^2+\cdots+x^{n-1}). \quad (4.0)$$

生成多项式的零点非常均匀地散布在单位圆周围，因此定理 4.27 的假设被极度地破坏了。尽管如此，该序列依然是单峰的；它在 $k = \binom{n}{2}/2$ 时稳步上升，随后稳步下降。

4.6 生成函数证明同余关系

(Generating Functions Prove Congruences)

在本节中，我们将给出一两个关于生成函数方法在证明组合数之间的**同余关系 (congruences)** 中强大力量的例子。两个生成函数之间的同余，意味着该同余对于它们每一对对应系数都成立。

例 4.32 (第一类斯特林数 / Stirling numbers of the first kind)

在第 3 章中，我们发现第一类斯特林数 $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ 的生成函数是：

$$\sum_k \left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] x^k = x(x+1)(x+2) \cdots (x+n-1). \quad (4.1)$$

假设我们有兴趣寻找某种准则，来决定这些数字的奇偶性 (evenness or oddness)。

如果我们在模 2 的意义下 (modulo 2) 读取 (4.61) 式，它变成了：

$$\begin{aligned} \sum_k \left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] x^k &\equiv x(x+1)x(x+1) \cdots \pmod{2} \\ &= x^{n/2}(x+1)^{n/2}. \end{aligned} \quad (4.2)$$

现在取两边 x^k 的系数并发现：

$$\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right] \equiv \begin{cases} x^k & \text{if } k \equiv n/2 \pmod{2} \\ 0 & \text{otherwise} \end{cases} \pmod{2}$$

定理 4.33 斯特林数 $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ 与二项式系数 $\binom{n/2}{k-n/2}$ 具有相同的奇偶性。特别地，如果 $k < n/2$ ， $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ 是一个偶数。

例 4.34 (另一类斯特林数 / The other Stirling numbers)

在计算集合划分的斯特林数 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 的情况下，我们在 (1.36) 中发现它们具有以下 ops 生成函数：

$$\sum_n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^n = \frac{x^k}{(1-x)(1-2x)\cdots(1-kx)}.$$

再一次，假设我们在模 2 的意义下读取该等式。那么我们会发现：

$$\begin{aligned} \sum_n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^n &= \frac{x^k}{(1-x)^{k/2}} \quad (2) \\ &= x^k \sum_h \binom{k/2 + h - 1}{h} x^h. \end{aligned}$$

现在在全式中提取 x^n 的系数。结果是：

定理 4.35 斯特林数 $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 与以下二项式系数具有相同的奇偶性：

$$\binom{k/2 + n - k - 1}{n - k}. \quad (4.4)$$

4.7 对称群的轮换指标

(The Cycle Index of the Symmetric Group)

我们已经研究过第一类斯特林数，它给出了包含恰好 k 个循环的 n 字母排列的数量。现在，我们将寻找关于排列循环的更为详细的信息。与其仅仅考虑一个排列拥有的循环的**总数**，我们将更感兴趣的是，它拥有的**每一种长度**的循环的数量。

因此，设 $\mathbf{a} = \{a_1, a_2, a_3, \dots\}$ 为一个给定的非负整数序列，使得 $n = a_1 + 2a_2 + 3a_3 + \dots$ 是一个有限数。有多少个 n 字母的排列恰好有 a_1 个长度为 1 的循环，恰好有 a_2 个长度为 2 的循环，等等？对于一个给定的排列，我们将向量 $\mathbf{a} = (a_1, a_2, a_3, \dots)$ 称为该排列的**循环类型 (cycle type)**。它告诉了我们拥有的各种长度的循环的数量。

令 $c(\mathbf{a})$ 表示所求的排列数量，并写下：

$$z_n(\mathbf{a}) = \sum_{\substack{a_1 + 2a_2 + \dots = n \\ a_1 \geq 0, a_2 \geq 0, \dots}} c(\mathbf{a}) x_1^{a_1} x_2^{a_2} \dots \quad (4.5)$$

那么多项式 $z_n(\mathbf{a})$ 就被称为对称群 S_n 的**轮换指标 (cycle index)**。如果我们能以某种方式求出 $z_n(\mathbf{a})$ ，那么每一个单项式的系数，就是其循环类型为 $\mathbf{a}$ 的 n 字母排列的数量。

我们将要去寻找“宏大 (grand)”的生成函数

$$Z(\mathbf{a}) = \sum_{n=1}^{\infty} z_n(\mathbf{a}) \frac{x^n}{n!} \quad (4.)$$

考虑到它包含了如此大量的信息，这个生成函数将被证明拥有令人惊讶的优雅形式（见下文 (4.69) 式）。

这个推导过程至少在某一个方面将是非同寻常的。最常见的情况是，生成函数是通往寻找某事物**精确公式**道路上的一个中转站。但在本问题中，我们将从寻找 $c()$ 的精确公式开始。然后，我们就能很容易地去验证，它的生成函数确实生成了这个序列。

那么，对于一个给定的 m ，有多少个排列以 m 为其循环类型呢？我们将首先证明一个引理，然后给出答案。

引理 4.36 给定整数 m, a, k 。从 m 个（互不相同的）给定字母中选出 ka 个字母，并将它们排列成 a 个长度为 k 的循环，其方法数为：

$$f(m, a, k) = \frac{m!}{(m - ka)!k^a a!}. \quad (4.7)$$

证明： 首先选出这些字母的一个有序 ka -元组，这有 $m!/(m - ka)!$ 种方法。然后将每 k 个连续字母组成的块排列成一个循环，这就给了我们包含了 a 个循环的集合。然而，我们断言，每一个固定的、由 a 个长度为 k 的循环构成的集合，在这个构造中将恰好出现 $k^a a!$ 次。事实上，该集合将出现在循环列表的每一次排序中（有 $a!$ 种排序）。此外，对于原本 ka -元组中由 k 个连续条目组成的块，在其内部元素进行圆排列（有 k 种可能）时，产生的将是同样的一组 a 个循环，即 k^a 次。

因此，如果我们给定 n 个字母，以及一个非负整数序列 $a_1, a_2, \dots$ 满足

$$a_1 + 2a_2 + 3a_3 + \dots = n,$$

那么用这些字母组成 a_1 个 1-循环和 a_2 个 2-循环，等等，其方法数显然是：

$$\begin{aligned} & f(n, a_1, 1)f(n - a_1, a_2, 2)f(n - a_1 - 2a_2, a_3, 3) \dots \\ &= \left(\frac{n!}{(n - a_1)!1^{a_1}a_1!} \right) \left(\frac{(n - a_1)!}{(n - a_1 - 2a_2)!2^{a_2}a_2!} \right) \dots \\ &= \frac{n!}{a_1!a_2! \dots 1^{a_1}2^{a_2}3^{a_3} \dots}, \end{aligned} \quad (4.)$$

这里使用了引理 4.36。这产生出了计算每一种给定循环类型的排列数量的明确公式。

定理 4.37 设 $\{j a_j\}$ 为满足 $\sum_j j a_j = n$ 的非负整数序列。那么其循环类型恰好为 $\{j a_j\}$ 的 n 字母排列的数量是：

$$c() = \frac{n!}{\prod_{j \geq 1} (j^{a_j} a_j!)}.$$

接下来，我们将寻找数量 $c()$ 的生成函数。由于存在无穷多个变量，所以当我们需要一个包含无穷多个变量的生成函数时，我们不应该感到惊讶。我们计算：

$$\begin{aligned}
(,) &= \sum_{n \geq 0} \frac{n!}{n!} n \\
&= \sum_{n \geq 0} \frac{n!}{n!} \sum_{\substack{a_1+2a_2+\dots=n \\ a_1 \geq 0, a_2 \geq 0, \dots}} c() \\
&= \sum_{n \geq 0} \frac{n!}{n!} \sum_{\substack{a_1+2a_2+\dots=n \\ a_1 \geq 0, a_2 \geq 0, \dots}} c() x_1^{a_1} x_2^{a_2} \dots \\
&= \left(\sum_{a_1 \geq 0} \frac{(x_1)^{a_1}}{1^{a_1} a_1!} \right) \left(\sum_{a_2 \geq 0} \frac{(x_2)^{a_2}}{2^{a_2} a_2!} \right) \dots \\
&= x_1^2 x_2 / 2^3 x_3 / 3 \dots \\
&= \left(\sum_{j \geq 1} \frac{x_j^j}{j} \right).
\end{aligned}$$

我们已经证明了以下结果。

定理 4.38 在下面的公式中，

$$(,) = \left(\sum_{j \geq 1} \frac{x_j^j}{j} \right) \quad (4.)$$

$n/n!$ 的系数就是 S_n 的轮换指标，即上面 (4.65) 式中生成 n 个字母拥有每一种可能循环类型的排列数的生成函数 $n()$ 。更加详细地说， $n/n!$ 的系数，正是其循环类型为 n 的 n 字母排列的数量。

如果这个结果让你想起了**指数公式 (exponential formula)**，并且如果你怀疑这中间肯定有什么联系的话，那么你是完全正确的。上一章习题 22 的结果正是定理 4.38 向一般指数族

(exponential families) 的推广。实际上，该定理只是那道习题结果的一个直接特例，但我们认为同时给出一个初等的证明也会很有趣。

因此，(4.69) 中的生成函数 $(,)$ 生成了所有对称群的轮换指标。现在我们将给出它在排列的概率理论中的一些应用。

(4.65) 中的多项式 $n()$ ，其系数给出了具有给定循环类型的 n 字母排列的数量。如果我们将它们除以 $n!$ ，如在 (4.66) 中那样，那么由于 $n!$ 是 n 字母排列的总数，我们将得到排列具有各种循环类型向量的**概率 (probabilities)**。因此

$$\begin{aligned}
(,) &= \sum_n \frac{n!}{n!} n \\
&= \sum_n n()^n
\end{aligned} \quad (4.70)$$

其中

$$n() = \sum_{\substack{a_1+2a_2+\dots=n \\ a_1 \geq 0, a_2 \geq 0, \dots}} (, n) \quad (4.71)$$

并且 $(, n)$ 是一个 n 字母的排列具有循环类型 的概率。

[翻译进度提示]

到这里，4.7节的前半部分已经翻译完毕。我们成功得出了极其优美的**对称群轮换指标的生成函数公式 (4.69)**。

在接下来的 4.7 节后半部分，我们将利用这个生成函数，去解答一些让人拍案叫绝的排列概率问题。比如：

- “在所有排列中，完全没有不动点（错排）的概率是多少？”（你会发现答案竟然是 $1/!$ ）
- “在一个随机排列中，没有任何循环长度是某个整数的平方的概率是多少？”（答案将惊人地出现圆周率！）

如果您准备好欣赏这些惊人的结论，请回复“继续”。

太棒了！接下来我们就要进入本书中最令人拍案叫绝、充满数学浪漫气息的部分了！

我们将利用上一节推导出的、那个包含无穷多个变量的**对称群轮换指标（Cycle Index）生成函数**，来回答一些乍一看“无从下手”、连参数 n 都没有的排列概率问题。你会发现，像自然对数底数、甚至圆周率，都会以极其神奇的方式出现在这些纯组合学的答案中！

下面为您翻译 《生成函数学》 第 4 章：4.7 节后半部分的内容。

就在我们的前方，躺着一些非常漂亮的定理。这些定理为一些似乎完全没有包含任何数量的问题，给出了极具**定量（quantitative）**色彩的答案。例如，看看这个问题，它是这类问题的一个简单例证：

一个排列没有任何不动点（fixed points）的概率是多少？

注意，这个问题甚至没有告诉我们要置换多少个字母。问题中根本没有出现“ n ”。但它却有一个不平凡的答案： $1/$ ，正如我们在 (4.16) 中发现的那样。要解释这样的一个问题，一个人应该按以下步骤进行：令 $f(n)$ 为没有不动点的 n 字母排列数量。那么 $f(n)/n!$ 就是一个 n 字母排列没有不动点的概率。由于在这个例子中，极限 $\lim_{n \rightarrow \infty} f(n)/n!$ 存在并且等于 $1/$ ，那么我们就可以在这个极其精确的意义上说：**一个排列没有不动点的概率是 $1/$** 。

关于排列，有许多美妙的问题听起来就像“一个排列具有……性质的概率是多少？”，在这些问题中，排列所作用的字母数量甚至没有被提及，而它们的答案全都是像上面的 $1/$ 这样非平凡的数字。我们马上就要一口气推导出几把这样的答案。但首先我们需要一个引理。

引理 4.39 设 $\sum_j b_j$ 是一个收敛级数。那么在函数

$$\frac{1}{1 - \sum_j b_j} = \sum_n n$$

的幂级数展开式中，我们有极限 l_n 存在，并且等于 $\sum_j b_j$ 。

证明： 根据 2.2 节的法则 5（将一个级数乘以 $1/(1 -)$ 相当于对其系数求前缀和）， n 是所有满足 $j \leq n$ 的 b_j 之和。后者显然趋向于定理所述的极限。

现在，令 S 为一个（有限或无限的）正整数集合，并具有以下属性：

$$\sum_{s \in S} \frac{1}{s} < \infty.$$

在生成函数 $(,)$ 中，对于所有的 $i \in S$ ，我们令 $x_i = 1$ 。这意味着，除了那些属于集合 S 的循环长度之外，我们声明自己对其余任何循环长度都没有兴趣，它们爱是什么长度就是什么长度。那么就变成了

$$\begin{aligned} (,) &= \left(\sum_{i \in S} x_i \frac{1}{i} + \sum_{i \notin S} \frac{1}{i} \right) \\ &= \left(\sum_{i \in S} \frac{(x_i - 1)}{i} + 1 \frac{1}{1 - \sum_{i \in S} \frac{1}{i}} \right) \\ &= \frac{1}{1 - \sum_{i \in S} \frac{1}{i}} \left(\sum_{i \in S} \frac{(x_i - 1)}{i} \right). \end{aligned}$$

根据上面这个引理 4.39，当 $n \rightarrow \infty$ 时，这最后一个表达式中 n 的系数将会趋于极限

$$\left(\sum_{i \in S} \frac{(x_i - 1)}{i} \right),$$

这就证明了以下定理。

定理 4.40 设 S 是一个满足级数 $\sum_{s \in S} 1/s$ 收敛的正整数集合，并设 $\mathbf{x}$ 是一个固定的循环类型向量。一个随机排列的循环类型向量在所有下标属于 S 的分量上都与 $\mathbf{x}$ 完全一致的概率是存在的，并且等于

$$-\left(\sum_{s \in S} \frac{1}{s}\right) \prod_{s \in S} \left(\frac{x_s}{s} \right) = \frac{1}{s^{\sum_{s \in S} s} a_s!}. \quad (4.72)$$

作为一个首要的例子，我们取 $S = \{1\}$ ，所以我们只对**不动点（fixed points）**感兴趣。根据 (4.72)，对于 $a \geq 0$ ，一个随机排列恰好拥有 a 个不动点的概率是 $1/(a!)$ 。

取 $S = \{r\}$ 。我们立刻就能看出，对于每个 $a = 0, 1, \dots$ ，一个随机排列恰好拥有 a 个 r -循环的概率是 $1/(\frac{1}{r} r^a a!)$ 。

令 $S = \{r, s\}$ 。因此，一个随机排列恰好拥有 a_r 个 r -循环和恰好 a_s 个 s -循环的概率是

$$\frac{e^{-1/r-1/s}}{r^{a_r} s^{a_s} r! s!}.$$

好了，现在蒙上你的眼睛，把手伸进一个装着世界上每一个排列的袋子里，随机抓出一个。这个排列的循环长度中，没有一个是某个整数的平方数的概率是多少？

我们断言，这个概率是 $e^{-2} = 0.13025\dots$ ，所以不发生这个事件的赔率大约是 5 比 1。实际上，这正是我们取 S 为所有整数平方数集合的情况，并使用了事实 $1 + 1/2^2 + 1/3^2 + \dots = 2$ 。

作为最后一个例子，一个随机选择的排列，其 1-循环的数量和 2-循环的数量完全相等的概率是多少？嗯，如果那个相等的数量是 j ，那么对于每个 $j = 0, 1, 2, \dots$ ，其概率是 $e^{-3/2}/(2^j j!^2)$ 。如果我们把所有这些 j 的情况加起来，我们发现所求的概率是

$$e^{-3/2} \sum_{j=0}^{\infty} \frac{1}{2^j j!^2} = 0.3444033\dots$$

我们可以使用**泊松分布 (Poisson distribution)** 的语言来重铸定理 4.40 中的结果。泊松分布是定义在非负整数 $j = 0, 1, 2, \dots$ 上的一个概率分布，它极其自然地出现在许多应用领域中，例如在排队论 (theory of waiting lines) 中。它由下式给出：

$$p(j) = \frac{e^{-\lambda} \lambda^j}{j!} \quad (j = 0, 1, 2, \dots)$$

其中 λ 是均值 (mean)。

那么定理 4.40 断言了以下内容：如果固定了一个使得 $\sum_{s \in S} 1/s < \infty$ 的集合 S ，那么对于一个随机选择的排列来说，对于每个 $s \in S$ ，长度为 s 的循环数量 a_s ，渐渐地趋向于相互独立的**泊松分布**，其中 s -循环的平均数量是 $1/s$ 。

4.8 有多少个排列拥有平方根？

(How Many Permutations Have Square Roots?)

设 σ 为一个排列。可能存在、也可能不存在这样一个排列 τ ，使得 $\sigma = \tau^2$ 。在接下来的意义下，我们想要描述并计算出**拥有平方根的那些**。更一般地，如果存在一个 τ 使得 $\sigma = \tau^k$ ，则 σ 有一个 k 次方根；并且同样地，我们想知道 n 个字母的排列中拥有 k 次方根的排列数量。

对这些问题的回答，涉及一些相当壮观的生成函数，且这些方法将极大地依赖于上一节的轮换指标 (cycle index) 结果，特别是定理 4.38。

我们从平方根问题开始。那么设 σ 是一个排列，考虑 σ 的单个循环，比如这一个：

3 13 1 7 12

当我们将 σ 进行平方时，那个循环会发生什么？映射 σ^2 将执行排列 σ 两次，所以它把 8 带到 13，把 13 带到 7，等等。因此，我们在这个 σ 的循环上跳跃着前进，每次拜访相隔一个的成员，直到我们回到起点。**一个偶数长度的循环，因此会散架 (falls apart)** 成两个长度为其一半的循环；而一个奇数长度的循环，仍然是一个相同长度的循环，尽管它变成了那个长度上的一个不同循环。

在上面的例子中，所展示的循环破裂成了两个循环，像这样：

$$(13\ 7) \quad \text{以及} \quad (3\ 1\ 12\ 3)$$

一般而言， σ 中长度为 $2m$ 的每个循环，将为 σ^2 贡献**两个**长度为 m 的循环。

因此， σ^2 中一个偶数长度的循环，只能是由 σ 中一个长度是其两倍的循环分裂成两个循环的结果。因此，**如果 σ 有一个平方根，那么对于每一个偶数长度，它所拥有的该长度的循环的数量，都必须是一个偶数。**

反过来，令 σ 为拥有此属性的任何排列。我们断言 $\sigma = \tau^2$ 对于至少一个 τ 成立。事实上，我们甚至可以构造出这样一个 τ （有多少种方法呢？）。为了做到这一点，选取一对相同偶数长度的循环，并将它们像串珠子一样交织在一起变成一个长度为其两倍的单个循环（正如在上面那两个例子中那样），其做法是交替地从一个循环中取一个字母，再从另一个循环中取一个字母。这些交织在一起的循环，就是正在被构造的排列 σ 的所有部分。

那我们如何处理 σ 的奇数循环呢？如果我们给定 σ 中一个奇数长度为 $2m + 1$ 的循环，我们按如下方式将其转换为 σ 中一个相同的奇数长度的循环。设 σ 的给定循环中的字母为：

$$a_1\ a_2\ a_3\ \cdots\ a_{2m}\ a_{2m+1}\ a_1$$

然后我们把它放进 τ 的这个循环中：

$$a_1\ a_{m+2}\ a_2\ a_{m+3}\ a_3\ a_{m+4}\ \cdots\ a_{2m+1}\ a_{m+1}$$

显然这个循环具有这样的性质：如果我们对它进行平方，它将回到 σ 中的原始顺序，这就完成了以下定理的证明（同时为我们提供了一个寻找排列平方根的算法！）。

定理 4.41 一个排列 σ 拥有平方根，当且仅当：对于每个偶数长度， σ 所拥有的那个长度的循环的数量都是偶数。

现在令 $f(n, 2)$ 为拥有平方根的 n 字母排列的数量。我们寻求该序列的生成函数。考虑这样一个排列的循环类型向量 λ 。其偶数下标的成分（即 $a_2, a_4, \dots$ ）必须是偶数，而奇数下标的成分是任意的。

根据定理 4.38，在乘积

$$x_1 x_2^2 / 2 x_3^3 / 3 \cdots$$

中 $x^\lambda / n!$ 的系数，是拥有循环类型 λ 的 n 字母排列的数量。我们在所有我们正在考虑的循环类型上（也就是那些偶数下标条目为偶数的）对这些系数求和，其方法如下：
在上述指数函数的乘积中，令 $x_1 = 1$ ，因为 a_1 的所有值都是被允许的。在第二个指数因子 $x_2^{2/2}$

中，**不要使用整个指数级数**。由于 x_2 只有偶数次幂是被允许的，我们要使用指数级数中由偶数次幂组成的子级数，即 级数，然后再令 $x_2 = 1$ 。接着令 $x_3 = 1$ 。然后使用 $(x_4^4/4)$ 级数并令 $x_4 = 1$ ，以此类推。

结果将是，具有平方根的 n 字母排列数量 $f(n, 2)$ 满足：

$$\begin{aligned} \sum_{n \geq 0} f(n, 2) \frac{t^n}{n!} &= e^t \cosh(t^2/2) e^{t^3/3} \cosh(t^4/4) \end{aligned}$$

因此序列 $\{f(n, 2)\}$ 开头为：

$$1, 1, 1, 3, 12, 0, 270, 10, 1420, \dots$$

推论 4.42 令 (n) 为一个 n 字母排列拥有平方根的概率。那么对于每个 $n = 0, 1, 2, \dots$ ，我们有 $(2n) = (2n + 1)$ 。

证明： 在 (4.73) 中的生成函数具有形式 $1/(1 -)$ 乘以一个关于 的**偶函数 (even function)**。因此 $f(n, 2)/n!$ 是一个偶函数系数序列的第 n 个部分和。

更一般地，概率序列实际上是微弱递减 (weakly decreasing) 的，即

$$(0) = (1) \geq (2) = (3) \geq (4) = (5) \geq \dots$$

这些事实的双射 (bijective) 证明由 Dennis White (私人通信) 发现。

[翻译进度提示]

太令人震撼了！4.7 节后半部分与 4.8 节前半部分的内容已经翻译完毕！生成函数如同魔法般，告诉我们“随机抓取一个排列没有任何循环长度是平方数”的概率竟然是 $^{-2/}$ ，并用奇函数、偶函数的级数奇妙地解开了“有多少排列拥有平方根”的千古难题！

接下来的 4.8 节后半段，作者会将这个结果推广到**任意 k 次方根 (k th roots)** 的情况。

紧接着，我们将进入 **4.9 节 Counting Polyominoes (计算多格骨牌/动物)**，以及 **4.10 节 Exact Covering Sequences (精确覆盖序列)**。

如果您准备好继续，请回复“**继续**”。

好的！接下来我们要将刚才的结论从“平方根”推广到“ **k 次方根 (k th roots)**”，然后我们将踏入组合数学中极其经典、又充满趣味的另外两个主题：**多格骨牌 (Polyominoes) 的计数**以及**精确覆盖序列 (Exact Covering Sequences)**。

下面为您翻译 《生成函数学》 第 4 章：4.8 节后半段至 4.10 节的内容。

现在让我们尝试解决 **k 次方根** 的问题。我们令 $f(n, k)$ 为拥有 k 次方根的 n 字母排列数量，并且我们寻求 $\{f(n, k)\}_{n \geq 0}$ 的 egf (指数生成函数)。这里引入一些额外的符号将会很有帮助。对于一个素数 和一个整数 n ，我们将写下 $(, n)$ 来表示**能整除 n 的 的最高次幂**。接下来，对于一对正

整数 m, k , 我们定义 $((m, k))$ 为

$$((m, k)) = \prod_m (k)^{(m)}.$$

首先, 如果给定了 k , 对于哪些排列, 存在一个排列使得 $=^k$ 成立呢? 定理 4.41 的推广如下:

定理 4.43 一个排列拥有 k 次方根当且仅当: 对于每一个 $m = 1, 2, \dots$, 所拥有的 m -循环的数量, 必定是 $((m, k))$ 的倍数。

为了证明这一点, 设 $=^k$ 是一个 n 字母的排列, 并且假设对于每个 $m = 1, 2, \dots$, 恰好有 m 个长度为 m 的循环。考虑 $=^k$ 中的一个长度为 r 的循环。在 $=^k$ 中, 这贡献了 (r, k) 个长度为 $r/(r, k)$ 的循环。

因此 $=^k$ 中 m 个长度为 m 的循环, 必定来自于 $=^k$ 中长度为 r 的循环, 其中 $r/(r, k) = m$ 。从这个方程 $r = m(r, k)$ 很容易看出, r 必须是 $m((m, k))$ 的倍数。因此 $=^k$ 中所有长度为 m 的循环, 全部来自于 $=^k$ 中长度为 $m((m, k))$ 整数倍的循环。但是 $=^k$ 中的每一个这种循环, 都会在 $=^k$ 中贡献一个 $((m, k))$ 倍数的 m -循环。因此, $=^k$ 中的 m -循环数量必定是 $((m, k))$ 的倍数。

为了证明这也是**充分**的, 令 $=^k$ 为一个满足该条件的排列。我们将构造出 $=^k$ 的一个 k 次方根。固定 m , 并写成 $g = ((m, k))$ 。由于 $=^k$ 中 m -循环的数量是 g 的倍数, 我们可以把它们每 g 个 m -循环绑成一捆, 然后对于每一捆, 我们如下构造出一个长度为 mg 的新的循环: 构造一个圆圈, 并在圆圈上连续标记出 mg 个位置。取出这捆中的第一个 m -循环, 并将其元素安排在标记好的位置上, 连续的元素之间相隔 g 个位置。然后对这捆中的第二个 m -循环做同样的事情, 等等。对每个 m 重复这个过程, 证明完毕。

定理 4.43 归功于 Arnold Knopfmacher 和 Richard Warlimont, 它纠正了本书早期印刷版本在这一讨论中出现的一个错误。

为了获得序列 $\{f(n, k)\}$ 的 egf, 我们按照上面 (4.8.1) [译注: 应为(4.73)段落的推导方式] 中的步骤进行。为那些出现的指数函数的子级数 (subseries) 起个名字会很方便。所以, 对于指数级数 $=^x$ 中, 仅仅选取那些 x 的幂次能被 k 整除的项所组成的子级数, 我们写为 $(x)_k$ 。也就是说

$$(x)_k = \sum_{j \geq 0} \frac{x^j}{(j)_k!} \quad (k = 1, 2, 3, \dots).$$

因此 $_1(x) = =^x$, $_2(x) = =^x$, $_3(x)$ 已在第 2 章方程 (2.30) 中显式给出, 等等。现在遵循引导我们得出 (4.73) 的相同论证, 我们获得了这个推广形式。

定理 4.44 令 $f(n, k)$ 为拥有 k 次方根的 n 字母排列数量。那么我们有

$$\sum_{n=0}^{\infty} f(n, k) \frac{x^n}{n!} = \prod_{m=1}^{\infty} ((m, k)) \left(\frac{x^m}{m} \right) \quad (k = 1, 2, 3, \dots). \quad (4.74)$$

邀请读者去检验一下: 当 $k = 1$ 时, 这退化为一个平凡的等式; 而当 $k = 2$ 时, 这就变成了 (4.73)。下表展示了一小部分 $f(n, k)$ ($1 \leq n \leq 10; 2 \leq k \leq 7$) 的值。

k	n=1	2	3	4	5	6	7	8	9	10
k=2:	1	1	3	12	60	270	1890	14280	128520	1096200
k=3:	1	2	4	16	80	400	2800	22400	181440	1814400
k=4:	1	1	3	9	45	225	1575	11130	100170	897750
k=5:	1	2	6	24	96	576	4032	32256	290304	2612736
k=6:	1	1	1	4	40	190	1330	8680	52920	340200
k=7:	1	2	6	24	120	720	4320	34560	311040	3110400

4.9 计算多格骨牌 (Counting Polyominoes)

在这个语境中，“细胞 (cell)”指的是 x - 平面内一个单位正方形的内部及其边界，前提是该正方形的顶点都位于网格点（坐标都是整数的点）上。令 P 是一组细胞的集合。我们把一个图 (graph) 与 P 关联起来，该图的顶点对应于 P 的细胞，并且如果两个细胞相交于一条线段（而不是相交于一个顶点，或者根本不相交），我们就在图中的两个顶点之间连一条边。如果与 P 关联的图是一个连通图，我们就说 P 是一个**连通的 (connected)** 细胞集合。

如果一个细胞集合 P 所有的细胞都位于第一象限内，且其中至少有一个与 y 轴相交，至少有一个与 x 轴相交，那么 P 就处于**标准位置 (standard position)**。

一个**多格骨牌 (polyomino)** 是一个处于标准位置的、连通的细胞集合。

以下是所有包含一、二或三个细胞的多格骨牌：

(此处是一幅插图：展示了俄罗斯方块中由 1 个、2 个、和 3 个小方块拼成的各种多格骨牌图形)

有时候多格骨牌被称为**动物 (animals)**。这是因为人们可以想象一个单细胞的动物通过在自身的一条边上萌发 (sprouting) 出新细胞来“生长”。然后，那个双细胞的动物又会在其某条边上长出新的细胞，依此类推。如果 $f(n)$ 是包含 n 个细胞的多格骨牌数量，那么从上面的图片我们看到 $\{f(n)\} = 1, 2, \dots$ 。如果能说“在这一节中我们要推导序列 $f(n)$ 的生成函数等等”，那该多好。然而，我们并不打算这么做。**该序列及其生成函数至今依然未知**，尽管人们在这个问题上已经投入了大量的努力。

然而，对于各种特殊类型的多格骨牌，人们已经计算出了它们相对于各种属性的数量。例如，在多格骨牌拥有的属性中，有人可能会提到它的**面积 (area)**，或者细胞数，以及它的**周长 (perimeter)**。所以，人们可能会问具有某种特殊类型的、面积为 n 的多格骨牌数量；或者周长为 m 的数量；或者面积为 n 且周长为 m 的数量，以及这些中任何一个的生成函数等等。关于这类问题最新进展的综述，请参见 [De1] 和 [De2]。

我们在本节要做的是计算一种特殊类型的多格骨牌，被称为**水平凸的** (**horizontally convex**，简称 **HC**) 多格骨牌。一个 HC-多格骨牌是指它的**每一行都由一个单一的、连续的细胞块组成**。下图展示了一个典型的 HC-多格骨牌。

(此处插图：展示了一个每一行都只有连续实心方块的水平凸多格骨牌)

另一种特殊类型的多格骨牌被称为**凸的** (**convex**)。如果一个多格骨牌既是水平凸的也是垂直凸的，那么它就是凸的。在多格骨牌理论中，最引人注目的结果之一是这样一个事实：恰好存在

$$(2n + 11)4^n - 4(2n + 1) \binom{2n}{n} \quad (4.75)$$

个周长为 $2n +$ 的凸多格骨牌。而面积为 n 的凸多格骨牌的准确数量目前仍是未知的。

在根据面积或周长计算 HC-多格骨牌方面，有一些有趣的问题。我们将在这里按**面积**来计算它们。值得注意的是，关于按**周长**枚举它们的问题也已经被解决 [De2]，并且其解涉及一个卓越非凡的生成函数，它看起来像这样：如果 c_n 是周长为 $2n + 2$ 的 HC-多格骨牌数量，那么

$$\sum_{n \geq 0} c_n x^n = \frac{\sqrt{-(A^{1/3} + D + x^{-1/3}) - F}}{2\sqrt{AH}} - \frac{H}{2\sqrt{2A}} - G$$

其中 $A, \dots, H$ 都是某些具体的函数。例如

$$A = 1^4(2^3 - 23^2 + 3 - 1)^2.$$

完整的列表见 [De2]。

我们回到按**面积**计算 HC-多格骨牌的问题，它与 2.2 节中枚举“硬币喷泉”的问题相似，并且我们的攻击方法也将类似。

令 $f(n, k, r)$ 为包含 n 个细胞、排成 k 行、其中顶行恰好包含 r 个细胞的 HC-多格骨牌数量。如果我们从这些多格骨牌之一上剥去顶行，留下来的将会是有 $n - r$ 个细胞，排成 $k - 1$ 行，其中顶行包含某个数量 $r' \geq 1$ 的细胞。因此在移除顶行之后，剩余部分的可能情况有 $f(n - r, k - 1, r')$ 种（对于某个 r' ）。然而，对于那些可能性中的每一种，都能通过在顶部附加一行 r 个细胞，并将它在原第二行上方的所有合法位置**向左和向右滑动**，来生成 $r + 1$ 个原始的 (n, k, r) 型 HC-多格骨牌。

因此我们得到

$$f(n, k, r) = \sum_{r' \geq 1} f(n - r, k - 1, r')(r + 1) \quad (k \geq 2; f(n, 1, r) = \delta_{n,r}). \quad (4.7)$$

如果我们定义生成函数 $F_k(x) = \sum_n f(n, k, r) x^n$ ，那么我们有 $F_1(x) = x$ ，对于 $k \geq 1$ ；且在将 (4.76) 乘以 x^n 并对 n 求和之后，我们获得

$$F_k(x) = x \sum_{r \geq 1} (r + 1) F_{k-1, r}(x) \quad (k \geq 2). \quad (4.77)$$

现在令 $U_k(x) = \sum_{r \geq 1} F_{k,r}(x)$ 且 $k(x) = \sum_{r \geq 1} r F_{k,r}(x)$ 。那么 $U_1(x) = x/(1-x)$ 且 $1(x) = x/(1-x)^2$ 。进一步，由 (4.77)，

$$F_k(x) = x(k_{-1}(x) + (-1)U_{k-1}(x)) \quad (k \geq 2), \quad (4.7)$$

如果我们对 k 求和，我们发现

$$U_k(x) = \frac{x}{1-x} k_{-1}(x) + \frac{x^2}{(1-x)^2} U_{k-1}(x) \quad (k \geq 2). \quad (4.7)$$

[翻译进度提示]

到这里，4.8 节关于排列的任意次方根的问题，以及 4.9 节关于多格骨牌的递推方程建立部分，已经全部翻译完毕！

我们看到，即便是像俄罗斯方块这种复杂的二维“动物 (Animals)”，只要限制条件合适（水平凸），我们就能写出巧妙的生成函数递推关系 ((4.78) 和 (4.79))。

在接下来的 4.9 节剩余部分，我们将通过极其华丽的代数操作，求出这组联立递推方程的生成函数解！接着将进入本章的最后一个小结 **4.10 节 Exact Covering Sequences (精确覆盖序列)**，看生成函数如何在数论的同余方程组中大放异彩。

如果您准备好继续，请回复“继续”。

好的！这绝对是一场脑力激荡的组合数学盛宴。接下来，我们将通过解开那组生成函数的联立递推方程，一窥**水平凸多格骨牌 (HC-Polyominoes)** 生成函数的真面目，并在最后通过**部分分式**求出它优美的渐近公式！

紧接着，我们将进入 **4.10 节 精确覆盖序列 (Exact Covering Sequences)**，看看生成函数如何将一个数论中的同余覆盖问题，完美地转化为**单位根 (roots of unity)** 和**分圆多项式 (cyclotomic polynomials)** 的问题。

下面为您翻译 《生成函数学》 第 4 章：4.9 节后半段至 4.11 节的内容。

如果我们首先将 (4.78) 乘以 x ，然后对 k 求和，我们发现

$$k(x) = \frac{x}{(1-x)^2} k_{-1}(x) + \frac{2x^2}{(1-x)^3} U_{k-1}(x) \quad (k \geq 2). \quad (4.0)$$

现在我们有**两个联立的递推方程 (simultaneous recurrences)** 需要解出序列 U_k 和 k 。为此，我们按照如下方法消去 k 序列：由 (4.79) 解出用 U_k 和 U_{k-1} 表示的 k_{-1} ，并将结果代入 (4.80)。化简之后，我们获得了一个关于 U 的单一的三项递推关系，即

$$\frac{1-x}{x}U_{k+1}(x) - \frac{x+1}{1-x}U_k(x) - \frac{x^2}{(1-x)^3}U_{k-1}(x) = 0 \quad (k \geq 1), \quad (4.1)$$

伴随着初始数据 $U_0(x) = 0$ 和 $U_1(x) = x/(1-x)$ 。

最后，为了解出 (4.81)，我们引入生成函数 $(x,) = \sum_{k \geq 0} U_k(x)^k$ 。然后，如果我们将 (4.81) 乘以 k 并对 $k \geq 1$ 求和，我们得到

$$\frac{1-x}{x}\{(x,) - U_1(x)\} - \frac{x+1}{1-x}(x,) - \frac{x^2}{(1-x)^3}(x,) = 0.$$

如果我们利用初始条件并解出，结果是

$$\begin{aligned} (x,) &= \sum_{k \geq 0} U_k(x)^k = \sum_{n,k,r} f(n, k, r) x^{nk} \\ &= \frac{x(1-x)^3}{(1-x)^4 - x(1-x-x^2+x^3+x^2)}. \end{aligned} \quad (4.2)$$

注意，这里关于 r 的求和并没有附带任何变量；它直接作用于 $f(n, k, r)$ 上，并且产生了包含 n 个细胞和 k 行的 HC-多格骨牌的数量，**而不考虑顶行有多少个细胞**。因此，如果 $g(n, k)$ 是那个数量，那么

$$\sum_{n,k} g(n, k) x^{nk} = \frac{x(1-x)^3}{(1-x)^4 - x(1-x-x^2+x^3+x^2)}. \quad (4.3)$$

关于序列 $\{f(n, k, r)\}$ 完整的三变量生成函数，请参见本章末尾的习题 21。

也许我们**只对** HC-多格骨牌的**总数**感兴趣，而不需要知道包含多少行。在那种情况下，我们在 (4.82) 中令 $= 1$ ，然后我们发现了以下由 D. Klarner 使用不同方法获得的结果。

定理 4.45 令 $f(n)$ 为包含 n 个细胞的 HC-多格骨牌数量。那么

$$\begin{aligned} \sum_{n \geq 1} f(n) x^n &= \frac{x(1-x)^3}{1-5x+7x^2-4x^3} \\ &= x + 2x^2 + x^3 + 1x^4 + 1x^5 + 1x + 2x^7 + 2017x \\ &\quad + 4x + 20727x^{10} + 441x^{11} + 2120x^{12} + \dots \end{aligned} \quad (4.4)$$

我们现在通过计算 $f(n)$ 的**渐近公式 (asymptotic formula)**，来预览一下第 5 章中的材料。为此，我们取 (4.84) 中的生成函数并在部分分式中将其展开。这给出了

$$\frac{x(1-x)^3}{1-5x+7x^2-4x^3} = -\frac{5}{1} + \frac{x}{4} + \frac{c_1}{1-\alpha_1 x} + \frac{c_2}{1-\alpha_2 x} + \frac{c_3}{1-\alpha_3 x},$$

其中 $\alpha_1 = 3.205543\dots$ 且 $\alpha_{2,3} = 0.7215 \pm 0.5457i$ 。

因此 HC-多格骨牌的数量，对于 $n \geq 2$ ，是

$$\begin{aligned} f(n) &= c_1 \alpha_1^n + c_2 \alpha_2^n + c_3 \alpha_3^n \\ &= c_1 \alpha_1^n + O(|\alpha_2|^n) \end{aligned}$$

这个公式的第一项给出的（例如） $f(12) = 2127.1$ ，相比于如 (4.84) 所示的精确值 212980。这极其精确！

4.10 精确覆盖序列 (Exact Covering Sequences)

每一个正整数 n 要么是 1 (2)，要么是 0 (4)，要么是 2 (4)，稍微思考一下就能确认这一点。因此，这三对余数和模数 $(a_1, b_1) = (1, 2)$, $(a_2, b_2) = (0, 4)$ 以及 $(a_3, b_3) = (2, 4)$ **精确地覆盖** (**exactly cover**) 了所有的正整数。

一个**精确覆盖序列 (ECS)** 是一个由非负整数构成的有序对的集合 (a_i, b_i) ($i = 1, \dots, k$)，它具有这样的性质：对于**每一个**非负整数 n ，存在**有且仅有一个** i 使得 $1 \leq i \leq k$ 且 $n \equiv a_i \pmod{b_i}$ 。

在本节中，我们将给出这类序列的基本理论，并用两三种不同的方法处理“如何判断给定的一个序列对是不是一个精确覆盖序列”这个问题。

这里就是生成函数对这一主题贡献。

假设 (a_i, b_i) ($i = 1, \dots, k$) 是一个精确覆盖序列。那么在级数

$$\sum_{i=1}^k \sum_{n \geq 0} x^{a_i + nb_i}$$

中，由于每一个非负整数 n 都会作为 x 的指数**恰好出现一次**，因此这个级数必定等于 $1/(1-x)$ 。如果我们对 x 进行求和，我们发现

$$\sum_{i=1}^k \frac{x^{a_i}}{1 - x^{b_i}} = \frac{1}{1 - x}. \quad (4.)$$

例如

$$\frac{x}{1 - x^2} + \frac{1}{1 - x^4} + \frac{x^2}{1 - x^4} = \frac{1}{1 - x}.$$

定理 4.46 一组数对 (a_i, b_i) ($i = 1, \dots, k$) 成为精确覆盖序列的充要条件是，关系式 (4.86) 成立。

我们立刻可以得出的一个结论是，在任何 ECS 中，必定有 $\sum_i 1/b_i = 1$ 。要看清这一点，只需将 (4.86) 乘以 $1 - x$ 然后令 $x \rightarrow 1$ 即可。但通过比较 (4.86) 左右两边的**部分分式展开式 (partial fraction expansions)**，我们能了解到更多。

对于左边，我们有

$$\sum_{i=1}^k \frac{x^{a_i}}{1-x^{b_i}} = \sum_{N=1} \frac{A()}{-x}$$

其中

$$\begin{aligned} A() &= \frac{1}{x}(-x) \sum_{i=1}^k \frac{x^{a_i}}{1-x^{b_i}} \\ &= \sum_{j: b_j=1} \frac{a_j+1}{b_j}, \end{aligned}$$

并且 $N = 1\{b_j\}$ 。如果我们比较 (4.86) 的右边，我们看到除了 $A(1) = 1$ 之外，所有的 $A()$ 必定都为 0（因为右边 $1/(1-x)$ 只有 $x = 1$ 这一个极点）。因此我们必定有

$$\sum_{j: b_j=1} \frac{a_j}{b_j} = \begin{cases} 1, & \text{如果 } = 1; \\ 0, & \text{其他.} \end{cases}$$

现在，除非 ϵ 是一次单位根，否则 $A()$ 肯定为零。所以设 $\epsilon = \epsilon^{2ir/s}$ 为一个**本原的 (primitive)** s 次单位根，其中 $s \geq 1$ 且 $(r, s) = 1$ 。那么我们的条件采取以下形式：

$$\sum_{j: sb_j} \frac{a_j}{b_j} = \begin{cases} 1 & \text{如果 } s = 1; \\ 0 & \text{其他.} \end{cases} \quad (4.7)$$

因此，与任意数对序列 $\{(a_i, b_i)\}_{i=1}^k$ 相关联，我们可以定义一些多项式：

$$s() = \sum_{j: sb_j} \frac{a_j}{b_j} \quad (s = 1, 2, 3, \dots). \quad (4.)$$

用这些多项式，我们可以如下重述我们的条件 (4.87)：给定的数对集合构成精确覆盖序列的必要且充分条件是，对于每一个 $s > 1$ ，多项式 s 应该在所有**本原的 s 次单位根**处为零（即有一个根），且 $1(1) = 1$ 。

然而，在所有本原 s 次单位根处均为零的任何多项式，**必定能被分圆多项式 (cyclotomic polynomial) 整除**（参见例 2.31）

$$s() = \prod_{\substack{r: (r,s)=1 \\ 0 < r < s}} (-\epsilon^{2ir/s}) \quad (s = 1, 2, 3, \dots)$$

因为 $s()$ 仅仅拥有那些根。前几个分圆多项式是

$$1 - \epsilon, 1 + \epsilon, 1 + \epsilon + \epsilon^2, 1 + \epsilon^2, 1 + \epsilon + \epsilon^2 + \epsilon^3 + \epsilon^4, 1 - \epsilon + \epsilon^2, \dots$$

如果我们把所有这些放在一起，我们就得到了以下结果。

定理 4.47 一组整数对 $(a_1, b_1), \dots, (a_k, b_k)$ （其中 a 为非负数且 b 为正数）是一个精确覆盖序列当且仅当 $\sum_j 1/b_j = 1$ ，并且对于每个 $s > 1$ ，(4.88) 中的多项式 $s()$ 能被分圆多项式 $s()$ 整除。

作为一个例子，取这些数对：

$$(0, 4), (2, 4), (1,), (3,), (5, 12), (11, 12).$$

那么 $\sum_j 1/b_j = 1/4 + 1/4 + 1/ + 1/ + 1/12 + 1/12 = 1$ ，并且定理中的整除性条件看起来像这样：

$$\begin{aligned} {}_2() &= (1 +) \text{整除} \frac{1}{4} + {}^2/4 + / + {}^3/ + {}^5/12 + {}^{11}/12; \\ {}_3() &= (1 + + {}^2) \text{整除} / + {}^3/ + {}^5/12 + {}^{11}/12; \\ {}_4() &= (1 + {}^2) \text{整除} \frac{1}{4} + {}^2/4 + {}^5/12 + {}^{11}/12; \\ () &= (1 - + {}^2) \text{整除} / + {}^3/ + {}^5/12 + {}^{11}/12; \\ {}_{12}() &= (1 - {}^2 + {}^4) \text{整除} {}^5/12 + {}^{11}/12. \end{aligned}$$

这些都能很容易地被检验出是正确的，因此给定的数对是一个 ECS（精确覆盖序列）。

定理 4.47 还有一些推论，其中一些被留作习题了。不过，其中有一个是相当清晰的。如果 $= \{b_j\}$ ，那么 在所有模数 $\{b_j\}$ 中**绝不可能只出现一次**。确实，如果我们在定理中取 $s =$ ，我们发现 $()$ 必定包含足够多的单项式，才能让它被 $()$ 整除，所以它肯定至少有两个单项式。

4.11 等待一个字符串 (Waiting for a String)

给定一个长度为 k 的单词，由 2 个字母的字母表 $\{H, \}$ 组成。我们反复抛掷一枚公平的硬币，从而获得一个由 H 和 组成的序列。字符串 首次出现在第 n 次抛掷开始时的概率是多少？也就是说，如果 $(a_1, a_2, \dots)$ 是抛掷结果的序列，那么使得 $(a_n, a_{n+1}, \dots, a_{n+k-1}) =$ 且对于所有 $j < n$ 都有 $(a_j, a_{j+1}, \dots, a_{j+k-1})$ 的概率是多少？

设 $_n$ 为所求的概率。进一步，令 A_i 表示“从 i 开始的连续 k 次抛掷的块刚好产生单词”这一事件。那么在事件 A_i 已经发生的条件下，事件 A_{i+u} 的概率与 i 无关。我们将后一个概率记为 g_u ，即 $g_u = P(A_{i+u} | A_i)$ 。

单词 出现在位置 n （即事件 A_n ）的概率是 2^{-k} 。事件 A_n 可以根据 首次出现的位置分解为：

$$\frac{1}{2^k} = \sum_{j=1}^n {}_j g_{n-j}, \quad (n = 1, 2, \dots; g_0 = 1) \quad (4.)$$

其中右侧第 j 项表示： 首次出现在时间 j 的概率，乘以，给定它首次出现在时间 j 时，它在时间 n 再次出现的概率。

[翻译进度提示]

太棒了！到这里，4.9 节关于**多格骨牌**（利用部分分式求渐近公式）和 4.10 节关于**精确覆盖序列**

(利用复变函数中的单位根极点) 的内容已经全部翻译完毕!

并且我们已经开启了 **4.11 节 Waiting for a String (等待一个字符串)** 的前奏。这部分是在讲抛硬币游戏中, 某个特定序列 (比如 "HHT") 首次出现的概率与等待时间的生成函数解法。紧接着还有 **4.12 节 Blocks of 1's (1 的区块)** 问题, 探讨长长的 01 串中连续 1 的问题。

如果您准备好了继续听抛硬币问题和 01 字符串问题的精彩解答, 请回复“继续”。

好的, 那我们就继续! 接下来我们将解决 **4.11 节“等待一个字符串”** 的后半部分, 看看如何用生成函数精准算出特定硬币抛掷序列首次出现的“期望等待时间”。

紧接着是 **4.12 节“1 的区块”(Blocks of 1's)**, 这是一个在概率论和计算机科学中非常经典的 01 字符串计数问题: 在长度为 n 、包含 m 个 1 的二进制字符串中, **不包含连续 k 个 1** 的字符串有多少个?

下面为您翻译 **《生成函数学》第 4 章: 4.11 节后半部分至 4.12 节** 的内容。

为了求解递推关系式 (4.89), 引入生成函数 $P(x) = \sum_{j \geq 1} p_j x^j$ 和 $G(x) = \sum_{j \geq 0} g_j x^j$ 。那么 (4.89) 意味着

$$\frac{x}{2^k(1-x)} = G(x)P(x),$$

也就是说,

$$P(x) = \frac{x}{2^k(1-x)G(x)}. \quad (4.0)$$

接下来我们注意到 $(1-x)G(x)$ 实际上是一个多项式, 因为对于所有的 $j \geq k$, 都有 $g_j = 2^{-k}$ 。因此 $G(x) = {}_{k-1}(x) + 2^{-k}x^k/(1-x)$, 其中 ${}_{k-1}$ 是一个次数为 $k-1$ 的多项式。因此 (4.90) 采取如下形式:

$$P(x) = \frac{x}{x^k + 2^k(1-x){}_{k-1}(x)}. \quad (4.1)$$

通过对生成函数求导, 我们能求出该字符串 首次出现的**期望时间 (expected time)** 为:

$$P'(1) = 2^k {}_{k-1}(1) - (k-1). \quad (4.2)$$

作为一个例子, 取字符串 $= HH$ 。此时 $k=2$, 且 $g_0=1, g_1=1/2$ 。因此 ${}_1(x) = 1 + x/2$, 根据 (4.91),

$$P(x) = \frac{x}{x^2 + 2^2(1-x){}_1(x)} = \frac{x}{4 - 2x - x^2} = \frac{x}{4} + \frac{x^2}{4} + \frac{x^3}{32} + \frac{3x^4}{32} + \dots$$

首次出现的期望时间是, 根据 (4.92), $4_1(1) - 1 = -1 = 5$ 。(译注: 也就是说, 平均抛掷 5 次硬币就会首次看到连续两个正面)。

4.12 1 的区块 (Blocks of 1's)

该问题是要确定：在由 n 个比特 (bits) 组成、其中 m 个是 1 的所有字符串中，**不包含任何连续 k 个 1** 的字符串数量 $f(n, m, k)$ 。

4.12.1 公式 (A Formula)

设 s 为这样的一个字符串。从左到右阅读它，并注意 0 和 1 的区块（连续块）的长度。为了具体起见，假设 s 以 1 开始并以 0 结束（其他三种情况类似）。那么当我们解析 s 时，我们会看到一个由 a_1 个 1 组成的区块，然后是一个由 b_1 个 0 组成的区块，然后是 a_2 个 1，接着是 b_2 个 0，以此类推，最后以 a_r 个 1 和 b_r 个 0 结束。

其中， a_i 们是将整数 m 分解为 r 个正数部分的**组合 (composition / 有序拆分)**，其中每个部分都 $< k$ ；而 b_i 们则是将 $n - m$ 分解为 r 个正数部分的组合。

将 $n - m$ 分解为 r 个正数部分的组合数量是 $\binom{n-m-1}{r-1}$ 。让我们将“将 m 分解为 r 个每个都 $< k$ 的正数部分的组合数量”记为 $g(m, r, k)$ ，为此我们显然有

$$\sum_{m \geq 0} g(m, r, k) x^m = (x + x^2 + \cdots + x^{k-1})^r = x^r \left(\frac{1 - x^{k-1}}{1 - x} \right)^r. \quad (4.3)$$

那么，对于这类以 1 开始、以 0 结束且恰好包含 r 个 1 的区块的单词 的数量为 $g(m, r, k) \binom{n-m-1}{r-1}$ ，其中 $r \geq 1$ 。

在算出了其他三种情况（即 1 开 1 结、0 开 0 结、0 开 1 结）之后，单词的总数是

$$\begin{aligned} f(n, m, k) &= m_{,0} + \sum_{r \geq 1} g(m, r, k) \left\{ \binom{n-m-1}{r-1} + \binom{n-m-1}{r-2} + \binom{n-m-1}{r} + \binom{n-m-1}{r-1} \right\} \\ &= m_{,0} + \sum_{r \geq 1} g(m, r, k) \binom{n-m+1}{r}. \end{aligned}$$

4.12.2 生成函数 (The Generating Function)

现在我们可以找到 f 的生成函数了。如果我们将 (4.93) 乘以 x^m 并对 r 求和，我们得到

$$\sum_{m \geq 0, r \geq 1} g(m, r, k) x^{mr} = \frac{(x - x^k)}{1 - x - (x - x^k)}.$$

因此，如果我们将 (4.94) 乘以 x^{nm} 并对 n, m 求和，我们发现

$$\begin{aligned} F_k(,) &= \sum_{n, m \geq 0} f(n, m, k) x^{nm} \\ &= \frac{1}{1 - x} + \frac{1}{(1 - x)} \sum_{r, m} g(m, r, k) x^{mr} \left(\frac{x - x^k}{1 - x} \right)^r \\ &= \frac{1}{1 - x} + \frac{x - x^k}{(1 - x)(1 - x + x^{k+1})}, \end{aligned}$$

或者最终得出

$$\sum_{n,m \geq 0} f(n, m, k)^{nm} = \frac{1 - k^k}{1 - (-1)^{k+1}k}. \quad (4.5)$$

4.12.3 显式公式 (An Explicit Formula)

从生成函数出发，我们现在可以找到 $f(n, m, k)$ 的一个显式公式。我们有

$$\begin{aligned} \frac{1}{1 - (-1)^{k+1}k} &= \frac{1}{1 - \left(\frac{1}{1 - (-1)^{k+1}k} \right)} \\ &= \sum_l \frac{1}{(1 - (-1)^{k+1}k)^{l+1}} (-1)^{kl} \\ &= \sum_l \sum_h \binom{h}{l}^h \sum_j \binom{l}{j}^{l-j} (-1)^{jk-jk-j}. \end{aligned}$$

nm 的系数为

$$c_{n,m} = \sum_{jn/k} (-1)^j \binom{n-kj}{n-m} \binom{n-m}{j}.$$

根据 (4.95)，我们的 $f(n, m, k)$ 是 $c_{n,m} - c_{n-k,m-k}$ ，这就得出

$$f(n, m, k) = \sum_{jn/k} (-1)^j \binom{n-kj}{n-m} \binom{n-m}{j} - \sum_{jn/k-1} (-1)^j \binom{n-k(j+1)}{n-m} \binom{n-m}{j}.$$

在将第二个求和中的 $j+1$ 替换为 j 之后，这可以被化简为

$$f(n, m, k) = \sum_{0 \leq j \leq m/k} (-1)^j \binom{n-m+1}{j} \binom{n-kj}{n-m} \quad (k \geq 1). \quad (4.)$$

4.12.4 递推公式 (A Recurrence)

由于

$$(1 - (-1)^{k+1}k) \sum_{n,m \geq 0} f(n, m, k)^{nm} = 1 - k^k,$$

通过画上等号并对比右侧和左侧单项式的系数，我们可以找到 f 的递推关系。其结果是 f 满足：

$$\begin{aligned} f(n, m, k) &= f(n-1, m, k) + f(n-1, m-1, k) \\ &\quad - f(n-k-1, m-k, k) + f(n-k, m-k, k). \end{aligned} \quad (4.7)$$

[翻译进度提示]

到这里，第 4 章的正文内容已经全部翻译完毕了！

在这一节的结尾 (4.12.5)，书中还延伸讨论了任意一个由 01 组成的字符串中，**1 的最大连续区块的大小的期望值**（一个非常有趣的概率论经典问题）。随后，便是一整页烧脑又精彩的 **4.13 节习题 (Exercises)**。

请问您希望我：

1. 继续翻译本章剩余的 **4.12.5 节和 4.13 课后习题**？
2. 或者直接进入本书解析方法的高潮——**第 5 章：解析与渐近方法 (Analytic and Asymptotic Methods)**？在这里我们将真正开始使用复变函数来解决硬核的渐近规模问题！

如果您准备好继续，请回复“**继续翻译习题**”或“**跳到第5章**”。

好的，让我们把第 4 章画上一个圆满的句号！接下来为您翻译 **《生成函数学》第 4 章：4.12.5 小节以及 4.13 节的全部课后习题**。

这些习题不仅包含了对生成函数计算均值、方差的应用（如优惠券收集问题、树上的随机游走），还包含了进一步使用“万金油 (Snake Oil)”方法和“筛法 (Sieve Method)”解决各种组合求和及排列图论问题的精彩练习。

4.12.5 进一步的展开 (Further Developments)

一个有趣且特殊的情况是计算 $h(n, k)$ ，即在长度为 n 的位串 (bit strings) 中，**最长全 1 区块 (longest block of 1's) 长度 $< k$ 的字符串数量**。为了计算它，我们只需在 (4.95) 中令 $x = 1$ ，然后就能发现：

$$\sum_{n \geq 0} h(n, k)^n = \frac{1 - x^k}{1 - 2x + x^{k+1}}.$$

当然，在 n 比特的字符串中，最长全 1 区块长度**至少为** k 的字符串数量 (n, k) 是 $2^n - h(n, k)$ ，其生成函数 (gf) 是：

$$\sum_{n \geq 0} (n, k)^n = \frac{1}{1 - 2} - \frac{1 - x^k}{1 - 2x + x^{k+1}} = \frac{x^k(1 - x)}{(1 - 2)(1 - 2x + x^{k+1})}.$$

但是，对于任何取值为整数的随机变量，其**期望值 (expected value)** 为：

$$= (\geq 0) + (\geq 1) + (\geq 2) + \dots$$

因此，如果 (n) 是长度为 n 比特的字符串中全 1 的**最大区块的大小 (size)**，那么 (n) 期望值的生成函数为：

$$\sum_{n \geq 0} (n)^n = \left(\frac{1 - x/2}{1 - x} \right) \sum_{k=0}^{\infty} \frac{x^k}{2^k \left(1 - x + \frac{x^{k+1}}{2^{k+1}} \right)}. \quad (4.)$$

4.13 习题 (Exercises)

1. 给定一枚硬币，其抛出“正面 (heads)”的概率为 p ，令 n 表示直到第 n 次抛掷才**首次 (for the first time)** 出现“正面”的概率。计算 n 以及序列 $\{n\}$ 的 opsgf (普通幂级数生成函数)。使用该 opsgf 找出直到首次出现“正面”所需抛掷次数的平均值 (mean) 以及该抛掷次数的标准差 (standard deviation)。

2. 在**优惠券收集者问题 (coupon collector's problem)** 中，我们想象我们想要获得一整套完整的电影明星照片。每次购买一盒麦片，我们都能获得一张这样的照片。当然，这张照片可能与我们的收藏中已有的照片重复。假设一套完整的收藏中包含 d 种不同的照片。令 n 表示为了首次凑齐一套完整的收藏，**恰好**需要购买 n 次的概率。

(a) 证明：

$$n = \frac{d! \begin{Bmatrix} n-1 \\ d-1 \end{Bmatrix}}{d^n},$$

其中 $\begin{Bmatrix} n \\ k \end{Bmatrix}$ 是第二类斯特林数 (见 1.6 节)。

(b) 令 $\{n\}$ 。证明：

$$(x) = \frac{(d-1)!x^d}{(d-x)(d-2x)\cdots(d-(d-1)x)}.$$

(c) 直接从生成函数 (x) 中计算出：为了收集齐所有 d 张优惠券所需的平均尝试次数。

(d) 同样地，使用 (x) 找出尝试次数的标准差。

(e) 在 $d = 10$ 的情况下，为了收集齐所有 10 种不同的照片，你预期需要购买多少盒麦片？

3. (树上的首次返回时间 / First return times on trees)

在图中进行**随机游走 (random walk)**，我们指的是：在图的顶点之间进行游走，当到达某个顶点时，下一步会走向一个完全从该顶点的所有邻居顶点中均匀随机选出的顶点。

如果 T 是一棵树，且 v 是 T 的一个顶点，令 $(j; v)$ 表示在 T 上从顶点 v 开始的随机游走，在恰好 j 步之后**首次返回**到 v 的概率。

现在设 T_1, T_2 是树，令 v_i 为 T_i 的一个顶点 (对于 $i = 1, 2$)，并设 T 是由这两棵树通过添加边 (v_1, v_2) 而形成的树。最后，令 $F_1(x; v_1), F_2(x; v_2), F(x; v_1)$ 分别为序列 $\{(j; v_1)\}_{j \geq 0}$, $\{(j; v_2)\}_{j \geq 0}$, 和 $\{(j; v_1)\}_{j \geq 0}$ 的 opsgf (普通幂级数生成函数)。

(a) 证明：

$$F(x; v_1) = \frac{1}{d_1 + 1} \left\{ d_1 F_1(x; v_1) + \frac{x^2}{d_2 + 1 - d_2 F_2(x; v_2)} \right\},$$

其中 d_i 是树 T_i 中顶点 v_i 的度数，对于 $i = 1, 2$ 。

(b) 令 (a) 表示在一个从 a 开始并在**首次**返回 a 时停止的随机游走中的**平均**步数。通过对 (a) 部分的答案求导，证明：

$$(1) = \frac{1}{d_1 + 1} (2 + d_{1_1}(1) + d_{2_2}(2)).$$

(c) 设树 是一条包含 $n + 1$ 个顶点的**路径 (path)**。证明：如果 是两个端点之一，从 开始的游走的平均返回时间是 $2n$ ，并且对于所有其他的 ，这个时间是 n (是不是有点令人惊讶?)。

(d) 同样，如果 是一条包含 n 个顶点的路径，且如果 $P_n(x)$ 表示 (a) 部分中的生成函数 $F(x; 1)$ ，其中 1 是 的一个端点，那么请找出一个计算 $P_n()$ 的显式公式。

4. 用 $N(x)$ 表示出计算那些**至多**拥有 m 个属性 (分别地，**至少**拥有 m 个属性) 的对象的序列 $\{m\}$ (分别地， $\{\geq m\}$) 的 opsgf。

5. 你应该使用什么样的**棋盘 (chessboard)** 来推导**没有不动点**的排列数量？利用棋盘方法重新推导这个数量的公式。

6. (邦费罗尼不等式 / Bonferroni's inequalities)

在筛法中，方程 (4.12) 计算了没有任何属性的对象数量。假设右边交替出现的级数在某个值 $= m$ 时被截断。证明：如果 m 是偶数，结果将**高估** 0 ；如果 m 是奇数，结果将**低估** 0 。要做到这一点，只需证明序列

$$m = \sum_{r \geq m} (-1)^{r-m} N_r \quad (m = 0, 1, 2, \dots)$$

拥有 opsgf: $0 + x \sum_{r \geq 0} r+1 (x+1)^r$ ，且它的系数显然都是非负的。

7. (邦费罗尼不等式，续)

不仅 0 被其筛法公式中交替的连续部分和时而低估、时而高估，对于每一个 k (即恰好具有 k 个属性的对象数) 来说，情况也是一样。为了用生成函数学的方式来证明这一点，对每个 $k, \geq 0$ ，定义

$$(k,) = (-1)^{+1} \left\{ k - \sum_j (-1)^j \binom{k+j}{j} N_{k+j} \right\}.$$

那么问题就是要证明所有的 $(k,) \geq 0$ 。

要做到这一点，

(a) 令 $(x,)$ 为 序列的双变量 opsgf。然后将 们的定义乘以 x^k ，对 $k, \geq 0$ 求和，并证明

$$(x,) = \frac{(x + (1 +)) - (x)}{(1 +)}.$$

(b) 从此可以得出 们都是非负的，实际上

$$(k,) = \sum_{r > k} \binom{r}{k} \binom{r-k-1}{r} \quad (k, \geq 0).$$

8. 证明：

$$\sum_r \binom{n}{r/2} x^r = (1+x)(1+x^2)^n.$$

然后使用**万金油 (Snake Oil)** 来计算：

$$\sum_k \binom{n}{k} \binom{n-k}{\frac{m-k}{2}} x^k$$

在 $x = \pm 2$ 时（归功于 D. E. Knuth）的显式值。并且无论 m 的值是什么，找到这些和式的生成函数。

9. 设 G 是一个包含 n 个顶点的图，并给定正整数 x 。令 $P(; x; G)$ 表示：为 G 的顶点分配 x 种给定颜色之一，使得 G 中恰好有 x 条边其两个端点颜色相同的方法数。

请将确定 P 表达为一个具有合适对象集合和属性集合的筛法问题。为 $P(; x; G)$ 找出一个公式，并观察到它是一个关于 n 和 x 这两个变量的多项式。图 G 的**色多项式 (chromatic polynomial)** 就是 $P(; 0; G)$ 。

10.

(a) 设 w 是一个在 k 个字母的字母表上的，长度为 m 个字母的单词。假设 w 的任何末尾子字符串都不等于 w 的起始子字符串。请使用筛法来计算：在该包含 k 个字母的字母表上，**不包含**子字符串的长度为 n 个字母的单词数量。

(b) 对你在 (a) 部分中求得的答案所在的那个和式，使用万金油 (Snake Oil) 方法进行计算。

11. 对以下所有的式子使用万金油方法 (Snake Oil method)：

(a) 为以下多项式找出一个不包含求和号的显式公式：

$$\sum_{k \geq 0} \binom{k}{n-k} x^k.$$

(b) 发明一个看起来非常恶心的、包含二项式系数的和式（不在我们在本章所做的范围之内），并以简单的形式计算出它。

(c) 计算：

$$\sum_k \binom{2n+1}{2+2k+1} \binom{n+k}{k},$$

并由此获得一个‘Moriarty identity’。

(d) 证明：

$$\sum_m \binom{r}{m} \binom{s}{-m} = \binom{r+s}{r}.$$

然后计算：

$$\sum_k \binom{n}{k}^2.$$

(e) 证明 (Graham 和 Riordan):

$$\sum_k \binom{2n+1}{2k} \binom{m+k}{2n} = \binom{2m+1}{2n}.$$

(f) 证明对于所有的 $n \geq 0$:

$$\sum_k \binom{n}{k} \binom{k}{j} x^k = \binom{n}{j} x^j (1+x)^{n-j}.$$

(g) 证明对于所有的 $n \geq 0$:

$$x \sum_k \binom{n+k}{2k} \left(\frac{x^2-1}{4} \right)^{n-k} = \left(\frac{x-1}{2} \right)^{2n+1} + \left(\frac{x+1}{2} \right)^{2n+1}.$$

(h) 证明对于 $n \geq 1$:

$$\sum_{k \geq 1} \binom{n+k-1}{2k-1} \frac{(x-1)^{2k} x^{n-k}}{k} = \frac{(x^n-1)^2}{n}.$$

12. 万金油方法不仅对涉及二项式系数的和式有效, 正如本练习所示, 它对各种各样的计数数字都有效。

(a) 设 $\{a_n\}$ 和 $\{b_n\}$ 是两个序列, 其 egf 分别是 $A(x)$, (x) 。假设这些序列通过以下关系连接:

$$b_n = \sum_k \begin{bmatrix} n \\ k \end{bmatrix} a_k \quad (n \geq 0),$$

其中 $\begin{bmatrix} n \\ k \end{bmatrix}$ 表示第一类斯特林数。证明它们的 egf 之间的联系为:

$$(x) = A\left(1 \frac{1}{1-x}\right).$$

(b) 设 b_n 为 $[n]$ 的有序划分数 (ordered partitions, 见 (5.17))。证明:

$$\sum_k \begin{bmatrix} n \\ k \end{bmatrix} b_k = n! 2^{n-1} \quad (n \geq 1).$$

(c) 令 $\{a_n\}$ 为 n 字母的错排 (即无不动点的排列) 数量, 且令 $\{b_n\}$ 像在 (a) 部分中那样定义。证明:

$$\frac{\{b_n\}_{gf}}{1 + 1(1-x)}.$$

(d) 在第二类斯特林数上重复 (a)-(c) 部分的过程，并发现几个你自己包含它们的恒等式。

(e) 将 (a)-(d) 部分推广到指数族。

13. 证明：

$$\sum_k (-1)^{n-k} \binom{2n}{k}^2 = \binom{2n}{n}$$

证明方法是：将其展示为一个具有两个自由参数的和式的一个特例，并在后者的和式上使用万金油方法。

14. 如果要计算一个具有如下形式的和式：

$$S(n) = \sum_k f(k)g(n-k),$$

最自然的方法是识别出 $S(n)$ 是 $[x^n]\{F(x)G(x)\}$ ，其中 F 和 G 分别是 $\{f_n\}$ 和 $\{g_n\}$ 的 opsgf。使用此方法来计算：

$$S(n) = \sum_k \frac{1}{k+1} \binom{2k}{k} \frac{1}{n-k+1} \binom{2n-2k}{n-k}.$$

15.

(a) 证明 (4.25) 的以下推广，并证明它确实是一个推广。对于所有的 $m, n, \geq 0$ ，我们有：

$$\sum_r \binom{m}{r} \binom{n-r}{n-r-r} (-1)^r = \sum_r \binom{m}{r} \binom{n-m}{n-r-r}^r.$$

(b) 雅可比多项式 (Jacobi polynomials) 可以对于 $n \geq 0$ 定义如下：

$$P_n^{(a,b)}(x) = \sum_k \binom{n+a}{k} \binom{n+b}{n-k} \left(\frac{x-1}{2}\right)^{n-k} \left(\frac{x+1}{2}\right)^k.$$

使用 (a) 部分的结果来证明：

$$P_n^{(a,b)}(x) = \sum_j \binom{n+a+b+j}{j} \binom{n+a}{j+a} \left(\frac{x-1}{2}\right)^j.$$

(c) 使用 (b) 部分的结果以及一点点“万金油 (Snake Oil)”，来证明：

$$P_n^{(a,b)}(x) = 2^{-n} (x-1)^{-a} [n+a+b] \left\{ \frac{(1+x-2)^{n+a}}{(1-)^{n+1}} \right\}.$$

16. 证明例 4.14 中和式的以下推广：如果两个序列 $\{f_n\}$ 和 $\{c_k\}$ 由方程连接：

$$f_n = \sum_k \binom{n+k}{m+2k} c_k \quad (n \geq 0),$$

其中 $m \geq 0$ 固定, 那么它们的 opsgf 之间的关系为:

$$F(x) = \frac{x^m}{(1-x)^{m+1}} \left(\frac{x}{(1-x)^2} \right).$$

准确说明在例 4.14 中所使用的序列 $\{c_k\}$ 有什么特别之处, 使得在那一个特定情况下结果显得如此整洁。

17. 本练习的目的是展示 [WZ] 方法与一些众所周知的连续现象之间的相似性。

(a) 设 $F(x), G(x)$ 为满足 $F_x = G$ 且对于某区间 $a < x < b$ 中的所有 x , 都有 $1_{\pm} G(x) = 0$ 的可导函数。证明我们有这样的“恒等式”:

$$F(x) d = n. \quad (a < x < b).$$

(b) 使用 (a) 部分的结果证明: 如果 $f()$ 在条形区域 $- < a < () < b <$ 内解析, 且如果 f_0 沿着该条形区域内的所有垂直线成立, 那么 (a) 部分的结论也成立, 其中 $F(x,)$ 是 $f()$ 的实部。

(c) 证明 (b) 部分所述的结论是正确的, 但不使用 (a) 部分的结果, 而是转而将柯西积分定理应用于一个适当的矩形上。

(d) 将这些结果应用于 $f() = e^{-x^2}$ 上, 并由此发现这样一个“恒等式”:

$$e^{-x^2} d = c e^{-x^2} \quad (x \text{ 为实数}),$$

该式指出函数 e^{-x^2} 是它自身的傅立叶变换 (Fourier transform)。求 c 。

18. 这个练习给出了关于 n 顶点树的数量的 Cayley 公式的一个巧妙证明, 它通过证明更多东西来实现: 即, 即使当所有顶点的度数都被指定时, 也存在一个漂亮公式可以计算出这类树的数量。

(a) 设 $d_1, \dots, d_n$ 是其总和为 $2n - 2$ 的正整数。证明: 对于所有 $i = 1, \dots, n$ 都满足顶点 i 的度数是 d_i 的 n 顶点标号树 的数量恰好是:

$$f_n(d_1, \dots, d_n) = \frac{(n-2)!}{(d_1-1)!(d_2-1)! \cdots (d_n-1)!}.$$

(对 n 使用归纳法进行证明。首先说明必定有至少一个 d_i 等于 1, 然后从那里开始。)

(b) 将生成函数

$$F_n(x_1, \dots, x_n) = \sum_{\substack{d_1 + \cdots + d_n = 2n-2 \\ d_1, \dots, d_n \geq 1}} f_n(d_1, \dots, d_n) x_1^{d_1} \cdots x_n^{d_n}$$

表示成一种不含任何求和号的、令人愉悦的显式形式。

(c) 令 $x_1 = x_2 = \cdots = x_n = 1$ 代入你 (b) 部分的答案, 从而证明 Cayley 的结果: 恰好有 n^{n-2} 个包含 n 个顶点的标号树。

(d) 使用筛法来证明: 如果 k 是包含 n 个顶点的树中, 有 k 个顶点是端点 (度数为 1 的顶点) 的树的数量, 那么:

$$\sum_k kx^k = \sum_r \binom{n}{r} r^{n-2} (x-1)^{n-r}.$$

(e) 证明 n 个顶点的树的平均端点数量为：

$$n \left(1 - \frac{1}{n}\right)^{n-2} \sim \frac{n}{e} \quad (n \rightarrow \infty),$$

即：一棵树中一个随机选择的顶点是端点的概率大约是 $1/e$ 。

19.

(a) 如果 $N(S)$ 是其属性集合包含在 S 之内的对象数量，那么证明对于所有集合 S ，其属性集合精确地是 $\sum_S (-1)^{|S|-|A|} N(A)$ 的对象数量为：

$$N(A) = \sum_S (-1)^{|S|-|A|} N(S).$$

(b) 设 S 为一个固定的正整数集合，并令 $h_n(S)$ 为某个特定有标号指数族中权重为 n 的手牌数量，它的所有卡牌大小都属于 S 。寻找 $\{h_n(S)\}$ 的 egf。

(c) 将其乘以 $(-1)^{|S|-|A|}$ 然后对所有的 S 进行求和，找出其组成卡牌的不同大小的集合恰好是 A 的手牌数量 $\{h_n(A)\}_{n \geq 0}$ 的 egf，形式如下 (d 's 是牌叠大小)：

$$\sum_{n \geq 0} \frac{h_n(A)}{n!} x^n = \prod_{d \in A} \left(e^{x/d} - 1 \right).$$

(d) 令 (n, k) 为权重为 n 、且具有恰好 k 种不同大小的卡牌（无论它们实际包含多少张卡牌！）的手牌数量。将 (c) 部分的结果在所有的 $|A| = k$ 上求和，以此类推，发现：

$$\sum_{n, k \geq 0} \frac{(n, k)}{n!} x^n = \prod_{k \geq 1} \left\{ 1 + \left(e^{x/k} - 1 \right) \right\}.$$

(e) 令 c_n 为 n 字母排列中出现不同循环大小的平均数量。证明 $\{c_n\}$ 的 opsgf 为：

$$\frac{1}{1-x} \sum_{k \geq 1} \left(1 - e^{-x/k} \right),$$

并找出 c_n 的显式公式。

20. 从集合 $\{1, 2, \dots, n\}$ 开始。将一枚硬币抛掷 n 次，集合中每一个成员对应一次。保留抛出“正面 (Heads)”的元素，并抛弃抛出“反面 (Tails)”的元素。现在你就拥有了原集合的一个子集 S 。将这整个过程称为一个“步骤 (step)”。现在对 S 采取“步骤”。也就是说，为 S 的每个元素抛一次硬币，保留出现“正面”的元素，得到一个子集 S' ，以此类推。当到达空集时游戏结束。令 $f(n, k, r)$ 为在 k 个步骤之后，恰好剩下 r 个对象的概率。

(a) 为 f 找出一个递推关系，为 f 找出生成函数，并找出 f 本身。

(b) 一场完整游戏的平均步骤数量是多少？

(c) 游戏步骤数量的标准差是多少？

21. 和 4.7 节中一样，令 $f(n, k,)$ 为拥有 n 个细胞、排成 k 层，且最高层恰好包含 n 个细胞的 HC-多格骨牌数量。证明这个“宏大”的三变量生成函数是：

$$\sum_{n,k} f(n, k,) x^{nk} = \frac{x(1-x)^2((1-x)(1-x)^2 + x^2(-1))}{(1-x)^2((1-x)^4 - x(1-x-x^2+x^3+x^2))}.$$

请注意，你在这里不需要从头开始，相反你可以使用 4.7 节的结果。

22. 设 $(a_1, b_1), \dots, (a_k, b_k)$ 为一个精确覆盖序列 (exact covering sequence)。证明

$$\sum_{j: a_j \text{ 为偶数}} \frac{1}{b_j} = 1/2 = \sum_{j: a_j \text{ 为奇数}} \frac{1}{b_j}.$$

将这个结果推广到对于 a_j 的其他同余类。

23. 一个随机排列其 r -循环和 s -循环拥有相等数量的概率是多少？用贝塞尔函数 (Bessel functions) 来表达你的答案 (参见第 2 章)。并制作一张关于你的答案 (作为 r 和 s 的函数) 的表格，对于 $1 < r < s$ 。

24. 找出其系数是关于 n 的多项式的一个三项递推关系，满足 (4.8.1)[译注：原书应为 (4.75)] 中显示的量，即周长为 $2n + 1$ 的凸多格骨牌的数量。

25. 要想使 $(a_i, b_i)_{i=1}^k$ 成为一个精确覆盖序列，其必要且充分条件是：对于所有的满足 $0 \leq n \leq N$ 的 n ， n 恰好仅与一个 i 同余于 $a_i \pmod{b_i}$ ，其中 N 是 $b_1, \dots, b_k$ 的最小公倍数。

26. (a) 发展出我们在 4.7 节中实质上在使用的指数公式的推广。准确地说，假设对于每个 $i = 1, 2, 3, \dots$ ，我们都给定了正整数集合 S_i 。令 $h(n)$ 为，如果我们在选择卡牌时被如下条件限制：对于每个 $i = 1, 2, 3, \dots$ ，为手牌选择的权重为 i 的卡牌数量必须在集合 S_i 中；那么在此限制下，能够从给定牌叠集合中组成的权重为 n 的手牌数量。请证明：

$$\sum_{n \geq 0} h(n) \frac{x^n}{n!} = \prod_{i=1}^{\infty} \sum_{s_i \in S_i} \left(\frac{d_i^{s_i}}{s_i!} x^{s_i} \right),$$

其中 $S_i(x)$ 是它的指数属于集合 S 的指数级数子级数，并且和在第 3 章一样， d_i 是第 i 个牌叠中的卡牌数量。

(b) 寻找 $\{f(n)\}$ 的 egf，其中 $f(n)$ 是满足以下条件的 $[n]$ 的划分数：大小为 2 的类的数量能被 2 整除，且大小为 3 的类的数量能被 3 整除，等等。

27. 使得余数和模数对 $(a_i, b_i)_{i=1}^k$ 成为精确覆盖序列的充要条件是对于整数 $r = 1, 2, 3, \dots$ 下式成立：

$$\sum_{i=1}^k b_i^{n-1} \left(\frac{a_i}{b_i} \right) = n \pmod{b_i} \quad (n = 0, 1, 2, \dots)$$

其中 $\{b_n\}$ 是伯努利数 (由 (2.40) 定义)，并且 $b_n(x)$ 是伯努利多项式 (Bernoulli polynomials)，定义为：

$$\frac{x}{-1} = \sum_{n=0} \frac{n(x)^n}{n!}.$$

28. 找出一个公式，计算一个排列拥有平方根的数量。什么样的排列拥有唯一的平方根？

29. 通过对 n 进行归纳，直接证明方程 (4.22)。

[翻译进度提示]

太棒了，令人脑洞大开的 第 4 章 的内容和所有课后习题已经全部翻译完成！在这一章我们见证了“生成函数”、“万金油”、“W-Z 证书算法”带来的无穷魅力。

接下来，我们将进入全书在分析上最精彩的压轴章节：**第 5 章：解析与渐近方法 (Analytic and Asymptotic Methods)**。在这里，我们将运用复变函数和柯西积分定理，使用**拉格朗日反演公式 (LIF)** 解隐函数方程（比如破解之前未解的 Cayley 树公式），用 **Darboux 达布定理** 解析极点与代数奇点，甚至用 **Hayman 鞍点法** 计算连奇点都没有的整函数的渐近系数规模（比如推导斯特林公式）。

如果您准备好了，请回复“继续”。

好的！欢迎来到本书在解析数学方面的最高潮——**第 5 章：解析与渐近方法 (Analytic and Asymptotic Methods)**。

在前面的章节中，我们已经见识了生成函数作为“代数对象（形式幂级数）”在组合计数上的惊人威力。现在，我们要把生成函数看作真正的**复变函数 (Complex Variables)**，利用复分析中的柯西积分公式、极点、分支点等工具，来获取序列的**渐近增长率 (Asymptotic growth rates)**。

首先登场的是一个极其优美、能直接解开复杂隐函数方程的工具：**拉格朗日反演公式 (Lagrange Inversion Formula, LIF)**。有了它，我们在第 3 章中留下的那个关于“有根树 (Rooted Trees)”的隐函数方程 $(x) = x^{(x)}$ 将被瞬间秒杀！

下面为您翻译《生成函数学》**第 5 章：5.1 节**的内容。

5 解析与渐近方法

(Analytic and Asymptotic Methods)

在前面的章节中，我们强调了生成函数理论的形式 (formal) 方面，而非解析 (analytic) 理论。然而，这门学科的吸引力之一就在于：人们可以多么轻易地转换思维方式，**从将生成函数仅仅视为挂载序列的晾衣绳，转变为将它们视为复变量的解析函数**。在后一种思维状态下，人们能够推导出所生成序列的许多属性，而这些属性是纯粹的形式方法所无法企及的。在这些属性中，最引人注目的就是序列的**渐近增长率 (asymptotic growth rates)**，这可能也是该理论在解析方面的

主要焦点。因此，在本章中，我们将发展一些对于这类研究来说具有无估量价值的解析机制。关于渐近分析的介绍以及渐近学所有符号的定义，请参见例如 [Wi1] 的第 4 章。

5.1 拉格朗日反演公式

(The Lagrange Inversion Formula)

拉格朗日反演公式 (Lagrange Inversion Formula, LIF) 是一个解决特定类型函数方程的非凡工具，在它发挥出最佳水平时，它甚至能给出显式公式，而此时其他方法往往会碰壁。LIF 能够提供帮助的函数方程具有以下形式：

$$u = (u). \quad (5.1)$$

在这里， (u) 是一个已知的、关于 u 的函数，我们考虑这个方程是为了确定 u 作为 (u) 的函数。也就是说，我们正在“用 (u) 来解出 u ”。

我们在 3.12 节中发现了一个这样的方程。在那里我们看到，如果 (x) 是各种顶点数量的有根标号树数量的 egf (指数生成函数)，那么 (x) 满足方程 $x = (x)$ ，这确实是 (5.1) 的形式，其中 $(u) = u$ 。

一般的问题是这样的：假设我们已知函数 (u) 的幂级数展开式，该级数在 (u) 平面的原点附近的某个邻域内收敛。我们如何才能找到 (5.1) 的解 $u = u()$ 在 (u) 平面的原点附近某个邻域内的幂级数展开式呢？**答案出人意料地明确，并且它甚至允许我们找到解 $u()$ 的某个函数的展开式。**

定理 5.1 (拉格朗日反演公式 / The Lagrange inversion formula)

设 $f(u)$ 和 (u) 为关于 u 的形式幂级数，且 $(0) = 1$ 。那么存在唯一的一个形式幂级数 $u = u()$ 满足 (5.1)。此外， f 在那个根 $u = u()$ 处的值 $f(u())$ ，当在 $u = 0$ 附近展开为关于 u 的幂级数时，满足：

$$[n]\{f(u())\} = \frac{1}{n}[u^{n-1}]\{f'(u)(u)^n\}. \quad (5.2)$$

证明：首先我们注意到，只需在 f 和 (u) 都是多项式的情况下证明该定理即可。实际上，如果固定了 n ，并且如果 f 和 (u) 都是完整的形式幂级数，那么假设我们通过丢弃所有涉及 $k > n$ 的幂 u^k 的项，来截断这两个级数。如果结论对于这些多项式成立，那么它对于原始的未截断级数依然成立，因为被丢弃的高阶项对于给定的 n ，根本不会影响到 (5.2) 式。

因此，我们现在假设 f 和 (u) 是多项式。

我们将首先进行一个形式计算，然后再讨论该结果有效的范围。我们有：

$$\begin{aligned}
\frac{1}{n}[u^{n-1}]\{f'(u)(u)^n\} &= \frac{1}{n}[u^{n-1}]\{f'(u)(u/)^n\} \\
&= \frac{1}{n}[u^{-1}]\{f'(u)/^n\} \\
&= \frac{1}{2i} \frac{f'(u)}{(u)^n} du \\
&= \frac{1}{2i} \int_{\gamma} f'(u) u' d \\
&= [^n]\{(d/d)f(u())\} \\
&= n[^n]f(u()).
\end{aligned} \tag{5.3}$$

在上述推导中，第一个等式来自 (5.1)（即 $(u) = u/$ ），第二个是显而易见的，第三个是复积分的**留数定理 (residue theorem)**，其中被积函数是 u 的函数，而积分路径 (contour) 可以是一个包含 0 的小圆。

第四个等式需要一些讨论。考虑函数 $g(u) = u/(u)$ 在原点附近的行为。由于 $(0) = 1$ ，函数在 0 的某个邻域内保持非零。因此 g 在那里是解析的，并且它具有形式为 $u + cu^2 + \dots$ 的幂级数展开。由此可见 g 在 0 附近是一个 **1-1 的共形映射 (conformal map)**。因此，它拥有一个本身在 0 附近也解析的、明确定义的逆映射。所以 (5.1) 在 $= 0$ 的某个邻域内具有唯一解 $u = u()$ ，且 u 在该处是 $$ 的解析函数。

如果在第四个等号后出现的积分中的积分路径，是一个位于 $$ 内的、围绕 $= 0$ 的圆，那么随着从 u 到 $$ 的积分变量替换，该等号就简单地成立了。

其余的等式都是不言而喻的，证明完毕。

例 5.2 在 3.12 节中，我们着手证明了定理 3.19，其结论是恰好存在 n^{n-2} 个具有 n 个顶点的标

号树。我们发现如果 $\{^n_x\}$ ，其中 $_n$ 是具有 n 个顶点的有根标号树的数量，那么 (x) 满足函数方程：

$$(x) = x^{(x)}. \tag{5.4}$$

现在，有了拉格朗日反演公式，我们**实际就能解出 (5.4) 了！** 因为这就是 (5.1) 中 $(u) = u$ 的情况。如果我们取 $f(u) = u$ ，那么根据 (5.2)，

$$\begin{aligned} [x^n] \mathcal{D}(x) &= (1/n) [u^{n-1}] \{\phi(u)^n\} \\ &= (1/n) [u^{n-1}] \{e^{nu}\} \end{aligned}$$

因此，

$$\frac{n}{n!} = \frac{n^{n-1}}{n!},$$

从而 $_n = n^{n-1}$ 。但 $_n$ 是 n 顶点的有根树数量，而每一棵无根的标号树由于有 n 个节点可作为根，从而贡献了 n 棵有根标号树，所以 n 顶点的标号树数量就是 n^{n-2} ，这就完成了定理 3.19 的证明。

例 5.3 在第 4 章的结尾，我们讨论了求和公式的反演对 (inverse pairs of summation formulas)，并给出了莫比乌斯反演公式之外的一些例子。在这里，我们将在 LIF 的帮助下推导出一个精致得多的例子。我们提议证明：如果两个序列 $\{a_n\}$ 和 $\{b_n\}$ 由下式关联：

$$b_n = \sum_k \binom{k}{n-k} a_k, \quad (5.)$$

那么我们有如下的反演：

$$na_n = \sum_k \binom{2n-k-1}{n-k} (-1)^{n-k} kb_k. \quad (5.7)$$

确实，如果 (5.6) 成立，那么正如我们在 4.3 节（万金油方法）中经常做的那样，乘以 x^n ，对 n 求和，并交换 k 和 n 的求和顺序，得到：

$$\begin{aligned} B(x) &= \sum_k a_k x^k \sum_n \binom{k}{n-k} x^{n-k} \\ &= \sum_k a_k x^k \end{aligned}$$

其中 A ，是这两个序列的 opsgf（普通幂级数生成函数）。现在我们可以通过令 $y = x + x^2$ 来解出用 y 表示的 A 。然后如果我们反向读取 (5.8)，我们发现

$$A(y) = (x(y)), \quad (5.)$$

其中 $x(y)$ 是二次方程 $y = x + x^2$ 的那个在 $y = 0$ 时消失（值为 0）的解。

现在我们当然可以显式地解这个二次方程。如果我们这样做（尽管我们不打算这么做），我们从 (5.9) 会发现

$$A(y) = \left(\frac{\sqrt{1+4y} - 1}{2} \right),$$

然后我们想要为右边 y^n 的系数找到一个漂亮的公式，这适用于每一个 n 。反过来，这需要为：

$$[y^n] \left\{ \frac{\sqrt{1+4y} - 1}{2} \right\}^k \quad (5.10)$$

（对于每一个 n 和 k ）找出一个漂亮的公式。与其试图通过显式地将花括号中的量提高到 k 次方并与平方根搏斗来处理 (5.10)，**让 LIF 来替我们干这些苦力活要优雅得多！**

我们首先以**隐式 (implicitly)** 而非显式的方式重新表述问题 (5.10)。我们想要的是：

$$[y^n] \{x(y)^k\},$$

其中 $x = x(y)$ 是 $y = x + x^2$ 的那个在 $y = 0$ 时为 0 的解。用 LIF 的术语来说，我们将这个方程写成

$$x = \frac{y}{1+x},$$

这恰好是 (5.1) 的形式，其中 $(u) = 1/(1+u)$ 。进一步，既然我们想要 $x(y)$ 的第 k 次方的系数，那么在 LIF 中的函数 $f(u)$ 现在就是 $f(u) = u^k$ 。

LIF 的结论 (5.2) 告诉我们

$$\begin{aligned}\{x()^k\} &= (1/n)[x^{n-1}] \left\{ \frac{kx^{k-1}}{(1+x)^n} \right\} \\ &= (k/n)[x^{n-k}] \frac{1}{(1+x)^n} \\ &= (k/n)(-1)^{n-k} \binom{2n-k-1}{n-k},\end{aligned}$$

于是我们就完全完成了 (5.7) 式的证明!

需要特别注意的是, **LIF 在计算未知函数的第 k 次幂的系数时, 就像计算未知函数本身的系数一样熟练。** 在 LIF 声明中的函数 f , 仅仅只是指定了: 我们想要知道其系数的那个关于未知函数的函数。然后, LIF 就会把这些系数交到我们手上。

[翻译进度提示]

到这里, 5.1 节关于**拉格朗日反演公式 (LIF)**的介绍及其两个极其精彩的应用已经翻译完毕! 我们看到了它是如何轻松解开带有 $x^{(x)}$ 形式的隐函数方程, 从而一举拿下 Cayley 的树公式的; 也看到了它是如何用隐式求解取代复杂的带根号多项式的展开, 从而优美地证明反演公式的。

接下来, 我们将进入 **5.2 节 Analyticity and Asymptotics (I): Poles (解析与渐近 (I): 极点)**。在这里, 我们将正式学习渐近分析的第一定律: “寻找距离原点最近的奇点”, 并看到如何用**主部分 (principal part)** 极其精确地估计系数的渐近规模。

如果您准备好继续, 请回复“**继续**”。

好的! 接下来, 我们将正式踏上“**利用复分析工具求序列渐近公式**”的旅程。在这条旅程中, 第一站是**极点 (Poles)**。

在这一节中, 作者将向我们传授渐近分析的“第一定律”: **寻找距离原点最近的奇点!** 并且, 我们将看到如何仅仅通过减去该奇点处的“主部分 (principal part)”, 就能让一个极其复杂的生成函数, 在无需展开所有项的情况下, 极其精确地暴露出其系数的渐近规模。

下面为您翻译 《生成函数学》 第 5 章: 5.2 节的内容。

5.2 解析与渐近 (I): 极点

(Analyticity and Asymptotics (I): Poles)

假设我们已经为某个我们感兴趣的组合数序列找到了生成函数 $f()$ 。接下来我们可能想要找到该序列的**渐近行为 (asymptotic behavior)**, 也就是说, 去寻找一个关于 n 的简单函数, 使得当 n 很大时, 这个简单函数能提供我们序列值的一个很好的近似。

做渐近分析的第一定律是：**寻找 $f()$ 距离原点最近的奇点 (singularity or singularities)**。原因是 $f()$ 恰恰在以原点为中心、**内部不包含任何奇点**的最大圆盘内是解析的 (analytic)，而我们通过寻找距离原点最近的奇点，就能找到那个圆的半径。一旦我们有了那个半径，我们也就有了幂级数 $f()$ 的收敛半径。一旦我们有了收敛半径，根据定理 2.25，我们就对 n 很大时系数的大小有所了解。通过对这个过程进行各种改良，我们就能发现更详细的信息。

因此，在本节及接下来的几节中，我们将研究解析函数的奇点对其系数的渐近行为的影响。将这些方法结合在一起，就提供了一种获取组合序列渐近性质的强大技术，同时这也为（如果还需要的话）采用生成函数方法提供了另一个理由。

在本节中，我们将集中讨论**仅具有极点 (poles) 作为奇点**的函数。

设 $f()$ 在复平面上包含原点的某个区域内解析，除了有限个奇点之外。如果 r 是这些奇点模长中的最小值，那么 f 在圆盘 $|z| < r$ 内是解析的，因此这正好就是其围绕原点的幂级数展开式收敛的圆盘。

反过来，如果某个生成函数 f 的幂级数展开式在圆盘 $|z| < r$ 内收敛，但在任何以原点为中心的更大圆盘内都不收敛，那么在圆周 $|z| = r$ 上必定存在函数 f 的一个或多个奇点。

处理系数序列的渐近增长问题，许多方法都依赖于以下策略：**寻找一个简单的函数 g ，使得它在圆周 $|z| = r$ 上拥有与 f 相同的奇点**。那么 $f - g$ 在某个更大的圆盘（假设半径为 $r' > r$ ）内就是解析的。

然后，根据定理 2.25（你可以去查阅）， $f - g$ 的幂级数系数在 n 很大时将小于 $(\frac{1}{r'} + \epsilon)^n$ ，因此它们将**远小于 (much smaller than)** f 本身的系数。后者根据同样的定理 2.25，将有无限多次像 $(\frac{1}{r} - \epsilon)^n$ 一样大。

因此，**通过观察 g 的系数的增长，我们就能够发现 f 的系数增长的最重要方面**。

所以，制胜的策略是：找到一个简单的函数来模仿你感兴趣的那个函数的奇点，然后利用简单函数系数的增长来做估计。

在**亚纯函数 (meromorphic function)** $f()$ （即除了有限个极点之外在整个区域内都解析的函数）的情况下，这些考虑表现得最为清晰。因此，我们将首先研究此类函数。其思想是，在一个极点 z_0 附近，一个亚纯函数可以很好地由其洛朗展开式 (Laurent expansion) 的**主部分 (principal part)** 来近似，也就是说，由级数中包含 $(z - z_0)^{-k}$ 的**负整数次幂**的那有限几项来近似。

例 5.4 函数 $f(z) = 1/(z-1)$ 在整个有限复平面上是亚纯的。它唯一的奇点在 $z_0 = 1$ 处，且在那个奇点处的主部分是 $1/(z-1)$ 。因此函数 $f(z) - 1/(z-1)$ 在整个平面上都是解析的。所以，如果 $\{c_n\}$ 是 f 围绕原点展开的幂级数系数，而 $\{d_n\}$ 是函数 $1/(z-1)$ 围绕原点展开的幂级数系数，那么当 n 很大时，差值 $c_n - d_n$ 是很小的。事实上，定理 2.25 保证：对于每一个 $\epsilon > 0$ ，对于所有足够大的 n ，我们都有 $|c_n - d_n| < \epsilon^n$ 。因此， $f(z)$ 中那些“未知的”系数，就被简单函数 $1/(z-1)$ 的系数极其良好地近似了。

很容易把这个特定的例子完全算出来，以看清机器究竟是如何运作的。 $1/(z-1)$ 围绕原点的展开式是（参见 2.2 节的法则 5）：

$$\frac{1}{(z-1)} = - \sum_{n \geq 0} \{1 + \frac{1}{2!} + \cdots + \frac{1}{n!}\} z^n.$$

另一方面， $1/(z-1)$ 的展开式是：

$$\frac{1}{(z-1)} = -1 - z - z^2 - z^3 - \cdots.$$

因此，用主部分替换原函数的这一行为，一步就得到了对 z^n 的真实系数的近似：该系数就是 -1 的幂级数的前 n 项部分和；而被近似成的就是 -1 本身，这无疑是一个**具有爆炸性优势**

(smashingly good) 的近似。在这个例子中取得巨大成功的原因是：仅仅从原函数中减去一个主部分，就将其解析圆盘的半径从 $|z| < 1$ 扩大到了半径 ∞ 。

看待这个例子的另一种方式是不去提及任何繁重的理论机制。考虑这个看起来很无辜的事实：

$$f(z) = \frac{1}{z-1} = \frac{1}{z-1} + \frac{1}{1-z}.$$

在第一项中，幂级数的所有系数都等于 -1 。第二项在整个有限复平面上没有任何奇点，也就是说，它是一个关于 z 的**整函数 (entire function)**。根据定理 2.25，对于每一个正数 n ，它的系数都是 $1/n!$ 的。因此 $f(z)$ 的系数是：

$$= -1 + \frac{1}{n!} \quad (n \geq 0)$$

对于任意 $n > 0$ 。

在不那么有利的情况下，人们可能需要减去**几个**主部分才能扩大解析圆盘的大小（例如，如果有几个极点都在收敛圆周上），并且即使如此，如果在一个稍微大一点的圆上还存在其他奇点的话，圆盘可能只会增大一点点。

如果 f 在区域 $|z - z_0| < r$ 中是亚纯的，令 z_0 为 f 的一个阶数为 r 的极点 ($1 \leq r < \infty$)。那么在某个以 z_0 为中心的去心圆盘内， f 拥有如下展开式：

$$f(z) = \sum_{j=1}^r \frac{a_{-j}}{(z-z_0)^j} + \sum_{j=0}^{\infty} a_j (z-z_0)^j. \quad (5.11)$$

上面两个和式中的第一个，也就是包含 $(z-z_0)^{-j}$ 负数次幂的那一个，被称为 f 围绕奇点 z_0 展开的**主部分 (principal part)**，我们将其记为 $PP(f; z_0)$ 。函数 $f - PP(f; z_0)$ 在 z_0 处是解析的。也就是说，我们可以通过减去主部分来**消除 (remove)** 这个奇点。

如果在同一个圆周 $|z - z_0| = r$ 上，除了 z_0 之外还存在 f 的其他极点，那么令 $z_1, \dots, z_s$ 为所有这样的极点。函数

$$h(z) = f(z) - PP(f; z_0) - PP(f; z_1) - \cdots - PP(f; z_s) \quad (5.12)$$

在每一个点 $\{z_j\}_0^s$ 处都是正则（解析）的。但 f 在该圆周上没有其他奇点了，所以 h 在一个以原点为中心、半径为 r （其中 $r > r_0$ ）的圆内是解析的。

根据定理 2.25，这意味着 h 围绕原点的幂级数系数，对于所有足够大的 n ，其增长速度不可能快于

$$\left(\frac{1}{r} + \epsilon\right)^n.$$

因此，如果 $\{a_n\}$ ，且如果

$$g(z) = PP(f; z_0) + PP(f; z_1) + \cdots + PP(f; z_s) \quad (5.13)$$

$$\{b_n\},$$

那么

$$a_n = b_n + \left(\left(\frac{1}{r} + \epsilon\right)^n\right) \quad (n \rightarrow \infty).$$

这样我们可能就已经在寻找 f 的系数渐近行为的道路上走得很远了。

实际上，现在让我们来研究一下 (5.13) 中显示的主部分之和的（围绕原点的）幂级数系数。如果 z_0 是一个重数为 r 的极点，我们有：

$$\begin{aligned} PP(f; z_0) &= \sum_{j=1}^r \frac{a_{-j}}{(z - z_0)^j} \\ &= \sum_{j=1}^r \frac{(-1)^{j-1} a_{-j}}{(z - z_0)^{j-1}} \end{aligned}$$

我们因此看到，函数 f 在 z_0 处的 r 阶极点，对 a_n 的系数贡献了

$$\sum_{j=1}^r \frac{(-1)^{j-1} a_{-j}}{n+j-1} \binom{n+j-1}{j-1} \quad (5.15)$$

这个基本定理，即断言“我们可以用亚纯函数在模长最小的极点处的主要部分的系数，来极好地近似该亚纯函数的系数”，可以被表述如下：

定理 5.5 设 f 在包含原点的区域 $|z| < R$ 内解析，除了有限个极点。设 $r_0 > 0$ 为模长最小的极点（或多个极点）的模，并设 $z_0, \dots, z_s$ 为 $f(z)$ 所有模长为 r_0 的极点。进一步，设 $r_1 > r_0$ 为 f 模长次小（next-smallest modulus）的极点（或多个极点）的模，并给定 $\epsilon > 0$ 。那么

$$[n]f(z) = [n] \left\{ \sum_{j=0}^s PP(f; z_j) \right\} + \left(\left(\frac{1}{r_1} + \epsilon \right)^n \right). \quad (5.1)$$

证明： 根据定理 2.25，只要我们确立了以下事实，本定理即可得证：如果从 $f(z)$ 中减去它在圆周 $|z| = r_0$ 上奇点的所有主部分之和，那么所得的函数在更大的圆盘 $|z| < r_1$ 内是解析的。考虑当我们从 $f(z)$ 中减去 $PP(f; z_0)$ 的那一刻。所得函数，比方说 g ，确实在 z_0 处解析。然而，接下来我们要从 g 中减去的是 $PP(g; z_1)$ ，而不是 $PP(f; z_1)$ 。我们声称这无关紧要，也就是说：

$$PP(f - PP(f; z_0); z_1) = PP(f; z_1).$$

要看出这一点，请观察：

$$PP(f - PP(f; 0); 1) = PP(f; 1) - PP(PP(f; 0); 1).$$

但等式右边的第二项消失了，因为 $PP(f; 0)$ 在 1 处是解析的。

通过对 s 进行归纳，结果得证。

例 5.6 (有序的贝尔数 / Ordered Bell numbers)

我们现在研究“有序贝尔数”的渐近行为。它们的定义如下：一个由 n 个元素组成的集合有 $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 种划分为 k 个类的划分。如果我们现在认为**类之间的顺序**是重要的，但各个类**内部**元素的顺序是不重要的，那么我们看到 $[n]$ 有 $k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 个有序的、分为 k 类的划分。

有序贝尔数 $b(n)$ 是 $[n]$ 的有序划分的总数，也就是说，它是 $\sum_k k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ 。

我们的问题是关于当 n 很大时 $\{b(n)\}$ 的增长。为了寻找这些数字的一个漂亮公式，将恒等式 (4.22) 的两边乘以 $-$ ，并从 0 到 ∞ 积分。这得出了一个整洁的结果：

$$b(n) = \sum_{r \geq 0} \frac{r^n}{2^{r+1}}.$$

那么有序贝尔数的指数生成函数 (egf) 就是

$$f() = \sum_{n \geq 0} \frac{b(n)}{n!} z^n = \frac{1}{2 - e^z}. \quad (5.17)$$

1 一定要自己把它算出来。

我们运气真好！生成函数 $f()$ 只有简单的极点 (simple poles)，也就是所有的点 $12 \pm 2ki$ (对所有整数 k)。在极点 $0 = 12$ 处的主部分是 $(-1/2)/(-12)$ 。单单那个主部分自己就对 n 的系数贡献了

$$\frac{1}{2(12)^{n+1}}$$

。在以原点为中心、半径为 12 的圆周上，没有 $f()$ 的其他奇点了。因此

$$h() = f() - \frac{(-1/2)}{(-12)}$$

在从原点一直延伸到 $12 + 2i$ 的更大的圆内是解析的。那个圆的半径是

$$= \sqrt{(12)^2 + 4^2} = .321 \dots$$

因此 $h()$ 的系数是 $((.1)^n)$ 。总而言之，我们已经证明了：**有序贝尔数 $b(n)$ 具有如下形式**

$$b(n) = \frac{1}{2(12)^{n+1}} n! + ((.1)^n n!), \quad (5.1)$$

对于投入这么少努力就能得到这样的结果，确实相当不错了。如果需要，可以根据 $f()$ 在剩余极点处的主部分（按照它们绝对值的非降序排列）得到渐近展开式的更多项。读者应该去研究一下接下来的那两个极点（它们互为复共轭）一起的贡献。

下面我们展示一张包含一些 $n, b(n)$, 以及 $n!/(2(12)^{n+1})$ 数值的表格。

n	1	2	3	5	10
$b(n)$	1	3	13	541	102247563
$n!/(2(12)^{n+1})$	1.04	3.002	12.997	541.002	102247563

这吻合得惊人地接近。基本上，我们所做的全部，就是用 $1/(2(12 -))$ 的泰勒级数系数作为 $1/(2 -)$ 级数系数的近似。然而我们却获得了一个极为出色的近似作为回报。

例 5.7 (没有小循环的排列 / Permutations with no small cycles)

固定一个正整数 q 。令 $f(n, q)$ 表示其所有循环的长度都 $> q$ 的 n 字母排列数量。我们想要知道 $f(n, q)$ 的渐近行为。

根据第 3 章的习题 11, $\{f(n, q)\}_0$ 的 egf 是

$$\begin{aligned} f_q(z) &= \exp \sum_{n > q} \frac{z^n}{n!} \\ &= \exp \left\{ \log \frac{1}{1-z} - \sum_{k=1}^q \frac{z^k}{k} \right\} \end{aligned}$$

$f()$ 在有限复平面内唯一的奇点是在 $z = 1$ 处的一个 1 阶极点，其主部分为 $-H/(1 - z)$ ，其中

$$H = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{q}$$

是第 q 个调和数。

这就是我们能获得极其精确的渐近估计的那种情况，因为该函数与其在 $z = 1$ 处主部分之间的差为：

$$\begin{aligned} h(z) &= f(z) - \frac{-H}{1-z} \\ &= \frac{-\{+\cdots+\} - H}{1-z}, \end{aligned}$$

并且它在整个平面上都是解析的，即，它是一个整函数 (entire function)。再一次根据定理 2.25，对于每一个 $\epsilon > 0$, $h()$ 的第 n 个系数在 n 大时是 $O(n^{-\epsilon})$ 。因此，

$$\frac{f(n,)}{n!} = -H + \binom{n}{n} \quad (n). \quad (5.20)$$

在这个估计中令人震惊的**极小误差项 (strikingly small error term)** 表明：对于每个固定的，一个 n -排列没有长度 n 的循环的概率，**应该非常接近于与 n 无关**。考虑 $n=1$ 的情况，以体验一下这里到底发生了什么。那么 $f(n,1)/n!$ 是一个 n -排列没有不动点（错排）的概率。但我们在 (4.16) 中看到

$$\begin{aligned} f(n,1)/n! &= \frac{1}{n!} = 1 - 1 + 1/2 - \cdots + (-1)^n/n! \\ &= -1 + (1/n!). \end{aligned}$$

的确，该概率几乎完全与 n 无关，并且在使用主部分时所涉及的误差对于每一个正数 n 来说都是 $(1/n!)$ 的。

上面的两个例子展示了该方法在非典型精确的情况下的运作。更常见的情况是，人们在整个平面内找到的不仅仅是一个 1 阶极点，而是许多具有各种重数的极点。在这些情况下，该方法保持不变，但可能需要大量额外的工作才能获得合理准确的估计。展示这种现象的一个例子在第 3.15 节中，关于换零钱问题那里已经给出了。实际上，舒尔定理（定理 3.24）的证明正是练习使用了亚纯函数在其极点处的主部分。在那个例子中，唯一的那个占主导地位（dominant）的奇点的重要性虽然小得多，但足以把定理证出来！

[翻译进度提示]

到这里，5.2 节关于**极点 (Poles)** 的内容已经翻译完毕！这就是我们渐近分析的核心思路：抛弃复杂的原函数，用其**最靠近原点的奇点处的主部分**来近似它！这个过程简单且异常精确（例如有序贝尔数的惊人近似，以及无小循环排列的 $-H$ 极限）。

接下来我们将进入 **5.3 节 Analyticity and Asymptotics (II): Algebraic Singularities (解析与渐近(II): 代数奇点)**。在这里我们将处理不是简单的“分母为 0”的极点，而是带有根号或者非整数幂的分支点 (Branch Points)！著名的 **Darboux 达布定理** 将在这里大显身手。

如果您准备好继续，请回复“**继续**”。

好的，接下来我们将进入渐近分析的另一个极其重要、也是极具挑战性的领域——**代数奇点 (Algebraic Singularities)**。

在上一节中，我们处理了像 $1/(1-x)^k$ 这样分母变成零的“极点 (poles)”。但如果生成函数中含有**根号**（或者说，非整数次幂），比如我们在图论或多格骨牌中经常遇到的 $\sqrt{1-x}$ 这种形式，我们该怎么估计它的系数呢？

此时，复变函数理论中的**达布定理 (Darboux's Theorem)** 将闪亮登场。我们将看到，即使面对分支点 (branch points)，我们依然可以用“围绕奇点展开”的同样思想，将极其复杂的生成函数转化为非常漂亮的渐近公式！

下面为您翻译 《生成函数学》第 5 章：5.3 节的内容。

5.3 解析与渐近 (II): 代数奇点

(Analyticity and Asymptotics (II): Algebraic Singularities)

同样地，设 $f()$ 在包含原点的某个区域内解析，但现在假设距离 0 最近的那个 f 的奇点 0 **不是一个极点 (pole)**，而是一个**代数奇点 (algebraic singularity / 分支点 branch point)**。那意味着 $f() = (0 -)g()$ ，其中 g 在 0 处解析，并且 **不是一个整数**，而是一个实数。

一个典型的例子在 (3.17) 中给出，我们在那里发现：其所有顶点度数都等于 2 的 n 顶点图数量的 egf (指数生成函数) 是

$$f() = \frac{-/2-^2/4}{\sqrt{1-}}. \quad (5.21)$$

在本节中，我们将推导出一个定理 (“**达布定理 / Darboux's Theorem**”)，它允许我们推断出具有这种生成函数的序列的渐近行为。归根结底，人们应该在这个情况下做与处理亚纯函数时**完全相同的事情 (exactly the same thing)**，然后正确的答案自然就会“掉出来”。然而，证明这确实是有效的，在目前的情况下要求更高。我们遵循 [KnW] 中的证明。

如有必要，通过考虑 $f(0)$ 而不是 $f()$ ，我们可以看到，在不失一般性的前提下，我们可以假设 $0 = 1$ 。因此，我们正在处理的是在单位圆盘内解析，且在 $= 1$ 处有一个分支点 (branch point) 的函数 f 。在得到进一步的通知之前，我们还将假设 $0 = 1$ 是 f 在某个圆盘 $|| < 1 +$ (其中 > 0) 内拥有的**唯一的奇点**。

经过了上一节关于亚纯函数的经验教训之后，这里说明了我们在这个情况下可能如何进行。首先我们有 $f() = (1 -)g()$ ，其中 g 在 $= 1$ 处解析。既然如此，我们可以将 g 在 $= 1$ 的邻域内展开成收敛的幂级数：

$$g() = \sum_{k \geq 0} g_k(1 -)^k.$$

因此 f 本身就拥有如下的展开式：

$$f() = \sum_{k \geq 0} g_k(1 -)^{k+}. \quad (5.22)$$

类比亚纯函数的处理过程，我们可能会期望：上述级数展开式中依次出现的每一项，都生成了 f 系数的渐近展开式的下一项。**这实际上是对的**。 $f()$ 中 n 系数占据主导地位的行为 (dominant behavior)，来自于 (5.22) 中的**第一项**。也就是说，简单的函数 $g_0(1 -)$ 的 n 系数，对 f 的那个系数构成了主要的贡献，以此类推。

我们现在将证明所有这些论断。

引理 5.8 设 $\{a_n\}, \{b_n\}$ 为满足以下条件的两个序列：(a) $a_n = (n^-)$ 且 (b) $b_n = (n)$ (其中 $0 < < 1$)。那么

定理 5.11 (达布定理 / Darboux) 设 $f(z)$ 在某个圆盘 $|z - a| < 1 + \delta$ 内解析, 并且假设在 $|z - a| = 1$ 的邻域内, 它拥有展开式 $f(z) = \sum_{j=0}^{\infty} a_j (z - a)^j$ 。令 $S = \{0, 1, 2, \dots\}$ 。那么

$$\begin{aligned}\{(1-z)^{-1}\} &= [n] \left\{ \sum_{j=0}^m j(1-z)^{+j} \right\} + (n^{-m-2}) \\ &= \sum_{j=0}^m j \binom{n-j-1}{n} + (n^{-m-2}).\end{aligned}\quad (5.25)$$

证明： 我们有

$$\begin{aligned}(1-z)^{-1} - \sum_{j=0}^m j(1-z)^{+j} &= \sum_{j>m} j(1-z)^{+j} \\ &= (1-z)^{+m+1},\end{aligned}$$

其中 $(1-z)^{-1}$ 的解析区域与 $(1-z)^{+m+1}$ 的解析区域相同。现在结果由引理 5.10 得出。

例 5.12 (2-正则图 / 2-regular graphs)

对于 (5.21) 中 n 个顶点 2-正则图数量的指数生成函数 $f(z)$ ，我们有 $f(z) = (1-z)^{-1/2}$ ，其中 $\alpha = -1/2$ 且 $\beta = \{-1/2 - 2/4\}$ 。 $f(z)$ 在 $z=1$ 附近展开式的前几项是：

$$z^{-1/2} = z^{-3/4} + z^{-3/4}(1-z) + \frac{1}{4}z^{-3/4}(1-z)^2 + \dots$$

然后，根据 (5.25)，这个 $f(z)$ 围绕 $z=1$ 的展开式“提升 (lifts)”成了 $f(z)$ 系数的渐近公式。在这个例子中，其系数是 $(n)/n!$ ，其中 (n) 是包含 n 个顶点的 2-正则图数量。如果我们使用具有 $m=2$ 的 (5.25)，我们获得

$$\frac{(n)}{n!} = z^{-3/4} \binom{n-1/2}{n} + z^{-3/4} \binom{n-3/2}{n} + \frac{1}{4} z^{-3/4} \binom{n-5/2}{n} + (n^{-7/2}). \quad (5.2)$$

如果我们愿意，我们可以利用已知的二项式系数渐近展开式进一步简化答案：

$$\binom{n-1/2}{n} \frac{n^{-1/2}}{(-1)^n} \left[1 + \frac{(+1)}{2n} + \frac{(+1)(+2)(3+1)}{24n^2} + \dots \right]. \quad (5.27)$$

如果将此代入 (5.26)，结果是：

$$(n) \frac{n!^{-3/4}}{\sqrt{n}} \left\{ 1 - \frac{5}{n} + \frac{1}{12n^2} + \dots \right\}. \quad (5.2)$$

我们刚才证明的这种形式的达布方法，适用于收敛圆周上**仅仅存在一个 (just one)** 代数奇点的情况。该方法可以扩展到多个此类奇点的情况。我们在不给出证明的情况下，引用关于这种形式的更一般的结果 ([Sz], 定理 8.4)：

定理 5.13 (Szegő) 设 $h(z)$ 在 $|z| < 1$ 内解析，并且假设它在 $|z| = 1$ 上有有限个奇点 $\{i_k\}_1^r$ 。假设在每个奇点 i_k 的邻域内，都存在如下展开式：

$$h(z) = \sum_{k \geq 0} c^{(k)} (1 - z^{i_k})^{k+k},$$

其中 $k > 0$ 。那么以下序列是 $h(z)$ 系数的完整的渐近级数 (complete asymptotic series)：

$$[n]h() \sum_{\geq 0} \sum_{k=1}^r c^{(k)} \binom{k+k}{n} (-i_k)^n.$$

[翻译进度提示]

到这里，5.3 节关于**代数奇点（Algebraic Singularities）**与 **Darboux 达布定理** 的内容已经全部翻译完毕！

在这节中，我们看到：只要生成函数的奇点形式可以提炼为 $(1 -)()$ （哪怕 不是整数！），我们都可以简单地将解析部分 $()$ 在奇点处泰勒展开，并且这个展开式将魔术般地“直接提升（lifts）”为原函数系数的精确渐近级数！这个方法让我们优美地得出了 2-正则图的渐近数量，甚至精细到了 $1/n^2$ 的修正项。

接下来，我们将面对最后一座大山：**如果生成函数在整个复平面上都没有奇点（它是一个整函数，Entire Function），我们该怎么办？** 例如 或者 。

这就是第 **5.4 节：Analyticity and Asymptotics (III): Hayman's Method (解析与渐近(III): Hayman 鞍点法)** 所要解决的问题。这个极具威力的方法不仅能算出斯特林公式，更能解决诸如对合数（involutions）的渐近数量！

如果您准备好继续，请回复“**继续**”。

好的！现在我们将进入本书中关于渐近分析的最后一块，也是最具挑战性、最有趣的一块拼图——**没有奇点的函数怎么办？**

如果一个生成函数在整个复平面上都没有奇点（它是一个**整函数，entire function**），我们之前学过的所有“找距离原点最近的奇点”、“在奇点处展开”的招数统统失效了。为了估算这种函数的系数，我们需要借助柯西积分公式，在复平面的一个“鞍点（saddle point）”上进行极其精细的积分估计。这就是著名的 **Hayman 方法（Hayman's Method）**。

下面为您翻译 《生成函数学》 第 5 章：5.4 节 的内容。

5.4 解析与渐近 (III): Hayman 方法

(Analyticity and Asymptotics (III): Hayman's Method)

在前面的两节中，我们已经看到了如何处理那些生成函数在有限复平面（finite plane）内具有奇点的序列的渐近性问题。本质上，人们去寻找距离原点最近的那个（些）奇点，找到在那些奇点附近行为与所讨论的生成函数相同的简单函数，然后证明：该生成函数的系数的渐近行为，与那些在奇点附近行为相同、并且拥有相同奇点的简单函数的系数是一样的。

但是，如果生成函数根本没有任何奇点，也就是说，如果它是一个“整函数（entire function）”，我们该怎么办呢？

例 5.14 (的系数 / The coefficients of)

考虑函数 e^{-z} 。中 z^n 的系数是 $1/n!$ 。我们能不能想出某种相当通用的、处理整函数渐近性的方法，并且在这个例子中，这个方法将能为我们推导出斯特林公式 (Stirling's formula) ？

这里是我们可能开始的方式。根据柯西公式 (Cauchy's formula) 我们有

$$\frac{1}{n!} = \frac{1}{2\pi i} \oint_{\gamma} \frac{e^{-z}}{z^{n+1}} dz,$$

其中积分的路径是某条包围原点的简单闭曲线。如果我们使用一条以原点为中心、半径为 r 的圆作为路径，那么通过取绝对值，我们发现

$$\begin{aligned} \frac{1}{n!} &\leq \frac{1}{2\pi} \oint_{\gamma} \left| \frac{e^{-z}}{z^{n+1}} \right| |dz| \\ &= \frac{1}{r^n}. \end{aligned}$$

由于 e^{-z} 是一个整函数，所以 $r > 0$ 的值完全由我们来定，所以我们不妨选择它来**最小化 (minimize)** 我们将要获得的那个上界。但是

$$\min_{r>0} \frac{1}{r^n} \tag{5.2}$$

是在 $r = n$ 时取到的，所以我们能从这个论证中得到的最好的可能估计是

$$\frac{1}{n!} \leq \left(\frac{e}{n}\right)^n.$$

如果我们把它与斯特林公式

$$\frac{1}{n!} \sim \frac{1}{\sqrt{2\pi n}} \left(\frac{e}{n}\right)^n$$

进行比较，我们发现我们做得其实还不算太糟，因为我们相当粗糙的估计与“真相”之间，仅仅只差了一个大约 $1/\sqrt{2\pi n}$ 的因子。

为了做得比这更好，我们在处理 e^{-z} 围绕积分路径的变化时，将不得不稍微给予它更多的尊重，而不是仅仅把它替换为它所能达到的最大绝对值。事实上，沿着圆周 $|z| = r$ 的绝大部分路程上， e^{-z} 的绝对值都**远远小于** r^n 。它只有在点 $z = r$ 的一个小邻域内，才差不多有那么大。

在 W. K. Hayman 1956 年的论文“斯特林公式的一个推广 (A Generalisation of Stirling's Formula)”[Hy] 中，他发展出了一套具有相当大威力的机制，用于更加精确地处理这种情况。此外，他的方法对于组合理论中出现的生成函数来说异常有用，因为这些函数倾向于具有**非负的实数系数 (nonnegative real coefficients)**。正因为如此，在任何以原点为中心的圆上，这种函数的模 (modulus) 在那个圆上的**正实数点 (positive real point)** 处将达到最大。Hayman 的方法恰恰在处理这类函数时最为强大。

Hayman 的机制不仅适用于整函数，而且适用于所有的解析函数，甚至包括那些在有限平面内有奇点的函数。在实践中，它最常被用于整函数，这主要是因为我们已经看到，当有限平面内存在奇点时，还有其他的方法可用。

设 $f()$ 在复平面的一个圆盘 $|| < \rho$ 内解析，其中 $0 < \rho < \infty$ 。进一步假设 $f()$ 对于该方法是一个**可容许函数 (admissible function)**。在操作层面上，这仅仅意味着 $f()$ 是一个该方法能够对其起作用的函数。我们将在下面给出一些满足可容许性的充分条件。

定义

$$M(r) = \max_{|z|=r} \{|f(z)|\}. \quad (5.30)$$

由可容许性条件将推导出一个结果，即对于所有足够大的 r ，

$$M(r) = f(r) \quad (5.31)$$

这如我们上面所指出的，是因为该方法针对的是那些在其正实数轴方向上取得最大值的函数。

接下来定义两个辅助函数 (auxiliary functions)，

$$a(r) = r \frac{f'(r)}{f(r)} \quad (5.32)$$

以及

$$b(r) = ra'(r) = r \frac{f'(r)}{f(r)} + r^2 \frac{f''(r)}{f(r)} - r^2 \left(\frac{f'(r)}{f(r)} \right)^2. \quad (5.33)$$

主要的结果如下：

定理 5.15 (Hayman) 设 $f(z) = \sum a_n z^n$ 为一个可容许函数。令 r_n 为方程 $a(r_n) = n$ 的正实数根，对于每个 $n = 1, 2, \dots$ ，其中 $a(r)$ 由上面的方程 (5.32) 给出。那么

$$a_n \sim \frac{f(r_n)}{r_n^n \sqrt{2b(r_n)}} \quad \text{当 } n \rightarrow \infty \text{ 时}, \quad (5.34)$$

其中 $b(r)$ 由上面的 (5.33) 给出。

值得注意的是，这个公式 (recipe) 本身应用起来非常直接。通常困难的地方在于判断函数 $f(z)$ 对于该方法是否可容许 (admissible)。在我们解释那个概念之前，让我们先把这个定理应用到 $f(z) = e^z$ 上，暂时先凭信心相信它是可容许的这一事实。

例 5.16 (函数 (续))

在这个例子中，我们首先从 (5.32) 中计算出 $a(r) = r$ 。那么确定 $\{r_n\}$ 的方程 $a(r_n) = n$ ，就单纯地变成了 $r_n = n$ 。这些数字 r_n 与我们之前从条件 (5.29) 中发现的是完全一样的。它们简单地就是让 $f(r)/r^n$ 取得极小值的那个 r 值。

接下来，从 (5.33) 中我们找到 $b(r) = r$ ，然后 Hayman 的结果 (5.34) 就读作

$$\frac{1}{n!} \sim \frac{n}{n^n \sqrt{2n}},$$

这又一次是斯特林公式，但这一次是它的**精确形式 (exact form)**。

接下来让我们给出可容许函数 (admissible functions) 类别的一个精确定义。设 $f() = \sum_{n \geq 0} a_n z^n$ 在 $|z| < \rho$ 内是正则的，其中 $0 < \rho$ 。假设：

- (a) 存在一个 $\rho_0 < \rho$ 使得对于 $\rho_0 < r < \rho$ ， $f(r) > 0$ ，并且
- (b) 存在一个函数 $b(r)$ 定义在 $\rho_0 < r < \rho$ 上，使得对于这些 r 有 $0 < b(r) < \rho$ ，并且使得当 $r \rightarrow \rho$ 时，对于 $|z| < b(r)$ **一致地 (uniformly)** 有

$$f(r^i) \sim f(r)^{ia(r) - \frac{1}{2}b(r)},$$

并且

- (c) 对于 $|z| < b(r)$ 一致地有

$$f(r^i) = \frac{(f(r))^i}{\sqrt{b(r)}} \quad (r \rightarrow \rho),$$

并且

- (d) 当 $r \rightarrow \rho$ 时，我们有 $b(r) \rightarrow \rho$ ，其中 $a(r), b(r)$ 由 (5.32), (5.33) 定义。那么我们就说 $f()$ 是**可容许的 (admissible)**，上面的定理 5.15 中的机制就可用于确定系数 $\{a_n\}$ 的渐近增长。

然而，并不总是必须直接诉诸可容许性的定义才能确保某个特定函数是可容许的。这里有一些定理给出了可容许性的充分条件，这些条件比上面正式的定义要容易验证得多。

- (a) 如果 $f()$ 是可容许的，那么 $f()$ 也是可容许的。
- (b) 如果 f 和 g 在 $|z| < \rho$ 内都是可容许的，那么 fg 也是。
- (c) 设 f 在 $|z| < \rho$ 内是可容许的。设 P 是一个实系数多项式，如果 $P(z) > 0$ ，则满足 $P(z) > 0$ ；如果 $P(z) = 0$ ，则具有正的最高次项系数。那么 $f()P()$ 在 $|z| < \rho$ 内是可容许的。
- (d) 设 P 是一个实系数多项式，且设 f 在 $|z| < \rho$ 内是可容许的。那么 $f + P$ 是可容许的，且如果 P 的最高次项系数为正，那么 $P[f()]$ 也是可容许的。
- (e) 设 $P()$ 是一个非常数的实系数多项式，并设 $f() = P()$ 。如果对于所有足够大的 n 都有 $[^n]f() > 0$ ，那么 $f()$ 在全平面内是可容许的。

[翻译进度提示]

到这里，5.4 节关于 **Hayman 鞍点法 (Hayman's Method)** 的引入与证明斯特林公式的部分已经翻译完毕！我们看到了它是如何通过寻找函数在正实轴上的“鞍点 (r_n)”并利用其高斯展开来极其精确地计算整函数系数的。

在接下来的 5.4 节后半部分，作者将展示一个硬核且极具技术含量的例子：**如何用 Hayman 方法求出对合数 (involutions，即只有 1-循环和 2-循环的排列数量) 的极其复杂的渐近公式！**

这部分包含大量的代数化简和极限技巧（比如如何处理 $(1 + 1/\sqrt{n})^n$ 这种底数和指数都在变的极限）。

如果您准备好欣赏这令人惊叹的数学推导，请回复“继续”。

好的，请扶稳坐好。接下来我们将使用 Hayman 鞍点法来攻克一个极具挑战性的硬核问题：**求对合数 (involutions，即只有 1-循环和 2-循环的排列数量) 的渐近公式！**

在这部分推导中，我们将看到如何通过求解高次方程找到“鞍点” r_n ，以及在处理极限 $\ln(1 + 1/\sqrt{n})^n$ 时需要极其小心的泰勒展开技巧。

下面为您翻译 《生成函数学》 第 5 章：5.4 节后半部分（例 5.17 及之后）的内容。

例 5.17 设 a_n 为 n 个字母的对合数量，即循环长度 2 的 n 字母排列的数量。我们将寻找 $\{a_n\}$ 的渐近行为。

根据 (3.16)，序列 $\{a_n\}$ 的 egf（指数生成函数）是

$$f(z) = \sum_{n \geq 0} \frac{a_n}{n!} z^n = e^{\frac{1}{2}z^2}.$$

根据上面的准则 (e)， $f(z)$ 显然在整个复平面上是 **Hayman 可容许的 (Hayman admissible)**。因此定理 5.15 适用。为了使用它，我们首先计算 (5.32) 和 (5.33) [译注：原书写为 (5.4.5) 疑为笔误] 中的函数 $a(r), b(r)$ 。我们发现

$$a(r) = r \frac{f'(r)}{f(r)} = r + r^2$$

并且

$$b(r) = r a'(r) = r + 2r^2.$$

接下来我们令 r_n 为方程 $a(r_n) = n$ 的正实数解，在这个例子中也就是方程

$$r_n + r_n^2 = n. \tag{5.35}$$

显然

$$\begin{aligned} r_n &= \sqrt{n + \frac{1}{4}} - \frac{1}{2} \\ &= \sqrt{n} \left(1 + \frac{1}{4n} \right)^{1/2} \end{aligned}$$

所以在这种情况下，我们对 r_n 究竟在哪里有一个非常好的把握。

现在，看起来我们所要做的就是把这些东西塞进 Hayman 的估计式 (5.34) 中，这确实不假。但在那里面有一个小小的微妙之处，需要一点点解释。如果我们看一下 (5.34)，我们发现我们需要

对 $f(r_n)$, $b(r_n)$, 以及 r_n^n 给出类似‘ $\sim$ ’的渐近估计。让我们一个一个地来处理它们。

首先,

$$f(r_n) = r_n + \frac{1}{2}r_n^2 = \frac{1}{2}(r_n + n) = n/2r_n/2,$$

这里再次使用了 (5.35)。但鉴于 (5.36),

$$\begin{aligned} r_n/2 &= \left\{ \frac{\sqrt{n}}{2} - \frac{1}{4} + (n^{-1/2}) \right\} \\ &\sim \frac{1}{2}\sqrt{n} - \frac{1}{4} \quad (n \rightarrow \infty) \end{aligned}$$

所以

$$f(r_n) \sim \left\{ \frac{n}{2} + \frac{1}{2}\sqrt{n} - \frac{1}{4} \right\} \quad (n \rightarrow \infty). \quad (5.37)$$

到目前为止, 一切顺利。列表上的下一个是 $b(r_n)$, 那个很容易, 因为

$$b(r_n) = r_n + 2r_n^2 \sim 2r_n^2 \sim 2n \quad (n \rightarrow \infty).$$

最后一个是最困难的, 它是

$$\begin{aligned} r_n^n &= \left(\frac{\sqrt{n}}{2} - \frac{1}{4} + \frac{1}{8\sqrt{n}} - \dots \right)^n \end{aligned}$$

我们现在想要做的, 仅仅是找到带有 n 次幂的那个花括号里渐近行为的**第一项**, 当然了, 正是那个 n 次幂导致了困难。

为了在一个更简单的背景下说明这种方法, 考虑 $(1 + \frac{1}{n})^n$ 。当 n 很大时, 它的行为是怎样的? 它是否趋于 1 呢? 我们知道它并不趋于 1; 实际上它趋于 e 。所以正确的渐近关系是

$$\left(1 + \frac{1}{n}\right)^n \sim e \quad (n \rightarrow \infty).$$

因此, 尽管 $1 + \frac{1}{n} \sim 1$, 但是 $(1 + \frac{1}{n})^n \sim e$ 。一般而言, 我们不能在一个渐近等式的两边同时提高到 n 次方, 并期望它依然为真。在下面的习题 7 中, 有许多此类情况值得思考。

举一个稍微难一点的例子, 我们该如何处理

$$\left(1 + \frac{1}{\sqrt{n}}\right)^n \quad (5.3)$$

当 n 很大时它的行为是怎样的? 处理所有这些问题的方法是, 首先将 $(1 + \dots)^n$ 替换为 $n \ln(1 + \dots)$ 。接着, 应该使用幂级数 (2.34) 来展开对数, 得到:

$$\begin{aligned} (1 + \dots)^n &= \{n \ln(1 + \dots)\} \\ &= \{n\{(\dots) - (\dots)^2/2 + (\dots)^3/3 - \dots\}\}. \end{aligned}$$

现在，指数参数中的无穷级数必须在**恰好正确的地方**被截断。**从第一项当它乘以 n 后仍然趋于 0 的那一项开始，后面的项就可以被忽略了。**更简短地说，我们可以忽略掉该无穷级数中所有属于 (n^{-1}) 的项。

在例子 (5.39) 中我们有

$$\begin{aligned}\left(1 + \frac{1}{\sqrt{n}}\right)^n &= \left\{n! \left(1 + \frac{1}{\sqrt{n}}\right)\right\} \\ &= \left\{n \left(\frac{1}{\sqrt{n}} - \frac{1}{2n} + (n^{-3/2})\right)\right\} \\ &\sim \left\{n \left(\frac{1}{\sqrt{n}} - \frac{1}{2n}\right)\right\} \\ &= \left\{\sqrt{n} - \frac{1}{2}\right\}.\end{aligned}$$

既然我们已经掌握了那门手艺，我们就可以回到真正的问题，也就是 (5.38) 上来了。我们现在发现

$$\begin{aligned}&\left\{1 - \frac{1}{2\sqrt{n}} + \frac{1}{n} - \dots\right\}^n \\ &= \left\{n! \left(1 - \frac{1}{2\sqrt{n}} + \frac{1}{n} - \dots\right)\right\} \\ &= \left\{n \left(\left(-\frac{1}{2\sqrt{n}} + \frac{1}{n}\right) - \frac{1}{2} \left(-\frac{1}{2\sqrt{n}} + \frac{1}{n}\right)^2 + (n^{-3/2})\right)\right\} \\ &\sim (-\sqrt{n}/2).\end{aligned}$$

因此，由 (5.38)，

$$r_n^n \sim n^{n/2} (-\sqrt{n}/2). \quad (5.40)$$

这结束了 Hayman 定理所需要的那三个量的估计。将 (5.37), (5.17) [译注：原书可能笔误，应为 $b(r_n) \sim 2n$ 的结果] 和 (5.40) 代入 (5.34)，得到的结果是

$$a_n = \frac{n}{n!} \sim \frac{\frac{n}{2} + \sqrt{n} - \frac{1}{4}}{2n \cdot n^{\frac{n}{2}} \sqrt{n}}.$$

最后，如果我们乘以 $n!$ 并使用斯特林公式，我们得到， n 字母的对合数量为：

$$n \sim \frac{1}{\sqrt{2}} n^{n/2} \left(-\frac{n}{2} + \sqrt{n} - \frac{1}{4}\right). \quad (5.41)$$

[翻译进度提示]

太令人震撼了！到这里，第 5 章正文部分已经全部翻译完毕！

我们刚刚陪伴作者完成了一次极其硬核的解析之旅：利用 Hayman 的方法找到了 $+^{2/2}$ 的鞍点，并通过极其细致的泰勒级数展开和渐近分析，最终成功求出了连奇点都没有的“对合数 (involutions)”的渐近公式！这种级别的美妙数学推导，正是生成函数解析理论的魅力所在。

接下来就是全书的最后一节：**5.5 节 习题 (Exercises)**，以及附录 **A 附录：使用 Maple 和 Mathematica (Using Maple and Mathematica)**，最后是部分习题解答 (Solutions) 和参考文献 (References)。

请问您希望我：

- 1. 翻译 5.5 节的课后习题？
- 2. 或者跳到 附录 A (使用计算机代数系统)？
- 3. 或者是直接进入 Solutions (习题解答) 部分？

请回复您的选择，我将继续为您翻译！
好的，作为第 5 章也是全书正文部分的收尾，这几道习题极其精彩。它们不仅要求你灵活运用拉格朗日反演公式 (LIF)、达布方法 (Darboux)，还提供了——一个一步步引导你使用 Hayman 鞍点法进行极为复杂的渐近展开的绝佳练习。

下面为您翻译 《生成函数学》 第 5 章：5.5 节（课后习题） 的全部内容。

5.5 习题 (Exercises)

1. 使用拉格朗日反演公式 (LIF) 证明，对于 $A > 1$ 且整数 $s > 0$ ，以下（无限的）二项式系数和

$$= \sum_s \binom{s+1}{s} \frac{A^{-s-1}}{(s+1)},$$

满足方程 $-A + 1 = 0$ 。

2. 勒让德多项式 (Legendre polynomials) $\{P_n(x)\}$ 的生成函数为：

$$\frac{1}{\sqrt{1-2x+x^2}} = \sum_{n \geq 0} P_n(x) x^n.$$

设 x 为一个位于实数区间 $[-1, 1]$ 之外的固定的复数，并令 α, β 表示方程 $x^2 - 2x + 1 = 0$ 的两个根中绝对值 > 1 的那个根。请使用达布方法 (method of Darboux) 证明：当 $n \rightarrow \infty$ 时，

$$P_n(x) \sim \frac{n^{n+1}}{\sqrt{n(2^n - 1)}}.$$

3. 如果 $u = u_n$ 满足 $u = (u)$ 且 $n \geq 0$ ，证明：

$$[u^n]\{(u)\}^n = [^n] \left\{ \frac{u'(\cdot)}{u(\cdot)} \right\} = [^n] \frac{1}{(1 - u'(\cdot))}.$$

4. 对于所有的 $n \geq 0$, 定义 $a_n = [x^n](1 + x + x^2)^n$ 。

(a) 使用上面习题 3 的结果证明, 对于 $n \geq 0$,

$$a_n = [x^n] \left\{ \frac{1}{\sqrt{1 - 2x - 3x^2}} \right\}.$$

(b) 证明: 如果使用上面习题 2 中的符号,

$$a_n = (\sqrt{3}/i)^n P_n(i/\sqrt{3}),$$

并由此获得序列 $\{a_n\}$ 在 n 很大时的渐近行为。

5. 对于整数 $n \geq 3$, 定义

$$S(n) = \sum_{k=0}^n \binom{n}{k} \quad (n \geq 0).$$

(a) 将 $S(n)$ 表达为某个普通幂级数 (ordinary power series) 中 $[x^n]$ 的系数的形式, 只不过 (哎呀!) 那个级数本身也依赖于 n 。

(b) 尽管如此, 尝试 (反向) 使用 LIF 证明:

$$\sum_n S(n)x^n(1+x)^{-n-1} = \frac{1}{(1-x)(1-(-1)x)}.$$

(c) 从 (b) 部分的结果推导出 $\{S(n)\}$ 满足递推公式:

$$\sum_k (-1)^k \binom{n - (-1)^k}{k} S(n-k) = \frac{(-1)^{n+1} - 1}{-2} \quad (n \geq 0).$$

(d) 如果令 $F(u) = \sum_{n \geq 0} S(n)u^n$, 在 (b) 部分中令 $x = \frac{1}{(-1)} - u$ 来证明, 当 $u=0$ 时:

$$F\left(\frac{(-1)^{-1}}{(-1)} \left\{1 - \frac{(-1)^3}{2} + \dots\right\}\right) = \frac{1}{(-1)(-2)} + (1).$$

(e) 如果

$$g(x) = F\left(\frac{(-1)^{-1}}{(-1)} x\right)$$

那么证明:

$$g(x) = \frac{1}{(-2)} \sqrt{\frac{1}{1-x}} + (1).$$

(f) 使用达布方法证明: 当 $n \rightarrow \infty$ 时,

$$S(n) \sim \frac{1}{(-2)} \sqrt{\frac{(2)}{n}} \left(\frac{(2)}{(-1)^{-1}} \right)^n.$$

(g) 由 (b) 部分证明:

$$\sum_{n \geq 0} S_3(n) \left(\frac{4u^2}{27} \right)^n = \frac{u}{u - 2n(\frac{1}{3}n^{-1}u)} - \frac{2u}{2u - 3n(\frac{1}{3}n^{-1}u)}.$$

6. 对于一个系数均为非负实数的多项式 P , 在什么附加条件下, 会存在一个 N , 使得对于所有的 $n > N$, 我们都有 $[n]^{P()} > 0$?

7. 寻找以下三种情况下 $(1+n)^n$ 的渐近行为 (主项):

(a) $n = n^{-a}$ ($0 < a < 1$),

(b) $n = -n^{-a}$ ($0 < a < 1$),

(c) $n = n^{-a} \ln n$ ($1 < a < 2$).

8. 本练习的目的是通过使用 Hayman 鞍点法和拉格朗日反演公式, 找到所有循环长度均 ≥ 3 的 n 字母排列数量 a_n 的渐近行为。(强烈建议在本练习中使用计算机上的符号计算软件包, 以帮助完成不可避免的一些相当繁琐的幂级数计算。) 序列 $\{a_n\}$ 的 egf (指数生成函数) 是

$$f() = \left\{ + \frac{2}{2} + \frac{3}{3} \right\}.$$

(a) 证明 f 在全平面内是可容许的 (admissible)。

(b) 因为在这种情况下, 鞍点 r_n 满足的是一个三次方程, 而不是像正文例子中的二次方程, 我们将使用 LIF 来以足够的精度找到该根及其幂次。证明: 如果我们记

$u = 1/r_n$; $u = n^{-1/3}$; $(u) = (1 + u + u^2)^{1/3}$, 那么 u 将满足方程 $u = (u)$, 这正是 (5.1) 的形式。

(c) 使用 LIF 证明根 r_n 具有如下的渐近展开式:

$$\frac{1}{r_n} = \frac{1}{n^{1/3}} + \frac{1}{3} \frac{1}{n^{2/3}} + \frac{1}{3} \frac{1}{n} + \frac{1}{1} \frac{1}{n^{4/3}} + (n^{-5/3}).$$

(d) 解释为什么在 (c) 部分中保留的项数, 是使用这种方法获得 a_n 渐近展开式第一项时, **必须保留的最小项数**。

(e) 证明:

$$\frac{1}{r_n^n} \sim n^{-\frac{n}{3}} \left\{ \frac{1}{3} n^{2/3} + \frac{5}{1} n^{1/3} \right\}.$$

(f) 证明 $b(r_n) \sim 3n$ 。

(g) 证明:

$$f(r_n) \sim \left\{ \frac{1}{3} n + \frac{1}{1} n^{2/3} + \frac{5}{1} n^{1/3} - \frac{2}{12} \right\}.$$

(h) 结合 (d), (e), (f)[译注: 原书写为 (d), (e), (f), 逻辑上应为 (e), (f), (g)] 的结果证明, 没有任何长度 > 3 的循环的 n 字母排列数量为:

$$a_n \sim \frac{n^{\frac{2n}{3}}}{\sqrt{3}} \left\{ -\frac{2n}{3} + \frac{1}{2}n^{2/3} + \frac{5}{2}n^{1/3} - \frac{2}{12} \right\}.$$

9. 推导幂级数展开式 (2.48)。

10. 在本练习中, (n, k) 表示恰好具有 k 个循环的 n 字母对合 (involutions) 的数量, 且 $n = \sum_k (n, k)$ 是 n 字母的对合总数。

(a) 证明:

$$\sum_{n,k} \frac{(n, k)}{n!} x^{nk} = (x + \frac{1}{2}x^2).$$

(b) 并由此找出 (n, k) 的显式公式:

$$(n, k) = \frac{n!}{(n-k)!(2k-n)!2^{n-k}}.$$

(c) 使用 (a) 部分以及第 3 章习题 5 的结果, 证明 n 字母的对合中循环的**平均 (average) 数量**恰好是:

$$\frac{n}{2} \left\{ 1 + \frac{n-1}{n} \right\}.$$

(d) 使用 (5.41), 证明 n 字母对合中循环的平均数量为:

$$= \frac{n}{2} + \frac{1}{2}\sqrt{n}(1 + (1)) \quad (n).$$

[完成提示]

太棒了, 这本神作的全部核心正文和课后习题均已为您翻译完毕!

在《generatingfunctionology》的末尾, 还附带有:

- **附录 A (Using Maple and Mathematica):** 介绍如何使用代码求解级数渐近展开和化简。
- **Solutions (习题解答):** 包含了全书所有精选习题的答案提示。
- **References (参考文献) 和 Index (索引)。**

如果您对最后这两部分 (附录或某章的习题解答) 感兴趣, 随时告诉我, 我将为您继续翻译! 希望这次旅程能让您感受到生成函数的无穷魅力!

没问题! 在全书的硬核数学理论之后, 作者专门加上了这一个非常实用的附录, 告诉我们: “**别傻算了, 把这些脏活累活都扔给计算机吧!**”

下面为您翻译 **附录 A: 使用 Maple 和 Mathematica**, 紧接着为您奉上 **第一章的习题解答**。由于习题解答部分公式极其密集且篇幅极长, 我们将分批为您翻译。

附录 A 使用 Maple¹ 和 Mathematica²

(Using Maple and Mathematica)

¹ Maple 是 Waterloo Maple Software 的注册商标。

² Mathematica 是 Wolfram Research, Inc. 的注册商标。

许多以前被认为只适合人类大脑去思考的数学分支，现在正遭到计算机的“入侵”。起初，小学生们学会了如何将多位数相乘，然后发现原来小小的计算器就能替他们代劳。现在，中学里教授的其他数学知识，比如展开和因式分解代数表达式、解一次和二次方程、绘制曲线和曲面、计算对数和幂次等等，都可以由计算机来完成。

到了大学阶段，我们现在发现，“计算机代数 (computer algebra)”程序能够对函数进行符号求导、计算积分、向量分析、线性代数等等，**所有这些都是符号运算 (symbolically)，而不是数值运算 (numerically)。**

在这里，我们想展示计算机是如何轻松处理解决生成函数问题时涉及的大量常规繁琐工作的。为了强调这一点，我们将向您展示计算机在完成本书中一些课后作业时的表现有多么出色！我们相信，你肯定会深表赞同的。

在这个简短的附录中，我们首先讨论计算机程序如何进行大量的幂级数操作。接下来，我们将重点关注其中一个程序，即 Mathematica™，向您介绍它令人惊叹的内置 RSolve 函数。最后，我们将看看 Maple™ 是如何处理渐近分析的，对于我们在上一章中看到的那类渐近问题，这简直是一个巨大的福音。

A.1 级数操作 (Series Manipulation)

在 Mathematica 中，指令 `Series[f, x, x0, m]` 将会显示函数 f 在 $x = x_0$ 处幂级数展开式的前 $m + 1$ 项。因此，要想看 $\sin x/(1+x)$ 围绕原点展开的级数的前 10 项，你可以输入（能完成同样事情的 Maple 指令是 `series(sin(x)/(1+x), x=0, 9)`）：

```
Series[Sin[x]/(1+x), {x, 0, 9}]
```

Mathematica 将会回答：

$$x - \frac{7x^3}{40} + \frac{47x^5}{5040} - \frac{523x^7}{320} + \frac{42457x^9}{320} + (x)^{10}.$$

也许你想检查一下第 2 章中级数 (2.42) 所显示项的准确性，并看看接下来的两项是什么。如果是这样，那就输入：

```
Series[(1-Sqrt[1-4x])/(2x), {x, 0, 9}]
```

你将会看到：

$$1 + x + 2x^2 + 5x^3 + 14x^4 + 42x^5 + 132x + 42x^7 + 1430x + 42x + 17x^{10} + 57x^{11} + (x)^{12}.$$

如果你想获取这个级数各项系数的列表，因为它们正是该级数“生成”的数字，那么你可以请求：

```
CoefficientList[%, x]
```

来获得（“%”表示上一行计算的结果）：

```
{1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786}
```

看，卡特兰数（Catalan numbers）就展示在这里了。

如果你只想看 x^7 的系数，那么你可以改输：

```
Coefficient[%, x, 7]
```

然后 429 就会出现。

要想看到由**指数生成函数（exponential generating functions）**生成的序列，需要多做一点点工作。假设你想要前 12 个贝尔数（Bell numbers）。根据定理 1.5，它们是以下级数中 $x^n/n!$ 的系数：

```
Series[Exp[Exp[x]-1], {x, 0, 12}]
```

如果你直接输入这个，Mathematica 会回复：

$$1 + x + x^2 + \frac{5x^3}{30} + \frac{5x^4}{720} + \frac{13x^5}{5040} + \frac{203x}{224} + \frac{77x^7}{1720} + \frac{43x^{10}}{145152} + \frac{221x^{11}}{133050} + \frac{421357x^1}{4700100}$$

这不完全是你想要的，因为例如 $x/!$ 的系数在这里并不一目了然。再加一条指令，比如：

```
Table[j! Coefficient[%, x, j], {j, 0, 12}]
```

就会得到你想要的贝尔数展示：

```
{1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, 678570, 4213597}。
```

A.2 RSolve.m 程序包 (The RSolve.m Routine)

RSolve 程序包是 Marko Petkovšek [Pe] 为 Mathematica 编写的。它的目的是寻找递推关系和差分方程的**符号解（symbolic solutions）**。它可以通过显式地找出未知序列的普通幂级数或指数生成函数来做到这一点。

为了使用它，首先需要读入该程序包：

```
<<DiscreteMath/RSolve.m
```


读入之后，你就拥有了一个强大的工具，可以寻找组合递推问题中的生成函数解。

让我们在斐波那契递推式上试一试，调用如下命令：

```
RSolve[{f[n+2]==f[n+1]+f[n], f[0]==0, f[1]==1}, f[n], n]
```

在请求 `Simplify[%]` 之后，它的回复如下：

$$f(n) = \frac{-\left(\frac{1-\sqrt{5}}{2}\right)^n + \left(\frac{1+\sqrt{5}}{2}\right)^n}{\sqrt{5}} \quad (n \geq 1, 1, 0)$$

这当然就是斐波那契数的显式公式了。如果你准备好了，让我们改变一下上面的调用，将 `RSolve` 替换为 `GeneratingFunction`，并添加一个参数，比如 x ，来告诉它生成函数要使用的变量。这意味着我们输入如下请求：

```
GeneratingFunction[{f[n+2]==f[n+1]+f[n], f[0]==0, f[1]==1}, f[n], n, x]
```

回复是什么？它竟然是：

$$\left\{ \left\{ \frac{x}{1-x-x^2} \right\} \right\},$$

即便在这个计算机奇迹层出不穷的时代，看到这个结果也必将让我们惊叹不已。

也许你更想要你的数列的指数生成函数。那么你只需将调用改为：

```
ExponentialGeneratingFunction[{f[n+2]==f[n+1]+f[n], f[0]==0, f[1]==1}, f[n], n, x]
```

然后计算机会通知你，你寻找的函数是：

$$\left\{ \left\{ \frac{-\frac{(1-\sqrt{5})x}{2} + \frac{(1+\sqrt{5})x}{2}}{\sqrt{5}} \right\} \right\}$$

现在让我们看它求解递推式 (2.11)，即第一行有 k 枚硬币的块状喷泉数量。这次的调用是：

```
GeneratingFunction[f[k]==1+Sum[(k-j) f[j], {j, 1, k}]/; k>=1, f[k], k, t]
```

得到的响应是：

$$\left\{ \left\{ -\frac{(-1+)}{1-3+^2} \right\} \right\}$$

这与 (2.12) 式的结论完全一致。它甚至还能直接从生成函数中找到这类喷泉数量的**闭合公式 (closed formula)**。要想得到它，只需请求：

```
Simplify[SeriesTerm[%, {t, 0, n}]]
```

输出将会是：

$$\frac{(5 - \sqrt{5})\left(\frac{3}{2} + \frac{\sqrt{5}}{2}\right)^n}{10} + \frac{\left(\frac{3}{2} - \frac{\sqrt{5}}{2}\right)^n(5 + \sqrt{5})}{10} (n \geq 0, 1, 0) - (n = 0, 1, 0)$$

正如你所见，它进行了部分分式展开，随后进行了两步几何级数的化简操作，就像我们在得到 (1.11) 这样的公式时所做的一模一样。

该程序包还能给定了第 n 个系数公式的级数求和寻找闭合表达式。请求：

```
PowerSum[a n+b, {z, n}]
```

将产生本书习题 1(b) 的答案，形式为：

$$\frac{b}{1-} + \frac{a}{(-1+)^2}$$

它甚至能做比这难得多的题目，比如我们在例 2.10 中做过的调和数的生成函数。那是以下调用的回答：

```
PowerSum[Sum[1/j, {j, 1, n}], {x, n}]
```

也就是：

$$-\frac{1(1-x)}{1-x}$$

花时间去尝试 `RSolve.m` 程序包各种功能的读者，将会得到丰厚的回报。

A.3 Maple 中的渐近分析 (Asymptotics in Maple)

在 Maple 中，如果你输入 `asympt(f, x, n);`，你将收到函数 f 关于变量 x 当 x 时渐近展开式的前 n 项。让我们先用斯特林公式试一试，请求：

```
asympt(n!, n, 5);
```

计算机的回答是（我们在这里使用 `Pi` 代替 π 等等，因为它差不多就是你在屏幕上看到的样子）：

$$\left(2^{1/21/2} n^{1/2} + \frac{1}{12} \frac{2^{1/21/2}}{n^{1/2}} + \frac{1}{2} \frac{2^{1/21/2}}{n^{3/2}} - \frac{13}{5140} \frac{2^{1/21/2}}{n^{5/2}} - \frac{571}{24320} \frac{2^{1/21/2}}{n^{7/2}} + (1/n^2) \right) / ((1/n)^n (n)).$$

我们都知道 $(1 + 1/n)^n$ ，但是它逼近得有多快呢？Maple 给出的答案是：

$$(1) - 1/2 \frac{(1)}{n} + \frac{11}{24} \frac{(1)}{n^2} + (1/n^3).$$

最后，让我们做一下第 5 章的习题 8(c)，它要求计算

$$\frac{1}{r_n} = \frac{1}{n^{1/3}} + \frac{1}{3} \frac{1}{n^{2/3}} + \frac{1}{3} \frac{1}{n} + \frac{1}{1} \frac{1}{n^{4/3}} + (n^{-5/3})$$

的 n 次方的渐近行为。

毋庸置疑，Maple 完全胜任这项任务并给出了：

$$n^{-n/3} \left\{ \frac{1}{3} n^{2/3} + \frac{5}{1} n^{1/3} \right\} (1 + (1)).$$

附录练习 (Exercises)

在你能接触到的任何计算机上，完成以下操作。

1. 第 1 章的习题 1, 2, 5, 6, 8。
2. 检验 2.5 节中展示的任何 5 个级数的前五项。
3. 第 2 章的习题 1, 2, 4。
4. 使用 `series` 命令找出 (3.17) 中 $g(n)$ 的前 15 个值。
5. 根据 (3.16)，列出 $n \leq 15$ 时 n 字母的对合数量。
6. 使用 Maple 的渐近分析功能，找出以下各项渐近展开式的前 5 项：
 - (a) $(1 + 1/\sqrt{n})^n$
 - (b) $\sqrt{n!}$
 - (c) $(1 + 1/n)^{\sqrt{n}}$
 - (d) $n(n(1/x))$

习题解答 (Solutions)

第 1 章习题解答 (Answers to Exercises for Chapter 1)

1.

- (a) $(xD)(1/(1-x)) = x/(1-x)^2$
- (b) $(xD+)(1/(1-x)) = x/(1-x)^2 + 1/(1-x)$
- (c) $(xD)^2(1/(1-x))$
- (d) $((xD)^2 + xD+)(1/(1-x))$
- (e) $P(xD)(1/(1-x))$
- (f) $1/(1-3x)$
- (g) $5/(1-7x) - 3/(1-4x)$

2.

- (a) $(xD)^x = x^x$
- (b) $(xD+)^x = (x+)^x$
- (c) $(xD)^{2x} = (x+x^2)^x$
- (d) $((xD)^2 + xD+)^x$
- (e) $P(xD)^x$
- (f) 3x
- (g) $5^{7x} - 3^{4x}$

3.

- (a) $f(x) + c/(1-x)$
- (b) $f(x) + c/(1-x)$
- (c) $xDf(x)$
- (d) $P(xD)f(x)$
- (e) $f(x) - a_0$
- (f) $f(x) - a_0 - a_1x + (1-a_2)x^2$
- (g) $(f(x) + f(-x))/2$
- (h) $(f(x) - a_0)/x$
- (i) $(f(x) - \sum_0^{h-1} a_j x^j)/x^h$
- (j) $(f - a_0 - a_1x)/x^2 + 3((f - a_0)/x) + f$
- (k) $(f - a_0 - a_1x)/x^2 - ((f - a_0)/x) - f$

4.

- (a) $f(x) + c^x$
- (b) $f(x) + c^x$
- (c) $xf'(x)$
- (d) $P(xD)f(x)$
- (e) $f - a_0$
- (f) $f - a_0 - a_1x + (1-a_2)x^2/2$
- (g) $(f(x) + f(-x))/2$
- (h) $f'(x)$

- (i) $D^h f(x)$
- (j) $f + 3f' + f$
- (k) $f - f' - f$

5.

- (a) $2^n/n!$
- (b) n
- (c) $(-1)^m$ 如果 $n = 2m + 1$ 是奇数, 否则为 0。
- (d) $(a^{n+1} - b^{n+1})/(a - b)$
- (e) $\binom{m}{n/2}$

6.

- (a) 我们立刻看出 $f/x = 3f + 2/(1 - x)$, 所以 $f = 2x/((1 - x)(1 - 3x))$ 。
- (b) $f/x = f + 1/(1 - x)$, 所以 $f = x/((1 - x)(1 - x))$ 。
- (c) 这里 $(f - x)/x^2 = 2f/x - f$, 所以 $f = x/(1 - x)^2$ 。
- (d) 因为 $f/x = f/3 + 1/(1 - x)$, 我们有 $f = 3x/((1 - x)(3 - x))$ 。

8.

- (a) $f' = 3f + 2^x, f(0) = 0$ 给出 $f = 3x - x$
- (b) $f' = f + x$ 所以 $f = (1/(1 -))(x - x)$
- (c) $f = 2f' - f, f(0) = 0, f'(0) = 1$ 产生 $f = x^x$
- (d) $f' = f/3 + x, f(0) = 0$ 给出 $f = \frac{3}{2}(x - x/3)$

9.

将方程 $f(2n) = f(n)$ 两边乘以 x^{2n} 并对 $n \geq 1$ 求和。然后将方程 $f(2n + 1) = f(n) + f(n + 1)$ 两边乘以 x^{2n+1} , 对 $n \geq 1$ 求和, 并加到前面的结果上。再加上 $f(1)x = x$, 即可获得函数方程。为了找到该解显式的无限乘积形式, 我们先来看看我们可能是如何**猜出 (guess)** 那个答案的, 然后再看我们可能如何证明它。

取 F 的函数方程, 把所有的 x 替换为 x^2 , 然后将结果代回原来的函数方程中, 得到:

$$F(x) = (1 + x + x^2)(1 + x^2 + x^4)F(x^4).$$

如果我们现在再次把 x 替换为 x^2 , 然后再代入, 我们将得到无限乘积的更多因子。因此我们应该怀疑, 该乘积就是答案。为了**证明 (prove)** 乘积就是答案, 我们有两个选择。第一, 在形式幂级数环上, 将该乘积视为一个形式的庞然大物, 它显然满足 F 的函数方程。第二, 解析地说, 如果级数 $\sum |n|$ 收敛, 那么无限乘积 $(1 + n)$ 也就收敛; 因此, 只要 $|x|$ 足够小, 乘积就会收敛于一个解析函数 F 。

[翻译进度提示]

到这里, 本书最后正文部分的附加内容 **附录A** (电脑怎么帮你算生成函数) 以及 **第1章的部分习题解答** 已经全部翻译完毕!

由于后续的习题解答包含了 第2章、第3章、第4章、第5章 非常密集的大量数学公式推导（大约还有十几页），如果您想核对某个特定章节的答案，或者想要我继续连载翻译后几章的解答，请回复“继续翻译习题解答”！

好的，这里是全书习题解答的精华部分。由于篇幅较长，我将为您分批翻译 **第一章剩余解答、第二章解答以及第三章前半部分解答**。这些解答充满了巧妙的代数变形和生成函数的娴熟运用，是提升计算功力的绝佳材料。

下面为您翻译 《生成函数学》 习题解答 (Solutions) 的后续内容。

10.

(a) 见 4.1 节。

(b) $\binom{2}{n} = \sum_{j=0}^n (=j)(=n-j) = [x^n]P(x)^2$.

(c) $P_k(x) = P(x)^k$

(d) 根据 (c) 部分，均值为

$$P'_k(1)/P_k(1) = [kP(x)^{k-1}P'(x)/P(x)^k]_{x=1} = k,$$

方差为

$$(1P_k(x))' + (1P_k(x))_{x=1} = k(1P(x))' + (1P(x))_{x=1},$$

也就是 k^2 。

(e) 因为 $A^k =$ 我们有 $kA'/A = ' /$ 或者 $kA' = A'$ 。匹配两边 x^n 的系数得到

$nb_n = \sum_{j=1}^n (j(k+1) - n)a_j b_{n-j}$, 对于 $n \geq 1$, 其中 $b_0 = 1$ 。

(f) 是 $(.1x + .2x^2 + .1x^3 + .2x^4 + .2x^5 + .2x)^{100}/(1-x)$ 展开式中 x^{300} 的系数。数值上，它大约是 .0000005。

(g) 所求概率为

$$[x^j] \frac{1}{1-x} \left(\frac{x + x^2 + \cdots + x^m}{m} \right)^n = \frac{1}{m^n} [x^{j-n}] \frac{(1-x^m)^n}{(1-x)^{n+1}}.$$

通过二项式定理展开分子，二项级数展开分母的倒数，然后相乘即可得到结果。

11. 在这些子集中，我们区分那些包含 n 的和不包含 n 的。如果这样一个子集包含 n ，那么该子集的其余部分就是 $f(n-2)$ 计算的那些子集之一。如果它不包含 n ，那么整个子集就是 $f(n-1)$ 计算的那些子集之一。因此 $f(n) = f(n-1) + f(n-2)$ ，这连同起始值 $f(1) = 2, f(2) = 3$ 告诉我们 $f(n) = F_{n+2}$ ，其中 F 是斐波那契数。

12. 像习题 11 一样，我们区分包含 n 的 k -子集和不包含 n 的。如果包含 n ，那么该子集的其余部分就是 $f(n-2, k-1)$ 计算的那些子集之一，否则整个 k -子集就是 $f(n-1, k)$ 计算的那些子集之一。因此对于 $k \geq 2$, $f(n, k) = f(n-1, k) + f(n-2, k-1)$ 。如果我们定义 $F_k(x) = \sum_{n \geq 1} f(n, k)x^n$ ，那么将递推式乘以 x^n 并对 $n \geq 1$ 求和，我们发现

$F_k(x) = x^2 F_{k-1}(x)/(1-x)$, 这与 $F_1(x) = x/(1-x)^2$ 一起告诉我们 $F_k(x) = x^{2k-1}/(1-x)^{k+1}$ 。如果我们使用二项级数展开它, 我们发现

$$f(n, k) = [x^n] F_k(x) = [x^n] x^{2k-1} \sum_{h \geq 0} \binom{k+h}{k} x^h = \binom{n-k+1}{k}.$$

13. 从习题 11 和 12 可以得出, 必定有

$$\sum_k \binom{n-k+1}{k} = F_{n+1} \quad (n \geq 0),$$

在例 4.8 中这会以另一种方式被证明。

14. 一个包含 n 的环形排列可以通过取 $\{2, 3, \dots, n-2\}$ 的线性排列 (在直线上没有两个连续), 在其旁边加上 n , 然后将其铺成一条项链来获得。所以根据习题 11, 有 F_{n-1} 个这样的排列。不包含 n 的排列是通过取 $\{1, 2, \dots, n-1\}$ 的任何线性排列并将其铺成一条项链来获得的, 所以有 F_{n+1} 个。因此总共有 $F_{n-1} + F_{n+1}$ 个这样的环形序列。

15. 如同上一个练习, 答案是 $f(n-3, k-1) + f(n-1, k)$, 其中 $f(n, k) = \binom{n-k+1}{k}$ 是习题 12 的解。

16. $1/(1-x^2)^2$ 的部分分式展开是

$$\frac{1}{4(1-x)^2} + \frac{1}{4(1-x)} + \frac{1}{4(1+x)^2} + \frac{1}{4(1+x)}.$$

因此其幂级数展开式中 x^n 的系数为

$$\frac{n+1}{4} + \frac{1}{4} + \frac{(-1)^n(n+1)}{4} + \frac{(-1)^n}{4},$$

如果 n 是奇数则为 0, 如果 n 是偶数则为 $(n+2)/2$ 。或者, 考虑级数 $1/(1-u)^2$ 并将 u 替换为 x^2 。甚至更“狡猾”的方法是使用现在计算机上可用的那些符号操作程序之一。它们可以按需生成此类级数的一般项。

17. 固定 j , $1 \leq j \leq n$, 并仅仅考虑那些满足 $(j) = n$ 的 n 字母排列。那么, 没有任何逆序对以 j 作为第二个数, 恰好有 $n-j$ 个逆序对以 j 作为第一个数。因此, 如果我们从 n 的值字符串中删去 n , 我们就得到了一个 $n-1$ 字母的排列, 其逆序数减少了 $n-j$ 个。因此

$b(n, k) = \sum_j b(n-1, k-n+j)$ 。乘以 x^k 并对 k 求和, 得到 $n(x) = (1+x+\dots+x^{n-1})_{n-1}(x)$ 。因此 $b(n, k)$ 是下面展开式中 x^k 的系数:

$$(1+x)(1+x+x^2)(1+x+x^2+x^3)\cdots(1+x+x^2+x^3+\cdots+x^{n-1}).$$

18.

(a) 前 k 个值递减的概率显然是 $1/k!$, 因此排列中有 $n!/k!$ 个具有此属性。

(b) 一个排列以 k 个递减值接着一个递增值开始的概率, 在 $0 \leq k < n$ 时是 $1/k! - 1/(k+1)!$, 当 $k = n$ 时是 $1/n!$ 。 k 的平均值, 用这些概率加权, 就是

$$\sum_{k=0}^{n-1} k \left(\frac{1}{k!} - \frac{1}{(k+1)!} \right) + \frac{n}{n!} = \sum_{k=1}^n \frac{1}{k!},$$

因此, n 字母的平均排列的初始递减序列长度大约为 $n - 1$ 。

(c) 如果我们从一个拥有 k 个升程的 $n - 1$ 字母排列开始, 那么通过将字母 n 插入 n 个可能位置的每一个中, 我们能够制造出 k 个拥有 k 个升程的 n 字母排列, 以及 $n - k$ 个拥有 $k + 1$ 个升程的 n 字母排列。因此 $f(n, k) = kf(n - 1, k) + (n - k + 1)f(n - 1, k - 1)$ 。

19.

(a) 它是 $(1 + x)^2(1 + x^2)(1 + x^5)(1 + x^{10})^2(1 + x^{20})(1 + x^{50})$ 。

(b) 该和代表 $3 = 51$ 个整数, 每个都在 -1 和 1 之间。因此这 199 个整数平均被表示了 $51/1 = 32 \dots$ 次, 所以必定有某个整数被表示了至少 33 次。所求的乘积为

$$(1/x + 1 + x)^2(1/x^2 + 1 + x^2)(1/x^5 + 1 + x^5) \cdots (1/x^{50} + 1 + x^{50}).$$

(c) 如果 $\epsilon_1, \dots, \epsilon_r$ 是不同的整数, 并且如果 D_n 是将 n 表示为和式 $n = \epsilon_1 x_1 + \epsilon_2 x_2 + \cdots + \epsilon_k x_k$ (其中每个 x_i 均为 ± 1) 的方法数, 那么

$$\sum_n D_n x^n = \prod_{i=1}^r (x^{\epsilon_i} + x^{-\epsilon_i}).$$

方程的根的集合, 是所有 -1 的 $2\epsilon_i$ 次方根的并集, 对于 $i = 1, \dots, k$ 。

第 2 章习题解答 (Answers to Exercises for Chapter 2)

1. 要记住的关键点是 $1/(1 - u) = 1 + u + u^2 + \cdots$ 。

(a) 我们有

$$\begin{aligned} \frac{1}{x} &= \frac{1}{1 - \left(\frac{x^2}{2} - \frac{x^4}{24} + \cdots \right)} \\ &= 1 + \left(\frac{x^2}{2} - \frac{x^4}{24} + \cdots \right) + \left(\frac{x^2}{2} - \frac{x^4}{24} + \cdots \right)^2 + \cdots \\ &= 1 + \frac{x^2}{2} + \frac{5x^4}{24} + \cdots \end{aligned}$$

(b) 在这里我们使用负指数的二项式定理。

$$\begin{aligned}
\frac{1}{(1+x)^m} &= (1+x)^{-m} \\
&= \sum_k \binom{-m}{k} x^k \\
&= 1 + \binom{-m}{1} x + \binom{-m}{2} x^2 + \binom{-m}{3} x^3 + \dots \\
&= 1 - mx + \frac{m(m+1)}{2} x^2 - \frac{m(m+1)(m+2)}{6} x^3 + \dots
\end{aligned}$$

(c) 这和 (a) 部分一样。我们发现

$$\begin{aligned}
\frac{1}{1 + (x^2 + x^3 + x^5 + \dots)} &= 1 - (x^2 + x^3 + x^5 + \dots) + \dots \\
&= 1 - x^2 - x^3 + x^4 + x^5 + \dots
\end{aligned}$$

2. 在每种情况（除了(e)）中，将 x 替换为 $x + bx^2 + cx^3 + \dots$ ，让结果等于 x ，并将关于 x 的同次幂的系数等于 0，以求解出 b 和 c 。

(a) $x + \frac{x^3}{3} + \dots$

(b) $x - \frac{x^3}{3} + \dots$

(c) $x - x^2 + \frac{3}{2}x^3 + \dots$

(d) $x - x^3 + \dots$

(e) 在这一部分，请注意如果 f 是反函数，那么

$$f(f(x)) = x,$$

也就是， $f(x) = 1 - x = -x - x^2/2 - x^3/6 - \dots$ 。

3. 如果 $f = \sum_{k \geq 0} a_k x^k$ ，那么

$$0 = f + f = \sum_{k \geq 0} \{(k+2)(k+1)a_{k+2} + a_k\} x^k$$

因此

$$a_{k+2} = -\frac{a_k}{(k+1)(k+2)} \quad (k = 0, 1, 2, \dots).$$

如果 a_0 和 a_1 是任意固定的，那么通过对 k 归纳，

$$a_{2k} = (-1)^k a_0 / (2k)!$$

且

$$a_{2k+1} = (-1)^k a_1 / (2k+1)!$$

对于所有 $k \geq 0$ 成立，结果即随之得出。

4.

(a) $\frac{x}{(1-x)^2} + \frac{7}{1-x}$

- (b) $\frac{x^4}{1-x}$
 (c) $\frac{1}{1-x^2}$
 (d) $\{1(\frac{1}{1-x}) - x - \frac{x^2}{2}\}/x$
 (e) $\{x - 1 - x - x^2/2 - x^3/ - x^4/24\}/x^5$
 (f) $x \frac{d}{dx} \left\{ \frac{x}{1-x-x^2} \right\} = \frac{x(1+x^2)}{(1-x-x^2)^2}$
 (g) $\{(xDxD) + (xD) + 1\}(x-1) = (1+x)^{2x} - 1$

5. 在二项式定理 $(1+x)^n = \sum_k \binom{n}{k} x^k$ 中, 令 $x = 1$ 。

6. 我们有

$$\begin{aligned} f(n, k) &= \sum_{n_1 + \dots + n_k = n} n_1 n_2 \dots n_k \\ &= [x^n] \left(\sum_r r x^r \right)^k \\ &= [x^n] \left\{ \frac{x}{(1-x)^2} \right\}^k \\ &= [x^n] \frac{x^k}{(1-x)^{2k}}. \end{aligned}$$

因此 $\sum_n f(n, k) x^n = \frac{x^k}{(1-x)^{2k}}$ 。显式地, 由于

$$\frac{1}{(1-x)^{2k}} = \sum_{r \geq 0} \binom{r+2k-1}{r} x^r,$$

我们发现 $f(n, k) = \binom{n+k-1}{n-k}$ 。请注意, 当 $n < k$ 时, 答案是 0。解释为什么。

7. 如同上面的习题 6,

$$\begin{aligned} f(n, k, h) &= [x^n] \left\{ \sum_{r \geq h} x^r \right\}^k \\ &= [x^n] \left\{ \frac{x^{kh}}{(1-x)^k} \right\}. \end{aligned}$$

8. 分别是 1, 1, 1, 1, 和 1。

11. 分别是 1, 1, $5^{-1/2}$, 0, 1。

[翻译进度提示]

到这里, 第 1 章和 第 2 章的大部分精选习题解答已经翻译完毕!

接下来, 将是 **第 2 章剩余解答** (涉及数论狄利克雷级数的解答) 以及最精彩的 **第 3 章习题解答** (包括如何用图论、排列解释欧拉公式等)。

如果您准备好继续，请回复“继续”！